STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.1
April 2026

STAR Development Team
Locked Inc.

Contents

1 Nanopb Protocol **13**
- 1.1 Executive Summary . . . 13
- 1.2 Frame Format . . . 13
- 1.3 Frame Types . . . 14
- 1.3.1 LOG_MESSAGE (0x20) . . . 14
- 1.4 CRC and Error Handling . . . 14
- 1.5 References . . . 14

2 Protocol Buffer Schema Architecture **15**
- 2.1 Architecture Overview . . . 15
- 2.2 Service Specifications . . . 15
- 2.2.1 MotorControlService . . . 15
- 2.2.2 TelemetryService . . . 16
- 2.2.3 BatteryManagementService . . . 17
- 2.2.4 ConfigurationService . . . 17
- 2.3 common.proto– Shared Types . . . 18
- 2.4 Code Generation and Integration . . . 18

3 Hardware Pin Connections **19**
- 3.1 RX72N Microcontroller Pinout– - 144 - Pin LFQFP(R5F572NNHxFB) . . . 19
- 3.2 Motor Control Architecture . . . 24
- 3.2.1 GPTW vs MTU Selection for PWM Generation . . . 24
- 3.2.2 Pin Allocation Strategy . . . 25
- 3.2.3 Communication Bandwidth Analysis . . . 26
- 3.2.4 DS18B20+ Temperature Sensor . . . 27
- 3.3 IMU . . . 28
- 3.4 Motor Driver Control Architecture . . . 29
- 3.5 RX72N Configuration Tables . . . 29
- 3.6 Raspberry Pi 5 Pinout . . . 30
- 3.7 Additional Peripheral Connections . . . 30

4 Protocol Buffer Style Guide **31**
- 4.1 Terminology Standard . . . 31
- 4.2 Protocol Buffer Naming Conventions(Boston Dynamics Style) . . . 31
- 4.3 Unit Suffixes(MKS System) . . . 31
- 4.4 General Documentation Standards . . . 32

5 C Style Guide **33**
- 5.1 General Guidelines for ASM and Inline ASM . . . 33
- 5.2 When to Use ASM and Inline ASM . . . 34
- 5.3 Guidelines . . . 34
- 5.4 Platform-Specific Error Checking Macros . . . 34
- 5.5 General Rules . . . 35
- 5.6 Control Structures . . . 35
- 5.6.1 For Loops . . . 35
- 5.6.2 While Loops . . . 36
- 5.7 Functions . . . 36

5.8 Structs and Enums . . . 37
5.9 Initializer Lists . . . 38
5.10 Closing Brace Placement . . . 38
5.11 Summary Table . . . 39
5.12 General Rules . . . 39
5.13 Basic Switch Structure . . . 39
5.14 Case Block Braces . . . 40
5.15 Break Statement Requirements . . . 41
5.16 Default Case Handling . . . 42
5.17 Switch on Enum Types . . . 43
5.18 Indentation . . . 44
5.19 Summary . . . 44
5.20 General Commenting Guidelines . . . 45
5.21 File Header Comments . . . 45
5.22 Source File Header . . . 46
5.23 Doxygen Documentation . . . 46
5.23.1 Function Documentation . . . 46
5.23.2 Parameter Direction Markers . . . 47
5.23.3 Code Examples with @code / @endcode . . . 48
5.23.4 Struct Member Documentation . . . 48
5.24 Special Comment Keywords . . . 49
5.24.1 Keyword Definitions . . . 49
5.25 Section Comments . . . 49
5.26 Aligned Comments . . . 50
5.27 Implementation Comments . . . 50
5.28 Private Function Documentation . . . 51
5.29 Summary . . . 52
5.30 RTOS Types, Mutexes, Semaphores, and Atomic Operations . . . 52
5.31 General Rules . . . 52
5.32 Basic Enum Structure . . . 53
5.33 Explicit Value Assignment . . . 53
5.34 Documentation . . . 53
5.35 Enum to String Functions . . . 54
5.36 Using the Count Sentinel . . . 54
5.37 Switch Statement Pattern . . . 55
5.38 Forward Declaration . . . 55
5.39 Naming Convention Summary . . . 56
5.40 General Principles . . . 56
5.41 Platform Error Codes . . . 56
5.42 Validation Macros . . . 57
5.42.1 RX_RETURN_ON_FALSE . . . 57
5.42.2 ESP_GOTO_ON_ERROR . . . 58
5.42.3 ESP_ERROR_CHECK . . . 59
5.43 NULL Safety Patterns . . . 59
5.44 Mutex Error Handling . . . 60
5.45 Error Interface Dependency Injection . . . 61
5.46 Cleanup Patterns . . . 61
5.47 Error Logging . . . 62

5.48 Summary . . . 62
5.49 Return Value Conventions . . . 63
5.50 Private and Exposed Private Functions . . . 63
5.50.1 Private Functions (static) . . . 63
5.50.2 Exposed Private Functions (priv_) . . . 64
5.51 Parameter Validation . . . 64
5.52 Function Declaration Formatting . . . 65
5.53 Doxygen Documentation . . . 66
5.54 Private Function Documentation . . . 66
5.55 Function Implementation Patterns . . . 66
5.55.1 Single Exit Point with Cleanup . . . 66
5.55.2 Factory Function Pattern . . . 67
5.56 Callback Functions . . . 68
5.57 Summary . . . 69
5.58 General Principles . . . 69
5.59 Static File-Scoped Variables (s_ prefix) . . . 69
5.60 Why static const Over #define . . . 70
5.61 When to Use #define . . . 70
5.62 Explicit Type Usage - Always Use Fixed - Width Integer Types . . . 71
5.62.1 Why Explicit Types Matter . . . 71
5.62.2 Correct Type Usage . . . 71
5.62.3 Special Case: Characters and Strings . . . 72
5.62.4 Loop Counters and Array Indices . . . 72
5.62.5 Function Parameters and Return Types . . . 72
5.62.6 Type Casting for Enum Comparisons . . . 73
5.62.7 Summary: Type Usage Rules . . . 73
5.63 True Global Variables (g_ prefix) . . . 73
5.64 Dependency Injection Alternative . . . 74
5.65 Static Variables for Module State . . . 74
5.66 Thread Safety for Shared State . . . 75
5.67 Summary . . . 76
5.68 General Principles . . . 76
5.69 Include Guard Naming Convention . . . 76
5.70 C++ Compatibility . . . 77
5.71 Header File Structure . . . 77
5.72 Include Order . . . 78
5.73 Forward Declarations . . . 79
5.74 Separating Types from Functions . . . 79
5.75 Doxygen File Documentation . . . 80
5.76 Complete Header Template . . . 81
5.77 Summary . . . 82
5.78 General Rules . . . 82
5.79 Space After Control Keywords . . . 82
5.80 No Space After Function Names . . . 83
5.81 Binary Operators . . . 83
5.82 Unary Operators . . . 84
5.83 Pointer Declarations . . . 84
5.84 Comma and Semicolon Spacing . . . 84

5.85 Parentheses Spacing . . . 85
5.86 Parameter Alignment . . . 85
5.87 Variable and Comment Alignment . . . 86
5.88 Struct Initializer Alignment . . . 86
5.89 Summary . . . 87
5.90 General Rules . . . 87
5.91 Include Order . . . 87
5.92 Complete Example from Codebase . . . 88
5.93 FreeRTOS Include Order . . . 88
5.94 System Includes . . . 89
5.95 Platform HAL Includes . . . 89
5.96 Project Includes . . . 89
5.97 Header File Includes . . . 90
5.98 Conditional Includes . . . 90
5.99 Include Comments . . . 90
5.100 Avoiding Circular Dependencies . . . 91
5.101 Summary . . . 91
5.102 General Rules . . . 92
5.103 Functions and Variables . . . 92
5.104 Module Prefixes . . . 92
5.105 Constants and Macros . . . 93
5.106 Type Names . . . 93
5.107 Enum Members . . . 94
5.108 Static Variables (File-Scoped) . . . 95
5.109 Global Variables . . . 95
5.110 Private Functions . . . 95
5.110.1 File-Scoped Static Functions (internal_ prefix) . . . 95
5.110.2 Exposed Private Functions (priv_ prefix) . . . 96
5.111 Summary Table . . . 97
5.112 Typedef Struct Pattern . . . 97
5.112.1 Standard Pattern . . . 97
5.112.2 Error Handler Example . . . 98
5.113 Union Usage for Protocol Configurations . . . 98
5.113.1 Protocol - Specific Structures . . . 99
5.114 Callback Function Types . . . 99
5.115 Interface Patterns (Dependency Injection) . . . 100
5.115.1 Interface Usage Pattern . . . 101
5.116 Opaque Handle Patterns . . . 102
5.117 Operations Structure Pattern . . . 102
5.118 Manager Structure Pattern . . . 103
5.119 Best Practices . . . 103
5.120 Library Directory Structure . . . 104
5.120.1 Example: star_bus Library . . . 104
5.121 CMakeLists.txt Structure . . . 104
5.122 Header File Organization . . . 105
5.122.1 Header File Template . . . 105
5.122.2 Header Section Order . . . 106
5.123 Include Guard Naming Convention . . . 107

5.124 Source File Organization . . . 107
5.124.1 Source File Template . . . 107
5.124.2 Source Section Order . . . 109
5.125 Types Header Separation . . . 109
5.125.1 Controller Header Pattern . . . 110
5.126 Project Root Structure . . . 110
5.127 Best Practices . . . 111
5.128 Overview . . . 111
5.129 SPI Bus Terminology . . . 111
5.129.1 COPI / CIPO(Not MOSI / MISO) . . . 111
5.130 I2C and General Bus Terminology . . . 112
5.130.1 Controller / Peripheral(Not Master / Slave) . . . 112
5.131 Mapping Platform Legacy APIs . . . 113
5.132 Documentation Guidelines . . . 114
5.133 Code Review Checklist . . . 114
5.134 Common Patterns . . . 115
5.135 Summary . . . 115
5.136 Dependency Injection (DIP) . . . 115
5.136.1 Interface Definition . . . 115
5.136.2 Interface Implementation . . . 116
5.136.3 Dependency Injection Pattern . . . 116
5.136.4 NULL Interface Handling . . . 117
5.137 Bus Abstraction Pattern . . . 117
5.137.1 Unified Configuration . . . 117
5.137.2 Factory Functions . . . 118
5.137.3 Manager Pattern . . . 118
5.138 Operations Structure Pattern . . . 119
5.139 Thread-Safe Callback Pattern . . . 120
5.140 Thread Safety Patterns . . . 120
5.140.1 Mutex Timeout Convention . . . 121
5.140.2 Mutex with Cleanup(goto pattern) . . . 121
5.141 Error Handler Ownership Pattern . . . 122
5.142 Linked List Management . . . 123
5.143 Best Practices Summary . . . 123
5.144 Prefer Alternatives to Macros . . . 124
5.145 When Macros Are Necessary . . . 124
5.146 Macro Safety Rules . . . 124
5.146.1 Wrap Statement - Like Macros in `do -while (0)` . . . 124
5.146.2 Parenthesize All Macro Arguments . . . 125
5.146.3 Beware of Side Effects and Multiple Evaluation . . . 125
5.147 Multi - line Macro Formatting . . . 125
5.148 Naming Conventions . . . 126
5.149 Include Guards . . . 126
5.150 Conditional Compilation . . . 126
5.151 Overview . . . 127
5.152 Benefits . . . 127
5.153 Standard Unit Suffixes . . . 128
5.154 Usage Examples . . . 128

5.154.1 Distance and Velocity . . . 128
5.154.2 Angular Measurements . . . 128
5.154.3 Temperature . . . 129
5.154.4 Time . . . 129
5.154.5 Electrical Measurements . . . 129
5.154.6 Percentage . . . 130
5.155 Protocol Buffer Integration . . . 130
5.156 Conversion Functions . . . 130
5.157 Summary . . . 131
5.158 No Dynamic Allocation . . . 131
5.158.1 Rationale . . . 131
5.158.2 Alternatives . . . 132
5.158.3 Platform Heap Functions . . . 132
5.159 Avoid Large Stack Allocations . . . 133
5.159.1 Stack Size Constraints . . . 133
5.159.2 Examples . . . 133
5.159.3 Stack Usage Guidelines . . . 134
5.160 Use `const` for Flash Storage . . . 134
5.160.1 Rationale . . . 134
5.160.2 Examples . . . 134
5.160.3 Pointer Declarations . . . 135
5.161 Validate All Buffers . . . 135
5.161.1 Rationale . . . 135
5.161.2 Validation Patterns . . . 135
5.161.3 DMA Buffer Validation . . . 136
5.161.4 Ring Buffer Validation . . . 137
5.162 Summary . . . 137
5.163 Interrupt Service Routines(ISRs) . . . 138
5.163.1 Minimize ISR Work . . . 138
5.163.2 ISR Constraints . . . 138
5.164 No Blocking Delays . . . 139
5.165 Real - Time and Safety . . . 140
5.165.1 Timing Constraints . . . 140
5.165.2 Watchdog Timers . . . 140
5.165.3 Sensor Timeouts . . . 141
5.165.4 Fail - Safe Design . . . 141
5.165.5 CRC Validation . . . 142
5.166 Hardware Abstraction . . . 143
5.166.1 Use Abstract Bus Interfaces . . . 143
5.166.2 Debounce Mechanical Inputs . . . 144
5.166.3 Enable Brown - Out Detection . . . 144
5.166.4 Flash Wear - Leveling . . . 144
5.166.5 Always Use Pull - Up / Pull - Down . . . 145
5.167 Performance and Real - Time . . . 145
5.167.1 Benchmark, Don’t Guess . . . 145
5.167.2 Avoid Blocking Operations . . . 146
5.168 Power Management . . . 146
5.168.1 Use Sleep Modes . . . 146

5.168.2 Use DMA to Reduce CPU Load . . . 147
5.169 Testing and Debugging . . . 147
5.169.1 Use Mock Drivers . . . 147
5.169.2 Meaningful Logging . . . 147
5.170 Reliability and Fault Tolerance . . . 147
5.170.1 Reset Peripherals if Stuck . . . 147
5.171 Summary . . . 148
5.172 nanopb Overview . . . 148
5.173 Why Static Allocation ? . . . 148
5.174 Field Size Configuration . . . 148
5.175 Standard Field Size Constraints . . . 149
5.176 Examples . . . 149
5.176.1 String Fields . . . 149
5.176.2 Repeated Fields . . . 149
5.176.3 Bytes Fields . . . 149
5.177 Sizing Guidelines . . . 150
5.177.1 String Sizes . . . 150
5.177.2 Repeated Field Counts . . . 150
5.177.3 Bytes Field Sizes . . . 150
5.178 Memory Impact . . . 151
5.178.1 Buffer Size Calculation . . . 151
5.178.2 RAM Usage Example . . . 151
5.179 Best Practices . . . 151
5.179.1 Right - Size Your Buffers . . . 151
5.179.2 Use Fixed - Size Types . . . 151
5.179.3 Validate Message Sizes . . . 152
5.180 Integration with Code Generation . . . 152
5.180.1 buf.gen.yaml Configuration . . . 152
5.180.2 Options File Naming . . . 152
5.181 Common Pitfalls . . . 152
5.181.1 Missing Options File . . . 152
5.181.2 Mismatched Field Names . . . 153
5.181.3 Forgetting Array Count . . . 153
5.182 Summary . . . 153
5.183 NASA Power of 10: Rules for Safety-Critical Code . . . 153
5.183.1 Compliance Status Legend . . . 154
5.183.2 Rule 1 : Simplify Control Flow . . . 154
5.183.3 Rule 2 : Fixed Loop Upper - Bounds . . . 154
5.183.4 Rule 3 : No Dynamic Memory After Initialization . . . 155
5.183.5 Rule 4 : Keep Functions Short . . . 155
5.183.6 Rule 5 : Use Assertions Generously . . . 156
5.183.7 Rule 6 : Declare Data at Smallest Scope . . . 156
5.183.8 Rule 7 : Check All Return Values and Parameters . . . 157
5.183.9 Rule 8 : Limit Preprocessor Use . . . 157
5.183.10 Rule 9 : Restrict Pointer Use . . . 158
5.183.11 Rule 10 : Compile with Maximum Warnings . . . 158
5.183.12 Summary: STAR Compliance Matrix . . . 159
5.183.13 Defensive Coding Measures . . . 159

5.183.14 References . . . 160

6 Gateway Architecture 161
6.1 Overview . . . 161
6.2 Protocol Stack Implementation . . . 161
6.3 Directory Structure . . . 161
6.4 Frame Package . . . 162
6.4.1 Frame Constants . . . 162
6.4.2 Frame Types . . . 162
6.5 Transport Package . . . 162
6.5.1 Available Transports . . . 162
6.5.2 SPI Configuration . . . 163
6.5.3 Transport Interface . . . 163
6.6 Link Layer (HARQ) . . . 163
6.6.1 SPILink . . . 163
6.6.2 CDCLink . . . 163
6.6.3 Shared Session State . . . 163
6.6.4 HARQ Parameters . . . 164
6.7 Transport Manager . . . 164
6.7.1 Transport Priorities . . . 164
6.7.2 Failover Behavior . . . 164
6.8 gRPC Services . . . 164
6.9 Build and Test . . . 165
6.10 Dependencies . . . 165

7 ROS2 Integration and Sensor Fusion Architecture 166
7.1 Overview . . . 166
7.2 Visual-LiDAR Sensor Fusion with RTAB-Map . . . 166
7.2.1 Fusion Architecture . . . 166
7.2.2 Mapping Strategy . . . 166
7.3 Coordinate Frames (REP-105) . . . 166
7.4 Custom ROS2 Nodes and Interfaces . . . 167
7.4.1 `star_spi_bridge` . . . 167
7.4.2 `star_gateway_bridge` . . . 167
7.4.3 `star_safety_monitor` . . . 167
7.5 Multi-Machine Communication . . . 167
7.6 Hardware Persistence (udev rules) . . . 167

8 ROS2 C++ Style Guide 168
8.1 Overview . . . 168
8.1.1 When to Use This Guide . . . 168
8.1.2 Key Differences . . . 168
8.2 Naming Conventions . . . 168
8.2.1 Classes and Types . . . 168
8.2.2 Methods and Functions . . . 169
8.2.3 Variables . . . 169
8.2.4 Namespaces . . . 169
8.3 File Naming and Organization . . . 170

8.3.1 Header Files . . . 170
8.3.2 Source Files . . . 170
8.4 Header Organization . . . 170
8.5 Code Formatting . . . 171
8.5.1 Line Length . . . 171
8.5.2 Indentation . . . 171
8.5.3 Braces . . . 171
8.6 ROS2-Specific Patterns . . . 172
8.6.1 Node Inheritance . . . 172
8.6.2 Publishers and Subscribers . . . 173
8.6.3 Timers . . . 173
8.7 Error Handling . . . 173
8.8 Logging . . . 174
8.9 Documentation . . . 174
8.10 Constants . . . 175
8.11 Formatting Enforcement . . . 175
8.11.1 Manual Formatting . . . 176
8.11.2 CI Enforcement . . . 176
8.12 References . . . 176
8.13 Integration with Existing Tools . . . 176
8.14 Summary . . . 176

9 Communication Stack **177**
9.1 Overview . . . 177
9.2 Why three paths . . . 177
9.3 Layers . . . 177
9.4 Log multiplexing . . . 178

10 USB Modes and Bench Behavior **178**
10.1 Transport Selection at the Gateway . . . 178
10.1.1 Simple USB mode . . . 178
10.1.2 Complex (full) USB mode – `force-usb` . . . 179
10.1.3 Forcing SPI – `force-spi` . . . 179
10.2 USB enumeration topology . . . 179
10.3 Re-enumeration: when ttyACM* changes . . . 179
10.4 Bench JTAG interaction (E2 Lite vs CY7C65213) . . . 180
10.5 Clock-source notes (TOM vs PROD) . . . 180
10.6 References . . . 180

11 Repository Layout: A Newcomer's Map **181**
11.1 Project shape, in one paragraph . . . 181
11.2 Top-level layout . . . 182
11.3 Top-level files . . . 183
11.4 Inside `star-rx72n-firmware/` . . . 184
11.4.1 The 26 firmware libraries . . . 185
11.4.2 Where firmware tasks live . . . 186
11.5 Inside `star-gateway/` . . . 186
11.6 Inside `star-ros2/` . . . 187

11.7 Inside `star-proto/` . . . 187
11.8 Inside `scripts/` . . . 188
11.9 Inside `final_docs/` . . . 188
11.10 Where do I look if I want to...? . . . 189
11.11 Branch + commit conventions . . . 189
11.12 Things this repository does not contain . . . 190

12 Boot Sequence and Clock Tree **190**
12.1 Power-on reset path . . . 190
12.2 Clock tree . . . 191
12.2.1 Which peripheral runs on which clock . . . 191
12.3 TOM vs PROD clock-source notes . . . 191
12.4 Reset sources . . . 191
12.5 Where to look in code . . . 192

13 Build System and CI **192**
13.1 Toolchain inventory . . . 193
13.2 Devcontainer (the canonical build environment) . . . 193
13.3 Top-level Makefile entry points . . . 193
13.4 Build-time feature flags . . . 194
13.5 Per-subproject build . . . 195
13.6 CI workflows . . . 195
13.7 What "green" means at PR time . . . 195
13.8 Reproducing CI locally . . . 196

14 Motor Control Loop **196**
14.1 Loop topology in one picture . . . 196
14.2 The loop body, step by step . . . 197
14.3 Where commands come from . . . 197
14.4 Where telemetry goes . . . 198
14.5 Concurrency model . . . 198
14.6 Safety interactions . . . 198
14.7 Tuning workflow . . . 198
14.8 Where to look in code . . . 199

15 Safety Architecture **199**
15.1 Defense in depth, in three layers . . . 199
15.2 The supervisor model . . . 200
15.3 Estop signal flow . . . 200
15.4 Watchdog architecture . . . 200
15.5 Hardware-level safety: DRV8263 + DRVOFF . . . 201
15.6 ThreadX FPU and stack safety . . . 201
15.7 Lifecycle nodes (ROS2 side) . . . 202
15.8 Where to look in code . . . 202

16 Test Strategy **202**
16.1 Layer 1: Firmware host unit tests (Unity) . . . 202
16.2 Layer 2: Cross-build verification . . . 203
16.3 Layer 3: Gateway Go tests (race + coverage) . . . 203

16.4 Layer 4: Hardware-in-the-Loop (HIL) . . . 203
16.5 Layer 5: ROS2 colcon tests . . . 203
16.6 Layer 6: Compliance engine tests . . . 204
16.7 Layer 7: Proto round-trip tests . . . 204
16.8 Layer 8: UI tests . . . 204
16.9 Layer 9: Static analysis . . . 204
16.10 What you should run before every commit . . . 204
16.11 Coverage gaps to be honest about . . . 205

17 Troubleshooting and Known Gotchas 205
17.1 USB / CDC enumeration . . . 205
17.1.1 ttyACM* indices change after re-flash . . . 205
17.1.2 No CY7C65213 UART bridge while E2 Lite is plugged in . . . 205
17.1.3 TOM enumerates fine on Pi 5 despite HUM warnings . . . 205
17.1.4 USB ISR not firing despite ICU/IER configured . . . 206
17.2 Bench-rig pitfalls . . . 206
17.2.1 AD2 Single-mode buffer is 4096 samples, period . . . 206
17.2.2 sci9_debug_puts hangs from a ThreadX task . . . 206
17.3 Build / toolchain . . . 207
17.3.1 Devcontainer rebuild loop . . . 207
17.3.2 Local rx-elf-gcc fails; should I use the devcontainer? . . . 207
17.4 Coding-policy reminders . . . 207
17.4.1 No NOLINT for readability-* clang-tidy checks . . . 207
17.4.2 Trust headers over comments . . . 207
17.4.3 Trust known-good bench tests over fresh code . . . 207
17.5 Common red herrings . . . 208
17.5.1 "It looks like the MCU halts 8 ms after rfp-cli disconnect" . . . 208
17.5.2 "USB enumeration is broken because BRR is wrong" . . . 208
17.5.3 "The peripheral isn't responding to register writes" . . . 208
17.6 Where to look in code . . . 208

18 Glossary 209

19 Compliance Engine 212
19.1 Executive summary . . . 212
19.2 Pipeline in one picture . . . 212
19.3 Detectors . . . 213
19.4 Engines . . . 214
19.5 Nodes (one ROS2 node per ADA check) . . . 215
19.6 Report generation . . . 217
19.7 Bring-up: `compliance.launch.py` . . . 217
19.8 Where to look in code . . . 218

20 Operator UI 218
20.1 Executive summary . . . 218
20.2 Layout overview . . . 219
20.3 Panels . . . 220
20.4 State management . . . 222

20.4.1 `store/dashboardStore.ts` . . . 222
20.4.2 `store/useWindowStore.ts` . . . 222
20.5 Custom hooks . . . 223
20.6 Service layer . . . 223
20.7 WebSocket protocol . . . 224
20.8 Build and run . . . 224
20.9 Theming and layout . . . 225
20.9.1 Theme . . . 225
20.9.2 Layout engine . . . 225
20.10 Testing . . . 225
20.11 Where to look in code . . . 226

21 Operations Runbook **226**
21.1 Scope . . . 226
21.2 Daily start sequence . . . 226
21.3 Daily stop sequence . . . 228
21.4 Dev mode . . . 229
21.5 Field deployment checklist . . . 230
21.6 Cockpit URLs and ports . . . 230
21.7 Systemd unit reference . . . 232
21.8 Wi-Fi and Ethernet helpers . . . 233
21.9 Common runtime issues . . . 233
21.10 Log locations . . . 234
21.11 Ops references . . . 234

22 Logging Architecture **235**
22.1 Scope . . . 235
22.2 Layers . . . 235
22.3 Log levels . . . 235
22.4 Tagging convention . . . 236
22.5 Three logging API tiers . . . 236
22.6 Banner: ISR vs task context . . . 237
22.7 The deferred-ISR-log machinery . . . 237
22.8 LOG_MESSAGE wire format . . . 238
22.9 Multi-transport log multiplexing . . . 238
22.10 Gateway-side log forwarding . . . 238
22.11 Operator-visible log paths . . . 239
22.12 Performance constraints . . . 239
22.13 Where to look in code . . . 240

1 Nanopb Protocol

1.1 Executive Summary

Overview: Protocol Buffers (nanopb) encapsulated in length-prefixed, CRC-32-protected frames. A single wire format is shared across three transport layers; the host can use any combination, and the firmware exposes all three concurrently.

- **USB CDC (USB0, primary)**: RX72N USB0 peripheral exposes a 3-port CDC composite device. The Pi5 enumerates it as `/dev/ttyACM{1,2,3}` (PROD board) or `/dev/ttyACM{4,5,6}` (TOM board). Port 1 carries the framed protocol (commands + telemetry); port 2 streams a decoded human-readable view; port 3 carries the runtime log stream. Bench-verified working on both boards 2026-04-24 (PROD via crystal, TOM via HOCO-PLL clock path).
- **UART-over-USB** (legacy / fallback): RX72N SCI9 → CY7C65213 USB-UART bridge → Raspberry Pi 5 `/dev/ttyUSB0` at 921 600 baud. Single-port; carries the same framed protocol when USB CDC is unavailable. Useful in bring-up because the bridge chip is independent of the RX72N's USB peripheral state.
- **SPI** (dedicated ROS2 link): RSPI2 → `/dev/spidev0.0` at 10 MHz, full-duplex. Terminated on the Pi5 by the `star_spi_bridge` ROS2 node for motor-control and telemetry paths that need sub-millisecond latency. HARQ (Chase Combining over a rate-$\frac{1}{2}$ K=7 convolutional code) protects against motor-driver EMI.

The three transports use the same wire format described below; the firmware multiplexes them via the comm-manager layer so the host can pick any one of them and see the same frame stream.

1.2 Frame Format

SYNC	SEQ	LEN	TYPE	FLAGS	PAYLOAD	CRC-32
2 B (LE)	2 B (LE)	2 B (LE)	1 B	1 B	0–1024 B	4 B (LE)

- **SYNC**: fixed `0x55AA`, wire bytes `0xAA 0x55`.
- **SEQ**: 16-bit monotonic sequence number (wraps at 65535).
- **LEN**: payload length in bytes, $0 \leq$ `LEN` ≤ 1024.
- **TYPE**: frame-type discriminator (see below).
- **FLAGS**: bitmask (`REQUIRES_ACK`, `RETRANSMIT`, `PRIORITY`, `FEC_ENABLED`, `SOFT_NACK`).
- **PAYLOAD**: nanopb-encoded protobuf message (for data frames) or raw ASCII (for `LOG_MESSAGE`).
- **CRC-32**: IEEE 802.3 polynomial `0x04C11DB7` computed over SYNC + header + payload.

Minimum frame: 12 bytes (header + CRC, empty payload). Maximum: 1036 bytes.

1.3 Frame Types

TYPE	Name	Purpose
0x00	PING	Liveness probe (either direction)
0x01	PONG	Liveness reply echoing payload
0x10	COMMAND	Pi5 → RX72N, protobuf command
0x11	RESPONSE	RX72N → Pi5, protobuf response / telemetry
0x12	ACK	SPI-only positive acknowledgment
0x13	NACK	SPI-only negative acknowledgment
0x20	LOG_MESSAGE	RX72N → Pi5, ASCII log chunk
0xFE	RESET_ACK	Session reset acknowledgment
0xFF	RESET	Session reset request

1.3.1 LOG_MESSAGE (0x20)

The RX72N's `rx_log_*` macros enqueue ASCII bytes into an in-RAM ring buffer (`rx_log_uart.c`). The communication task drains the ring on every 10 ms poll tick, wraps the drained bytes in a `LOG_MESSAGE` frame, and transmits over the active transport (USB CDC port 3 when the 3-port composite device is enumerated; otherwise SCI9 UART or the SPI log channel). The gateway's frame dispatcher (`manager.go`) extracts the payload and prints it to `stderr`/log file with an `[RX72N]` prefix.

Framing the log bytes (rather than writing raw ASCII) prevents the log text from containing the `0x55AA` sync word and fooling the frame decoder.

1.4 CRC and Error Handling

CRC-32 is computed once per frame and mismatched frames are discarded. On SPI, the link layer (`rx_spi_link`) additionally runs a Chase-Combining HARQ loop with rate-$\frac{1}{2}$ convolutional FEC and Viterbi decoding to recover from EMI bursts. UART relies on the CRC alone plus bounded resync (`rx_frame_decode_with_resync`); the bridge chip and USB CDC transport on the host provide sufficient lossless transport for the gateway path.

1.5 References

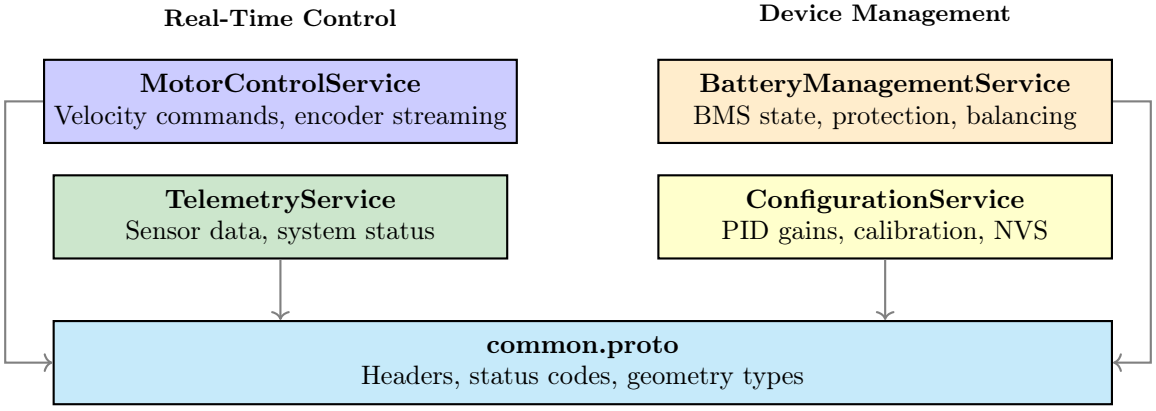
- `star-rx72n-firmware/libs/rx_frame/inc/rx_frame.h` – frame format reference (bit-exact with gateway).
- `star-rx72n-firmware/libs/rx_usb/` – USB0 USBb peripheral driver (3-port CDC composite, primary transport).
- `star-rx72n-firmware/libs/rx_uart_comm/` – UART framer (SCI9 + CY7C65213 fallback path).
- `star-rx72n-firmware/libs/rx_spi_link/` – SPI HARQ link.
- `star-rx72n-firmware/libs/rx_comm_manager/` – multiplex across USB / UART / SPI.
- `star-gateway/internal/frame/frame.go` – Go reference.
- `star-ros2/src/star_spi_bridge/` – ROS2 SPI consumer.

2 Protocol Buffer Schema Architecture

This section documents the Protocol Buffer schema architecture for RPi5↔RX72N communication. All schemas follow the **Boston Dynamics Protobuf Style Guide** for consistency, maintainability, and industry best practices.

2.1 Architecture Overview

Overview: Five gRPC services define the complete API surface for robot control, organized by functional domain. Services communicate via the nanopb-serialized frames described in Section 1.



Key Design Principles

- **Versioned Package :** All services use `star.v1` namespace for API evolution
- **Request/Response Headers:** Every RPC includes tracing, timestamps, and status
- **Streaming Support:** Continuous data uses server streaming (up to 100Hz)
- **Static Allocation:** All messages sized for nanopb constraints (no dynamic memory)

Note: Naming conventions, unit suffixes, and coding standards are consolidated in Section 4 (Style Guide and Conventions).

2.2 Service Specifications

2.2.1 MotorControlService

Purpose: Low-latency differential drive control with real-time encoder feedback.

Target Latency: <10ms round-trip for velocity commands.

Table 1: MotorControlService RPCs

RPC	Type	Description
SetVelocity	Unary	Set left / right wheel velocities(m / s)
EmergencyStop	Unary	Immediate stop, priority over all commands
SetMotorPower	Unary	Direct PWM control(bypass PID, testing only)
StreamEncoders	Server stream	Encoder ticks and velocity at 1 - 100Hz
ControlStream	Bidirectional	Velocity in, encoder feedback out

Key Messages:

- `VelocityCommand` –Left / right velocity(m / s), sequence number, timestamp
- `EncoderData` –Motor ID, tick count(341.2 PPR), velocity, timestamp
- `MotorStatus` –Duty cycle, velocity, temperature, current, fault flags
- `PidConfig` –Kp, Ki, Kd gains with output saturation limits

2.2.2 TelemetryService

Purpose : Real - time sensor data aggregation and system health monitoring.

Table 2: TelemetryService RPCs

RPC	Type	Description
GetTelemetry	Unary	Snapshot of all sensor data
StreamTelemetry	Server stream	Continuous sensor updates at 1 - 100Hz
GetSystemStatus	Unary	Firmware version, uptime, connection states

Key Messages:

- `TelemetryData` –IMU, GPS, battery %, WiFi signal, CPU usage, temperature
- `ImuData` –Pitch / roll / yaw(rad), acceleration(m / s^2), angular velocity(rad / s)
- `GpsData` –Lat / lon(degrees), altitude(m), accuracy, satellite count, fix type
- `SystemStatus` –Connection states, operating mode, free heap, uptime

Enumerations:

- `RobotMode` –IDLE, MANUAL, AUTONOMOUS, MAPPING, EMERGENCY_STOP
- `ConnectionStatus` –CONNECTED, DISCONNECTED, CONNECTING, ERROR
- `GpsFix` –NONE, 2D, 3D, DGPS, RTK_FLOAT, RTK_FIXED

2.2.3 BatteryManagementService

Purpose: BQ7850 BMS monitoring for lithium-ion battery pack safety and management.
Hardware: TI BQ7850 16-cell BMS with I2C interface.

Table 3: BatteryManagementService RPCs

RPC	Type	Description
GetBatteryState	Unary	Complete battery snapshot(cells, temps, SOC)
StreamBatteryState	Server stream	Continuous battery monitoring
Get / SetProtectionThresholds	Unary	OV / UV / OC / OT protection limits
Enable / DisableCellBalancing	Unary	Per - cell passive balancing control
ControlFets	Unary	Charge / discharge FET enable / disable
GetDeviceInfo	Unary	Manufacturer, serial, chemistry, versions

Key Messages:

- `BatteryState` –Composite of all battery data
- `CellData` –Individual cell voltages(mV), pack voltage, valid cell count
- `BatteryTemperatureData` –Thermistor readings(0.1°C), average
- `StateOfChargeData` –Relative / absolute SOC(%), capacity(mAh), cycle count
- `ProtectionThresholds` –OV / UV voltage limits, OC current limits, OT temperatures

2.2.4 ConfigurationService

Purpose : Runtime parameter management with NVS persistence across power cycles.

Table 4: ConfigurationService RPCs

RPC	Type	Description
GetConfiguration	Unary	Read current system configuration
SetConfiguration	Unary	Apply configuration(optional NVS persist)
ResetToDefaults	Unary	Restore factory defaults
ValidateConfiguration	Unary	Validate without applying
SaveConfiguration	Unary	Persist in - memory config to NVS
Get / SetMotorPidConfig	Unary	Per - motor PID tuning

Key Messages:

- `SystemConfiguration` –Complete config with version and CRC32
- `MotorPidConfig` –Per - motor PID gains(4 motors supported)
- `CurrentSensorCalibration` – Offset and scale for current sensing

- `SafetyThresholds` –Overcurrent, thermal shutdown, cell imbalance limits
- `TimingConfig` –Motor period, telemetry period, communication timeout

2.3 common.proto– Shared Types

All services import `common.proto` for standardized request / response handling.

Table 5: Standard Headers

Message	Fields
RequestHeader	request_id(UUID), client_timestamp, client_version, timeout
ResponseHeader	request_id(echo), server_timestamp, status, error_message, latency_us

Status Codes : UNKNOWN, OK, INVALID_REQUEST, INTERNAL_ERROR, NOT_FOUND, TIMEOUT, INVALID_STATE, ESTOP_ACTIVE

Geometric Types:

- `Vector3` –Generic 3D vector(x, y, z)
- `Quaternion` –Orientation(w, x, y, z)
- `SE2Pose` –2D pose(x_m, y_m, angle_rad)
- `SE2Velocity` –2D velocity(vx_mps, vy_mps, omega_rad_per_s)

2.4 Code Generation and Integration

Table 6: Code Generation Targets

Target	Generator	Output	Usage
Go	protoc-gen-go + grpc	`gen/go/`	star-gateway(RPi5)
nanopb	nanopb_generator	`gen/nanopb/`	RX72N firmware

nanopb Constraints: RX72N requires static allocation. Field sizes are configured in `.options` files:

- String fields: `max_size:64` (request_id), `max_size:128` (error_message)
- Repeated fields: `max_count:16` (cell voltages), `max_count:4` (motors)
- Bytes fields: `max_size:1024` (firmware chunks)

Build Integration:

- **Linting:** `buf lint` enforces style guide compliance
- **Breaking Changes:** `buf breaking` detects API incompatibilities
- **CI Pipeline :** Automatic generation and testing on PR

3 Hardware Pin Connections

3.1 RX72N Microcontroller Pinout– - 144 - Pin LFQFP(R5F572NNHxFB)

Color Legend:

Function Category	Color
PWM Motor Control (GPTW) / Motor Driver GPIO (DRVOFF, nSLEEP)	Blue
Encoder Inputs (MTU / TPU)	Green
Host SPI / IRQ Communication (RSPI2)	Orange
I2C Communication (RIIC0 / RIIC1 / IMU SCL,SDA)	Violet
ADC Current Sensing	Red
Debug / JTAG / Boot / USB	Yellow
Power / Ground	Gray
GTETRG PWM Ext Trig / Temp / IMU INT,RST	Lime
Ultrasonic Sonar (HC-SR04)	Cyan
Status LEDs	Teal
Unused / Available	White

Pin	Pin Name	Connected To	Note / Comment
1	AVSS0	Ground	Analog ground
2	P05 / IRQ13	Unused	Available GPIO (IRQ13)
3	AVCC1	Boards 3v3	Analog power supply(3.3V, connect to VCC via branch with 0.1uF bypass to AVSS1)
4	P03/IRQ11	Sonar 0 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
5	AVSS1	Ground	Analog ground
6	P02/IRQ10	Sonar 1 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
7	P01/IRQ9	Sonar 2 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
8	P00/IRQ8	Sonar 3 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
9	PF5	Sonar 0 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
10	EMLE	Emulator Configuration jumper	See Table **??**
11	PJ5	Sonar 1 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
12	VSS	Ground	Power supply
13	PJ3	Sonar 2 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
14	VCL	220nF cap to ground	Voltage regulator
15	VBATT	Boards 3v3	Battery backup
16	MD / FINED	E1E2 - Emulator - Interface, DIP switch	See Table **??**.Default : OFF, OFF
17	XCIN	10k resistor to ground	RTC crystal input(not used)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
18	XCOUT	Floating	RTC crystal output(not used)
19	RES#	E1E2 - Emulator - Interface, Reset button	MCU reset
20	P37 / XTAL	24MHz crystal(to pin 22)	System clock crystal
21	VSS	Ground	Power supply
22	P36 / EXTAL	24MHz crystal(to pin 20)	System clock crystal
23	VCC	Boards 3v3	Power supply
24	P35/NMI/UPSEL	UPSEL configuration jumper	Can be used for NMI (not used). See Table **??**
25	P34 / TRST#	E1E2 - Emulator - Interface	JTAG(TRST#)
26	P33	Sonar 3 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
27	P32	IMU INT	IMU interrupt line (active-low, IRQ input)
28	P31 / TMS	E1E2 - Emulator - Interface	JTAG(TMS)
29	P30 / TDI	E1E2 - Emulator - Interface	JTAG(TDI)
30	P27 / TCK	E1E2 - Emulator - Interface	JTAG(TCK)
31	P26 / TDO	E1E2 - Emulator - Interface	JTAG(TDO)
32	P25 / MTCLKB	Encoder 0 Phase B	MTU1 encoder(quadrature counting)
33	P24 / MTCLKA	Encoder 0 Phase A	MTU1 encoder(quadrature counting)
34	P23/GTIOC0A	Motor 0 `IN2`	`GPTW` PWM channel 0A (32-bit)
35	P22/GTIOC1A	Motor 1 `IN2`	`GPTW` PWM channel 1A (32-bit)
36	P21/SCL1	IMU SCL	IMU I2C clock (RIIC1)
37	P20/SDA1	IMU SDA	IMU I2C data (RIIC1)
38	P17/GTIOC0B	Motor 0 `IN1`	`GPTW` PWM channel 0B (32-bit)
39	P87	Unused	Available GPIO
40	P16 / USB0_VBUS	USB0 VBUS sense	5 V detect on USB-C, gate for USBb peripheral
41	P86/GTIOC2B	Motor 2 `IN1`	`GPTW` PWM channel 2B (32-bit)
42	P15/GTETRGA	PWM Ext Trig 0	GPTW external trigger A (GTETRGA)
43	P14/GTETRGD	PWM Ext Trig 3	GPTW external trigger D (GTETRGD)
44	P13/SDA0	Host I2C Data	RIIC0 for RPi5 communication (FM+ capable)
45	P12/SCL0	Host I2C Clock	RIIC0 for RPi5 communication (FM+ capable)
46	VCC_USB	Boards 3v3	USB power supply
47	USB0_DM	USB D-	USB0 differential pair (3-port CDC)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
48	USB0_DP	USB D+	USB0 differential pair (3-port CDC)
49	VSS_USB	Ground	USB ground
50	P56	Unused	Available GPIO
51	P55	Unused	Available GPIO
52	P54	Unused	Available GPIO
53	P53	Unused	Not available as GPIO when external bus enabled(BCLK)
54	P52	Unused	Available GPIO
55	P51	DS18B20 DQ	1 - Wire temperature sensor data pin
56	P50	Unused	Available GPIO
57	VSS	Ground	Power supply
58	P83	IMU RST	IMU reset (active-low, GPIO output)
59	VCC	Boards 3v3	Power supply
60	PC7 / UB	E1E2 - Emulator - Interface, DIP switch	Operation mode configuration.See Table **??**
61	PC6/GTIOC3B	Motor 3 `IN1`	`GPTW` PWM channel 3B (32-bit)
62	PC5 / MTCLKD	Encoder 1 Phase B	MTU2 encoder(quadrature counting)
63	P82	Unused	Available GPIO(GTIOC2A, SCI10, Ethernet)
64	P81	Unused	Available GPIO(GTIOC0B, SCI10, Ethernet, QSPI)
65	P80	Unused	Available GPIO(SCK10, Ethernet, QSPI)
66	PC4/GTETRGC	PWM Ext Trig 2	GPTW external trigger C (GTETRGC)
67	PC3/GTIOC1B	Motor 1 `IN1`	`GPTW` PWM channel 1B (32-bit)
68	P77	Unused	Available GPIO(SCI11, Ethernet, QSPI, SDHI)
69	P76	Unused	Available GPIO(SCI11, Ethernet, QSPI, SDHI)
70	PC2 / TCLKA	Encoder 2 Phase A	TPU1 encoder(quadrature counting, 16 - bit)
71	P75	Unused	Available GPIO(SCI11, Ethernet, SDHI)
72	P74/CS4#	Unused	Available GPIO
73	PC1	Unused	Available GPIO
74	VCC	Boards 3v3	Power supply
75	PC0 / TCLKC	Encoder 3 Phase A	TPU2 encoder(quadrature counting, 16 - bit)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
76	VSS	Ground	Power supply
77	P73	Unused	Available GPIO(Ethernet)
78	PB7 / TXD9	Debug SCI USB - C RX	UART crossed.See Table **??**
79	PB6 / RXD9	Debug SCI USB - C TX	UART crossed.See Table **??**
80	PB5	Unused	Available GPIO
81	PB4	Unused	Available GPIO
82	PB3 / TCLKD	Encoder 3 Phase B	TPU2 encoder(quadrature counting, 16 - bit)
83	PB2	LED5	Status LED(GPIO output)
84	PB1	LED4	Status LED(GPIO output)
85	P72	LED3	Status LED(GPIO output)
86	P71	LED2	Status LED(GPIO output)
87	PB0	LED1	Status LED(GPIO output)
88	PA7	LED0	Status LED(GPIO output)
89	PA6/GTETRGB	PWM Ext Trig 1	GPTW external trigger B (GTETRGB)
90	PA5	Unused	Available GPIO(GTIOC0A, RSPI_A SCLK, Ethernet)
91	VCC	Boards 3v3	Power supply
92	PA4	Unused	Available GPIO(RSPI_A CS, Ethernet, IRQ5)
93	VSS	Ground	Power supply
94	PA3 / TCLKB	Encoder 2 Phase B	TPU1 encoder(quadrature counting, 16 - bit)
95	PA2	Unused	Available GPIO(GTIOC1A, SCI5 RX)
96	PA1 / MTCLKC	Encoder 1 Phase A	MTU2 encoder(quadrature counting)
97	PA0	Unused	Available GPIO(GTIOC0B, Ethernet)
98	P67 / IRQ15	HOST_IRQ	Host interrupt output to RPi5(active - low, data ready)
99	P66	Unused	Available GPIO(GTIOC2B, CAN2 TX)
100	P65	Unused	Available GPIO(external bus CKE)
101	PE7/GTIOC3A	Motor 3 `IN2`	`GPTW` PWM channel 3A (32-bit)
102	PE6/GTIOC3B	Unused	Available GPIO (GTIOC3B)
103	VCC	Boards 3v3	Power supply
104	P70	Unused	Available GPIO(SDCLK)
105	VSS	Ground	Power supply
106	PE5/GTIOC0A	Unused	Available GPIO (GTIOC0A)
107	PE4/GTIOC1A	Unused	Available GPIO (GTIOC1A)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
108	PE3/GTIOC2A	Motor 2 IN2	GPTW PWM channel 2A (32-bit)
109	PE2/GTIOC0B	Motor 3 DRVOFF	DRV8263H motor 3 drive output disable (GPIO output)
110	PE1/GTIOC1B	Motor 3 nSLEEP	DRV8263H motor 3 sleep control (GPIO output, active-low)
111	PE0/GTIOC2B	Motor 2 DRVOFF	DRV8263H motor 2 drive output disable (GPIO output)
112	P64	Motor 2 nSLEEP	DRV8263H motor 2 sleep control (GPIO output, active-low)
113	P63	Motor 1 DRVOFF	DRV8263H motor 1 drive output disable (GPIO output)
114	P62	Motor 1 nSLEEP	DRV8263H motor 1 sleep control (GPIO output, active-low)
115	P61	Motor 0 DRVOFF	DRV8263H motor 0 drive output disable (GPIO output, active-high; held HIGH at boot to keep bridge disabled until firmware ready)
116	VSS	Ground	Power supply
117	P60	Motor 0 nSLEEP	DRV8263H motor 0 sleep control (GPIO output, active-low)
118	VCC	Boards 3v3	Power supply
119	PD7	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ7, AN107)
120	PD6	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ6, AN106)
121	PD5	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ5, AN113)
122	PD4 / SSLC0	RPi5 RSPI2 CS	RSPI2 - A chip select
123	PD3 / RSPCKC	RPi5 RSPI2 SCLK	RSPI2 - A clock
124	PD2 / MISOC	RPi5 RSPI2 CIPO	RSPI2 - A Controller In Peripheral Out
125	PD1 / MOSIC	RPi5 RSPI2 COPI	RSPI2 - A Controller Out Peripheral In
126	PD0	Unused	Available GPIO(GTIOC1B, IRQ0, AN108)
127	P93	Unused	Available GPIO(SCI7, Ethernet, AN117)
128	P92/SMISO7	Unused	Available GPIO (SCI7)
129	P91/SCK7	Unused	Available GPIO (SCI7)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
130	VSS	Ground	Power supply
131	P90/SMOSI7	Unused	Available GPIO (SCI7)
132	VCC	Boards 3v3	Power supply
133	P47 / AN007	Motor 0 Current Sense	ADC Unit 0, 12 - bit, ~3.20A max at 3.3V ADC reference (`IPROPI` from `DRV8263H`, 5.1kΩ sense to GND, 202 µA/A mirror)
134	P46 / AN006	Motor 1 Current Sense	ADC Unit 0, 12 - bit, ~3.20A max at 3.3V ADC reference (`IPROPI` from `DRV8263H`, 5.1kΩ sense to GND, 202 µA/A mirror)
135	P45 / AN005	Motor 2 Current Sense	ADC Unit 0, 12 - bit, ~3.20A max at 3.3V ADC reference (`IPROPI` from `DRV8263H`, 5.1kΩ sense to GND, 202 µA/A mirror)
136	P44 / AN004	Motor 3 Current Sense	ADC Unit 0, 12 - bit, ~3.20A max at 3.3V ADC reference (`IPROPI` from `DRV8263H`, 5.1kΩ sense to GND, 202 µA/A mirror)
137	P43	Unused	Available GPIO(IRQ11, AN003)
138	P42	Unused	Available GPIO(IRQ10, AN002)
139	P41	Unused	Available GPIO(IRQ9, AN001)
140	VREFL0	Ground	ADC reference low
141	P40	Unused	Available GPIO(IRQ8, AN000)
142	VREFH0	3.3V	ADC reference high
143	AVCC0	Boards 3v3	ADC power supply
144	P07	Unused	Floating

3.2 Motor Control Architecture

3.2.1 GPTW vs MTU Selection for PWM Generation

The RX72N provides two timer peripherals suitable for motor control PWM generation: the General PWM Timer (GPTW) and the Multi-Function Timer Unit (MTU). After detailed analysis, **GPTW was selected** for the following technical reasons :

Resolution and Performance:

- **GPTW:** 32 - bit timer resolution enabling precise duty cycle control
- **MTU:** 16 - bit timer resolution(65, 536 maximum count)

- At 20 kHz PWM frequency with 120 MHz peripheral clock :
  - GPTW provides 6, 000 discrete duty cycle steps(120 MHz / 20 kHz)
  - MTU limited to maximum 3, 276 steps before counter overflow

Channel Architecture:

- **GPTW:** 4 independent channels, each with 2 complementary outputs(GTIOC0A / B through GTIOC3A / B)
- **Perfect match** for 4 motors in `IN2` / `IN1` (sign-magnitude) control mode
- Each motor driver(`DRV8263H`) requires exactly 2 PWM signals

Peripheral Separation:

- Using GPTW for PWM provides dedicated high-resolution motor control
- Encoders use MTU (front wheels) and TPU (rear wheels) for phase counting
- No resource conflicts between PWM generation and encoder reading

Encoder Peripheral Selection:

- **Front encoders (0, 1)**: MTU1/MTU2 - 32-bit counters with phase counting support
- **Rear encoders (2, 3)**: TPU1/5, TPU2/4 - 16-bit counters with phase counting support
- **IMPORTANT**: MTU3 and MTU4 do NOT support phase counting mode per RX72N Manual Ch24 §24.1.2
- Only MTU1 and MTU2 have phase counting capability, hence TPU required for rear encoders
- TPU provides adequate performance for 210 RPM max speed (13.7s overflow period with 16-bit counter)
- Software overflow detection at 100 Hz control loop provides 137× safety margin

Pin Flexibility:

- GPTW signals available on multiple pin locations via MPC(Multi - Function Pin Controller)
- Example : GTIOC0A available on pins 34, 90, 106, 123
- Enabled conflict - free allocation by selecting alternate pin groups

3.2.2 Pin Allocation Strategy

Design Philosophy:

1. **Peripheral Grouping :** Place related functions on contiguous port pins where possible
   - `GPTW` PWM (`IN1`/`IN2`) : Motors spread across PORT E, PORT C, and PORT P (pins 34, 35, 38, 41, 61, 67, 101, 108)
   - Motor Driver GPIO (`DRVOFF`/`nSLEEP`) : PORT E and PORT P6x (pins 109-115, 117)

- Host SPI to RPi5 : PD1 - PD4 (RSPI2_A, pins 122 - 125)
- MTU Encoders : P24 / P25 (MTU1), PA1 / PC5 (MTU2)
- TPU Encoders : PC2 / PA3 (TPU1), PC0 / PB3 (TPU2)
- I2C : P12 - P13 (RIIC0 Host RPi5), P20 - P21 (RIIC1 IMU)

2. **Conflict Resolution via Alternate Functions:**
   - 144-pin package provides more pins, reducing conflicts vs. 100-pin
   - Table 1.9 analysis revealed all peripherals have 2-4 alternate pin locations
   - GTETRG inputs used as PWM external triggers (GTETRGA/B/C/D on pins 42, 89, 66, 43)
   - RIIC1 (P20/P21) repurposed for IMU after BMS removal
   - Motor driver GPIO (`DRVOFF`/`nSLEEP`) placed on PORT E and PORT P6x to keep routing compact
3. **PCB Routing Optimization:**
   - Host SPI signals on PD1-PD4 (pins 122-125) for compact layout
   - Motor PWM (`IN1`/`IN2`) and driver GPIO (`DRVOFF`/`nSLEEP`) on PORT E and PORT P6x
   - Sonar sensors grouped on pins 4-13 (top of package)
   - ADC current sense on P44-P47 (pins 133-136) for short analog traces
4. **Future Expandability:**
   - 43 GPIO pins available after critical allocations (of 144 total)
   - Unused Port D pins (PD5-PD7) available for additional QSPI/SDHI
   - Port E pins split between motor PWM (PE3, PE7), motor GPIO (PE0-PE2), and available GPIO (PE4-PE6)
   - Ethernet pins (Port B/P7x) available for future network connectivity
   - Additional ADC channels available (AN000-AN003, AN100-AN117)

3.2.3 Communication Bandwidth Analysis

SPI Configuration: 10 Mbps is Optimal

Peak bandwidth calculation for RPi5 $\leftrightarrow$ RX72N communication:

- **Velocity Commands(100 Hz) :**
  - Message size : 34 bytes(Nanopb encoded MotorVelocityCommand)
  - Bandwidth : 34 bytes $\times$ 100 Hz $\times$ 8 = 27.2Kbps
- **Encoder Feedback(100 Hz) :**
  - Message size : 18 bytes(4 encoders $\times$ 4 bytes + header)
  - Bandwidth : 18 bytes $\times$ 100 Hz $\times$ 8 = 14.4Kbps
- **Telemetry(50 Hz) :**

  - Message size : 64 bytes(battery, motor current, temperature, status)
  - Bandwidth : 64 bytes $\times$ 50 Hz $\times$ 8 = 25.6Kbps

- **ARQ Protocol Overhead(estimated 40%) :**
  - Headers : 8 bytes per frame(sequence, CRC - 32, flags)
  - Retransmissions : Average 5% packet loss $\times$ 3 retry attempts
  - Total overhead : $(27.2 + 14.4 + 25.6) \times 1.4 = 94.1$Kbps

Total Peak Bandwidth :

$$\text{Peak} = 27.2 + 14.4 + 25.6 + 94.1 = \mathbf{161.3\ Kbps}$$

SPI Utilization :

$$\frac{161.3\text{Kbps}}{10\text{Mbps}} = \mathbf{1.6\%\ utilization}$$

Margin Analysis:

- Available headroom: $10,000/161.3 = 62\times$ current requirements
- No need for Ethernet (would add PHY chip, 9+ pins, PCB complexity)
- No need for QSPI (RPi5 doesn't support quad-lane SPI natively)
- 10 Mbps provides excellent margin for future features (camera data, additional sensors)

3.2.4 DS18B20+ Temperature Sensor

The DS18B20+ is a digital 1-Wire temperature sensor connected to P51 (pin 55).

Specifications:

- **Temperature Range:** -55°C to +125°C
- **Accuracy:** ±0.5°C from -10°C to +85°C
- **Resolution:** 9-bit to 12-bit configurable (0.5°C to 0.0625°C)
- **Interface:** 1-Wire protocol on DQ pin (P51)
- **Power:** Parasite power or separate VDD (3.0V to 5.5V)

Pin Connections:

- **DQ (Data):** P51 (pin 55) - 1-Wire data line
- **VDD:** 3.3V power supply
- **GND:** Ground

Required External Components:

- 4.7kΩ pull-up resistor on DQ line (P51 to 3.3V)
- 0.1μF decoupling capacitor near VDD pin

- Optional: 100nF ceramic capacitor if using parasite power mode

Firmware Implementation:

- Use 1-Wire protocol library (timing-critical, requires precise delays)
- Typical read cycle: 750ms for 12-bit conversion
- Multiple sensors can share same 1-Wire bus (each has unique 64-bit ROM code)

Use Cases:

- Ambient temperature monitoring
- Motor driver thermal management (supplement `DRV8263H` internal temp)
- System enclosure temperature

3.3 IMU

The IMU is connected to the RX72N via I2C with a dedicated interrupt and reset line.

Pin Connections:

- **SCL (Clock):** P21 (pin 36) — I2C clock (RIIC1)
- **SDA (Data):** P20 (pin 37) — I2C data (RIIC1)
- **INT (Interrupt):** P32 (pin 27) — active-low interrupt output from IMU to RX72N IRQ input
- **RST (Reset):** P83 (pin 58) — active-low reset, driven as GPIO output by RX72N
- **VDD:** 3.3V power supply
- **GND:** Ground

Required External Components:

- 4.7kΩ pull-up resistors on SCL and SDA lines (to 3.3V)
- 100nF decoupling capacitor near VDD pin

Firmware Implementation:

- Configure INT pin as IRQ input (falling-edge triggered)
- Assert RST low for $\geq$10 ms at startup to perform a hardware reset before initialisation
- Use I2C to read sensor data registers on each INT assertion

3.4 Motor Driver Control Architecture

The DRV8263H motor drivers are controlled via three interfaces:

Real-Time Control (PWM):

- **Signals:** IN1 and IN2 distributed across GPTW channels 0-3
- **Purpose:** Motor speed/direction control at 20 kHz PWM frequency
- **Update rate:** 100 Hz velocity commands from RPi5
- **Pin mapping:** IN2 = GTIOC_A, IN1 = GTIOC_B (sign-magnitude PWM)

Driver Enable/Sleep Control (GPIO):

- DRVOFF **signals:** PE2 (Motor 3), PE0 (Motor 2), P63 (Motor 1), P61 (Motor 0)
- nSLEEP **signals:** PE1 (Motor 3), P64 (Motor 2), P62 (Motor 1), P60 (Motor 0)
- DRVOFF **purpose:** Disable H-bridge outputs without entering sleep mode (GPIO output)
- nSLEEP **purpose:** Put driver into low-power sleep mode when pulled low (GPIO output, active-low)
- **Update rate:** On demand for enable/disable/sleep transitions

External PWM Triggers (GTETRG):

- **Signals:** GTETRGA (pin 42), GTETRGB (pin 89), GTETRGC (pin 66), GTETRGD (pin 43)
- **Purpose:** GPTW external trigger inputs for synchronised PWM control

3.5 RX72N Configuration Tables

Table 8: Emulator Configuration Jumper Settings

Jumper Position	Configuration
Shorted Pin 1-2 (normal Operation)	E1/E2 Lite normal debugging (use this mode for normal operation). Microcontroller single operation (without emulator)
Shorted Pin 2-3 (debug with E1/E2)	E1/E2 Lite debugging with Hot plug-in
All open	DO NOT SET

Table 9: Operation Configuration Mode DIP Switch Settings

Note: Both USB-C ports are active: the SCI (UART-over-USB via CY7C65213) port carries fallback / bring-up traffic, and the USB0 port carries the primary 3-port CDC composite device (PROTO / DECODED / LOG). Boot flashing normally goes through the E2 Lite debugger over the JTAG header; SCI boot mode and USB0 boot mode are preserved in the silicon as recovery paths.

Pin1	Pin2	UPSEL_CFG	OP Mode
OFF	X	X	Single Chip Mode
ON	OFF	X	SCI Boot Mode (SCI1)
ON	ON	1/2	USB (Bus Powered) (USB0)
ON	ON	2/3	USB (Self Powered) (USB0)

Table 10: UPSEL Power Configuration for USB Boot Mode

Jumper Position	Power Configuration for USB Boot Mode
Shorted Pin 1-2 (No battery pack)	Bus Powered
Shorted Pin 2-3 (Battery Pack)	Self Powered

Table 11: Required Pin States Upon Reset Release

To enter a mode capable of flashing firmware, the following pin configurations must be held when the **RES#pin** goes from Low to High:

Desired Flashing Mode	MD Pin Level	UB Pin Level	Interface Used
Boot Mode (SCI)	Low	Low	Serial (SCI1)
Boot Mode (USB)	Low	High	USB
Boot Mode (FINE)	Low → High*	Low	FINE Interface

*For FINE interface, the MD pin must be Low at reset release and then switched to High within 20 to 100 ms.

Table 12: SCI9 Firmware Configuration(MPC Settings)

Note: Pins are multiplexed. Firmware must explicitly configure the Multi-Function Pin Controller (MPC) as follows:

Step	Configuration
1. UNLOCK MPC	Unlock registers using the Write-Protect Register (PWPR)
2. PIN FUNCTION SELECT	**PIN 78 (PB7) → TXD9:** Set PB7PFS.PSEL[5:0] = 001010b (0x0A) **PIN 79 (PB6) → RXD9:** Set PB6PFS.PSEL[5:0] = 001010b (0x0A)
3. ENABLE PERIPHERAL	Set Port Mode Register (PMR) bit to '1' for both PB7 and PB6

USB-UART IC Configuration: Use Cypress configuration utility to enable BUSDETECT.

3.6 Raspberry Pi 5 Pinout

Status: Pin assignments are pending hardware verification and will be documented once validated.

3.7 Additional Peripheral Connections

Status: Peripheral connections are pending hardware verification and will be documented once validated.

== == == == == == == == == == == == = =

4 Protocol Buffer Style Guide

This section defines naming standards and conventions for all Protocol Buffer definitions in the STAR project, following Boston Dynamics style guidelines.

4.1 Terminology Standard

The project uses inclusive terminology following OSHWA (Open Source Hardware Association) standards.

Table 13: Communication Bus Terminology

Use	Avoid	Context
Controller / Peripheral	Master / Slave	I2C, SPI, 1 - Wire bus roles
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
Primary / Main	Master	Configuration structures

4.2 Protocol Buffer Naming Conventions(Boston Dynamics Style)

The STAR project follows Boston Dynamics' Protocol Buffer style guide for all .proto definitions.

Table 14: Protocol Buffer Naming Conventions

Element	Convention	Example
Package	Versioned namespace	`star.v1`
Messages	PascalCase	`VelocityCommand`, `BatteryState`
Fields	snake_case + unit suffix	`velocity_mps`, `temperature_celsius`
Enums	SCREAMING_SNAKE_CASE	`MOTOR_STATE_RUNNING`
Enum Zero Value	_UNKNOWN suffix	`STATUS_UNKNOWN = 0`
Enum Values	Type name prefix	`ROBOT_MODE_MANUAL`
RPC Pattern	{Verb} Request/Response	`SetVelocityRequest`

4.3 Unit Suffixes(MKS System)

All numeric fields use SI units with explicit suffixes .This eliminates unit ambiguity and enables automatic documentation generation.

Table 15: Standard Unit Suffixes

Suffix	Unit	Proto Example	C Example
`_m`	meters	`x_m`	`distance_m`
`_mps`	meters / second	`velocity_mps`	`speed_mps`
`_mps2`	m / s^2	`accel_x_mps2`	`acceleration_mps2`
`_rad`	radians	`angle_rad`	`heading_rad`
`_rad_per_s`	rad / second	`omega_rad_per_s`	`angular_vel_rad_per_s`
`_celsius`	degrees C	`temperature_celsius`	`motor_temp_celsius`
`_deci_celsius`	0.1°C	`temp_deci_celsius`	(BMS native format)
`_us`	microseconds	`timestamp_us`	`elapsed_us`
`_ms`	milliseconds	`timeout_ms`	`period_ms`
`_ma`	milliamps	`current_ma`	`motor_current_ma`
`_mv`	millivolts	`cell_mv`	`pack_voltage_mv`
`_mw`	milliwatts	`power_mw`	`consumption_mw`
`_percent`	0 –100%	`soc_percent`	`duty_cycle_percent`

4.4 General Documentation Standards

- Use Doxygen-compatible comments for all public APIs
- Document the “why” not just the “what”
- Keep configuration centralized in header files
- Avoid magic numbers – use named constants with comments
- Version hardware and firmware together using git tags

== == == == == == == == == == == == = =

5 C Style Guide

This section defines comprehensive style guidelines for C firmware development in the STAR project. All firmware code for the RX72N and future platforms must follow these conventions to ensure consistency, maintainability, and safety in embedded systems development.

In this project, **Assembly code** and **Inline Assembly** should be used with extreme caution. While it can offer performance improvements or enable access to low-level hardware features, its use can make code harder to understand, maintain, and port. Always prefer writing in C unless absolutely necessary.

5.1 General Guidelines for ASM and Inline ASM

- **Avoid ASM When Possible** : Always try to write in C before resorting to assembly language.C compilers are highly optimized and can often produce efficient machine code that rivals hand - written assembly.
- **Document ASM Code Thoroughly** : Assembly code is often harder to understand than C, so it should be accompanied by thorough comments that explain **why** ASM was used, as well as what the code does.

```
    /* Set register to zero using ASM */
    mov r0,
    #0 /* Initialize r0 to 0 */
```

- **Use Inline ASM Sparingly** : Inline assembly allows you to embed assembly instructions directly in C code.It should only be used when absolutely necessary, such as when interacting with hardware registers or performing CPU - specific optimizations.
- **Constrain the Scope of Inline ASM** : When using inline assembly, constrain its scope to as few lines as possible and avoid intermixing it with complex C logic .This helps isolate potential issues and reduces the risk of bugs.

```
void set_flag(void) {
  asm volatile("mov r0, #1" : : : "r0"); /* Set flag in register r0 */
}
```

- **Always Use `volatile` for Inline ASM** : Inline assembly should be marked as `volatile` to prevent the compiler from optimizing it out.
- **Do Not Use Inline ASM for Optimizations** : Modern C compilers perform aggressive optimizations. Inline ASM should not be used to try and optimize code performance unless absolutely necessary, as it is often more error-prone than compiler optimizations.
- **Use Constraints Correctly**: When using inline ASM in C, use the appropriate constraints to inform the compiler how the assembly instructions interact with C variables.

```c
int add(int a, int b) {
  int result;
  asm volatile("add %[r], %[a], %[b]"
               : [r] "=r"(result)       /* Output */
               : [a] "r"(a), [b] "r"(b) /* Inputs */
  );
  return result;
}
```

5.2 When to Use ASM and Inline ASM

- **Hardware Access**: Use ASM when direct hardware access is required and C alone cannot provide the necessary control (e.g., setting or clearing specific registers).
- **Performance Critical Code**: ASM may be justified in performance-critical sections where C compiler optimizations fall short, but this should be documented and used as a last resort.
- **CPU-Specific Instructions**: Use ASM for executing instructions that are unique to the target CPU (e.g., specific ARM or x86 instructions that are not exposed in C).

In this project, **assertions** are used as a tool to catch programming errors and invalid states during development. They help ensure that assumptions made in the code are correct, and they provide valuable debugging information when things go wrong. However, assertions should never be relied on to handle runtime errors or expected conditions in production code.

5.3 Guidelines

- **Use Assertions for Debugging** : Assertions are intended to catch conditions that should never occur during normal program execution. Use them to assert assumptions about the state of the program, such as parameter validation or invariants within algorithms.

```c
#include <assert.h>

        void
        process_data(int *data) {
      assert(data != NULL); /* Assert that data is not NULL */
      /* Continue processing */
   }
```

5.4 Platform-Specific Error Checking Macros

Different platforms provide error checking macros for development and debugging:

- **ESP32 (ESP-IDF)**: `ESP_ERROR_CHECK` aborts on error, `ESP_ERROR_CHECK_WITHOUT_ABORT` logs but continues
- **RX72N**: Custom error checking macros following similar patterns
- **General Pattern**: Check return values and handle errors appropriately for your platform

Note : While platform - specific macros are useful during development, production code should use explicit error handling with proper cleanup and recovery mechanisms .

Proper brace placement makes code cleaner and easier to read.This section establishes consistent brace conventions used throughout the STAR firmware codebase.

5.5 General Rules

- **Always Use Braces for Control Statements** : Every `if` , `for` , `while`, or similar control structure must have braces, even for single statements.
- **Same Line for Control Structures and Blocks** : For control structures like `if` , `for` , `while` , and blocks like structs and enums, always place the opening brace `{` on the same line as the statement.
- **Functions Have Braces on a New Line**: The only exception to the same-line rule is for functions, where the opening brace goes on its own line.

5.6 Control Structures

Opening brace on the **same line** as the control statement.

```
        /* CORRECT - if/else with braces on same line */
        if (ret != RX_OK) {
      RX_LOG_ERROR(s_TAG, "Operation failed: %s", rx_err_to_name(ret));
      return ret;
  }

   if (manager == NULL) {
     return RX_ERR_INVALID_ARG;
  } else if (manager->mutex == NULL) {
     return RX_ERR_INVALID_STATE;
  } else {
     /* Proceed with operation */
  }

   /* CORRECT - Single statement still requires braces */
   if (pin < 0 || pin >= GPIO_NUM_MAX) {
     return RX_OK; /* Not an error, pin is not used */
  }

   /* WRONG - Missing braces */
   if (ret != RX_OK)
     return ret;

   /* WRONG - Brace on new line */
   if (ret != RX_OK) {
     return ret;
  }
```

5.6.1 For Loops

```
        /* CORRECT - for loop with brace on same line */
        for (star_bus_config_t *curr = manager->buses; curr;
             curr = curr->next) {
      if (curr->name && strcmp(curr->name, name) == 0) {
        found_bus = curr;
        break;
      }
    }

    for (uint32_t i = 0; i < SPI_HOST_MAX; ++i) {
      manager->spi_host_initialized[i] = false;
      manager->spi_device_count[i] = 0;
    }

    /* WRONG */
    for (uint32_t i = 0; i < count; ++i)
      process_item(i); /* Missing braces */
```

5.6.2 While Loops

```
        /* CORRECT - while loop with brace on same line */
        while (curr) {
      star_bus_config_t *next = curr->next;
      esp_err_t destroy_result = star_bus_config_destroy(curr);
      if (destroy_result != RX_OK && first_error == RX_OK) {
        first_error = destroy_result;
      }
      curr = next;
    }

    while (error_handler_can_retry(&handler)) {
      ret = attempt_operation();
      if (ret == RX_OK) {
        break;
      }
    }

    /* WRONG */
    while (curr) {
      process_node(curr); /* Brace on new line */
      curr = curr->next;
    }
```

5.7 Functions

Opening brace on a **new line** for function definitions.

```
          /* CORRECT - Function brace on new line */
          rx_err_t
          star_bus_manager_init(star_bus_manager_t * manager, const
              char *tag,
                                star_error_interface_t *error_iface,
```

```
                                    star_pin_interface_t *pin_iface) {
        RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                           "Manager pointer is NULL");

        manager->buses = NULL;
        manager->error_iface = error_iface;
        manager->pin_iface = pin_iface;

        /* ... */

        return RX_OK;
    }

    static rx_err_t internal_register_bus_pin(
        star_pin_interface_t * pin_iface, gpio_num_t pin,
        const char *bus_name, const char *usage, bool can_be_shared) {
      if (pin < 0 || pin >= GPIO_NUM_MAX) {
        return RX_OK;
      }
      /* ... */
    }

    /* WRONG - Function brace on same line */
    rx_err_t star_bus_manager_init(star_bus_manager_t * manager, ...) {
      /* ... */
    }
```

5.8 Structs and Enums

Opening brace on the **same line** as the typedef / struct / enum keyword.

```
        /* CORRECT - Struct brace on same line */
        typedef struct star_bus_manager {
      star_bus_config_t *buses;
      SemaphoreHandle_t mutex;
      const char *tag;
      star_error_interface_t *error_iface;
      star_pin_interface_t *pin_iface;
      bool spi_host_initialized[SPI_HOST_MAX];
      int spi_device_count[SPI_HOST_MAX];
    } star_bus_manager_t;

    /* CORRECT - Enum brace on same line, C23 typed enum */
    typedef enum : uint8_t {
      k_star_bus_type_none,
      k_star_bus_type_i2c,
      k_star_bus_type_spi,
      k_star_bus_type_gpio,
      k_star_bus_type_adc,
      k_star_bus_type_count,
    } star_bus_type_t;

    /* CORRECT - Union brace on same line */
    union {
```

```
        star_i2c_bus_config_t i2c;
        star_spi_bus_config_t spi;
        star_gpio_bus_config_t gpio;
        star_adc_bus_config_t adc;
    } proto;

    /* WRONG - Brace on new line */
    typedef struct star_bus_manager {
        /* ... */
    } star_bus_manager_t;
```

5.9 Initializer Lists

Opening brace on the **same line** as the assignment.

```
        /* CORRECT - Initializer brace on same line */
        star_bus_event_t event = {
        .bus_type = k_star_bus_type_i2c,
        .bus_name = config->name,
        .i2c = {
            .port = port, .address = address, .is_write = true, .len =
                len}};

    spi_device_interface_config_t spi_dev_cfg = {
        .clock_speed_hz = 10 * 1000 * 1000,
        .mode = 0,
        .spics_io_num = GPIO_NUM_5,
        .queue_size = 7,
        .pre_cb = NULL,
        .post_cb = NULL,
    };

    /* CORRECT - Short initializers can be on one line */
    i2c_config_t i2c_conf = {.mode = I2C_MODE_MASTER};

    /* WRONG - Brace on new line */
    star_bus_event_t event = {
        .bus_type = k_star_bus_type_i2c,
        /* ... */
    };
```

5.10 Closing Brace Placement

Closing braces are always on their own line, except for `else`, `else if` , and `while` in `do -while` .

```
    /* CORRECT - else on same line as closing brace */
    if (ret != RX_OK) {
  handle_error(ret);
}
else {
  continue_operation();
}
```

```

/* CORRECT - do-while */
do {
  ret = try_operation();
} while (ret == RX_ERR_TIMEOUT && retries-- > 0);

/* CORRECT - else if chain */
if (config->type == k_star_bus_type_i2c) {
  /* Handle I2C */
} else if (config->type == k_star_bus_type_spi) {
  /* Handle SPI */
} else {
  /* Handle other */
}
```

5.11 Summary Table

Construct	Opening Brace Position
Functions	New line
if/else/else if	Same line
for	Same line
while	Same line
do-while	Same line
switch	Same line
struct	Same line
enum	Same line
union	Same line
Initializer lists	Same line

Switch statements follow specific formatting rules to ensure consistency and readability throughout the STAR firmware codebase.

5.12 General Rules

- **Same Line for Switch and Opening Brace** : The opening brace `{` for a `switch` statement should be on the same line as the `switch` keyword.
- **Braces for Case Blocks** : Each `case` should have its own block of code enclosed in braces `{}`.
- **Break Statements**: Every case must end with `break`, `return`, or a fall-through comment.
- **Default Case**: Always include a `default` case, even if it only logs a warning.

5.13 Basic Switch Structure

```
      /* CORRECT - from star_bus_manager.c */
      switch (config->type) {
        case k_star_bus_type_i2c: {
  ret =
```

```
      internal_register_bus_pin(pin_iface,
          config->proto.i2c.config.sda_io_num,
                                config->name, "I2C SDA", true);
  if (ret != RX_OK && first_error == RX_OK) {
    first_error = ret;
 }

  ret =
      internal_register_bus_pin(pin_iface,
          config->proto.i2c.config.scl_io_num,
                                config->name, "I2C SCL", true);
  if (ret != RX_OK && first_error == RX_OK) {
    first_error = ret;
 }
  break;
}

case k_star_bus_type_spi: {
  ret = internal_register_bus_pin(pin_iface,
      config->proto.spi.copi_io_num,
                                  config->name, "SPI COPI", true);
  if (ret != RX_OK && first_error == RX_OK) {
    first_error = ret;
 }
  /* ... more pin registrations ... */
  break;
}

default: {
  RX_LOG_DEBUG(s_TAG, "No pin registration for bus type: %d",
      config->type);
  break;
}
}
```

5.14 Case Block Braces

Each case must have its own braces.This prevents variable scope issues and improves readability.

```
        /* CORRECT - Each case has braces */
        switch (bus_type) {
          case k_star_bus_type_i2c: {
      i2c_port_t port = config->proto.i2c.port; /* Local variable */
      uint8_t address = config->proto.i2c.address;
      RX_LOG_INFO(s_TAG, "I2C bus on port %d, address 0x%02X", port,
          address);
      break;
   }
    case k_star_bus_type_spi: {
      spi_host_device_t host = config->proto.spi.host; /* Different scope
          */
      RX_LOG_INFO(s_TAG, "SPI bus on host %d", host);
      break;
   }
```

```
    case k_star_bus_type_gpio: {
      uint8_t pin_count = config->proto.gpio.pin_count;
      RX_LOG_INFO(s_TAG, "GPIO bus with %d pins", pin_count);
      break;
  }
    default: {
      RX_LOG_WARN(s_TAG, "Unknown bus type: %d", bus_type);
      break;
  }
  }

    /* WRONG - Missing braces */
    switch (bus_type) {
    case k_star_bus_type_i2c:
      handle_i2c();
      break;
    case k_star_bus_type_spi:
      handle_spi();
      break;
  }
```

5.15 Break Statement Requirements

Every case must have a clear exit: `break`, `return`, or explicit fall - through.

```
                          /* CORRECT - Using break */
                          switch (config->type) {
    case k_star_bus_type_i2c: {
      process_i2c(config);
      break;
  }
    case k_star_bus_type_spi: {
      process_spi(config);
      break;
  }
    default: {
      break;
  }
  }

    /* CORRECT - Using return */
    const char *star_bus_type_to_string(star_bus_type_t type) {
      switch (type) {
      case k_star_bus_type_none: {
        return "NONE";
    }
      case k_star_bus_type_i2c: {
        return "I2C";
    }
      case k_star_bus_type_spi: {
        return "SPI";
    }
      case k_star_bus_type_gpio: {
        return "GPIO";
```

```
    }
     case k_star_bus_type_adc: {
       return "ADC";
    }
     case k_star_bus_type_count: {
       return "COUNT";
    }
     default: {
       return "UNKNOWN";
    }
    }
  }

   /* CORRECT - Explicit fall-through (rare, must be commented) */
   switch (priority) {
   case PRIORITY_CRITICAL: {
     log_critical(message);
     /* FALLTHROUGH */
  }
   case PRIORITY_HIGH: {
     notify_admin(message);
     /* FALLTHROUGH */
  }
   case PRIORITY_NORMAL: {
     log_message(message);
     break;
  }
   default: {
     break;
  }
  }
```

5.16 Default Case Handling

Always include a default case. Use it for error handling or logging unexpected values.

```
/* CORRECT - Default case logs warning */
switch (curr->type) {
  case k_star_bus_type_i2c: {
  reg_result =
      register_pin_if_valid(curr->proto.i2c.config.sda_io_num, bus_name,
                            "I2C SDA", reg_func, user_ctx, true);
  /* ... */
  break;
}
case k_star_bus_type_spi: {
  reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
     bus_name,
                                     "SPI COPI", reg_func, user_ctx,
                                        true);
  /* ... */
  break;
}
default: {
```

```
  RX_LOG_WARN(manager->tag,
              "Skipping pin registration for "
              "unsupported bus type: %d",
              curr->type);
  break;
}
}

/* CORRECT - Default case returns error for invalid input */
rx_err_t process_bus_type(star_bus_type_t type) {
  switch (type) {
  case k_star_bus_type_i2c: {
    return process_i2c();
 }
  case k_star_bus_type_spi: {
    return process_spi();
 }
  default: {
    RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
    return RX_ERR_INVALID_ARG;
 }
 }
}
```

5.17 Switch on Enum Types

When switching on an enum, consider handling all values explicitly.

```
    /* CORRECT - All enum values handled */
    switch (config->type) {
case k_star_bus_type_none: {
  RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
  break;
}
case k_star_bus_type_i2c: {
  /* Initialize I2C */
  break;
}
case k_star_bus_type_spi: {
  /* Initialize SPI */
  break;
}
case k_star_bus_type_gpio: {
  /* Initialize GPIO */
  break;
}
case k_star_bus_type_adc: {
  /* Initialize ADC */
  break;
}
case k_star_bus_type_count: {
  /* Sentinel value - should never be used */
  RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
  return RX_ERR_INVALID_ARG;
```

```
}
  /* No default needed when all enum values are handled */
  /* Compiler warns if new enum values are added */
}
```

5.18 Indentation

Case labels are at the same indentation level as the switch.Case body is indented one level.

```
    /* CORRECT indentation */
    switch (state) {
case STATE_IDLE: {
  /* Case body indented */
  if (condition) {
    /* Nested code further indented */
 }
  break;
}
case STATE_RUNNING: {
  process_running();
  break;
}
default: {
  break;
}
}

/* WRONG - Case labels indented */
switch (state) {
case STATE_IDLE: {
  break;
}
}
```

5.19 Summary

- Opening brace on same line as `switch`
- Each `case` has its own braces `{ ...}`
- Every case ends with `break`, `return`, or fall - through comment
- Always include `default` case
- Case labels at switch indentation level
- Case body indented one level inside braces

The maximum line length is **80 characters** .Lines longer than this should be split.

- **Break at Natural Points**: Break lines at natural points, such as after operators.
- **Indentation for Continuation Lines** : When breaking lines, ensure the continued line is indented properly.

All functions must include Doxygen comments. This section establishes comprehensive commenting standards for the STAR firmware codebase.

5.20 General Commenting Guidelines

- **Use Block Comments (`/* */`)**: Always prefer block comments over line comments (`//`).
- **Explain Why, Not What** : Comments should explain the reasoning behind code, not what the code does(which should be clear from the code itself) .
- **Keep Comments Updated** : Outdated comments are worse than no comments .
- **English Only** : All comments must be in English.

```
      /* CORRECT - Block comment style */
      /* Initialize the mutex for thread-safe access */
      manager->mutex = xSemaphoreCreateMutex();

  /* WRONG - Line comment style */
  // Initialize the mutex for thread-safe access
  manager->mutex = xSemaphoreCreateMutex();
```

5.21 File Header Comments

Every source and header file should begin with a file path comment, followed by include guards (for headers) and a `@file` Doxygen block for headers with significant content.

```
      /* lib/star_bus/include/star_bus_manager.h */

#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

#include "esp_err.h"
#include "star_bus_manager_types.h"

  /**
   * @file star_bus_manager.h
   * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
      protocols
   *
   * The bus manager provides a central registry for managing multiple bus
   * configurations. It supports dependency injection of error handlers
      and
   * pin validators for loose coupling.
   *
   * Key Features:
   * - Thread-safe bus management with mutex protection
   * - Support for multiple bus types (I2C, SPI, GPIO, DHT22, etc.)
   * - Automatic pin registration and conflict detection
```

```
25     * - Safe bus access via callback pattern (prevents use-after-free)
26     *
27     * Example Usage:
28     * @code
29     * // === Basic Bus Manager Setup ===
30     *
31     * #include "star_bus_manager.h"
32     * #include "star_bus_config.h"
33     * #include "star_error_handler.h"
34     *
35     * // 1. Create error handler and pin validator interfaces
36     * error_handler_t error_handler;
37     * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
38     *
39     * star_error_interface_t error_iface;
40     * star_pin_interface_t pin_iface;
41     * error_handler_get_interface(&error_iface, &error_handler);
42     * pin_validator_get_interface(&pin_iface);
43     *
44     * // 2. Initialize bus manager with dependency injection
45     * star_bus_manager_t bus_manager;
46     * star_bus_manager_init(&bus_manager, "main", &error_iface,
       &pin_iface);
47     * @endcode
48     */
```

5.22 Source File Header

Source files begin with a file path comment.

```
1          /* lib/star_bus/src/star_bus_manager.c */
2
3  #include "star_bus_manager.h"
4
5  #include "freertos/FreeRTOS.h"
6  #include "freertos/semphr.h"
7        /* ... */
```

5.23 Doxygen Documentation

5.23.1 Function Documentation

All public functions must have Doxygen documentation in the header file.

```
1        /**
2         * @brief Initialize a bus manager instance.
3         *
4         * Allocates resources (like a mutex) for the manager. Must be
            called
5         * before any other manager functions.
6         *
7         * @param[out] manager     Pointer to bus manager structure to
            initialize.
```

```
8         *                              Must not be NULL.
9         * @param[in]  tag               Optional tag string for logging
              associated with
10        *                              this manager instance. If NULL or empty,
              a
11        *                              default tag is used.
12        * @param[in]  error_iface       Optional error handler interface (can be
              NULL).
13        * @param[in]  pin_iface         Optional pin validator interface (can be
              NULL).
14        * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
15        *                    is NULL, ESP_ERR_NO_MEM if mutex creation
              fails.
16        */
17       rx_err_t
18       star_bus_manager_init(star_bus_manager_t * manager, const char *tag,
19                             star_error_interface_t *error_iface,
20                             star_pin_interface_t *pin_iface);
```

5.23.2 Parameter Direction Markers

Use `[in]`, `[out]`, or `[in,out]` to indicate parameter direction.

```
1  /**
2  * @brief Write data to an I2C device register.
3  *
4  * @param[in]  config        Pointer to I2C bus configuration.
5  * @param[in]  data          Pointer to data buffer to write.
6  * @param[in]  len           Number of bytes to write.
7  * @param[in]  reg_addr      Target register address.
8  * @param[out] bytes_written Pointer to store actual bytes written.
9  *                           Can be NULL if not needed.
10 * @return rx_err_t RX_OK on success.
11 */
12 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
       config,
13 const uint8_t*           data,
14 size_t                   len,
15 uint8_t                  reg_addr,
16 size_t*                 bytes_written);
17
18   /**
19    * @brief Record an error and manage retry/reset logic
20    *
21    * @param[in,out] handler     Pointer to error handler structure
22    * @param[in]     error       Error code that occurred
23    * @param[in]     description Human-readable description of the error
24    * @param[in]     file        Source file where error occurred
25    * @param[in]     line        Line number where error occurred
26    * @param[in]     func        Function name where error occurred
27    * @return RX_OK if error was handled, or the original error code.
28    */
29   rx_err_t error_handler_record_error(error_handler_t * handler, rx_err_t
         error,
```

```
                                        const char *description, const char
                                            *file,
                                        int32_t line, const char *func);
```

5.23.3 Code Examples with @code / @endcode

Include usage examples in documentation blocks.

```
      /**
       * @file star_error_interface.h
       * @brief Error handler interface for dependency inversion
       *
       * Example Usage:
       * @code
       * // === Creating and Using Error Interface ===
       *
       * #include "star_error_interface.h"
       * #include "star_error_handler.h"
       *
       * // 1. Create concrete error handler
       * error_handler_t error_handler;
       * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
       *
       * // 2. Get interface from implementation
       * star_error_interface_t error_iface;
       * error_handler_get_interface(&error_iface, &error_handler);
       *
       * // 3. Inject into components that need error handling
       * star_bus_manager_init(&bus_manager, "main", &error_iface,
           &pin_iface);
       *
       * // 4. Use the interface
       * if (error_iface.can_retry(error_iface.ctx)) {
       *     // Retry operation...
       *}
       * @endcode
       */
```

5.23.4 Struct Member Documentation

Document struct members inline with Doxygen.

```
      /**
       * @brief Bus configuration structure (common part).
       */
      typedef struct star_bus_config {
    const char *name;     /**< Unique name for this bus/device instance */
    star_bus_type_t type; /**< Type of bus (I2C, SPI, etc.) */
    bool initialized;     /**< Whether this bus has been initialized */
    void *handle;         /**< Driver/device handle */
    void *user_ctx;       /**< User context pointer for callbacks */

    union {
```

```
        star_i2c_bus_config_t i2c;   /**< I2C-specific configuration */
        star_spi_bus_config_t spi;   /**< SPI-specific configuration */
        star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
        star_adc_bus_config_t adc;   /**< ADC-specific configuration */
    } proto;

    struct star_bus_config *next; /**< Next bus config in linked list */
} star_bus_config_t;
```

5.24 Special Comment Keywords

Use these keywords to mark special comments for tracking.

```
/* TODO: Implement retry logic for I2C bus reset */

/* FIXME: Memory leak when bus removal fails mid-operation */

/* BUG: Race condition possible if mutex timeout is too short */

/* XXX: This code assumes little-endian byte order */

/* NOTE: ESP-IDF uses mosi_io_num internally - we map to COPI here */

/* WARNING: Must hold mutex before calling this function */
```

5.24.1 Keyword Definitions

- **TODO** : Work that needs to be done but is not urgent.
- **FIXME** : Known bugs or issues that need to be fixed.
- **BUG** : Confirmed bugs in the code.
- **XXX** : Dangerous or hacky code that works but should be reviewed.
- **NOTE** : Important information about the code.
- **WARNING** : Critical information that must be understood before modifying.

5.25 Section Comments

Use section comments to organize code within files.

```
    /* === Includes === */
#include "freertos/FreeRTOS.h"
#include "star_bus_manager.h"

    /* === Constants === */
    static const char *s_TAG = "STAR_BUS_MANAGER";

/* === Private Function Prototypes === */
static esp_err_t internal_register_bus_pin(...);

```

```
/* === Public Functions === */
esp_err_t star_bus_manager_init(...) { /* ... */}

/* === Private Functions === */
static rx_err_t internal_register_bus_pin(...) { /* ... */}

/* --- Pin Registration Helpers --- */
/* Subsection within a section */
```

5.26 Aligned Comments

Align comments after declarations for improved readability.

```
typedef struct error_handler {
  uint32_t error_count;          /**< Total errors recorded */
  uint32_t max_retries;          /**< Max retry attempts */
  uint32_t current_retry;        /**< Current retry counter */
  uint32_t base_retry_delay;     /**< Base delay (ms) */
  uint32_t max_retry_delay;      /**< Max delay (ms) */
  uint32_t current_retry_delay; /**< Current delay with backoff */
  rx_err_t last_error;           /**< Last recorded error code */
  bool in_error_state;           /**< Whether in error state */
  void *reset_context;           /**< Context for reset function */
  SemaphoreHandle_t mutex;       /**< Mutex for thread-safety */

  rx_err_t (*reset_fn)(void *context); /**< Optional reset function */
} error_handler_t;
```

5.27 Implementation Comments

Comments within function bodies explain non - obvious logic.

```
rx_err_t star_bus_manager_add_bus(
        star_bus_manager_t * manager, star_bus_config_t * config) {
  /* Validate inputs using ESP-IDF macro */
  RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                     "Manager or config pointer is NULL");

  rx_err_t ret = RX_OK;

  /* Acquire mutex with timeout to prevent deadlock */
  if (xSemaphoreTake(manager->mutex,
                     pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
                        !=
      pdTRUE) {
    RX_LOG_ERROR(manager->tag, "Timeout acquiring mutex");
    return RX_ERR_TIMEOUT;
 }

  /* Check for duplicate name - bus names must be unique */
  for (star_bus_config_t *curr = manager->buses; curr; curr = curr->next)
     {
    if (curr->name && strcmp(curr->name, config->name) == 0) {
```

```
      ret = RX_ERR_INVALID_STATE;
      goto add_give_mutex; /* Single exit point pattern */
    }
  }

  /* Initialize hardware before adding to list
   * This ensures manager's SPI host tracking is updated correctly */
  if (!config->initialized) {
    ret = star_bus_config_init(config, manager);
    if (ret != RX_OK) {
      goto add_give_mutex;
    }
  }

  /* Add to head of list (O(1) insertion) */
  config->next = manager->buses;
  manager->buses = config;

add_give_mutex:
  xSemaphoreGive(manager->mutex);
  return ret;
}
```

5.28 Private Function Documentation

Static functions should have brief documentation in the source file.

```
    /**
     * @brief Helper to register a single pin via the interface
     */
    static rx_err_t
    internal_register_bus_pin(star_pin_interface_t * pin_iface,
       gpio_num_t pin,
                              const char *bus_name, const char *usage,
                              bool can_be_shared) {
  if (pin < 0 || pin >= GPIO_NUM_MAX) {
    return RX_OK; /* Not an error, pin is not used */
  }

  if (!pin_iface || !pin_iface->register_pin) {
    return RX_OK; /* No interface, skip registration */
  }

  /* Create description: "BusName: Usage" (e.g., "I2C0: SDA") */
  char desc[64];
  snprintf(desc, sizeof(desc), "%s: %s", bus_name, usage);

  return pin_iface->register_pin(pin_iface->ctx, pin, desc,
     can_be_shared);
}
```

5.29 Summary

- Use block comments (`/* */`), never line comments (`//`)
- Document all public functions with Doxygen in headers
- Use `[in]`, `[out]`, `[ in, out ]` for parameter direction
- Include `@code`/`@endcode` examples in file and complex function docs
- Use TODO, FIXME, BUG, XXX, NOTE, WARNING keywords for tracking
- Align comments after struct members for readability
- Use section comments to organize source files

This section outlines the best practices for ensuring thread-safe and efficient concurrent programming.

5.30 RTOS Types, Mutexes, Semaphores, and Atomic Operations

- **RTOS Types**: Use RTOS types when developing for embedded systems like ESP32.
- **Mutexes**: Use a mutex when you need **exclusive access** to a shared resource.
- **Semaphores**: Use semaphores when you need to manage **shared resources** that multiple threads can access simultaneously.
- **Atomic Operations**: Use atomic operations when working with **simple data types** such as counters or flags.

This section establishes the standards for defining and using enumerations in the STAR firmware codebase.

5.31 General Rules

- **Enum Names Should End in `_t`** : All enum types should end with `_t` .
- **Use Snake Case for Enum Values** : All enum values should follow `snake_case` naming conventions.
- **Enum Values Should Start with `k_`**: To distinguish enum members from other constants, values should start with `k_`.
- **Use typedef**: Always use `typedef` to define the enum type.
- **Include Count Sentinel**: Include a `_count` sentinel value for iteration.

5.32 Basic Enum Structure

```
        /* CORRECT - C23 typed enum with explicit underlying type */
        typedef enum : uint8_t {
          k_star_bus_type_none  = 0, /**< No bus type specified */
          k_star_bus_type_i2c   = 1, /**< I2C bus */
          k_star_bus_type_spi   = 2, /**< SPI bus */
          k_star_bus_type_gpio  = 3, /**< GPIO bus */
          k_star_bus_type_adc   = 4, /**< ADC bus */
          k_star_bus_type_count = 5, /**< Number of bus types (sentinel)
             */
       } star_bus_type_t;

    /* WRONG - Various style violations */
    typedef enum {                  /* Missing ': uint8_t' */
      STAR_BUS_TYPE_I2C,  /* Missing k_ prefix */
      StarBusTypeSpi,     /* PascalCase not allowed */
      star_bus_type_gpio, /* Missing k_ prefix */
   } bad_enum_style;     /* Missing _t suffix */
```

5.33 Explicit Value Assignment

Assign explicit values when needed for stability or compatibility.

```
/* CORRECT - C23 typed enum with explicit values */
typedef enum : uint8_t {
  k_error_none  = 0,  /**< No error */
  k_error_minor = 1,  /**< Minor error, operation can continue */
  k_error_major = 2,  /**< Major error, operation should stop */
  k_error_fatal = 3,  /**< Fatal error, system needs reset */
  k_error_count = 4,  /**< Number of error levels */
} error_level_t;

/* CORRECT - Bit flags with explicit underlying type */
typedef enum : uint8_t {
  k_bus_flag_none = 0x00,        /**< No flags */
  k_bus_flag_initialized = 0x01, /**< Bus is initialized */
  k_bus_flag_busy = 0x02,        /**< Bus is busy */
  k_bus_flag_error = 0x04,       /**< Bus has error */
  k_bus_flag_dma = 0x08,         /**< DMA enabled */
} bus_flags_t;
```

5.34 Documentation

Document each enum value with Doxygen comments.

```
    /**
     * @brief Types of supported bus interfaces.
     */
    typedef enum : uint8_t {
      k_star_bus_type_none,  /**< No bus type specified */
      k_star_bus_type_i2c,   /**< I2C bus */
      k_star_bus_type_spi,   /**< SPI bus */
```

```
      k_star_bus_type_gpio,  /**< GPIO bus */
      k_star_bus_type_adc,   /**< ADC bus */
      k_star_bus_type_count, /**< Number of bus types */
  } star_bus_type_t;
```

5.35 Enum to String Functions

Provide a string conversion function for each enum type.

```
/* In header file */
const char* star_bus_type_to_string(star_bus_type_t type);

/* In source file */
const char *star_bus_type_to_string(star_bus_type_t type) {
  switch (type) {
  case k_star_bus_type_none: {
    return "NONE";
 }
  case k_star_bus_type_i2c: {
    return "I2C";
 }
  case k_star_bus_type_spi: {
    return "SPI";
 }
  case k_star_bus_type_gpio: {
    return "GPIO";
 }
  case k_star_bus_type_adc: {
    return "ADC";
 }
  case k_star_bus_type_count: {
    return "COUNT";
 }
  default: {
    return "UNKNOWN";
 }
 }
}

/* Usage */
RX_LOG_INFO(s_TAG, "Added bus config: %s (Type: %s)", config->name,
            star_bus_type_to_string(config->type));
```

5.36 Using the Count Sentinel

The `_count` value enables iteration and array sizing.

```
    /* Array indexed by enum */
    static const char *s_bus_type_names[k_star_bus_type_count] = {
    [k_star_bus_type_none] = "NONE", [k_star_bus_type_i2c] = "I2C",
    [k_star_bus_type_spi] = "SPI",   [k_star_bus_type_gpio] = "GPIO",
    [k_star_bus_type_adc] = "ADC",
};
```

```

/* Iteration using count */
for (star_bus_type_t type = k_star_bus_type_none; type <
    k_star_bus_type_count;
      ++type) {
  RX_LOG_INFO(s_TAG, "Bus type %d: %s", type, s_bus_type_names[type]);
}

/* Validation using count */
if (type >= k_star_bus_type_count) {
  RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
  return RX_ERR_INVALID_ARG;
}
```

5.37 Switch Statement Pattern

Always handle all enum values or include a default case.

```
        /* All values handled explicitly */
        switch (config->type) {
          case k_star_bus_type_none: {
      RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
      break;
   }
    case k_star_bus_type_i2c: {
      ret = init_i2c_bus(config);
      break;
   }
    case k_star_bus_type_spi: {
      ret = init_spi_bus(config);
      break;
   }
    case k_star_bus_type_gpio: {
      ret = init_gpio_bus(config);
      break;
   }
    case k_star_bus_type_adc: {
      ret = init_adc_bus(config);
      break;
   }
    case k_star_bus_type_count: {
      /* Sentinel value should never be used */
      RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
      return RX_ERR_INVALID_ARG;
   }
      /* No default - compiler warns if new enum values added */
}
```

5.38 Forward Declaration

Enums can be forward-declared for header dependency reduction.

```
/* Define enum with explicit underlying type (C23) before typedef */
```

```c
enum star_bus_type : int32_t {
  k_star_bus_type_none,
  k_star_bus_type_i2c,
  /* ... */
};
typedef enum star_bus_type star_bus_type_t;

/* Or forward-declare in header, define in source (C23 with underlying
    type) */
/* In header: */
enum star_bus_type : int32_t;  /* Forward declaration with explicit type
    */
typedef enum star_bus_type star_bus_type_t;

/* In source: */
enum star_bus_type : int32_t {
  k_star_bus_type_none,
  k_star_bus_type_i2c,
  /* ... */
};
```

5.39 Naming Convention Summary

Element	Convention	Example
Type name	`snake_case_t`	`star_bus_type_t`
Member prefix	`k_`	`k_star_bus_type_i2c`
Member naming	`snake_case`	`k_star_bus_type_i2c`
Count sentinel	`_count`	`k_star_bus_type_count`
String function	`_to_string`	`star_bus_type_to_string`

This section establishes error handling patterns for the STAR firmware codebase (RX72N and future targets).

5.40 General Principles

- **Return Error Codes**: Functions should return error codes via `esp_err_t`.
- **Check All Return Values**: Never ignore return values from functions that can fail.
- **Use `goto` for Clean - up**: Use `goto` to jump to a clean-up section when multiple resources need release.
- **Fail Fast**: Return early on invalid arguments or precondition failures.

5.41 Platform Error Codes

Use platform-specific error codes consistently. STAR defines generic error types that map to platform implementations.

```c
/* Generic STAR error codes (map to platform-specific implementations) */
RX_OK                    /* Success */
RX_ERR_FAIL              /* Generic failure */
```

```
RX_ERR_NO_MEM            /* Out of memory */
RX_ERR_INVALID_ARG       /* Invalid argument */
RX_ERR_INVALID_STATE     /* Invalid state for operation */
RX_ERR_INVALID_SIZE      /* Invalid size */
RX_ERR_NOT_FOUND         /* Requested resource not found */
RX_ERR_NOT_SUPPORTED     /* Operation not supported */
RX_ERR_TIMEOUT           /* Operation timed out */

/* Usage example */
rx_err_t star_bus_manager_init(star_bus_manager_t* manager, ...)
{
      if (manager == NULL) {
        return RX_ERR_INVALID_ARG;
     }

      manager->mutex = xSemaphoreCreateMutex();
      if (manager->mutex == NULL) {
        return RX_ERR_NO_MEM;
     }

      return RX_OK;
}
```

5.42 Validation Macros

Use platform-provided validation macros for parameter checking and early returns. These provide consistent error handling patterns across the codebase.

5.42.1 RX_RETURN_ON_FALSE

Returns an error if condition is false, with logging. ESP32 platforms use `ESP_RETURN_ON_FALSE`, RX72N uses `RX_RETURN_ON_FALSE`.

```
    /* Platform-specific: RX72N uses rx_check.h */
#include "rx_check.h"

rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*              tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      /* Validate required parameter with automatic logging */
      RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                         "Manager pointer is NULL");

      /* Multiple validations */
      RX_RETURN_ON_FALSE(manager->mutex, RX_ERR_INVALID_STATE,
         manager->tag,
                         "Manager mutex not initialized");

      RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
                             config->type < k_star_bus_type_count,
                         RX_ERR_INVALID_ARG, manager->tag,
```

```
                        "Invalid bus type in config");

    return RX_OK;
}
```

5.42.2 ESP_GOTO_ON_ERROR

Jumps to a label on error, with logging. Use for cleanup patterns.

```
static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
    config,
const uint8_t*           data,
size_t                   len,
uint8_t                  reg_addr,
size_t*                  bytes_written)
{
    rx_err_t ret = RX_OK;
    i2c_cmd_handle_t cmd = NULL;

    cmd = i2c_cmd_link_create();
    RX_RETURN_ON_FALSE(cmd, RX_ERR_NO_MEM, s_TAG,
                       "Failed to create I2C command link");

    ret = i2c_master_start(cmd);
    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': START
        failed: %s",
                      config->name, rx_err_to_name(ret));

    ret = i2c_master_write_byte(cmd, (address << 1) | I2C_MASTER_WRITE,
        true);
    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
                      "Write '%s': Address (W) failed: %s",
                         config->name,
                      rx_err_to_name(ret));

    ret = i2c_master_write(cmd, data, len, true);
    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
                      "Write '%s': Data write failed: %s", config->name,
                      rx_err_to_name(ret));

    ret = i2c_master_stop(cmd);
    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': STOP failed:
        %s",
                      config->name, rx_err_to_name(ret));

    /* Execute command */
    ret = i2c_master_cmd_begin(port, cmd, pdMS_TO_TICKS(1000));
    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': CMD failed",
                      config->name);

    /* Success path */
    i2c_cmd_link_delete(cmd);
    if (bytes_written) {
      *bytes_written = len;
```

```
    }
      return RX_OK;

    write_fail:
      if (cmd) {
        i2c_cmd_link_delete(cmd);
    }
      return ret;
}
```

5.42.3 ESP_ERROR_CHECK

Use during development to catch fatal errors.

```
/* Development only - aborts on error */
/* Platform-specific error checking example (ESP32 shown) */
ESP_ERROR_CHECK(i2c_driver_install(port, I2C_MODE_MASTER, 0, 0, 0));

/* Generic pattern: check return value and handle errors */
rx_err_t ret = optional_init();
if (ret != RX_OK) {
      RX_LOG_WARN(TAG, "Optional init failed: %d", ret);
}
```

5.43 NULL Safety Patterns

Always validate pointer parameters.

```
/* At function start - validate all pointers */
rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
const char*                name,
const uint8_t*             data,
size_t                     len,
uint8_t                    reg_addr,
size_t*                    bytes_written)
{
      RX_RETURN_ON_FALSE(manager && name && data, RX_ERR_INVALID_ARG,
          s_TAG,
                         "Manager, name, or data is NULL");
      RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
                         "Write length must be > 0");

      /* Clear output parameter initially */
      if (bytes_written) {
        *bytes_written = 0;
    }

      /* ... rest of implementation ... */
}

/* Check optional interface before use */
if (manager->error_iface && manager->error_iface->record_error) {
      manager->error_iface->record_error(manager->error_iface->ctx, ret,
```

```
                                            "Bus hardware initialization
                                               failed",
                                            __FILE__, __LINE__, __func__);
}

    /* Use convenience macro for safe interface calls */
#define STAR_IFACE_RECORD_ERROR(iface, err, msg)
    \\
    ((iface) && (iface)->record_error
                                                  \\
        ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
                \\
                                __LINE__, __func__)
                                                                \\
        : (err))

/* Usage */
STAR_IFACE_RECORD_ERROR(manager->error_iface, ret, "Operation failed");
```

5.44 Mutex Error Handling

Handle mutex acquisition failures properly.

```
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t*  config)
{
      /* Validate arguments first */
      RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                         "Invalid arguments");
      RX_RETURN_ON_FALSE(manager->mutex, ESP_ERR_INVALID_STATE,
          manager->tag,
                         "Mutex not initialized");

      rx_err_t ret = RX_OK;

      /* Acquire mutex with timeout */
      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        RX_LOG_ERROR(manager->tag,
                     "Timeout acquiring mutex for adding bus '%s'",
                     config->name);
        return RX_ERR_TIMEOUT;
     }

    /* Critical section */
    /* ... operations ... */

    /* Single exit point ensures mutex release */
    add_give_mutex:
      xSemaphoreGive(manager->mutex);
      return ret;
}
```

5.45 Error Interface Dependency Injection

Components receive error handlers through dependency injection.

```
/* Interface definition */
typedef struct star_error_interface {
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
         *message,
                               const char *file, int line, const char
                                  *func);
      bool (*can_retry)(void *ctx);
      rx_err_t (*reset_state)(void *ctx);
      void *ctx;
} star_error_interface_t;

/* Injecting error handler into component */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*              tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      manager->error_iface = error_iface; /* Can be NULL */
      /* ... */
}

/* Using the error interface */
if (ret != RX_OK) {
      if (manager->error_iface && manager->error_iface->record_error) {
        manager->error_iface->record_error(manager->error_iface->ctx, ret,
                                           "Bus initialization failed",
                                           __FILE__, __LINE__, __func__);
     }
}
```

5.46 Cleanup Patterns

Use goto for consistent cleanup when multiple resources are acquired.

```
rx_err_t complex_init(complex_t* obj)
{
      rx_err_t ret = RX_OK;

      /* Allocate resources */
      obj->buffer = malloc(BUFFER_SIZE);
      if (!obj->buffer) {
        ret = RX_ERR_NO_MEM;
        goto cleanup;
     }

      obj->mutex = xSemaphoreCreateMutex();
      if (!obj->mutex) {
```

```
        ret = RX_ERR_NO_MEM;
        goto cleanup;
    }

    ret = hardware_init(obj);
    if (ret != RX_OK) {
        goto cleanup;
    }

    return RX_OK;

cleanup:
    /* Release resources in reverse order */
    if (obj->mutex) {
        vSemaphoreDelete(obj->mutex);
        obj->mutex = NULL;
    }
    if (obj->buffer) {
        free(obj->buffer);
        obj->buffer = NULL;
    }
    return ret;
}
```

5.47 Error Logging

Use appropriate log levels for different error severities.

```
/* RX_LOG_ERROR - Errors that prevent operation */
RX_LOG_ERROR(s_TAG, "Failed to initialize bus '%s': %s",
config->name, rx_err_to_name(ret));

/* RX_LOG_WARN - Warnings that don't prevent operation */
RX_LOG_WARN(manager->tag, "Failed to register pins for bus '%s': %s "
"(bus still added)", config->name, rx_err_to_name(ret));

/* RX_LOG_INFO - Informational messages */
RX_LOG_INFO(manager->tag, "Added bus config: %s (Type: %s)",
config->name, star_bus_type_to_string(config->type));

/* RX_LOG_DEBUG - Debug messages (compiled out in release) */
RX_LOG_DEBUG(s_TAG, "Write Success: %zu bytes to '%s' (Addr 0x%02X)",
len, config->name, address);
```

5.48 Summary

- Return `rx_err_t` from all functions that can fail
- Use `ESP_RETURN_ON_FALSE` for parameter validation
- Use `ESP_GOTO_ON_ERROR` for cleanup patterns
- Always validate pointers before dereferencing

- Handle mutex timeouts as `ESP_ERR_TIMEOUT`
- Use dependency injection for error handlers
- Release resources in reverse order of acquisition
- Log errors with appropriate severity levels

This section establishes standards for function design, implementation, and documentation in the STAR firmware codebase.

5.49 Return Value Conventions

- If the name of a function is an action (or an imperative command), it should return an **error-code integer** (`esp_err_t`).
- If the name is a predicate (a function that returns true/false), it should return a **succeeded boolean**.

```
/* Action functions return rx_err_t */
rx_err_t star_bus_manager_init(...);
rx_err_t star_bus_manager_add_bus(...);
rx_err_t star_bus_config_destroy(...);

/* Predicate functions return bool */
bool error_handler_can_retry(error_handler_t* handler);
bool is_bus_initialized(const star_bus_config_t* config);

/* Getter functions return the value or pointer */
star_bus_config_t* star_bus_manager_find_bus(...);
const char* star_bus_type_to_string(star_bus_type_t type);
```

5.50 Private and Exposed Private Functions

5.50.1 Private Functions (static)

Functions internal to a single source file should be declared `static` with the `internal_` prefix.

```
/* In .c file - private static functions */
static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage,
bool                  can_be_shared)
{
      if (pin < 0 || pin >= GPIO_NUM_MAX) {
        return RX_OK; /* Not an error, pin is not used */
     }
      /* ... implementation ... */
}

static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
   config,
```

```
i2c_port_t                port,
i2c_cmd_handle_t          cmd,
const char*               operation_name)
{
      /* ... implementation ... */
}

/* Declare prototypes at top of file */
static rx_err_t internal_register_bus_pin(...);
static rx_err_t internal_unregister_bus_pin(...);
static rx_err_t internal_register_config_pins(...);
```

5.50.2 Exposed Private Functions (priv_)

If a function cannot be `static` and must be exposed across files within a module (but not publicly), prefix its name with `priv_`.

```
/* In internal header (e.g., star_bus_internal.h) */
rx_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
void      priv_star_bus_release_mutex(star_bus_manager_t* manager);
```

5.51 Parameter Validation

Validate all parameters at the start of public functions.

```
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t*  config)
{
      /* Validate required parameters */
      ESP_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                          "Manager or config pointer is NULL");

      /* Validate config contents */
      ESP_RETURN_ON_FALSE(config->name && strlen(config->name) > 0,
                          RX_ERR_INVALID_ARG, manager->tag,
                          "Config name is NULL or empty");

      /* Validate enum range */
      RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
                             config->type < k_star_bus_type_count,
                         RX_ERR_INVALID_ARG, manager->tag,
                         "Invalid bus type in config");

      /* Validate state */
      RX_RETURN_ON_FALSE(manager->mutex, RX_ERR_INVALID_STATE,
         manager->tag,
                         "Manager mutex not initialized");

      /* ... rest of implementation ... */
}

rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
```

```
const char*                 name,
const uint8_t*              data,
size_t                      len,
uint8_t                     reg_addr,
size_t*                     bytes_written)
{
      /* Validate all required pointers */
      RX_RETURN_ON_FALSE(manager && name && data, RX_ERR_INVALID_ARG,
          s_TAG,
                         "Manager, name, or data is NULL");

      /* Validate numeric constraints */
      RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
                         "Write length must be > 0");

      /* Clear output parameters initially */
      if (bytes_written) {
        *bytes_written = 0;
     }

      /* ... rest of implementation ... */
}
```

5.52 Function Declaration Formatting

Align parameters vertically when they span multiple lines.

```
/* CORRECT - Aligned parameters */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*              tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);

star_bus_config_t* star_bus_config_create_spi_device(
const char*                          name,
spi_host_device_t                    host,
gpio_num_t                           copi_pin,
gpio_num_t                           cipo_pin,
gpio_num_t                           sclk_pin,
int32_t                              dma_chan,
const spi_device_interface_config_t* dev_cfg);

/* Function pointers in structs */
typedef struct star_error_interface {
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
          *message,
                               const char *file, int line, const char
                                  *func);
      bool (*can_retry)(void *ctx);
      rx_err_t (*reset_state)(void *ctx);
      void *ctx;
} star_error_interface_t;
```

5.53 Doxygen Documentation

All public functions must have complete Doxygen documentation in header files.

```
/**
* @brief Add a new bus/device configuration to the manager.
*
* Adds a previously created configuration (via star_bus_config_create_*)
* to the manager's internal list. The manager takes ownership conceptually,
* but the configuration still needs to be destroyed eventually (typically via
* star_bus_manager_remove_bus or star_bus_manager_deinit).
*
* The configuration must have a unique name. This function is thread-safe.
*
* @param[in] manager Pointer to the initialized bus manager.
*                    Must not be NULL.
* @param[in] config  Pointer to the bus configuration structure to add.
*                    Must not be NULL and must have a valid name.
* @return rx_err_t RX_OK on success,
*                  ESP_ERR_INVALID_ARG if manager or config is invalid,
*                  ESP_ERR_INVALID_STATE if a bus with the same name
*                      already exists or mutex error,
*                  ESP_ERR_TIMEOUT if mutex times out.
*/
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t*  config);
```

5.54 Private Function Documentation

Static functions should have brief documentation in the source file.

```
/**
* @brief Helper: Execute I2C command with retry logic
*
* Wraps i2c_master_cmd_begin() with automatic retry for transient errors.
*
* @param[in] config         Pointer to I2C bus configuration
* @param[in] port           I2C port number
* @param[in] cmd            I2C command handle
* @param[in] operation_name Name of operation for logging
* @return rx_err_t Final result after retries
*/
static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
    config,
i2c_port_t               port,
i2c_cmd_handle_t         cmd,
const char*              operation_name);
```

5.55 Function Implementation Patterns

5.55.1 Single Exit Point with Cleanup

```c
rx_err_t star_bus_manager_remove_bus(star_bus_manager_t* manager,
const char*         name)
{
      ESP_RETURN_ON_FALSE(manager && name && strlen(name) > 0,
                          RX_ERR_INVALID_ARG, s_TAG, "Invalid arguments");

      star_bus_config_t *to_remove = NULL;
      star_bus_config_t *prev = NULL;
      rx_err_t ret = RX_ERR_NOT_FOUND;

      /* Acquire mutex */
      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      /* Find and unlink */
      for (star_bus_config_t *curr = manager->buses; curr;
           prev = curr, curr = curr->next) {
        if (curr->name && strcmp(curr->name, name) == 0) {
          to_remove = curr;
          if (prev == NULL) {
            manager->buses = curr->next;
         } else {
            prev->next = curr->next;
         }
          to_remove->next = NULL;
          ret = RX_OK;
          break;
       }
     }

      if (ret == ESP_ERR_NOT_FOUND) {
        goto remove_give_mutex;
     }

      /* Cleanup operations */
      rx_err_t destroy_result = star_bus_config_destroy(to_remove);
      if (destroy_result != RX_OK) {
        ret = destroy_result;
     }

    remove_give_mutex:
      xSemaphoreGive(manager->mutex);
      return ret;
}
```

5.55.2 Factory Function Pattern

```c
star_bus_config_t* star_bus_config_create_i2c(const char* name,
```

```
i2c_port_t  port,
uint8_t     address,
gpio_num_t  sda_pin,
gpio_num_t  scl_pin,
uint32_t    clk_speed)
{
      if (name == NULL || strlen(name) == 0) {
        RX_LOG_ERROR(s_TAG, "Invalid name for I2C bus config");
        return NULL;
     }

      star_bus_config_t *config = calloc(1, sizeof(star_bus_config_t));
      if (config == NULL) {
        RX_LOG_ERROR(s_TAG, "Failed to allocate memory for I2C config");
        return NULL;
     }

      /* Initialize common fields */
      config->name = strdup(name);
      config->type = k_star_bus_type_i2c;
      config->initialized = false;
      config->handle = NULL;
      config->next = NULL;

      /* Initialize I2C-specific fields */
      config->proto.i2c.port = port;
      config->proto.i2c.address = address;
      config->proto.i2c.config.sda_io_num = sda_pin;
      config->proto.i2c.config.scl_io_num = scl_pin;
      config->proto.i2c.config.master.clk_speed = clk_speed;

      /* Set default operations */
      star_bus_i2c_init_default_ops(&config->proto.i2c.ops);

      return config;
}
```

5.56 Callback Functions

```
/* Callback type definition */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
void*              user_ctx);

/* Function using callback */
rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
const char*         name,
star_bus_callback_t callback,
void*               user_ctx)
{
      ESP_RETURN_ON_FALSE(manager && name && callback, RX_ERR_INVALID_ARG,
                          s_TAG, "Invalid arguments");

      /* ... find bus and invoke callback while holding mutex ... */
```

```
        rx_err_t callback_result = callback(found_bus, user_ctx);

        xSemaphoreGive(manager->mutex);
        return callback_result;
}

/* Using the callback pattern */
rx_err_t my_bus_callback(star_bus_config_t* bus, void* ctx) {
        ESP_LOGI(TAG, "Bus type: %s", star_bus_type_to_string(bus->type));
        return RX_OK;
}

star_bus_manager_with_bus(&manager, "sensor_i2c", my_bus_callback, NULL);
```

5.57 Summary

- Action functions return `esp_err_t`; predicates return `bool`
- Use `static` with `internal_` prefix for file-local functions
- Use `priv_` prefix for exposed-but-not-public functions
- Validate all parameters at function start
- Align parameters vertically in multi-line declarations
- Document all public functions with complete Doxygen
- Use single exit point with cleanup for complex functions
- Use factory functions for object creation

Global variables should be avoided when possible. This section establishes patterns for when they are necessary.

5.58 General Principles

- **Avoid Globals**: Global variables are strongly discouraged. Use dependency injection instead.
- **File-Scoped When Necessary**: If a global is needed, limit its scope to a single file using `static`.
- **Use Prefix Conventions**: `s_` for static file-scoped, `g_` for true globals.
- **Prefer `static const` Over `#define`**: For typed constants, use `static const`.

5.59 Static File-Scoped Variables (s_ prefix)

Use `static` with `s_` prefix for variables that should be accessible only within a single source file.

```
/* CORRECT - Static file-scoped variables */

/* Logging tag - most common pattern */
static const char* s_TAG = "STAR_BUS_MANAGER";
```

```c

/* Configuration constants - prefer static const over #define */
static const uint32_t s_i2c_timeout_ms = 1000;
static const uint32_t s_default_retry_count = 3;

/* State variables scoped to this file */
static bool s_module_initialized = false;

/* From star_bus_i2c.c */
static const char* s_TAG = "STAR_BUS_I2C";
static const uint32_t s_i2c_timeout_ms = 1000;

/* From star_bus_config.c */
static const char* s_TAG = "STAR_BUS_CONFIG";
```

5.60 Why static const Over #define

`static const` provides type safety and scope control that `#define` lacks.

```c
/* PREFERRED - Type-safe, scoped, debugger-visible */
static const uint32_t s_timeout_ms = 1000;
static const size_t   s_buffer_size = 256;

/* Use the constant with type checking */
uint32_t timeout = s_timeout_ms;  /* Compiler checks type */

    /* AVOID - No type safety, global namespace pollution */
#define TIMEOUT_MS 1000
#define BUFFER_SIZE 256

/* Issues with #define: */
/* 1. No type checking */
/* 2. Can collide with other defines */
/* 3. Not visible in debugger */
/* 4. Can have unexpected macro expansion */
```

5.61 When to Use #define

Use `#define` for:

- Compile-time constants for array sizes
- Conditional compilation
- Macros that cannot be functions

```c
    /* Array sizes must be compile-time constants */
#define STAR_BUS_NAME_MAX_LEN (32)

char bus_name[STAR_BUS_NAME_MAX_LEN];  /* OK - compile-time constant */

    /* Conditional compilation */
#ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
```

```
RX_LOG_DEBUG(s_TAG, "Debug info: %d", value);
#endif

    /* Macros with special features */
#define RECORD_ERROR(handler, code, desc)
   \\
      do {

         \\
        error_handler_record_error((handler), (code), (desc), __FILE__,
                 \\
                                    __LINE__, __func__);
                                                          \\
     } while (0)
```

5.62 Explicit Type Usage - Always Use Fixed - Width Integer Types

CRITICAL RULE: Never use implicit types like `int`, `char`, `short`, or `long`. Always use explicit fixed-width types from `<stdint.h>`.

5.62.1 Why Explicit Types Matter

- **Platform Independence**: `int` can be 16-bit or 32-bit depending on platform
- **Clear Intent**: `uint32_t` explicitly shows the variable is unsigned and 32-bit
- **Prevents Bugs**: Implicit integer promotion and sign extension bugs are avoided
- **Embedded Safety**: Embedded systems require precise control over data sizes
- **Code Portability**: Code works identically on ESP32, RX72N, and future platforms

5.62.2 Correct Type Usage

```
#include <stdbool.h>
#include <stdint.h>

/* CORRECT - Always use explicit fixed-width types */

/* Unsigned integers */
uint8_t   byte_value    = 0xFF;      /* 8-bit unsigned (0 to 255) */
uint16_t  word_value    = 0xFFFF;    /* 16-bit unsigned (0 to 65535) */
uint32_t  counter       = 0;          /* 32-bit unsigned */
uint64_t  timestamp_us  = 0;          /* 64-bit unsigned */

/* Signed integers */
int8_t    temperature   = -40;        /* 8-bit signed (-128 to 127) */
int16_t   offset        = -1000;      /* 16-bit signed */
int32_t   position      = 0;          /* 32-bit signed */
int64_t   delta_time_us = 0;          /* 64-bit signed */

/* Boolean (not int!) */
bool      is_enabled    = false;      /* Use stdbool.h */
```

```

/* Size and pointer types */
size_t    buffer_size   = 256;        /* For sizes and counts */
uintptr_t address       = 0x20000000; /* For storing addresses */

/* WRONG - Never use these implicit types */
int       bad_counter;     /* Platform-dependent size! */
unsigned  bad_value;       /* Ambiguous width! */
char      bad_byte;        /* Is it signed or unsigned? */
short     bad_small;       /* 16-bit? Not guaranteed! */
long      bad_large;       /* 32-bit or 64-bit? Depends on platform! */
```

5.62.3 Special Case: Characters and Strings

Only use `char` for actual character strings, never for byte data.

```
/* CORRECT - char only for strings */
const char* message = "Hello, STAR!";
char        buffer[64];
char        letter = 'A';

/* WRONG - Don't use char for byte data */
char data[256];  /* Should be uint8_t data[256] */
char byte = 0xFF;  /* Should be uint8_t byte = 0xFF */

/* CORRECT - uint8_t for byte data */
uint8_t data_buffer[256];
uint8_t byte_value = 0xFF;
```

5.62.4 Loop Counters and Array Indices

Use `uint32_t` or `size_t` for loop counters and array indices.

```
/* CORRECT - Explicit types for loops */
for (uint32_t i = 0; i < array_length; i++) {
      process_element(array[i]);
}

/* For sizes from sizeof() or string length */
for (size_t i = 0; i < buffer_size; i++) {
      buffer[i] = 0;
}

/* WRONG - Never use int for loops */
for (int i = 0; i < count; i++) {  /* What if count > INT_MAX? */
      /* ... */
}
```

5.62.5 Function Parameters and Return Types

Always use explicit types in function signatures.

```c
/* CORRECT - Explicit types in function signatures */
uint32_t calculate_checksum(const uint8_t* data, size_t length);
int32_t  read_temperature_celsius(uint8_t sensor_id);
bool     is_motor_running(uint8_t motor_id);

/* WRONG - Implicit types */
int calculate_value(char* data, int len);  /* Ambiguous! */
```

5.62.6 Type Casting for Enum Comparisons

When comparing enums of different types, cast to the appropriate fixed-width type.

```c
typedef enum : uint8_t {
  k_channel_0 = 0,
  k_channel_1 = 1,
  k_channel_max = 2,
} channel_t;

/* CORRECT - Cast to explicit type for comparison */
if ((int32_t)channel >= k_channel_max) {
      return ERROR_INVALID_CHANNEL;
}

/* WRONG - Direct enum comparison can trigger warnings */
if (channel >= k_channel_max) {  /* -Werror=enum-compare */
      return ERROR_INVALID_CHANNEL;
}
```

5.62.7 Summary: Type Usage Rules

- **ALWAYS**: Use `uint8_t`, `int32_t`, etc. from `<stdint.h>`
- **NEVER**: Use `int`, `unsigned`, `short`, `long`
- **ONLY**: Use `char` for actual character strings
- **USE**: `bool` from `<stdbool.h>` for boolean values
- **USE**: `size_t` for sizes and counts from sizeof()
- **CAST**: Enums to `int32_t` when comparing different enum types

5.63 True Global Variables (g_ prefix)

If a truly global variable is unavoidable, use the `g_` prefix. This should be rare.

```c
/* DISCOURAGED but sometimes necessary */
/* g_ prefix for true globals (visible across files) */
g_system_state_t g_system_state;

/* Prefer dependency injection instead */
/* BETTER approach: */
typedef struct app_context {
```

```
      star_bus_manager_t bus_manager;
      error_handler_t error_handler;
      /* ... */
} app_context_t;

/* Pass context to functions that need it */
void app_process_sensors(app_context_t* ctx);
```

5.64 Dependency Injection Alternative

Replace globals with dependency injection.

```
/* WRONG - Global error handler */
error_handler_t g_error_handler;  /* Global - bad */

void some_function(void) {
      if (error) {
        RECORD_ERROR(&g_error_handler, err, "Failed"); /* Uses global */
     }
}

/* CORRECT - Dependency injection */
typedef struct star_bus_manager {
      star_error_interface_t *error_iface; /* Injected dependency */
      /* ... */
} star_bus_manager_t;

rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,  /* Injected */
star_pin_interface_t*   pin_iface)    /* Injected */
{
      manager->error_iface = error_iface;
      /* ... */
}

/* Usage */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
error_handler_get_interface(&error_iface, &error_handler);

star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, NULL);
```

5.65 Static Variables for Module State

When a module needs to maintain state, prefer explicit initialization and deinitialization.

```
/* Module-private state */
static struct {
      bool initialized;
```

```
      uint32_t transaction_count;
      /* ... */
} s_module_state = {
  .initialized = false,
  .transaction_count = 0,
};

rx_err_t module_init(void)
{
      if (s_module_state.initialized) {
        return RX_ERR_INVALID_STATE;
     }

      /* Initialize module */
      s_module_state.initialized = true;
      return RX_OK;
}

rx_err_t module_deinit(void)
{
      if (!s_module_state.initialized) {
        return RX_ERR_INVALID_STATE;
     }

      /* Clean up */
      s_module_state.initialized = false;
      s_module_state.transaction_count = 0;
      return RX_OK;
}
```

5.66 Thread Safety for Shared State

If static variables are accessed from multiple threads, protect with mutex.

```
/* Thread-safe module state */
static struct {
      bool initialized;
      SemaphoreHandle_t mutex;
      uint32_t counter;
} s_shared_state;

rx_err_t safe_increment_counter(void)
{
      if (xSemaphoreTake(s_shared_state.mutex, pdMS_TO_TICKS(1000)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      s_shared_state.counter++;

      xSemaphoreGive(s_shared_state.mutex);
      return RX_OK;
}
```

5.67 Summary

Type	Prefix	Usage
Static file-scoped	`s_`	Prefer this for module-internal state
True global	`g_`	Avoid; use dependency injection
`static const`	`s_`	Type-safe constants
`#define`	None	Array sizes, macros, conditional compilation

- Use `static const` for type-safe constants
- Use `s_TAG` for module logging tags
- Prefer dependency injection over global variables
- Protect shared state with mutexes
- Limit variable scope to the smallest necessary

This section establishes standards for header file organization and content in the STAR firmware codebase.

5.68 General Principles

- **Header Files Should Only Contain Declarations**: No implementations except for inline functions.
- **Minimal Includes in Headers**: Include only what is necessary for the declarations.
- **Self-Contained Headers**: Each header should compile independently.
- **One Purpose Per Header**: Split large headers into focused sub-headers.

5.69 Include Guard Naming Convention

All header files must use include guards following the pattern: `STAR_COMPONENT_FILENAME_H`

```
    /* CORRECT - from star_bus_manager.h */
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

        /* ... declarations ... */

#endif /* STAR_COMPONENT_BUS_MANAGER_H */

    /* CORRECT - from star_bus_config.h */
#ifndef STAR_COMPONENT_BUS_CONFIG_H
#define STAR_COMPONENT_BUS_CONFIG_H
    /* ... */
#endif /* STAR_COMPONENT_BUS_CONFIG_H */

    /* CORRECT - from star_error_interface.h */
#ifndef STAR_ERROR_INTERFACE_H
#define STAR_ERROR_INTERFACE_H
    /* ... */
```

```
#endif /* STAR_ERROR_INTERFACE_H */

    /* CORRECT - from star_pin_interface.h */
#ifndef STAR_PIN_INTERFACE_H
#define STAR_PIN_INTERFACE_H
    /* ... */
#endif /* STAR_PIN_INTERFACE_H */
```

5.70 C++ Compatibility

All headers must include C++ extern "C" blocks for compatibility with C++ compilers.

```
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

        /* All declarations go here */

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.71 Header File Structure

Headers should follow this organization:

```
        /* lib/star_bus/include/star_bus_manager.h */

#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

    /* === 1. Includes === */
#include "esp_err.h"
#include "star_bus_manager_types.h"

    /* === 2. File Documentation === */
    /**
     * @file star_bus_manager.h
     * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
        protocols
     *
     * The bus manager provides a central registry for managing multiple
     * bus configurations. It supports dependency injection of error
        handlers
```

```
     * and pin validators for loose coupling.
     *
     * Example Usage:
     * @code
     * // Initialize bus manager with dependency injection
     * star_bus_manager_t bus_manager;
     * star_bus_manager_init(&bus_manager, "main", &error_iface,
        &pin_iface);
     * @endcode
     */

    /* === 3. Forward Declarations (if needed) === */
    /* Forward declarations reduce header dependencies */

    /* === 4. Macros and Constants === */
#define STAR_BUS_NAME_MAX_LEN (32)
     /* === 5. Type Definitions === */
     /* (Usually in separate _types.h header) */

     /* === 6. Function Declarations === */

     /**
      * @brief Initialize a bus manager instance.
      *
      * @param[out] manager     Pointer to bus manager to initialize.
      * @param[in]  tag         Optional tag string for logging.
      * @param[in]  error_iface Optional error handler interface.
      * @param[in]  pin_iface   Optional pin validator interface.
      * @return rx_err_t RX_OK on success.
      */
     rx_err_t star_bus_manager_init(
         star_bus_manager_t * manager, const char *tag,
         star_error_interface_t *error_iface, star_pin_interface_t
            *pin_iface);

         /* ... more function declarations ... */

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.72 Include Order

Includes should be ordered in groups, separated by blank lines:

```
    /* 1. Primary header (in .c file, include corresponding .h first) */
#include "star_bus_manager.h"

    /* 2. FreeRTOS includes */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"
#include "freertos/task.h"
```

```

    /* 3. Standard library includes */
#include <inttypes.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

    /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */
#include "driver/gpio.h"
#include "driver/i2c.h"
    /* Platform-specific: RX72N uses rx_check.h */
#include "esp_err.h"
#include "esp_log.h"
#include "rx_check.h"
#include "sdkconfig.h"

    /* 5. Project includes */
#include "star_bus_config.h"
#include "star_bus_i2c.h"
#include "star_bus_types.h"
#include "star_error_interface.h"
```

5.73 Forward Declarations

Use forward declarations to minimize header dependencies.

```
/* In star_bus_config.h - forward declare instead of including */
/* Forward declaration */
struct star_bus_manager;  /* Don't need full definition */

/* Function using forward-declared type */
rx_err_t star_bus_config_init(star_bus_config_t*      config,
struct star_bus_manager* manager);

/* In star_bus_common_types.h */
/* Forward declarations for pointer-only usage */
typedef struct star_bus_config  star_bus_config_t;
typedef struct star_bus_manager star_bus_manager_t;
```

5.74 Separating Types from Functions

For complex modules, split type definitions into dedicated headers.

```
/* File organization for star_bus module */
star_bus/include/
star_bus_manager.h        /* Main API (functions) */
star_bus_manager_types.h  /* Types for manager */
star_bus_common_types.h   /* Shared types */
star_bus_protocol_types.h /* Protocol-specific types */
star_bus_function_types.h /* Function pointer types */
```

```
star_bus_event_types.h    /* Event structures */
star_bus_types.h          /* Controller - includes all */

    /* Controller header pattern */
    /* star_bus_types.h */
#ifndef STAR_COMPONENT_BUS_TYPES_H
#define STAR_COMPONENT_BUS_TYPES_H

#ifdef __cplusplus
extern "C" {
#endif

    /* Include all type headers */
#include "star_bus_common_types.h"
#include "star_bus_event_types.h"
#include "star_bus_function_types.h"
#include "star_bus_manager_types.h"
#include "star_bus_protocol_types.h"

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_TYPES_H */
```

5.75 Doxygen File Documentation

Include `@file` documentation for headers with significant content.

```
/**
* @file star_error_handler.h
* @brief Error handling with retry logic and exponential backoff
*
* This module provides a concrete implementation of the error interface
* with automatic retry management, exponential backoff delays, and
* optional reset function callbacks.
*
* Key Features:
* - Configurable max retries and backoff delays
* - Exponential backoff between retries
* - Optional custom reset function for recovery
* - Thread-safe with mutex protection
*
* Example Usage:
* @code
* error_handler_t error_handler;
* error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
*
* if (operation_failed()) {
  *     RECORD_ERROR(&error_handler, ESP_ERR_TIMEOUT, "Timeout");
  *}
  * @endcode
  */
```

5.76 Complete Header Template

```c
        /* lib/star_module/include/star_module.h */

#ifndef STAR_COMPONENT_MODULE_H
#define STAR_COMPONENT_MODULE_H

#ifdef __cplusplus
extern "C" {
#endif

    /* === Includes === */
#include <stdbool.h>
#include <stdint.h>

#include "esp_err.h"
      /* === File Documentation === */
      /**
       * @file star_module.h
       * @brief Brief description of this module
       *
       * Detailed description of what this module provides and how to use
          it.
       */

      /* === Forward Declarations === */
      typedef struct star_module star_module_t;

    /* === Macros === */
#define STAR_MODULE_DEFAULT_TIMEOUT_MS (1000)

      /* === Type Definitions === */
      typedef struct star_module {
        bool initialized;
        uint32_t timeout_ms;
        void *user_ctx;
     } star_module_t;

      /* === Function Declarations === */

      /**
       * @brief Initialize the module.
       *
       * @param[out] module  Pointer to module structure to initialize.
       * @param[in]  timeout Timeout in milliseconds.
       * @return rx_err_t RX_OK on success.
       */
      esp_err_t star_module_init(star_module_t * module, uint32_t
          timeout);

      /**
       * @brief Deinitialize the module.
       *
```

```
50          * @param[in,out] module Pointer to module structure to
                deinitialize.
51          * @return rx_err_t RX_OK on success.
52          */
53         esp_err_t star_module_deinit(star_module_t * module);
54
55 #ifdef __cplusplus
56 }
57 #endif
58
59 #endif /* STAR_COMPONENT_MODULE_H */
```

5.77 Summary

- Use include guards: `STAR_COMPONENT_FILENAME_H`
- Include C++ extern "C" blocks
- Order includes: RTOS, standard library, platform HAL, project
- Use forward declarations to minimize dependencies
- Split types into separate _types.h headers for complex modules
- Document files with `@file` Doxygen blocks
- Keep headers focused on declarations, not implementations

This section establishes standards for horizontal spacing in the STAR firmware codebase.

5.78 General Rules

- **Space After Keywords**: Add space after `if`, `for`, `while`, `switch`.
- **Binary Operators**: Add a single space around binary operators.
- **No Space After Function Names**: No space between function name and opening parenthesis.
- **Alignment**: Align related declarations and comments.

5.79 Space After Control Keywords

Add a space between keywords and opening parenthesis.

```
1 /* CORRECT - Space after keywords */
2 if (ret != RX_OK) {
3       return ret;
4 }
5
6 for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
7       process(curr);
8 }
9
```

```c
while (retry_count > 0) {
      attempt_operation();
      retry_count--;
}

switch (config->type) {
    case k_star_bus_type_i2c: {
      /* ... */
      break;
   }
}

/* WRONG - No space after keywords */
if(ret != RX_OK) {          /* Missing space */
      return ret;
}

for(int i = 0; i < 10; i++) {  /* Missing space */
      /* ... */
}
```

5.80 No Space After Function Names

Do not add space between function name and opening parenthesis.

```c
/* CORRECT - No space after function name */
rx_err_t ret = star_bus_manager_init(&manager, "main", NULL, NULL);
RX_LOG_INFO(s_TAG, "Initialized bus manager");
free(buffer);

/* WRONG - Space after function name */
esp_err_t ret = star_bus_manager_init (&manager, "main", NULL, NULL);
RX_LOG_INFO (s_TAG, "Initialized bus manager");
free (buffer);
```

5.81 Binary Operators

Add a single space around binary operators.

```c
/* CORRECT - Spaces around binary operators */
int result = a + b;
bool is_valid = (count > 0) && (size <= MAX_SIZE);
uint32_t mask = flags | FLAG_ENABLED;
int shifted = value << 2;
size_t remaining = total - processed;

/* Assignment operators */
count += 1;
flags |= FLAG_INITIALIZED;
value >>= 4;

/* Comparison operators */
if (ret != RX_OK) { /* ... */}
```

```
if (count >= threshold) { /* ... */}
if (ptr == NULL) { /* ... */}

/* WRONG - Missing spaces */
int result = a+b;
bool is_valid = (count>0)&&(size<=MAX_SIZE);
if (ret!=ESP_OK) { /* ... */}
```

5.82 Unary Operators

No space between unary operator and operand.

```
/* CORRECT - No space with unary operators */
count++;
--remaining;
bool is_null = !ptr;
int negative = -value;
uint32_t complement = ~mask;
size_t size = sizeof(star_bus_config_t);
int* ptr = &value;
int deref = *ptr;

/* WRONG - Space with unary operators */
count ++;
-- remaining;
bool is_null = ! ptr;
```

5.83 Pointer Declarations

Place asterisk with the type, not the variable name.

```
/* CORRECT - Asterisk with type */
star_bus_config_t* config;
const char* name;
void* user_ctx;

/* Function parameters */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*              tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);

/* Multiple pointers on same line (avoid when possible) */
int *a, *b;  /* Both are pointers */

/* WRONG - Various incorrect styles */
star_bus_config_t *config;     /* Asterisk with variable */
star_bus_config_t * config;    /* Spaces around asterisk */
```

5.84 Comma and Semicolon Spacing

Space after comma and semicolon, not before.

```c
/* CORRECT */
star_bus_manager_init(&manager, "main", &error_iface, &pin_iface);

for (int i = 0; i < count; i++) {
      /* ... */
}

int array[] = {1, 2, 3, 4, 5};

/* WRONG */
star_bus_manager_init(&manager ,"main" ,&error_iface ,&pin_iface);
for (int i = 0 ;i < count ;i++) { /* ... */}
int array[] = {1 ,2 ,3 ,4 ,5};
```

5.85 Parentheses Spacing

No space after opening parenthesis or before closing parenthesis.

```c
/* CORRECT */
if (ret != RX_OK) {
      return ret;
}

result = calculate(a, b, c);

/* WRONG - Spaces inside parentheses */
if ( ret != RX_OK ) {
      return ret;
}

result = calculate( a, b, c );
```

5.86 Parameter Alignment

Align parameters vertically in multi-line function declarations and calls.

```c
/* CORRECT - Aligned parameters */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);

star_bus_config_t* star_bus_config_create_spi_device(
const char*                         name,
spi_host_device_t                   host,
gpio_num_t                          copi_pin,
gpio_num_t                          cipo_pin,
gpio_num_t                          sclk_pin,
int32_t                             dma_chan,
const spi_device_interface_config_t* dev_cfg);

/* Long function calls */
```

```
ESP_RETURN_ON_FALSE(manager && config,
ESP_ERR_INVALID_ARG,
s_TAG,
"Manager or config pointer is NULL");
```

5.87 Variable and Comment Alignment

Align related variables and their comments.

```
/* CORRECT - Aligned declarations and comments */
typedef struct error_handler {
      uint32_t error_count;          /**< Total errors recorded */
      uint32_t max_retries;          /**< Max retry attempts */
      uint32_t current_retry;        /**< Current retry counter */
      uint32_t base_retry_delay;     /**< Base delay (ms) */
      uint32_t max_retry_delay;      /**< Max delay (ms) */
      uint32_t current_retry_delay; /**< Current delay with backoff */
      rx_err_t last_error;           /**< Last recorded error code */
      bool in_error_state;           /**< Whether in error state */
      void *reset_context;           /**< Context for reset function */
      SemaphoreHandle_t mutex;       /**< Mutex for thread-safety */
} error_handler_t;

/* Aligned local variables */
i2c_port_t       port    = config->proto.i2c.port;
uint8_t          address = config->proto.i2c.address;
rx_err_t        ret     = RX_OK;
i2c_cmd_handle_t cmd     = NULL;
```

5.88 Struct Initializer Alignment

Align designated initializers.

```
/* CORRECT - Aligned initializers */
star_bus_event_t event = {
  .bus_type = k_star_bus_type_i2c,
  .bus_name = config->name,
  .i2c      = {
    .port     = port,
    .address  = address,
    .is_write = true,
    .len      = len
 }
};

spi_device_interface_config_t spi_cfg = {
  .clock_speed_hz = 10 * 1000 * 1000,
  .mode           = 0,
  .spics_io_num   = GPIO_NUM_5,
  .queue_size     = 7,
  .pre_cb         = NULL,
  .post_cb        = NULL,
};
```

5.89 Summary

Context	Rule
After `if`, `for`, `while`, `switch`	Space
After function name	No space
Around binary operators	Space
With unary operators	No space
After comma/semicolon	Space
Inside parentheses	No space
Pointer asterisk	With type
Multi-line parameters	Align vertically

This section establishes standards for include statement organization and ordering in the STAR firmware codebase.

5.90 General Rules

- **Use Angle Brackets for System/Library Headers**: `<stdio.h>`.
- **Use Quotation Marks for Project Headers**: `"my_project.h"`.
- **Minimal Includes in Header Files**: Only include what is necessary.
- **Group and Order Includes**: Follow consistent ordering.

5.91 Include Order

Includes should be ordered in these groups, separated by blank lines:

1. Primary header (in .c files)
2. FreeRTOS includes
3. Standard library includes
4. Platform HAL includes
5. Project includes

```
    /* lib/star_bus/src/star_bus_i2c.c */

    /* 1. Primary header - always first in .c file */
#include "star_bus_i2c.h"

    /* 2. FreeRTOS includes */
#include "freertos/FreeRTOS.h"

    /* 3. Standard library includes */
#include <inttypes.h>
#include <string.h>

    /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */
    /* Platform-specific: RX72N uses rx_check.h */
```

```
#include "esp_log.h"
#include "rx_check.h"

    /* 5. Project includes */
#include "star_bus_manager.h"
#include "star_bus_types.h"
```

5.92 Complete Example from Codebase

```
    /* lib/star_bus/src/star_bus_manager.c */

    /* Primary header */
#include "star_bus_manager.h"

    /* FreeRTOS */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"

    /* Standard library */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

    /* Platform HAL (ESP-IDF, Renesas, etc.) */
    /* Platform-specific: RX72N uses rx_check.h */
#include "esp_log.h"
#include "rx_check.h"
#include "sdkconfig.h"

    /* Project includes */
#include "star_bus_adc.h"
#include "star_bus_config.h"
#include "star_bus_gpio.h"
#include "star_bus_i2c.h"
#include "star_bus_spi.h"
```

5.93 FreeRTOS Include Order

FreeRTOS requires specific include ordering. Always include `FreeRTOS.h` first.

```
    /* CORRECT - FreeRTOS.h must come first */
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "freertos/task.h"

    /* WRONG - Other FreeRTOS headers before FreeRTOS.h */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h" /* Error: FreeRTOS.h not included first */
```

5.94 System Includes

Use angle brackets for system and standard library headers.

```
    /* Standard C library */
#include <inttypes.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

5.95 Platform HAL Includes

Platform HAL headers (ESP-IDF, Renesas, etc.) use quotation marks and are grouped separately.

```
    /* Platform-specific driver includes (example: ESP32) */
#include "driver/gpio.h"
#include "driver/i2c.h"
#include "driver/spi_master.h"
#include "driver/uart.h"

    /* Platform system includes */
    /* Platform-specific: RX72N uses rx_check.h */
#include "platform_system.h"
#include "rx_check.h"
#include "rx_err.h" /* Platform error types */
#include "rx_log.h" /* Platform logging */

    /* Platform peripheral includes (if needed) */
#include "esp_adc/adc_oneshot.h"
#include "platform_adc.h"

    /* Configuration */
#include "sdkconfig.h"
```

5.96 Project Includes

Use quotation marks for project headers. Alphabetize within each group.

```
    /* Project includes - alphabetized */
#include "star_bus_adc.h"
#include "star_bus_config.h"
#include "star_bus_gpio.h"
#include "star_bus_i2c.h"
#include "star_bus_manager.h"
#include "star_bus_spi.h"
#include "star_bus_types.h"
#include "star_error_interface.h"
#include "star_pin_interface.h"
```

5.97 Header File Includes

Header files should minimize includes and use forward declarations when possible.

```
    /* star_bus_manager.h - Header file includes */
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

    /* Only include what's needed for declarations */
#include "esp_err.h"
#include "star_bus_manager_types.h"
      /* Use forward declarations for pointer-only usage */
      struct star_bus_config; /* Don't include star_bus_config.h */

      /* Function using forward-declared type */
      rx_err_t star_bus_manager_add_bus(star_bus_manager_t * manager,
                                        struct star_bus_config * config);

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.98 Conditional Includes

Use `#ifdef` for platform-specific or optional includes.

```
    /* Platform-specific includes */
#ifdef CONFIG_IDF_TARGET_ESP32S3
#include "esp32s3/rom/gpio.h"
#else
#include "esp32/rom/gpio.h"
#endif

    /* Optional feature includes */
#ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
#include "esp_debug_helpers.h"
#endif
```

5.99 Include Comments

Add comments for non-obvious includes explaining why they are needed.

```
#include "star_bus_manager.h"

#include "freertos/FreeRTOS.h"

#include <inttypes.h>
#include <string.h>
```

```

    /* Platform-specific: RX72N uses rx_check.h */
#include "esp_log.h"
#include "rx_check.h"
#include "star_bus_manager.h" /* Needed for star_bus_manager_find_bus */
#include "star_bus_types.h"   /* Needed for star_bus_config_t and union */
```

5.100 Avoiding Circular Dependencies

Structure your includes to avoid circular dependencies.

```
/* Problem: Circular dependency */
/* star_bus_manager.h includes star_bus_config.h */
/* star_bus_config.h includes star_bus_manager.h */

/* Solution: Use forward declarations */
/* star_bus_config.h */
struct star_bus_manager;  /* Forward declare, don't include */

rx_err_t star_bus_config_init(star_bus_config_t* config,
struct star_bus_manager* manager);

    /* star_bus_config.c - Include full header here */
#include "star_bus_config.h"
#include "star_bus_manager.h" /* Full definition needed in .c */
```

5.101 Summary

Order	Category	Style
1	Primary header (.c files)	`"..."`
2	FreeRTOS	`"freertos/..."`
3	Standard library	`<...>`
4	Platform HAL	`"..."`
5	Project	`"..."`

- Always include `FreeRTOS.h` before other FreeRTOS headers
- Separate groups with blank lines
- Alphabetize within each group
- Minimize includes in headers; use forward declarations
- Add comments for non-obvious include dependencies

This project follows the rule of **2 spaces** for indentation, without using tabs.
Unix-style line endings ('LF', represented as '\n') are mandatory.
Logging is handled using platform-specific logging macros (e.g., `RX_LOG_*` for STAR, `ESP_LOG*` for ESP-IDF compatibility layer).

- **RX_LOG_ERROR / ESP_LOGE** - Error

- **RX_LOG_WARN / ESP_LOGW** - Warning
- **RX_LOG_INFO / ESP_LOGI** - Info
- **RX_LOG_DEBUG / ESP_LOGD** - Debug
- **RX_LOG_VERBOSE / ESP_LOGV** - Verbose

This section establishes comprehensive naming standards for all identifiers in the STAR firmware codebase. Consistent naming improves readability, maintainability, and reduces cognitive load when navigating the codebase.

5.102 General Rules

- **Language**: All identifiers must be in English.
- **Clarity**: Names should be descriptive and self-documenting.
- **Abbreviations**: Avoid abbreviations unless they are widely understood (e.g., `GPIO`, `I2C`, `SPI`).

5.103 Functions and Variables

Functions and variables use `snake_case`—all lowercase with words separated by underscores.

```
/* CORRECT - snake_case for functions and variables */
esp_err_t star_bus_manager_init(star_bus_manager_t* manager, const char*
    tag);
uint32_t  error_count;
size_t    bytes_written;
bool      is_initialized;

/* WRONG - other naming styles */
rx_err_t StarBusManagerInit();   /* PascalCase */
rx_err_t starBusManagerInit();   /* camelCase */
uint32_t  ErrorCount;             /* PascalCase */
```

5.104 Module Prefixes

All public functions must use the module prefix pattern: `star_[module]_[function]`.

```
/* From star_bus_manager.h */
rx_err_t star_bus_manager_init(...);
rx_err_t star_bus_manager_add_bus(...);
rx_err_t star_bus_manager_remove_bus(...);
rx_err_t star_bus_manager_deinit(...);

/* From star_bus_config.h */
star_bus_config_t* star_bus_config_create_i2c(...);
star_bus_config_t* star_bus_config_create_spi_device(...);
rx_err_t           star_bus_config_destroy(...);

/* From star_bus_i2c.h */
rx_err_t star_bus_i2c_write(...);
```

```
rx_err_t star_bus_i2c_read(...);
void      star_bus_i2c_init_default_ops(...);

/* From star_error_handler.h */
rx_err_t error_handler_init(...);
rx_err_t error_handler_record_error(...);
bool      error_handler_can_retry(...);
```

5.105 Constants and Macros

Constants and macros use `SCREAMING_SNAKE_CASE`—all uppercase with words separated by underscores.

```
    /* CORRECT - SCREAMING_SNAKE_CASE for macros and constants */
#define STAR_BUS_NAME_MAX_LEN (32)
#define CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS (1000)
#define I2C_MASTER_WRITE 0
#define I2C_MASTER_READ 1
#define GPIO_NUM_MAX 48

    /* Convenience macro with proper naming */
#define RECORD_ERROR(handler, code, desc)
   \\
      do {

         \\
        error_handler_record_error((handler), (code), (desc), __FILE__,
                 \\
                                    __LINE__, __func__);
                                                                \\
     } while (0)

#define STAR_IFACE_RECORD_ERROR(iface, err, msg)
   \\
      ((iface) && (iface)->record_error
                                                \\
           ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
                    \\
                                    __LINE__, __func__)
                                                          \\
           : (err))

    /* WRONG */
#define star_bus_name_max_len 32 /* lowercase */
#define StarBusNameMaxLen 32     /* PascalCase */
```

5.106 Type Names

All custom types must use `snake_case` with the `_t` suffix.

```
/* CORRECT - snake_case_t for type names */
typedef struct star_bus_config  star_bus_config_t;
typedef struct star_bus_manager star_bus_manager_t;
```

```c
typedef struct error_handler    error_handler_t;
typedef enum star_bus_type      star_bus_type_t;

/* Callback function types also use _t suffix */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus, void*
    user_ctx);
typedef rx_err_t (*pin_registration_function_t)(gpio_num_t  pin_num,
const char* bus_name,
const char* pin_usage,
void*       user_ctx);

/* WRONG */
typedef struct StarBusConfig StarBusConfig;  /* PascalCase */
typedef struct star_bus_config star_bus_config;  /* missing _t suffix */
```

5.107 Enum Members

Enum members use the `k_` prefix followed by `snake_case`.

```c
/* CORRECT - k_prefix with snake_case, C23 typed enum */
typedef enum : uint8_t {
  k_star_bus_type_none,    /**< No bus type specified */
  k_star_bus_type_i2c,     /**< I2C bus */
  k_star_bus_type_spi,     /**< SPI bus */
  k_star_bus_type_gpio,    /**< GPIO bus */
  k_star_bus_type_adc,     /**< ADC bus */
  k_star_bus_type_count,   /**< Number of bus types (sentinel) */
} star_bus_type_t;

/* Usage in switch statements */
switch (config->type) {
    case k_star_bus_type_i2c: {
      /* Handle I2C */
      break;
   }
    case k_star_bus_type_spi: {
      /* Handle SPI */
      break;
   }
    default: {
      RX_LOG_WARN(s_TAG, "Unknown bus type: %d", config->type);
      break;
   }
}

/* WRONG */
typedef enum {
  STAR_BUS_TYPE_I2C,    /* SCREAMING_SNAKE_CASE without k_ */
  StarBusTypeI2c,       /* PascalCase */
  starBusTypeI2c,       /* camelCase */
} bad_enum_t;
```

5.108 Static Variables (File-Scoped)

Static file-scoped variables use the `s_` prefix followed by `snake_case`.

```
/* CORRECT - s_ prefix for static file-scoped variables */
static const char* s_TAG = "STAR_BUS_MANAGER";
static const uint32_t s_i2c_timeout_ms = 1000;

/* From star_bus_i2c.c */
static const char* s_TAG = "STAR_BUS_I2C";
static const uint32_t s_i2c_timeout_ms = 1000;

/* From star_bus_config.c */
static const char* s_TAG = "STAR_BUS_CONFIG";

/* WRONG - missing s_ prefix */
static const char* TAG = "MODULE";
static uint32_t timeout_ms = 1000;
```

5.109 Global Variables

Global variables use the `g_` prefix followed by `snake_case`. However, global variables are **strongly discouraged**. Prefer dependency injection or static file-scoped variables.

```
/* CORRECT (but discouraged) - g_ prefix for globals */
g_system_state_t g_system_state;
uint32_t g_error_count;

/* PREFERRED - use dependency injection instead */
typedef struct {
      star_error_interface_t *error_iface; /* Injected */
      star_pin_interface_t *pin_iface;     /* Injected */
} star_bus_manager_t;

/* Initialize with injected dependencies */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface);
```

5.110 Private Functions

5.110.1 File-Scoped Static Functions (internal_ prefix)

Static functions that are internal to a single source file use the `internal_` prefix.

```
/* From star_bus_manager.c - internal helper functions */
static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage,
bool                  can_be_shared)
{
      if (pin < 0 || pin >= GPIO_NUM_MAX) {
```

```
        return RX_OK; /* Not an error, pin is not used */
    }
    /* ... implementation ... */
}

static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
    pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage)
{
    /* ... implementation ... */
}

static rx_err_t internal_register_config_pins(star_pin_interface_t*
    pin_iface,
const star_bus_config_t* config)
{
    /* ... implementation ... */
}

/* From star_bus_i2c.c */
static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
    config,
const uint8_t*          data,
size_t                  len,
uint8_t                 reg_addr,
size_t*                 bytes_written);

static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
    config,
i2c_port_t              port,
i2c_cmd_handle_t        cmd,
const char*             operation_name);
```

5.110.2 Exposed Private Functions (priv_ prefix)

Functions that must be exposed across files within a module (but are not part of the public API) use the `priv_` prefix.

```
/* In private header (e.g., star_bus_internal.h) */
/* These functions are used by multiple .c files in the same module
but should not be called by external code */

rx_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
void     priv_star_bus_release_mutex(star_bus_manager_t* manager);

/* Used in implementation files within the module */
rx_err_t star_bus_config_init(star_bus_config_t* config,
struct star_bus_manager* manager)
{
    /* Validate using private helper */
```

```
      esp_err_t ret = priv_star_bus_validate_config(config);
      if (ret != RX_OK) {
        return ret;
      }
      /* ... rest of implementation ... */
}
```

5.111 Summary Table

Identifier Type	Convention	Example
Functions	`snake_case`	`star_bus_manager_init`
Variables	`snake_case`	`bytes_written`
Constants/Macros	`SCREAMING_SNAKE_CASE`	`STAR_BUS_NAME_MAX_LEN`
Type names	`snake_case_t`	`star_bus_config_t`
Enum members	`k_snake_case`	`k_star_bus_type_i2c`
Static variables	`s_snake_case`	`s_TAG`
Global variables	`g_snake_case`	`g_system_state`
Internal functions	`internal_snake_case`	`internal_register_bus_pin`
Private functions	`priv_snake_case`	`priv_star_bus_validate_config`
Module prefix	`star_[module]_`	`star_bus_`, `star_error_`

This section describes the patterns and conventions for defining custom types in the STAR firmware codebase. Consistent type definition patterns improve code readability and enable better tooling support.

5.112 Typedef Struct Pattern

All structures should be defined using the typedef pattern with the `_t` suffix.

5.112.1 Standard Pattern

```
/* Forward declaration (in header for dependencies) */
typedef struct star_bus_config  star_bus_config_t;
typedef struct star_bus_manager star_bus_manager_t;

/* Full definition (in header or implementation) */
typedef struct star_bus_config {
      const char *name;     /**< Unique name for this bus/device instance
         */
      star_bus_type_t type; /**< Type of bus (I2C, SPI, etc.) */
      bool initialized;     /**< Whether this bus has been initialized */
      void *handle;         /**< Driver/device handle */
      void *user_ctx;       /**< User context pointer for callbacks */

      /* Protocol-specific configuration (see union section) */
      union {
        star_i2c_bus_config_t i2c;
        star_spi_bus_config_t spi;
        star_gpio_bus_config_t gpio;
        star_adc_bus_config_t adc;
```

```
    } proto;

    struct star_bus_config *next; /**< Linked list pointer */
} star_bus_config_t;
```

5.112.2 Error Handler Example

```
/* From star_error_handler.h */
typedef struct error_handler {
    uint32_t error_count;          /**< Total number of errors recorded */
    uint32_t max_retries;          /**< Maximum retry attempts allowed */
    uint32_t current_retry;        /**< Current retry attempt counter */
    uint32_t base_retry_delay;     /**< Base delay between retries (ms) */
    uint32_t max_retry_delay;      /**< Maximum retry delay (ms) */
    uint32_t current_retry_delay; /**< Current delay with backoff */
    rx_err_t last_error;           /**< Last recorded error code */
    bool in_error_state;           /**< Whether in error state */
    void *reset_context;           /**< Context for reset function */
    SemaphoreHandle_t mutex;       /**< Mutex for thread-safety */

    rx_err_t (*reset_fn)(void *context); /**< Optional reset function */
} error_handler_t;
```

5.113 Union Usage for Protocol Configurations

Unions are used to store protocol-specific configurations efficiently. This pattern allows a single configuration structure to support multiple bus types.

```
/* From star_bus_manager_types.h */
typedef struct star_bus_config {
    const char *name;
    star_bus_type_t type; /* Discriminator for the union */
    bool initialized;
    void *handle;
    void *user_ctx;

    /* Union holds protocol-specific configuration */
    union {
      star_i2c_bus_config_t i2c;   /**< I2C-specific configuration */
      star_spi_bus_config_t spi;   /**< SPI-specific configuration */
      star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
      star_adc_bus_config_t adc;   /**< ADC-specific configuration */
    } proto;

    struct star_bus_config *next;
} star_bus_config_t;

/* Accessing union members based on type discriminator */
switch (config->type) {
    case k_star_bus_type_i2c: {
```

```
        i2c_port_t port = config->proto.i2c.port;
        uint8_t address = config->proto.i2c.address;
        /* ... */
        break;
    }
     case k_star_bus_type_spi: {
        spi_host_device_t host = config->proto.spi.host;
        gpio_num_t copi_pin = config->proto.spi.copi_io_num;
        /* ... */
        break;
    }
        /* ... other cases ... */
}
```

5.113.1 Protocol - Specific Structures

```
/* I2C bus configuration (from star_bus_protocol_types.h) */
typedef struct star_i2c_bus_config {
        i2c_port_t port;                /**< I2C port number */
        i2c_config_t config;            /**< Platform I2C configuration */
        uint8_t address;                /**< 7-bit I2C device address */
        star_i2c_callbacks_t callbacks; /**< User callbacks */
        star_i2c_ops_t ops;             /**< Operation function pointers */
} star_i2c_bus_config_t;

/* SPI bus configuration */
typedef struct star_spi_bus_config {
        spi_host_device_t host;                 /**< SPI host number */
        bool is_peripheral;                     /**< True if SPI peripheral
            mode */
        spi_bus_config_t bus_cfg;               /**< Platform SPI bus config
            */
        spi_device_interface_config_t dev_cfg; /**< Device config */
        star_spi_callbacks_t callbacks;
        star_spi_ops_t ops;

        /* Inclusive terminology pin names (see Terminology section) */
        gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
        gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
        gpio_num_t sclk_io_num; /**< Serial Clock */
        gpio_num_t cs_io_num;   /**< Chip Select */
        /* ... */
} star_spi_bus_config_t;
```

5.114 Callback Function Types

Callback function pointer types follow a consistent pattern with the `_t` suffix.

```
/* From star_bus_common_types.h - Pin registration callback */
typedef rx_err_t (*pin_registration_function_t)(gpio_num_t  pin_num,
const char* bus_name,
const char* pin_usage,
```

```
void*       user_ctx);

/* From star_bus_manager.h - Bus access callback */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
void*              user_ctx);

/* From star_bus_function_types.h - I2C operation callbacks */
typedef rx_err_t (*star_i2c_write_fn_t)(const star_bus_config_t* config,
const uint8_t*           data,
size_t                   len,
uint8_t                  reg_addr,
size_t*                  bytes_written);

typedef rx_err_t (*star_i2c_read_fn_t)(const star_bus_config_t* config,
uint8_t*                 data,
size_t                   len,
uint8_t                  reg_addr,
size_t*                  bytes_read);

/* Event callbacks */
typedef void (*star_i2c_transfer_complete_cb_t)(const star_bus_event_t*
   event,
void*                   user_ctx);

typedef void (*star_i2c_error_cb_t)(const star_bus_event_t* event,
rx_err_t               error,
void*                   user_ctx);

/* Grouping related callbacks in a struct */
typedef struct star_i2c_callbacks {
      star_i2c_transfer_complete_cb_t on_transfer_complete;
      star_i2c_error_cb_t on_error;
} star_i2c_callbacks_t;
```

5.115 Interface Patterns (Dependency Injection)

Interfaces use structs with function pointers and an opaque context pointer. This pattern enables dependency injection and testability.

```
/* From star_error_interface.h */
typedef struct star_error_interface {
      /**
       * @brief Record an error
       * @param ctx Implementation context
       * @param error Error code
       * @param message Error message
       * @param file Source file
       * @param line Line number
       * @param func Function name
       * @return ESP_OK on success
       */
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
         *message,
```

```
                                const char *file, int line, const char
                                    *func);

        /**
         * @brief Check if retry is possible
         * @param ctx Implementation context
         * @return true if retry is allowed
         */
        bool (*can_retry)(void *ctx);

        /**
         * @brief Reset error state
         * @param ctx Implementation context
         * @return ESP_OK on success
         */
        rx_err_t (*reset_state)(void *ctx);

        /**
         * @brief Implementation context (opaque pointer)
         */
        void *ctx;
} star_error_interface_t;

/* From star_pin_interface.h */
typedef struct star_pin_interface {
        rx_err_t (*register_pin)(void *ctx, int pin, const char
            *description,
                                 bool shared);
        rx_err_t (*unregister_pin)(void *ctx, int pin, const char
            *description);
        rx_err_t (*validate)(void *ctx);
        void *ctx;
} star_pin_interface_t;
```

5.115.1 Interface Usage Pattern

```
/* Creating an interface from concrete implementation */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
error_handler_get_interface(&error_iface, &error_handler);

/* Injecting interfaces into components */
star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

/* Using interface through function pointers */
if (error_iface.can_retry(error_iface.ctx)) {
        /* Retry operation */
}

/* Or using the convenience macro */
```

```
STAR_IFACE_RECORD_ERROR(&error_iface, ESP_ERR_TIMEOUT, "Operation timed
    out");
```

5.116 Opaque Handle Patterns

For complex objects that should not be directly manipulated, use opaque handles.

```
/* Forward declaration creates an opaque type */
typedef struct star_bus_manager star_bus_manager_t;

/* Public API only uses pointers to the opaque type */
esp_err_t star_bus_manager_init(star_bus_manager_t* manager, ...);
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager, ...);

/* Full definition is only in the .c file or private header */
/* Users cannot access internal fields directly */

/* Example usage - users allocate storage but don't access internals */
star_bus_manager_t bus_manager;  /* Stack allocation */
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

/* Or heap allocation */
star_bus_manager_t* manager_ptr = malloc(sizeof(star_bus_manager_t));
star_bus_manager_init(manager_ptr, "main", &error_iface, &pin_iface);
```

5.117 Operations Structure Pattern

Group related function pointers in an operations structure for pluggable implementations.

```
/* From star_bus_protocol_types.h */
typedef struct star_i2c_ops {
      star_i2c_write_fn_t write;                 /**< Write with register
          */
      star_i2c_read_fn_t read;                   /**< Read with register
          */
      star_i2c_write_command_fn_t write_command; /**< Write command byte
          */
      star_i2c_read_raw_fn_t read_raw;           /**< Read raw bytes */
} star_i2c_ops_t;

typedef struct star_spi_ops {
      star_spi_transmit_fn_t transmit; /**< Transmit only */
      star_spi_receive_fn_t receive;   /**< Receive only */
      star_spi_transfer_fn_t transfer; /**< Full-duplex transfer */
} star_spi_ops_t;

typedef struct star_gpio_ops {
      rx_err_t (*write_digital)(const struct star_bus_config *config,
                                uint8_t pin_index, uint8_t value);
      rx_err_t (*read_digital)(const struct star_bus_config *config,
                               uint8_t pin_index, uint8_t *value);
} star_gpio_ops_t;

```

```c
/* Initializing default operations */
void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
{
      if (ops == NULL) {
        RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
        return;
     }
      ops->write = internal_star_bus_i2c_write;
      ops->read = internal_star_bus_i2c_read;
      ops->write_command = internal_star_bus_i2c_write_command;
      ops->read_raw = internal_star_bus_i2c_read_raw;
}
```

5.118 Manager Structure Pattern

Manager structures coordinate multiple resources with thread-safe access.

```c
/* From star_bus_manager_types.h */
typedef struct star_bus_manager {
      star_bus_config_t *buses;            /**< Linked list of bus
          configs */
      SemaphoreHandle_t mutex;             /**< Mutex for thread-safety */
      const char *tag;                     /**< Logging tag */
      star_error_interface_t *error_iface; /**< Injected error handler */
      star_pin_interface_t *pin_iface;     /**< Injected pin validator */

      /* Resource tracking */
      bool spi_host_initialized[SPI_HOST_MAX];
      int spi_device_count[SPI_HOST_MAX];
} star_bus_manager_t;
```

5.119 Best Practices

- **Always use _t suffix**: Makes types easily distinguishable from variables.
- **Use forward declarations**: Minimize header dependencies and compilation time.
- **Document all fields**: Use Doxygen comments for struct members.
- **Keep unions discriminated**: Always pair unions with a type field.
- **Prefer static const over #define**: For typed constants within structures.
- **Use opaque types for encapsulation**: Hide implementation details from users.
- **Group related function pointers**: Use ops structs for pluggable implementations.

This section describes the standard file and directory structure for the STAR firmware codebase. Consistent organization improves navigability and maintainability.

5.120 Library Directory Structure

Each library follows a standard component structure (PlatformIO/ESP-IDF on ESP32, CMake on RX72N):

```
lib/star_component/
    CMakeLists.txt       # CMake build configuration
    library.json         # PlatformIO library metadata
    include/             # Public headers (API)
        star_component.h
        star_component_types.h
    src/                 # Implementation files
        star_component.c
```

5.120.1 Example: star_bus Library

```
lib/star_bus/
    CMakeLists.txt
    library.json
    include/
        star_bus_adc.h
        star_bus_common_types.h
        star_bus_config.h
        star_bus_event_types.h
        star_bus_function_types.h
        star_bus_gpio.h
        star_bus_i2c.h
        star_bus_manager.h
        star_bus_manager_types.h
        star_bus_protocol_types.h
        star_bus_spi.h
        star_bus_types.h
    src/
        star_bus_adc.c
        star_bus_config.c
        star_bus_gpio.c
        star_bus_i2c.c
        star_bus_manager.c
        star_bus_spi.c
        star_bus_types.c
```

5.121 CMakeLists.txt Structure

```
#lib / star_bus / CMakeLists.txt

idf_component_register(
SRCS
"src/star_bus_adc.c"
"src/star_bus_config.c"
```

```
"src/star_bus_gpio.c"
"src/star_bus_i2c.c"
"src/star_bus_manager.c"
"src/star_bus_spi.c"
"src/star_bus_types.c"
INCLUDE_DIRS
"include"
REQUIRES
driver
esp_adc
freertos
star_core
)
```

5.122 Header File Organization

Header files must follow a consistent internal structure.

5.122.1 Header File Template

```
        /* lib/star_bus/include/star_bus_manager.h */

#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H

#ifdef __cplusplus
extern "C" {
#endif

    /* === Includes === */
#include "esp_err.h"
#include "star_bus_manager_types.h"
      /**
       * @file star_bus_manager.h
       * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
          protocols
       *
       * The bus manager provides a central registry for managing multiple
       * bus configurations. It supports dependency injection of error
          handlers
       * and pin validators for loose coupling.
       *
       * Key Features:
       * - Thread-safe bus management with mutex protection
       * - Support for multiple bus types (I2C, SPI, GPIO, etc.)
       * - Automatic pin registration and conflict detection
       * - Safe bus access via callback pattern
       *
       * Example Usage:
       * @code
       * // Initialize bus manager with dependency injection
       * star_bus_manager_t bus_manager;
```

```
31          * star_bus_manager_init(&bus_manager, "main", &error_iface,
                &pin_iface);
32          *
33          * // Create and add an I2C bus
34          * star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
35          *     "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22,
                400000);
36          * star_bus_manager_add_bus(&bus_manager, i2c_bus);
37          * @endcode
38          */
39
40        /* === Forward Declarations === */
41        /* (if needed beyond what's in included headers) */
42
43        /* === Public Functions === */
44
45        /**
46         * @brief Initialize a bus manager instance.
47         *
48         * @param[out] manager     Pointer to bus manager to initialize.
                Must not
49         * be NULL.
50         * @param[in]  tag         Optional tag string for logging. NULL
                uses
51         * default.
52         * @param[in]  error_iface Optional error handler interface (can be
                NULL).
53         * @param[in]  pin_iface   Optional pin validator interface (can be
                NULL).
54         * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
                is
55         * NULL, ESP_ERR_NO_MEM if mutex creation fails.
56         */
57        rx_err_t star_bus_manager_init(
58            star_bus_manager_t * manager, const char *tag,
59            star_error_interface_t *error_iface, star_pin_interface_t
                *pin_iface);
60
61            /* ... more function declarations ... */
62
63  #ifdef __cplusplus
64  }
65  #endif
66
67  #endif /* STAR_COMPONENT_BUS_MANAGER_H */
```

5.122.2 Header Section Order

1. **File path comment**: Identifies the file location
2. **Include guard**: `#ifndef STAR_COMPONENT_FILENAME_H`
3. **C++ extern "C" opening**: For C++ compatibility

4. **Includes**: System, then platform HAL, then project headers
5. **File documentation**: `@file` Doxygen block with overview
6. **Forward declarations**: If needed
7. **Constants and macros**: Public defines
8. **Type definitions**: Enums, structs, typedefs
9. **Function declarations**: Grouped by category
10. **C++ extern "C" closing**: Matches opening
11. **Include guard closing**: `#endif /* ... */`

5.123 Include Guard Naming Convention

Include guards follow the pattern: `STAR_COMPONENT_FILENAME_H`

```
    /* star_bus_manager.h */
#ifndef STAR_COMPONENT_BUS_MANAGER_H
#define STAR_COMPONENT_BUS_MANAGER_H
    /* ... */
#endif /* STAR_COMPONENT_BUS_MANAGER_H */

    /* star_bus_config.h */
#ifndef STAR_COMPONENT_BUS_CONFIG_H
#define STAR_COMPONENT_BUS_CONFIG_H
    /* ... */
#endif /* STAR_COMPONENT_BUS_CONFIG_H */

    /* star_error_interface.h */
#ifndef STAR_ERROR_INTERFACE_H
#define STAR_ERROR_INTERFACE_H
    /* ... */
#endif /* STAR_ERROR_INTERFACE_H */

    /* star_pin_interface.h */
#ifndef STAR_PIN_INTERFACE_H
#define STAR_PIN_INTERFACE_H
    /* ... */
#endif /* STAR_PIN_INTERFACE_H */
```

5.124 Source File Organization

Source files follow a consistent internal structure.

5.124.1 Source File Template

```
    /* lib/star_bus/src/star_bus_manager.c */

    /* === Primary Header === */
#include "star_bus_manager.h"
```

```

    /* === System Includes === */
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

    /* === Platform HAL Includes === */
    /* Platform-specific: RX72N uses rx_check.h */
#include "esp_log.h"
#include "rx_check.h"
#include "sdkconfig.h"

    /* === Project Includes === */
#include "star_bus_adc.h"
#include "star_bus_config.h"
#include "star_bus_gpio.h"

/* === Constants === */

static const char* s_TAG = "STAR_BUS_MANAGER";

/* === Private Function Prototypes === */

static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage,
bool                  can_be_shared);

static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
   pin_iface,
gpio_num_t            pin,
const char*           bus_name,
const char*           usage);

/* === Public Functions === */

rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                         "Manager pointer is NULL");

      manager->buses = NULL;
      manager->error_iface = error_iface;
      manager->pin_iface = pin_iface;

      /* ... rest of implementation ... */

```

```
58        return RX_OK;
59  }
60
61  /* ... more public functions ... */
62
63  /* === Private Functions === */
64
65  static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
66  gpio_num_t             pin,
67  const char*            bus_name,
68  const char*            usage,
69  bool                   can_be_shared)
70  {
71        if (pin < 0 || pin >= GPIO_NUM_MAX) {
72          return RX_OK; /* Not an error, pin is not used */
73       }
74        /* ... implementation ... */
75  }
```

5.124.2 Source Section Order

1. **File path comment**: Identifies the file location
2. **Primary header**: The corresponding .h file for this .c file
3. **System includes**: FreeRTOS, standard library (`<...>`)
4. **Platform HAL includes**: Platform-specific components
5. **Project includes**: Other project headers (`"..."`)
6. **Constants**: `static const` variables, including `s_TAG`
7. **Private function prototypes**: Forward declarations of static functions
8. **Public functions**: Implementation of API functions
9. **Private functions**: Implementation of static helper functions

5.125 Types Header Separation

For complex modules, separate type definitions into dedicated headers:

```
star_bus/include/
    star_bus_manager.h        # Main API (functions)
    star_bus_manager_types.h  # Types for manager
    star_bus_common_types.h   # Shared types (enums, forward decls)
    star_bus_protocol_types.h # Protocol-specific types
    star_bus_function_types.h # Function pointer types
    star_bus_event_types.h    # Event structures
    star_bus_types.h          # Controller header (includes all)
```

5.125.1 Controller Header Pattern

```c
    /* star_bus_types.h - includes all type headers */
#ifndef STAR_COMPONENT_BUS_TYPES_H
#define STAR_COMPONENT_BUS_TYPES_H

#ifdef __cplusplus
extern "C" {
#endif

    /* Controller include file for all public types */
#include "star_bus_common_types.h"
#include "star_bus_event_types.h"
#include "star_bus_function_types.h"
#include "star_bus_manager_types.h"
#include "star_bus_protocol_types.h"

#ifdef __cplusplus
}
#endif

#endif /* STAR_COMPONENT_BUS_TYPES_H */
```

5.126 Project Root Structure

```
star-esp32-firmware/
    CLAUDE.md              # Claude Code instructions
    platformio.ini         # PlatformIO configuration
    sdkconfig.defaults     # ESP-IDF default config
    lib/                   # Libraries/components
        star_core/         # Core interfaces
        star_bus/          # Bus abstraction
        star_error_handler/
        star_pin_validator/
        star_sensor_*/     # Sensor drivers
        star_motor/
        star_pid/
    src/                   # Main application
        main.c
    partitions/            # Partition tables
        partitions_4mb.csv
        partitions_16mb.csv
    docs/                  # Documentation
        style_guide.tex
        style_guide_sections/
    scripts/               # Build/utility scripts
    test/                  # Test files
```

5.127 Best Practices

- **One header per component**: Each component has its primary public header.
- **Minimize header dependencies**: Use forward declarations to reduce includes.
- **Self-contained headers**: Each header should compile independently.
- **Consistent naming**: Files match the component they contain.
- **Group related files**: Keep related headers and sources together.
- **Separate types from functions**: For complex modules, split into _types.h.
- **Document file purpose**: Use `@file` Doxygen blocks.

This section establishes the OSHWA (Open Source Hardware Association) inclusive terminology standards used throughout the STAR firmware codebase. These standards replace legacy terminology with more inclusive alternatives.

5.128 Overview

The STAR project follows inclusive terminology guidelines to ensure our codebase is welcoming to all contributors and avoids language with harmful historical connotations. This is particularly relevant in embedded systems where certain legacy terms have been deeply entrenched.

5.129 SPI Bus Terminology

5.129.1 COPI / CIPO(Not MOSI / MISO)

STAR Term	Legacy Term	Description
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
SCLK	SCLK	Serial Clock (unchanged)
CS	SS	Chip Select (unchanged)

```
/* CORRECT - OSHWA terminology */
typedef struct star_spi_bus_config {
      gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
      gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
      gpio_num_t sclk_io_num; /**< Serial Clock */
      gpio_num_t cs_io_num;   /**< Chip Select */
      /* ... */
} star_spi_bus_config_t;

/* Factory function uses inclusive naming */
star_bus_config_t* star_bus_config_create_spi_device(
const char*       name,
spi_host_device_t host,
gpio_num_t        copi_pin,   /* NOT mosi_pin */
gpio_num_t        cipo_pin,   /* NOT miso_pin */
gpio_num_t        sclk_pin,
int32_t           dma_chan,
```

```
const spi_device_interface_config_t* dev_cfg);

/* In documentation and comments */
/* CORRECT */
reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
bus_name,
"SPI COPI",  /* Descriptive usage */
reg_func,
user_ctx,
true);

/* WRONG */
reg_result = register_pin_if_valid(curr->proto.spi.mosi_io_num,
bus_name,
"SPI MOSI",  /* Legacy terminology */
reg_func,
user_ctx,
true);
```

5.130 I2C and General Bus Terminology

5.130.1 Controller / Peripheral(Not Master / Slave)

STAR Term	Legacy Term	Description
Controller	Master	Device that initiates communication
Peripheral	Slave	Device that responds to controller
Primary	Master	For configuration structures
Main	Master	For general reference

```
/* CORRECT - Controller/Peripheral terminology */
typedef struct star_spi_bus_config {
      spi_host_device_t host;
      bool is_peripheral; /**< True if operating as SPI peripheral */
      /* ... */
} star_spi_bus_config_t;

/* Factory function for peripheral mode */
star_bus_config_t* star_bus_config_create_spi_peripheral(
const char*       name,
spi_host_device_t host,
gpio_num_t        copi_pin,
gpio_num_t        cipo_pin,
gpio_num_t        sclk_pin,
gpio_num_t        cs_pin,
int32_t           queue_size,
uint8_t           mode);

/* In comments and documentation */
/**
* @brief I2C Controller operations
*
* The controller device initiates all transactions on the bus.
```

```
* Peripheral devices respond when addressed by the controller.
*/

/* WRONG */
bool is_slave;            /* Use is_peripheral */
star_bus_config_create_spi_slave();  /* Use _peripheral */
```

5.131 Mapping Platform Legacy APIs

Some platform HALs (like ESP-IDF) still use legacy terminology internally. When interfacing with these APIs, document the mapping clearly.

```
/* From star_bus_protocol_types.h */
typedef struct star_spi_bus_config {
      /* ... */

      /* SPI pin naming follows OSHWA standard:
       * - COPI (Controller Out, Peripheral In) - replaces MOSI
       * - CIPO (Controller In, Peripheral Out) - replaces MISO
       * This modern terminology removes master/slave language.
       * Some platforms (like ESP-IDF) use legacy mosi/miso terminology,
       * which we map here.
       */
      gpio_num_t
          copi_io_num; /**< GPIO for COPI (maps to platform mosi_io_num)
              */
      gpio_num_t
          cipo_io_num; /**< GPIO for CIPO (maps to platform miso_io_num)
              */
      /* ... */
} star_spi_bus_config_t;

/* When setting up platform HAL structures (example: ESP32) */
static rx_err_t internal_setup_spi_bus(star_bus_config_t* config)
{
      spi_bus_config_t bus_cfg = {
          /* Map our inclusive terminology to platform legacy names */
          .mosi_io_num = config->proto.spi.copi_io_num,
          .miso_io_num = config->proto.spi.cipo_io_num,
          .sclk_io_num = config->proto.spi.sclk_io_num,
          .quadwp_io_num = -1,
          .quadhd_io_num = -1,
          .max_transfer_sz = config->proto.spi.max_transfer_sz,
     };
      /* ... */
}

/* Similarly for I2C */
/* Some platforms use I2C_MODE_SLAVE/MASTER, we document as
   peripheral/controller */
i2c_config_t i2c_conf = {
  .mode = I2C_MODE_MASTER,  /* Controller mode in STAR terminology */
  /* or */
```

```
  .mode = I2C_MODE_SLAVE,    /* Peripheral mode in STAR terminology */
  /* ... */
};
```

5.132 Documentation Guidelines

When writing comments, documentation, or user-facing text:

```
/* CORRECT - Use inclusive terminology in all documentation */

/**
* @brief Configure ESP32 as SPI peripheral device
*
* The peripheral will respond to transactions initiated by an
* external SPI controller. Data flows:
* - COPI: Data received from controller
* - CIPO: Data sent to controller
*
* @param[in] name Unique name for this SPI peripheral instance
* @param[in] host SPI host device
* @param[in] copi_pin GPIO for Controller Out, Peripheral In
* @param[in] cipo_pin GPIO for Controller In, Peripheral Out
* ...
*/
star_bus_config_t* star_bus_config_create_spi_peripheral(...);

/* WRONG - Legacy terminology */
/**
* @brief Configure ESP32 as SPI slave device
*
* The slave will respond to transactions initiated by the
* SPI master. Data flows:
* - MOSI: Data from master to slave
* - MISO: Data from slave to master
*/
```

5.133 Code Review Checklist

When reviewing code, check for:

- **Variable names**: No “master”, “slave”, “mosi”, “miso”
- **Function names**: Use “controller”, “peripheral”, “copi”, “cipo”
- **Comments**: Update any legacy terminology
- **Documentation**: Ensure Doxygen uses inclusive terms
- **String literals**: Check log messages and error strings
- **Platform HAL mapping**: Document when interfacing with legacy APIs

5.134 Common Patterns

```
/* Logging with inclusive terminology */
RX_LOG_INFO(s_TAG, "SPI peripheral initialized on host %d", host);
RX_LOG_INFO(s_TAG, "I2C controller configured on port %d", port);

/* Error messages */
ESP_LOGE(s_TAG, "Failed to initialize SPI peripheral: %s",
rx_err_to_name(ret));
ESP_LOGE(s_TAG, "I2C controller transaction failed");

/* Pin registration descriptions */
internal_register_bus_pin(pin_iface, config->proto.spi.copi_io_num,
config->name, "SPI COPI", true);
internal_register_bus_pin(pin_iface, config->proto.spi.cipo_io_num,
config->name, "SPI CIPO", true);
```

5.135 Summary

Use This	Not This
Controller	Master
Peripheral	Slave
COPI	MOSI
CIPO	MISO
Primary	Master (for configs)
Main	Master (general)
Allowlist	Whitelist
Blocklist	Blacklist

Following these guidelines ensures our codebase is inclusive and welcoming while maintaining technical clarity. The terminology accurately describes the functional roles of devices without relying on harmful historical language.

This section describes the key architectural patterns used throughout the STAR firmware codebase. These patterns enable loose coupling, testability, and maintainability in embedded systems.

5.136 Dependency Injection (DIP)

The Dependency Inversion Principle (DIP) is the foundational architectural pattern of the STAR framework. High-level modules depend on abstract interfaces rather than concrete implementations.

5.136.1 Interface Definition

Interfaces are defined as structs containing function pointers and an opaque context pointer.

```
/* From star_error_interface.h */
typedef struct star_error_interface {
      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
         *message,
                               const char *file, int line, const char
                                  *func);
      bool (*can_retry)(void *ctx);
```

```
      rx_err_t (*reset_state)(void *ctx);
      void *ctx; /* Opaque context pointer */
} star_error_interface_t;

/* From star_pin_interface.h */
typedef struct star_pin_interface {
      rx_err_t (*register_pin)(void *ctx, int pin, const char
         *description,
                               bool shared);
      rx_err_t (*unregister_pin)(void *ctx, int pin, const char
         *description);
      rx_err_t (*validate)(void *ctx);
      void *ctx;
} star_pin_interface_t;
```

5.136.2 Interface Implementation

Concrete implementations provide the actual functionality.

```
/* Create error handler and get interface */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
error_handler_get_interface(&error_iface, &error_handler);

/* Implementation of get_interface */
void error_handler_get_interface(star_error_interface_t* iface,
error_handler_t* handler)
{
      iface->record_error = adapter_record_error; /* Adapter function */
      iface->can_retry = adapter_can_retry;
      iface->reset_state = adapter_reset_state;
      iface->ctx = handler; /* Concrete handler as context */
}
```

5.136.3 Dependency Injection Pattern

Components receive their dependencies through initialization functions.

```
/* Component declares interface dependencies */
typedef struct star_bus_manager {
      star_bus_config_t *buses;
      SemaphoreHandle_t mutex;
      const char *tag;
      star_error_interface_t *error_iface; /* Injected */
      star_pin_interface_t *pin_iface;     /* Injected */
      /* ... */
} star_bus_manager_t;

/* Initialization receives dependencies */
rx_err_t star_bus_manager_init(star_bus_manager_t*     manager,
const char*             tag,
```

```
star_error_interface_t* error_iface,
star_pin_interface_t*   pin_iface)
{
      RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
                         "Manager pointer is NULL");

      manager->buses = NULL;
      manager->error_iface = error_iface; /* Can be NULL */
      manager->pin_iface = pin_iface;     /* Can be NULL */
      /* ... */
}

/* Usage with injected dependencies */
error_handler_t error_handler;
error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);

star_error_interface_t error_iface;
star_pin_interface_t pin_iface;
error_handler_get_interface(&error_iface, &error_handler);
pin_validator_get_interface(&pin_iface);

star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
```

5.136.4 NULL Interface Handling

Components gracefully handle NULL interfaces.

```
/* Check interface before use */
if (manager->error_iface && manager->error_iface->record_error) {
      manager->error_iface->record_error(manager->error_iface->ctx, ret,
                                         "Operation failed", __FILE__,
                                            __LINE__,
                                         __func__);
}

    /* Or use convenience macro */
#define STAR_IFACE_RECORD_ERROR(iface, err, msg)
   \\
      ((iface) && (iface)->record_error
                                                \\
           ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
                   \\
                                   __LINE__, __func__)
                                                          \\
           : (err))
```

5.137 Bus Abstraction Pattern

The bus abstraction provides a unified interface for different communication protocols.

5.137.1 Unified Configuration

```c
/* Single config type with protocol union */
typedef struct star_bus_config {
      const char *name;
      star_bus_type_t type; /* Discriminator */
      bool initialized;
      void *handle;
      void *user_ctx;

      union {
        star_i2c_bus_config_t i2c;
        star_spi_bus_config_t spi;
        star_gpio_bus_config_t gpio;
        star_adc_bus_config_t adc;
     } proto;

      struct star_bus_config *next;
} star_bus_config_t;
```

5.137.2 Factory Functions

Factory functions create configurations for specific protocols.

```c
/* I2C bus factory */
star_bus_config_t* star_bus_config_create_i2c(
const char* name,
i2c_port_t  port,
uint8_t     address,
gpio_num_t  sda_pin,
gpio_num_t  scl_pin,
uint32_t    clk_speed);

/* SPI device factory */
star_bus_config_t* star_bus_config_create_spi_device(
const char*                          name,
spi_host_device_t                    host,
gpio_num_t                           copi_pin,
gpio_num_t                           cipo_pin,
gpio_num_t                           sclk_pin,
int32_t                              dma_chan,
const spi_device_interface_config_t* dev_cfg);

/* GPIO bus factory */
star_bus_config_t* star_bus_config_create_gpio(
const char* name,
gpio_num_t* pins,
uint8_t     pin_count);
```

5.137.3 Manager Pattern

The bus manager coordinates multiple bus configurations.

```c
/* Initialize manager */
```

```
star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

/* Add buses */
star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
"sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22, 400000);
star_bus_manager_add_bus(&bus_manager, i2c_bus);

/* Find and use buses */
star_bus_config_t* bus = star_bus_manager_find_bus(&bus_manager,
   "sensor_i2c");

/* Safe access with callback (prevents use-after-free) */
rx_err_t my_callback(star_bus_config_t* bus, void* ctx) {
      /* Access bus safely while mutex is held */
      return RX_OK;
}
star_bus_manager_with_bus(&bus_manager, "sensor_i2c", my_callback, NULL);
```

5.138 Operations Structure Pattern

Group related function pointers for pluggable implementations.

```
/* Define operations structure */
typedef struct star_i2c_ops {
      star_i2c_write_fn_t write;
      star_i2c_read_fn_t read;
      star_i2c_write_command_fn_t write_command;
      star_i2c_read_raw_fn_t read_raw;
} star_i2c_ops_t;

/* Initialize with default implementations */
void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
{
      if (ops == NULL) {
        RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
        return;
     }
      ops->write = internal_star_bus_i2c_write;
      ops->read = internal_star_bus_i2c_read;
      ops->write_command = internal_star_bus_i2c_write_command;
      ops->read_raw = internal_star_bus_i2c_read_raw;
}

/* Use through function pointers */
rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
const char* name, ...)
{
      star_bus_config_t *bus_config = star_bus_manager_find_bus(manager,
         name);
      /* ... validation ... */
      return bus_config->proto.i2c.ops.write(bus_config, data, len,
         reg_addr,
                                             bytes_written);
```

```
}
```

5.139 Thread-Safe Callback Pattern

Safely access shared resources through callbacks.

```
/* Callback type */
typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
void* user_ctx);

/* Thread-safe access function */
rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
const char*         name,
star_bus_callback_t callback,
void*               user_ctx)
{
      ESP_RETURN_ON_FALSE(manager && name && callback, RX_ERR_INVALID_ARG,
                          s_TAG, "Invalid arguments");

      /* Acquire mutex */
      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      /* Find bus while holding mutex */
      star_bus_config_t *found_bus = NULL;
      for (star_bus_config_t *curr = manager->buses; curr; curr =
          curr->next) {
        if (curr->name && strcmp(curr->name, name) == 0) {
          found_bus = curr;
          break;
       }
     }

      if (found_bus == NULL) {
        xSemaphoreGive(manager->mutex);
        return ESP_ERR_NOT_FOUND;
     }

      /* Invoke callback while holding mutex - prevents use-after-free */
      rx_err_t callback_result = callback(found_bus, user_ctx);

      xSemaphoreGive(manager->mutex);
      return callback_result;
}
```

5.140 Thread Safety Patterns

5.140.1 Mutex Timeout Convention

Standard mutex timeout is 1000ms (configurable via Kconfig).

```
/* Standard mutex acquisition pattern */
if (xSemaphoreTake(manager->mutex,
pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
!= pdTRUE) {
      ESP_LOGE(manager->tag, "Timeout acquiring mutex");
      return ESP_ERR_TIMEOUT;
}

/* Critical section */
/* ... protected operations ... */

xSemaphoreGive(manager->mutex);
```

5.140.2 Mutex with Cleanup(goto pattern)

```
rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
star_bus_config_t* config)
{
      RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
                         "Invalid arguments");

      rx_err_t ret = RX_OK;

      if (xSemaphoreTake(
              manager->mutex,
              pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
          pdTRUE) {
        return RX_ERR_TIMEOUT;
     }

      /* Check for duplicate name */
      for (star_bus_config_t *curr = manager->buses; curr; curr =
          curr->next) {
        if (curr->name && strcmp(curr->name, config->name) == 0) {
          ret = RX_ERR_INVALID_STATE;
          goto add_give_mutex; /* Single exit point */
       }
     }

      /* Initialize bus hardware */
      ret = star_bus_config_init(config, manager);
      if (ret != RX_OK) {
        goto add_give_mutex;
     }

      /* Add to list */
      config->next = manager->buses;
      manager->buses = config;

```

```
    add_give_mutex:
      xSemaphoreGive(manager->mutex);
      return ret;
}
```

5.141 Error Handler Ownership Pattern

Components track whether they own their error handler for proper cleanup.

```
/* Sensor handle with ownership tracking */
typedef struct mpu6050_handle {
      star_bus_manager_t *bus_manager;
      const char *bus_name;
      star_error_interface_t *error_iface;
      bool owns_error_handler; /* Ownership flag */
      SemaphoreHandle_t mutex;
      bool initialized;
      /* ... */
} mpu6050_handle_t;

/* Initialization with optional interface */
rx_err_t star_sensor_mpu6050_init(mpu6050_handle_t* handle,
star_bus_manager_t* bus_manager,
const char* bus_name,
star_error_interface_t* error_iface,
const mpu6050_config_t* config)
{
      if (error_iface != NULL) {
        /* Use provided interface - we don't own it */
        handle->error_iface = error_iface;
        handle->owns_error_handler = false;
     } else {
        /* Create default - we own it */
        handle->error_iface = star_error_interface_create_default();
        handle->owns_error_handler = true;
     }
      /* ... */
}

/* Cleanup respects ownership */
rx_err_t star_sensor_mpu6050_deinit(mpu6050_handle_t* handle)
{
      /* Clean up error interface if we own it */
      star_error_interface_cleanup(handle->error_iface,
                                   handle->owns_error_handler);
      handle->error_iface = NULL;
      /* ... */
}

/* Helper for cleanup */
void star_error_interface_cleanup(star_error_interface_t* error_iface,
bool owns_handler)
{
      if (!owns_handler || !error_iface) {
```

```
        return; /* Not our responsibility */
    }
    error_handler_t *handler = (error_handler_t *)error_iface->ctx;
    error_handler_destroy_default(handler);
    free(error_iface);
}
```

5.142 Linked List Management

Bus configurations are managed as a singly-linked list.

```
/* Add to head of list (O(1)) */
config->next = manager->buses;
manager->buses = config;

/* Remove from list (O(n)) */
star_bus_config_t* prev = NULL;
for (star_bus_config_t* curr = manager->buses; curr; prev = curr, curr =
    curr->next) {
    if (strcmp(curr->name, name) == 0) {
        if (prev == NULL) {
            manager->buses = curr->next; /* Remove head */
        } else {
            prev->next = curr->next; /* Remove middle/tail */
        }
        curr->next = NULL;
        /* ... cleanup curr ... */
        break;
    }
}

/* Iterate through list */
for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
    /* Process each configuration */
}
```

5.143 Best Practices Summary

- **Use dependency injection**: Components receive interfaces, not concrete types.
- **Handle NULL interfaces**: Always check before dereferencing.
- **Track ownership**: Know who is responsible for cleanup.
- **Use callbacks for safe access**: Prevents use-after-free with shared resources.
- **Consistent mutex timeout**: 1000ms standard timeout.
- **Single exit point with goto**: Ensures mutex release on all paths.
- **Factory functions**: Create configurations with proper initialization.
- **Operations structures**: Enable pluggable implementations.

5.144 Prefer Alternatives to Macros

Rule: Avoid macros unless absolutely necessary. Use modern C alternatives:

- `static const` for constants
- `inline` functions for simple operations
- `enum` for related constants

Rationale: Macros lack type safety, scope control, and debugger visibility.
Good:

```
static const uint32_t k_max_buffer_size = 256;
static const float k_pi = 3.14159F;

static inline int max(int a, int b) {
      return (a > b) ? a : b;
}
```

Bad:

```
#define MAX_BUFFER_SIZE 256
#define PI 3.14159
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

5.145 When Macros Are Necessary

Use macros only for:

1. Array sizes (compile-time constants)
2. Conditional compilation (`#ifdef`, `#if`)
3. Token pasting and stringification
4. Platform-specific code paths

5.146 Macro Safety Rules

5.146.1 Wrap Statement - Like Macros in `do -while (0)`

Rule: Always wrap multi-statement macros in `do { ... } while (0)`.
Why: Prevents bugs when used in if-statements without braces.
Correct:

```
#define LOG_ERROR(msg)                                          \\
      do {                                                        \\
        log_timestamp();                                          \\
        log_message(msg);                                         \\
     } while (0)
```

Incorrect(causes bugs) :

```
#define LOG_ERROR(msg)                                                  \\
      log_timestamp();                                                      \\
      log_message(msg)

/* Bug example: */
if (error)
    LOG_ERROR("fail");  /* Only log_timestamp() is conditional! */
else
    handle_success();
```

5.146.2 Parenthesize All Macro Arguments

Rule: Wrap all argument uses and the entire expression in parentheses.

Correct:

```
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Incorrect(causes bugs) :

```
#define SQUARE(x) x *x

int result = SQUARE(2 + 3);  /* Expands to: 2 + 3 * 2 + 3 = 11, not 25! */
```

5.146.3 Beware of Side Effects and Multiple Evaluation

Rule: Document if macro arguments are evaluated multiple times. Never pass arguments with side effects.

Dangerous:

```
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int i = 5;
int result = MAX(i++, 10);  /* i is incremented TWICE! */
```

Solution: Use inline functions or add warning comment:

```
    /* WARNING: Arguments evaluated multiple times - no side effects! */
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

5.147 Multi - line Macro Formatting

Rules:

- Use backslash continuation aligned consistently
- Indent continuation lines for readability
- Place opening brace on same line as `do`

Example:

```
#define RECORD_ERROR(handler, code, desc)                                    \\
      do {                                                                       \\
        if ((handler)->error_iface != NULL) {                                    \\
          (handler)->error_iface->record_error((code), (desc), __FILE__,       \\
                                                __LINE__);                       \\
       }                                                                        \\
     } while (0)
```

5.148 Naming Conventions

Rule: Use `SCREAMING_SNAKE_CASE` for all macros.

Examples:

```
    /* Simple constants */
#define MAX_RETRY_COUNT (3)
#define MOTOR_PWM_FREQ_HZ (20000)

    /* Function-like macros (always use parentheses) */
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

    /* Multi-statement macros (always use do-while(0)) */
#define LOG_ERROR(msg)                                                       \\
      do {                                                                       \\
        log_timestamp();                                                         \\
        log_message(msg);                                                        \\
     } while (0)

#define RECORD_ERROR(handler, code, desc)                                    \\
      do {                                                                       \\
        error_handler_record_error((handler), (code), (desc), __FILE__,      \\
                                   __LINE__, __func__);                        \\
     } while (0)
```

5.149 Include Guards

Use traditional include guards for maximum portability:

```
#ifndef STAR_COMPONENT_FILENAME_H
#define STAR_COMPONENT_FILENAME_H

        /* File contents */

#endif /* STAR_COMPONENT_FILENAME_H */
```

Do not use `#pragma once` (see Section 12 for details).

5.150 Conditional Compilation

Best Practices:

- Document why conditionals exist
- Keep conditional blocks small
- Use configuration headers instead of scattered `#ifdef`

Good:

```
    /* Platform-specific GPIO initialization */
#if defined(CONFIG_STAR_BOARD_ESP32_S3)
    configure_esp32s3_gpio();
#elif defined(CONFIG_STAR_BOARD_ESP32_WROOM)
    configure_esp32_wroom_gpio();
#else
#error "Unsupported board configuration"
#endif
```

This section establishes the unit suffix naming standard used throughout the STAR firmware codebase. All numeric fields use SI (International System of Units) units with explicit suffixes to eliminate unit ambiguity and enable automatic documentation generation.

5.151 Overview

The firmware uses the MKS (Meter-Kilogram-Second) system as the foundation for all physical measurements. Every numeric variable that represents a physical quantity must include a unit suffix that clearly identifies its meaning.

5.152 Benefits

- **Eliminates Unit Ambiguity**: No confusion about whether a value is in meters or millimeters, degrees or radians
- **Self-Documenting Code**: Variable names explicitly state their units
- **Protocol Buffer Compatibility**: Matches naming conventions used in `star-proto/` messages
- **Prevents Conversion Errors**: Explicit units reduce the risk of unit mismatch bugs

5.153 Standard Unit Suffixes

Table 16: Standard Unit Suffixes for STAR Firmware

Suffix	Unit	Proto Example	C Example
_m	meters	x_m	distance_m
_mps	meters/second	velocity_mps	speed_mps
_mps2	m/s^2	accel_x_mps2	acceleration_mps2
_rad	radians	angle_rad	heading_rad
_rad_per_s	rad/second	omega_rad_per_s	angular_vel_rad_per_s
_celsius	degrees C	temperature_celsius	motor_temp_celsius
_deci_celsius	0.1°C	temp_deci_celsius	(BMS native format)
_us	microseconds	timestamp_us	elapsed_us
_ms	milliseconds	timeout_ms	period_ms
_ma	milliamps	current_ma	motor_current_ma
_mv	millivolts	cell_mv	pack_voltage_mv
_mw	milliwatts	power_mw	consumption_mw
_percent	0–100%	soc_percent	duty_cycle_percent

5.154 Usage Examples

5.154.1 Distance and Velocity

```
/* CORRECT - Distance and velocity with units */
float distance_m = 1.5F;            /* 1.5 meters */
float velocity_mps = 2.0F;          /* 2.0 meters per second */
float acceleration_mps2 = 9.81F;    /* 9.81 m/s^2 (gravity) */

/* Motor velocity tracking */
float target_velocity_mps = 0.5F;
float current_velocity_mps;
star_encoder_get_velocity_mps(&encoder, 10.0F, 500,
   &current_velocity_mps);

/* WRONG - No unit suffix */
float distance = 1.5F;      /* What unit? Meters? Millimeters? Feet? */
float velocity = 2.0F;      /* Meters per second? RPM? */
```

5.154.2 Angular Measurements

```
/* CORRECT - Angular measurements in radians */
float heading_rad = 1.57F;              /* 90 degrees in radians */
float angular_velocity_rad_per_s = 3.14F; /* pi radians per second */

/* Robot pose */
typedef struct {
      float x_m;
      float y_m;
      float theta_rad; /* Heading angle */
```

```
} robot_pose_t;

/* WRONG - Ambiguous angle units */
float heading = 90.0F;  /* Degrees or radians? */
float omega = 3.14F;    /* What unit? */
```

5.154.3 Temperature

```
/* CORRECT - Temperature with units */
float motor_temp_celsius;
star_sensor_ds18b20_read_temp(&temp_sensor, &motor_temp_celsius);

/* BMS uses deci-celsius (0.1 degree resolution) */
int16_t cell_temp_deci_celsius = 235;  /* 23.5 degrees C */

    /* Thermal shutdown threshold */
static const float s_motor_thermal_shutdown_celsius = 85.0F;

if (motor_temp_celsius > s_motor_thermal_shutdown_celsius) {
      rx_motor_stop(&motor, true); /* Emergency brake */
}

/* WRONG */
float motor_temp = 75.0F;  /* Celsius? Fahrenheit? Kelvin? */
```

5.154.4 Time

```
/* CORRECT - Time units */
uint32_t timeout_ms = 1000;        /* 1 second timeout */
uint64_t timestamp_us;             /* Microsecond timestamp */
star_encoder_get_timestamp_us(&encoder, &timestamp_us);

/* Timing for control loops */
const uint32_t control_period_ms = 1;     /* 1ms = 1000Hz */
const uint32_t sensor_poll_us = 10000;    /* 10ms in microseconds */

/* WRONG */
uint32_t timeout = 1000;    /* Milliseconds? Seconds? Ticks? */
uint64_t timestamp;         /* What resolution? */
```

5.154.5 Electrical Measurements

```
/* CORRECT - Electrical units */
uint32_t motor_current_ma;
rx_motor_get_current_ma(&motor, &motor_current_ma);

uint32_t cell_voltage_mv = 3700;   /* 3.7V in millivolts */
uint32_t pack_voltage_mv = 11100;  /* 11.1V (3S LiPo) */
uint32_t power_mw = 7400;          /* 7.4W in milliwatts */

```

```
    /* Current limiting */
typedef enum : uint32_t {
  k_max_motor_current_ma = 2000, /**< 2A motor current limit */
} motor_current_limits_t;

if (motor_current_ma > k_max_motor_current_ma) {
      ESP_LOGW(s_TAG, "Motor current limit exceeded: %lu mA",
         motor_current_ma);
      rx_motor_set_duty(&motor, 0.0F);
}

/* WRONG */
uint32_t motor_current = 1500;  /* Milliamps? Amps? */
uint32_t voltage = 3700;        /* Millivolts? Volts? */
```

5.154.6 Percentage

```
/* CORRECT - Percentage (0-100 range) */
float duty_cycle_percent = 75.0F;      /* 75% duty cycle */
uint8_t battery_soc_percent = 85;      /* 85% state of charge */
float throttle_percent = 50.0F;        /* 50% throttle */

rx_motor_set_duty(&motor, duty_cycle_percent);

/* WRONG */
float duty_cycle = 0.75F;    /* Normalized (0-1)? Percentage (0-100)? */
uint8_t battery_soc = 85;    /* Percentage? Raw ADC value? */
```

5.155 Protocol Buffer Integration

The unit suffix convention matches the naming used in Protocol Buffer messages:

```
// From star-proto/proto/robot_command.proto
message VelocityCommand {
      float velocity_mps = 1;               // Meters per second
      float angular_velocity_rad_per_s = 2; // Radians per second
}

message BatteryState {
      repeated uint32 cell_mv = 1;   // Cell voltages in millivolts
      uint32 current_ma = 2;         // Pack current in milliamps
      uint8 soc_percent = 3;         // State of charge (0-100%)
      int16 temperature_celsius = 4; // Pack temperature
}
```

5.156 Conversion Functions

When converting between units, use descriptive function names:

```
/* CORRECT - Clear unit conversion */
static const float s_seconds_per_minute = 60.0F;

```

```c
float rpm_to_mps(float rpm, float wheel_circumference_m)
{
      float revolutions_per_second = rpm / s_seconds_per_minute;
      return revolutions_per_second * wheel_circumference_m;
}

static const float s_deci_celsius_per_celsius = 10.0F;

float celsius_to_deci_celsius(float temp_celsius)
{
      return temp_celsius * s_deci_celsius_per_celsius;
}

static const float s_mv_per_volt = 1000.0F;

float mv_to_volts(uint32_t voltage_mv)
{
      return voltage_mv / s_mv_per_volt;
}

/* WRONG - Ambiguous conversions */
float convert_rpm(float rpm);  /* Convert to what? */
float scale_temp(float temp);  /* What units? */
```

5.157 Summary

- **Always use unit suffixes** for physical quantities
- **Match Protocol Buffer conventions** for consistency
- **Use SI base units** when possible (meters, radians, seconds)
- **Document conversions** clearly in function names
- **Prefer millivolts/milliamps** over floating-point volts/amps for integer precision

This convention eliminates an entire class of bugs caused by unit ambiguity and makes the codebase more maintainable.

This section establishes memory management rules for STAR firmware across all platforms. Embedded platforms have limited RAM (e.g., RX72N: 1MB SRAM), making proper memory management critical for system stability and performance.

5.158 No Dynamic Allocation

Rule: Never use `malloc()`, `calloc()`, `realloc()`, or `free()` in production firmware.

5.158.1 Rationale

- **Heap Fragmentation**: Repeated allocation/deallocation causes memory fragmentation over time
- **Memory Leaks**: Forgotten `free()` calls lead to gradual RAM exhaustion

- **Unpredictable Timing**: Allocation time is non-deterministic (breaks real-time constraints)
- **Out-of-Memory Failures**: No recovery mechanism when heap is exhausted
- **Debugging Difficulty**: Memory corruption bugs are extremely hard to track down

5.158.2 Alternatives

Use static allocation instead:

```
/* WRONG - Dynamic allocation */
void process_sensor_data(void)
{
      float *samples = (float *)malloc(1000 * sizeof(float));
      if (samples == NULL) {
        RX_LOG_ERROR(s_TAG, "Out of memory!");
        return;
     }

      /* Process samples... */

      free(samples); /* Easy to forget! */
}

/* CORRECT - Static allocation */
static float s_samples[1000];  /* Allocated once at compile time */

void process_sensor_data(void)
{
      /* Use s_samples directly - no allocation, no leaks */
      for (int i = 0; i < 1000; i++) {
        s_samples[i] = read_sensor();
     }
}
```

5.158.3 Platform Heap Functions

Some platforms provide specialized heap functions (e.g., ESP-IDF's `heap_caps_malloc`) that are **only acceptable** for:

- **One-time initialization**: Allocating resources during startup that persist for the lifetime of the program
- **FreeRTOS task stacks**: Created once during task initialization
- **Third-party libraries**: When required by external dependencies (document clearly)

```
/* ACCEPTABLE - One-time allocation during init */
rx_err_t sensor_driver_init(sensor_handle_t* handle)
{
      /* Allocate persistent buffer once */
      handle->rx_buffer = heap_caps_malloc(DMA_BUFFER_SIZE,
         MALLOC_CAP_DMA);
```

```
    if (handle->rx_buffer == NULL) {
      return ESP_ERR_NO_MEM;
    }
    handle->owns_buffer = true; /* Track ownership */
    return RX_OK;
}

/* Must have corresponding cleanup */
rx_err_t sensor_driver_deinit(sensor_handle_t* handle)
{
    if (handle->owns_buffer && handle->rx_buffer) {
      free(handle->rx_buffer);
      handle->rx_buffer = NULL;
    }
    return RX_OK;
}
```

5.159 Avoid Large Stack Allocations

Rule: Never allocate large arrays on the stack. Use static or global storage instead.

5.159.1 Stack Size Constraints

- Typical embedded task stack: **2KB–8KB** (platform and RTOS dependent)
- ISR stack: Even smaller (˜1KB)
- Stack overflow causes silent corruption or hard faults

5.159.2 Examples

```
/* WRONG - Large stack allocation */
void process_image(void)
{
    uint8_t frame_buffer[640 * 480]; /* 307KB on stack! */
    /* Stack overflow guaranteed! */
}

/* WRONG - Nested large allocations */
void motor_control_loop(void)
{
    float pid_state[100];          /* 400 bytes */
    uint32_t encoder_samples[500]; /* 2000 bytes */
    /* Total: 2400 bytes - may overflow 3KB stack */
}

/* CORRECT - Use static storage */
static uint8_t s_frame_buffer[640 * 480];  /* In .bss, not stack */

void process_image(void)
{
    /* Use s_frame_buffer safely */
    memset(s_frame_buffer, 0, sizeof(s_frame_buffer));
```

```
}

/* CORRECT - Small stack allocations are fine */
void calculate_checksum(const uint8_t* data, size_t len)
{
      uint32_t crc = 0; /* 4 bytes - perfectly fine on stack */
      /* ... */
}
```

5.159.3 Stack Usage Guidelines

Table 17: Stack Allocation Guidelines

Size	Allowed	Storage Location
$<$ 256 bytes	Yes	Stack (automatic)
256 bytes – 1 KB	Caution	Prefer static if persistent
$>$ 1 KB	No	Static or global only
$>$ 10 KB	Never	Static with explicit comment

5.160 Use `const` for Flash Storage

Rule: Place all read-only data in flash/ROM using the `const` keyword.

5.160.1 Rationale

- Embedded platforms typically have large flash but limited RAM (e.g., RX72N: 4MB flash / 1MB RAM)
- `const` data is stored in flash and accessed directly (no RAM copy)
- Saves precious SRAM for runtime variables

5.160.2 Examples

```
/* WRONG - Lookup table in RAM (wastes 256 bytes) */
static uint8_t sine_lookup[256] = {
  0, 3, 6, 9, 12, /* ... */
};

/* CORRECT - Lookup table in flash (zero RAM usage) */
static const uint8_t sine_lookup[256] = {
  0, 3, 6, 9, 12, /* ... */
};

/* WRONG - String literals without const */
static char* error_messages[] = {
  "Sensor timeout",
  "Invalid checksum",
  "Bus error"
};

```

```c
/* CORRECT - Strings in flash */
static const char* const error_messages[] = {
  "Sensor timeout",
  "Invalid checksum",
  "Bus error"
};

/* CORRECT - Default configuration in flash */
static const star_pid_config_t default_pid_config = {
  .kp = 1.0F,
  .ki = 0.5F,
  .kd = 0.1F,
  .output_min = -100.0F,
  .output_max = 100.0F,
};

/* Usage: Copy to RAM only when needed */
star_pid_config_t runtime_config;
memcpy(&runtime_config, &default_pid_config, sizeof(runtime_config));
```

5.160.3 Pointer Declarations

Be careful with pointer `const` placement:

```c
/* Pointer to const data (data in flash, pointer in RAM) */
const char* error_msg = "Timeout";

/* Const pointer to data (pointer in flash, data in RAM) */
char* const buffer_ptr = buffer;

/* Const pointer to const data (both in flash) */
const char* const error_table[] = {"Error 1", "Error 2"};
```

5.161 Validate All Buffers

Rule: Always validate buffer sizes before `memcpy()`, DMA operations, or ring buffer manipulations.

5.161.1 Rationale

- Buffer overruns cause memory corruption
- Corruption may manifest much later, making debugging extremely difficult
- Real-time systems have no memory protection (unlike Linux)

5.161.2 Validation Patterns

```c
/* WRONG - Unchecked memcpy */
void receive_packet(uint8_t* data, size_t len)
{
      memcpy(rx_buffer, data, len); /* What if len > buffer size? */
}
```

```

    /* CORRECT - Validate before copy */
#define RX_BUFFER_SIZE (512)
static uint8_t rx_buffer[RX_BUFFER_SIZE];

void receive_packet(const uint8_t* data, size_t len)
{
      if (data == NULL || len > RX_BUFFER_SIZE) {
        RX_LOG_ERROR(s_TAG, "Invalid packet: len=%zu (max=%d)", len,
                     RX_BUFFER_SIZE);
        return;
      }
      memcpy(rx_buffer, data, len);
}

/* CORRECT - Clamp to buffer size */
void receive_packet_safe(const uint8_t* data, size_t len,
size_t* bytes_copied)
{
      size_t copy_len = (len < RX_BUFFER_SIZE) ? len : RX_BUFFER_SIZE;
      memcpy(rx_buffer, data, copy_len);
      *bytes_copied = copy_len;

      if (len > RX_BUFFER_SIZE) {
        RX_LOG_WARN(s_TAG, "Packet truncated: %zu bytes lost",
                    len - RX_BUFFER_SIZE);
      }
}
```

5.161.3 DMA Buffer Validation

```
/* DMA buffers must be validated AND aligned */
rx_err_t start_dma_transfer(const uint8_t* src, size_t len)
{
      /* Validate buffer pointer */
      if (src == NULL) {
        return RX_ERR_INVALID_ARG;
      }

      /* Validate length */
      if (len == 0 || len > MAX_DMA_TRANSFER_SIZE) {
        RX_LOG_ERROR(s_TAG, "Invalid DMA length: %zu", len);
        return ESP_ERR_INVALID_SIZE;
      }

      /* Check alignment (DMA requires 4-byte alignment) */
      if (((uintptr_t)src & 0x3) != 0) {
        RX_LOG_ERROR(s_TAG, "DMA buffer not 4-byte aligned");
        return RX_ERR_INVALID_ARG;
      }

      /* Proceed with DMA */
      return spi_device_transmit(spi_handle, &transaction);
```

```
}
```

5.161.4 Ring Buffer Validation

```
/* CORRECT - Ring buffer with overflow protection */
typedef struct {
      uint8_t buffer[256];
      size_t head;
      size_t tail;
      size_t count;
} ring_buffer_t;

rx_err_t ring_buffer_write(ring_buffer_t* rb, const uint8_t* data,
size_t len)
{
      if (rb == NULL || data == NULL) {
        return RX_ERR_INVALID_ARG;
     }

      /* Check available space */
      size_t available = sizeof(rb->buffer) - rb->count;
      if (len > available) {
        RX_LOG_WARN(s_TAG, "Ring buffer overflow: %zu bytes lost",
                    len - available);
        return ESP_ERR_NO_MEM;
     }

      /* Safe to write */
      for (size_t i = 0; i < len; i++) {
        rb->buffer[rb->head] = data[i];
        rb->head = (rb->head + 1) % sizeof(rb->buffer);
        rb->count++;
     }

      return RX_OK;
}
```

5.162 Summary

Table 18: Memory Management Rules Summary

Rule	Implementation
No dynamic allocation	Use static/global buffers instead of malloc/free
Avoid large stack arrays	Use static storage for arrays $>$ 256 bytes
Use `const`	Place lookup tables, strings, defaults in flash
Validate all buffers	Check lengths before memcpy, DMA, ring buffers

Following these rules ensures stable, predictable memory usage and prevents an entire class of memory-related bugs that are extremely difficult to debug in embedded systems.

This section establishes best practices for real-time embedded systems development across all STAR platforms. These guidelines are critical for motor control (1000Hz loops), sensor fusion, and safety-critical robotics applications.

5.163 Interrupt Service Routines(ISRs)

5.163.1 Minimize ISR Work

Rule: Keep ISRs as short as possible. Set flags or queue work to background tasks instead of performing complex operations.

```
/* WRONG - Too much work in ISR */
static void IRAM_ATTR gpio_isr_handler(void* arg)
{
      /* Reading I2C sensor blocks for milliseconds! */
      float temperature = read_i2c_sensor();
      update_pid_controller(temperature);
      adjust_motor_speed();
      log_telemetry_to_sd_card(); /* File I/O in ISR! */
}

/* CORRECT - Minimal ISR, queue work to task */
static volatile bool s_encoder_event = false;
static QueueHandle_t s_encoder_queue;

static void IRAM_ATTR encoder_isr_handler(void* arg)
{
      /* Only set flag and queue event */
      s_encoder_event = true;

      encoder_event_t event = {.timestamp_us = esp_timer_get_time()};
      xQueueSendFromISR(s_encoder_queue, &event, NULL);

      /* ISR done in microseconds, not milliseconds */
}

/* Background task processes events */
void encoder_task(void* arg)
{
      encoder_event_t event;
      while (1) {
        if (xQueueReceive(s_encoder_queue, &event, portMAX_DELAY)) {
          /* Heavy processing here, not in ISR */
          process_encoder_event(&event);
       }
     }
}
```

5.163.2 ISR Constraints

- **No blocking operations**: No mutexes, delays, or I/O
- **Use `IRAM_ATTR`**: Places ISR in IRAM for faster execution

- **FromISR variants**: Use `xQueueSendFromISR()`, not `xQueueSend()`
- **Keep stack usage low**: ISR stack is typically 1KB or less
- **No floating-point**: Context save/restore is expensive

5.164 No Blocking Delays

Rule: Never use `vTaskDelay()` or busy-wait loops in time-critical code. Use event-driven design.

```
/* WRONG - Blocking delay in motor control */
void motor_control_loop(void)
{
      while (1) {
        read_encoder();
        compute_pid();
        update_motor();

        vTaskDelay(pdMS_TO_TICKS(1)); /* Blocks, jitter from scheduler */
    }
}

/* CORRECT - Timer-driven execution */
static void motor_control_timer_callback(void* arg)
{
      /* Called exactly every 1ms by hardware timer */
      read_encoder();
      compute_pid();
      update_motor();
}

void setup_motor_control(void)
{
      esp_timer_create_args_t timer_args = {
          .callback = motor_control_timer_callback, .name = "motor_ctrl"};

      esp_timer_handle_t timer;
      esp_timer_create(&timer_args, &timer);
      esp_timer_start_periodic(timer, 1000); /* 1ms = 1000us */
}

/* CORRECT - Event-driven sensor reading */
void sensor_task(void* arg)
{
      while (1) {
        /* Wait for event, no polling */
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

        /* Event occurred, process it */
        process_sensor_data();
    }
}
```

5.165 Real - Time and Safety

Table 19: Real - Time Best Practices

Practice	Implementation
Know timing constraints	Profile worst-case execution times (WCET)
Use watchdog timers	Automatic reset on firmware lockup
Handle sensor timeouts	Never assume sensors always respond
Fail safe	On error, disable actuators and enter safe state
CRC critical data	Validate packets and stored configuration

5.165.1 Timing Constraints

```
/* Profile execution time using GPIO toggle */
void motor_control_loop(void)
{
      gpio_set_level(DEBUG_PIN, 1); /* Pin HIGH */

      /* Critical timing section */
      read_encoder();
      compute_pid();
      update_motor();

      gpio_set_level(DEBUG_PIN, 0); /* Pin LOW */

      /* Measure with oscilloscope:
       * Pulse width = worst-case execution time (WCET) */
}

/* Or use platform-specific cycle counter (ESP32, RX72N, etc.) */
uint32_t start_cycles = esp_cpu_get_cycle_count();

critical_function();

uint32_t end_cycles = esp_cpu_get_cycle_count();
uint32_t elapsed_us = (end_cycles - start_cycles) / (CPU_FREQ_MHZ);
RX_LOG_INFO(s_TAG, "Execution time: %lu us", elapsed_us);
```

5.165.2 Watchdog Timers

```
    /* Initialize task watchdog */
#include "esp_task_wdt.h"

void motor_control_task(void* arg)
{
      /* Subscribe to watchdog (5 second timeout) */
      esp_task_wdt_add(NULL);

      while (1) {
        /* Kick watchdog to prove we're alive */
```

```
        esp_task_wdt_reset();

        /* Control loop */
        motor_control_iteration();

        vTaskDelay(pdMS_TO_TICKS(1));
    }
}

/* If task hangs, watchdog resets system after timeout
   (platform-specific) */
```

5.165.3 Sensor Timeouts

```
/* WRONG - Assumes sensor always responds */
float read_temperature(void)
{
      send_i2c_command(TEMP_SENSOR_ADDR, READ_TEMP_CMD);
      return receive_i2c_data(TEMP_SENSOR_ADDR); /* Hangs if sensor fails
         */
}

/* CORRECT - Timeout and error handling */
rx_err_t read_temperature_safe(float* temp_celsius)
{
      rx_err_t ret;
      uint8_t data[2];

      /* I2C read with 100ms timeout */
      ret = star_bus_i2c_read(bus_manager, "temp_i2c", data, 2,
         TEMP_READ_REG,
                              NULL);
      if (ret != RX_OK) {
        RX_LOG_WARN(s_TAG, "Temperature sensor timeout");
        return ret;
    }

      *temp_celsius = (data[0] << 8 | data[1]) / 10.0F;
      return RX_OK;
}

/* Use sensor timeout for safety */
rx_err_t ret = read_temperature_safe(&motor_temp);
if (ret != RX_OK) {
      /* Sensor failed - enter safe state */
      rx_motor_stop(&motor, true);
      system_enter_safe_mode();
}
```

5.165.4 Fail - Safe Design

```
/* Safe state: all motors stopped, systems halted */
typedef enum : uint8_t {
  k_num_motors = 4, /**< Number of motors on STAR platform */
} motor_count_t;

void system_enter_safe_mode(void)
{
      RX_LOG_ERROR(s_TAG, "ENTERING SAFE MODE");

      /* Stop all motors */
      for (uint8_t i = 0; i < k_num_motors; i++) {
        rx_motor_stop(&motors[i], true); /* Brake */
     }

      /* Disable all actuators */
      gpio_set_level(RELAY_ENABLE_PIN, 0);

      /* Set error LED */
      gpio_set_level(ERROR_LED_PIN, 1);

      /* Log error to flash */
      log_critical_error_to_nvs("Safe mode triggered");

      /* Infinite loop - require manual reset */
      while (1) {
        vTaskDelay(pdMS_TO_TICKS(100));
     }
}

/* Trigger safe mode on critical errors */
typedef enum : uint16_t {
  k_battery_critical_mv = 3000, /**< Critical per-cell voltage threshold
     (3.0V) */
} battery_limits_t;

static const float s_motor_thermal_shutdown_celsius = 85.0F;

if (cell_voltage_mv < k_battery_critical_mv) {
      system_enter_safe_mode();
}

if (motor_temp_celsius > s_motor_thermal_shutdown_celsius) {
      system_enter_safe_mode();
}
```

5.165.5 CRC Validation

```
/* Validate critical configuration with CRC */
typedef struct {
      star_pid_config_t pid;
      uint32_t crc32; /* CRC of pid structure */
} stored_config_t;
```

```

rx_err_t save_config_to_nvs(const star_pid_config_t* config)
{
      stored_config_t stored;
      memcpy(&stored.pid, config, sizeof(star_pid_config_t));

      /* Calculate CRC-32 */
      stored.crc32 =
          esp_crc32_le(0, (uint8_t *)&stored.pid,
             sizeof(star_pid_config_t));

      /* Save to NVS */
      return nvs_set_blob(nvs_handle, "pid_config", &stored,
                          sizeof(stored_config_t));
}

rx_err_t load_config_from_nvs(star_pid_config_t* config)
{
      stored_config_t stored;
      size_t len = sizeof(stored_config_t);

      esp_err_t ret = nvs_get_blob(nvs_handle, "pid_config", &stored,
         &len);
      if (ret != RX_OK) {
        return ret;
     }

      /* Verify CRC */
      uint32_t calculated_crc =
          esp_crc32_le(0, (uint8_t *)&stored.pid,
             sizeof(star_pid_config_t));
      if (calculated_crc != stored.crc32) {
        RX_LOG_ERROR(s_TAG, "Config CRC mismatch! Flash corrupted?");
        return ESP_ERR_INVALID_CRC;
     }

      memcpy(config, &stored.pid, sizeof(star_pid_config_t));
      return RX_OK;
}
```

5.166 Hardware Abstraction

5.166.1 Use Abstract Bus Interfaces

```
/* CORRECT - Driver depends on bus abstraction, not platform HAL */
rx_err_t mpu6050_init(mpu6050_handle_t* handle,
star_bus_manager_t* bus_manager,
const char* bus_name)
{
      handle->bus_manager = bus_manager;
      handle->bus_name = bus_name;

      /* Access I2C through bus manager */
```

```
      uint8_t whoami;
      esp_err_t ret = star_bus_i2c_read(bus_manager, bus_name, &whoami, 1,
                                        MPU6050_WHO_AM_I, NULL);

      /* Easy to mock for testing */
      return ret;
}
```

5.166.2 Debounce Mechanical Inputs

```
/* Software debounce for button */
typedef struct {
      gpio_num_t pin;
      bool state;
      uint32_t last_change_ms;
      uint32_t debounce_ms;
} debounced_button_t;

bool button_read(debounced_button_t* btn)
{
      bool raw_state = gpio_get_level(btn->pin);
      uint32_t now_ms = xTaskGetTickCount() * portTICK_PERIOD_MS;

      if (raw_state != btn->state) {
        /* State change detected */
        if (now_ms - btn->last_change_ms > btn->debounce_ms) {
          /* Stable for debounce period */
          btn->state = raw_state;
          btn->last_change_ms = now_ms;
       }
     }

      return btn->state;
}
```

5.166.3 Enable Brown - Out Detection

```
/* In sdkconfig or menuconfig:
* CONFIG_ESP32_BROWNOUT_DET=y
* CONFIG_ESP32_BROWNOUT_DET_LVL_SEL_7=y  (3.08V threshold)
*/

/* ESP32 automatically resets on voltage dip below threshold */
/* Prevents flash corruption and erratic behavior */
```

5.166.4 Flash Wear - Leveling

```
    /* Use NVS (Non-Volatile Storage) for wear-leveling */
#include "nvs_flash.h"

```

```
rx_err_t init_nvs(void)
{
      esp_err_t ret = nvs_flash_init();
      if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
          ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        /* Erase and retry */
        /* Platform-specific: handle flash erase error appropriately */
        nvs_flash_erase();
        ret = nvs_flash_init();
     }
      return ret;
}

/* NVS automatically rotates writes across sectors */
nvs_handle_t nvs_handle;
nvs_open("storage", NVS_READWRITE, &nvs_handle);
nvs_set_u32(nvs_handle, "boot_count", boot_count);
nvs_commit(nvs_handle);
```

5.166.5 Always Use Pull - Up / Pull - Down

```
/* WRONG - Floating input */
gpio_config_t io_conf = {
  .pin_bit_mask = (1ULL << GPIO_NUM_35),
  .mode = GPIO_MODE_INPUT,
  /* No pull-up or pull-down! Reads random values */
};

/* CORRECT - Enable pull-up */
gpio_config_t io_conf = {
  .pin_bit_mask = (1ULL << GPIO_NUM_35),
  .mode = GPIO_MODE_INPUT,
  .pull_up_en = GPIO_PULLUP_ENABLE,
  .pull_down_en = GPIO_PULLDOWN_DISABLE,
};
gpio_config(&io_conf);

/* For active-low signals (like nFAULT), use pull-up */
/* For active-high signals, use pull-down */
```

5.167 Performance and Real - Time

5.167.1 Benchmark, Don't Guess

```
    /* Use GPIO toggle to measure timing on oscilloscope */
#define BENCHMARK_START() gpio_set_level(BENCHMARK_PIN, 1)
#define BENCHMARK_END() gpio_set_level(BENCHMARK_PIN, 0)

          void
          pid_control_loop(void) {
        BENCHMARK_START();

```

```
        float velocity_rpm;
        star_encoder_get_velocity_rpm(&encoder, 1.0F, 500, &velocity_rpm);

        float output;
        star_pid_compute(&pid, setpoint, velocity_rpm, 0.001F, &output);

        star_motor_set_duty(&motor, output);

        BENCHMARK_END();
        /* Oscilloscope shows pulse width = execution time */
    }
```

5.167.2 Avoid Blocking Operations

```
          /* WRONG - Blocking I2C read in control loop */
          void
          control_loop(void) {
        while (1) {
          uint8_t data[6];
          i2c_master_read_from_device(I2C_NUM_0, SENSOR_ADDR, data, 6,
                                      100 / portTICK_PERIOD_MS);
          /* Blocks for up to 100ms! */

          process_data(data);
      }
    }

     /* CORRECT - Non-blocking with DMA */
     void setup_nonblocking_i2c(void) {
       /* Configure I2C interrupt + DMA */
       /* When data ready, ISR triggers */
       /* Background task processes results */
    }
```

5.168 Power Management

5.168.1 Use Sleep Modes

```
          /* Light sleep between sensor reads */
          void
          sensor_polling_task(void *arg) {
        while (1) {
          read_sensors();
          process_data();

          /* Sleep for 100ms instead of busy-wait */
          esp_sleep_enable_timer_wakeup(100000); /* 100ms in us */
          esp_light_sleep_start();
      }
    }
```

5.168.2 Use DMA to Reduce CPU Load

```
        /* DMA-enabled SPI transfer (CPU-free) */
        spi_device_transmit(spi_handle, &transaction);
    /* DMA handles transfer, CPU does other work */
```

5.169 Testing and Debugging

5.169.1 Use Mock Drivers

```
        /* Create mock bus for unit testing */
        star_bus_manager_t mock_bus_manager;
    star_bus_manager_init(&mock_bus_manager, "mock", NULL, NULL);

    /* Test driver without hardware */
    mpu6050_handle_t sensor;
    mpu6050_init(&sensor, &mock_bus_manager, "mock_i2c");
```

5.169.2 Meaningful Logging

```
        /* Include timestamp, module, and error codes */
        ESP_LOGE(s_TAG,
                 "[%lu] Motor fault detected: current=%lu mA (limit=%d
                    mA)",
                 esp_timer_get_time() / 1000, /* Timestamp in ms */
                 current_ma, MAX_MOTOR_CURRENT_MA);
```

5.170 Reliability and Fault Tolerance

5.170.1 Reset Peripherals if Stuck

```
        /* I2C bus recovery */
        rx_err_t
        recover_i2c_bus(i2c_port_t port) {
      ESP_LOGW(s_TAG, "Recovering I2C bus %d", port);

      /* Delete driver */
      i2c_driver_delete(port);

      /* Wait */
      vTaskDelay(pdMS_TO_TICKS(10));

      /* Re-initialize */
      i2c_config_t conf = {/* ... */};
      i2c_param_config(port, &conf);
      return i2c_driver_install(port, conf.mode, 0, 0, 0);
  }
```

5.171 Summary

Following these real - time and embedded best practices ensures :

- **Predictable timing** for motor control and sensor fusion
- **Robust error handling** for fault tolerance
- **Safe operation** in robotics applications
- **Maintainable code** through proper abstractions
- **Testability** with mock drivers and measurement tools

These practices are the foundation of reliable embedded systems for robotics.

This section establishes configuration requirements for Protocol Buffer messages used in STAR embedded firmware. The STAR project uses **nanopb** (Nano Protocol Buffers) for embedded systems, which requires static memory allocation.

5.172 nanopb Overview

nanopb is a small, portable Protocol Buffers implementation designed for embedded systems:

- *Static Allocation* : All buffers allocated at compile time(no malloc)
- *Small Code Size* : Minimal flash / RAM footprint
- *C Language* : Compatible with C99(optional C11 features; not C++)
- *Field Size Constraints* : Maximum sizes configured in `.options` files

5.173 Why Static Allocation ?

STAR firmware follows strict "no dynamic allocation" rules(see Section 25) .nanopb field sizes must be explicitly configured to enable compile - time buffer allocation .

5.174 Field Size Configuration

Field sizes are configured in `.options` files located alongside `.proto` files :

```
#File : star - proto / proto / star.options
            star.v1.RequestHeader.request_id max_size :
                64 star.v1.ResponseHeader.error_message
            max_size : 128 star.v1.FirmwareChunk.data max_size : 1024
```

5.175 Standard Field Size Constraints

Table 20: nanopb Field Size Constraints for Embedded Platforms

Field Type	Constraint	Example
String(UUID)	`max_size : 64`	`request_id`
String(error)	`max_size : 128`	`error _message`
String(version)	`max_size : 32`	`firmware_version`
Repeated(cells)	`max_count : 16`	`cell_mv[]`
Repeated(motors)	`max_count : 4`	`motor_status[]`
Bytes(firmware)	`max_size : 1024`	`chunk_data`

5.176 Examples

5.176.1 String Fields

```
    // File: robot_command.proto
    message RequestHeader {
  string request_id = 1; // UUID (36 chars + null terminator)
}

// File: robot_command.options
RequestHeader.request_id max_size : 64
```

Generated C code :

```
typedef struct _RequestHeader {
  char request_id[64]; /* Static allocation */
} RequestHeader;
```

5.176.2 Repeated Fields

```
    // File: battery_state.proto
    message BatteryState {
  repeated uint32 cell_mv = 1; // Cell voltages in millivolts
}

// File: battery_state.options
BatteryState.cell_mv max_count : 16
```

Generated C code :

```
typedef struct _BatteryState {
  uint32_t cell_mv[16];    /* Static array */
  pb_size_t cell_mv_count; /* Actual element count */
} BatteryState;
```

5.176.3 Bytes Fields

```
    // File: firmware_update.proto
    message FirmwareChunk {
  bytes data = 1; // Binary firmware data
}

// File: firmware_update.options
FirmwareChunk.data max_size : 1024
```

Generated C code :

```
typedef struct _FirmwareChunk {
  pb_bytes_array_t data; /* Contains uint8_t bytes[1024] */
} FirmwareChunk;
```

5.177 Sizing Guidelines

5.177.1 String Sizes

Table 21: Recommended String Field Sizes

Purpose	Size	Example
UUID(RFC 4122)	64 bytes	`request_id, session_id`
Error messages	128 bytes	`error _message`
Version strings	32 bytes	`firmware_version`
Short names	32 bytes	`component_name`
Log messages	256 bytes	`log_text`

5.177.2 Repeated Field Counts

Table 22: Recommended Repeated Field Counts

Purpose	Count	Example
Battery cells(3S LiPo)	3	`cell_mv[]`
Motors(quadruped)	4	`motor_status[]`
Motors(hexapod)	8	`motor_status[]`
Sensor array	8 –16	`temperature_sensors[]`
Telemetry samples	32 –64	`velocity_samples[]`

5.177.3 Bytes Field Sizes

Table 23: Recommended Bytes Field Sizes

Purpose	Size	Example
Firmware chunk	1024 bytes	`chunk_data`
Image thumbnail	4096 bytes	`thumbnail`
Small binary data	256 bytes	`calibration_blob`
Cryptographic key	64 bytes	`aes_key`

5.178 Memory Impact

5.178.1 Buffer Size Calculation

Each message allocates memory for the largest possible size:

```
/* Example: BatteryState with 3 cells (3S LiPo) */
typedef struct {
  uint32_t cell_mv[3];      /* 3 * 4 = 12 bytes */
  pb_size_t cell_mv_count; /* 4 bytes */
  uint32_t current_ma;      /* 4 bytes */
  uint8_t soc_percent;      /* 1 byte */
  /* Total: ~21 bytes + padding */
} BatteryState;

/* Even if only 1 cell used, full array is allocated */
```

5.178.2 RAM Usage Example

```
    /* Static allocation of message buffers */
    static RequestHeader s_request;    /* 64 bytes */
static ResponseHeader s_response;      /* 128 bytes */
static BatteryState s_battery_state;   /* 24 bytes */
static FirmwareChunk s_firmware_chunk; /* 1024 bytes */

/* Total: ~1240 bytes of static RAM */
```

5.179 Best Practices

5.179.1 Right - Size Your Buffers

- **Don't over-allocate**: `max_size:1024` for a 20-byte string wastes 1004 bytes
- **Don't under-allocate**: Truncated data causes silent bugs
- **Measure actual usage** : Profile message sizes in production

5.179.2 Use Fixed - Size Types

```
        /* CORRECT - Fixed-size integer types */
        message MotorCommand {
      int32 speed_percent = 1; // Always 4 bytes
      uint64 timestamp_us = 2; // Always 8 bytes
   }

    /* AVOID - Variable-length encoding */
    message MotorCommand {
      sint32 speed_percent = 1; // 1-5 bytes (varint encoding)
   }
```

5.179.3 Validate Message Sizes

```
        /* Check encoded message size before transmission */
        uint8_t buffer[256];
    pb_ostream_t stream = pb_ostream_from_buffer(buffer, sizeof(buffer));

    bool status = pb_encode(&stream, RequestHeader_fields, &request);
    if (!status) {
      RX_LOG_ERROR(s_TAG, "Encoding failed: %s", PB_GET_ERROR(&stream));
      return RX_ERR_FAIL;
   }

    size_t message_length = stream.bytes_written;
    RX_LOG_INFO(s_TAG, "Encoded message: %zu bytes", message_length);

    /* Verify it fits in SPI transaction buffer */
    if (message_length > SPI_MAX_TRANSFER_SIZE) {
      RX_LOG_ERROR(s_TAG, "Message too large for SPI: %zu > %d",
          message_length,
                  SPI_MAX_TRANSFER_SIZE);
      return ESP_ERR_INVALID_SIZE;
   }
```

5.180 Integration with Code Generation

5.180.1 buf.gen.yaml Configuration

```
#File : star - proto / buf.gen.yaml
        version : v2 plugins : -local : nanopb_generator out : gen /
                                                               nanopb opt
        : - --extension =.pb
```

5.180.2 Options File Naming

```
#Options file must match.proto file name
        proto /
        robot_command.proto->robot_command.options battery_state.proto
            ->battery_state.options firmware_update.proto->firmware_update
            .options
```

5.181 Common Pitfalls

5.181.1 Missing Options File

```
        /* ERROR: No .options file for string field */
        // robot_command.proto
        message Command {
      string name = 1; // No max_size specified
   }

    /* Generated code uses default (usually too small or error) */
```

Fix: Always create `.options` file for messages with strings, bytes, or repeated fields.

5.181.2 Mismatched Field Names

```
#WRONG - Field name mismatch
#robot_command.options
    Command.command_name max_size : 32 #Field is 'name',
    not 'command_name'

#CORRECT
        Command.name max_size : 32
```

5.181.3 Forgetting Array Count

```
    /* Common mistake: Forgetting to set count field */
    BatteryState battery = {0};
battery.cell_mv[0] = 3700;
battery.cell_mv[1] = 3720;
battery.cell_mv[2] = 3710;
/* BUG: battery.cell_mv_count not set! Encoder sends 0 cells */

/* CORRECT */
battery.cell_mv_count = 3; /* Set actual element count (3S LiPo) */
```

5.182 Summary

- **Always create.options files** for strings, bytes, repeated fields
- **Right-size buffers** based on actual usage (measure, don't guess)
- **Use fixed-size types** to avoid varint encoding overhead
- **Validate encoded sizes** before transmission
- **Set array count fields** for repeated fields

Properly configured nanopb constraints enable reliable, predictable Protocol Buffer communication on resource-constrained embedded hardware.

== == == == == == == == == == == == = = =

5.183 NASA Power of 10: Rules for Safety-Critical Code

This section documents the NASA/JPL Power of 10 coding rules developed by Gerard J. Holzmann in 2006 for safety-critical embedded systems. These rules prioritize verification and predictability over coding flexibility.

STAR Compliance Philosophy: The STAR project follows the Power of 10 rules, with one **intentional architectural deviation** for Dependency Inversion Principle (DIP) – a conscious trade-off for testability and modularity.

5.183.1 Compliance Status Legend

Symbol	Meaning
✓ COMPLIANT	STAR follows this rule
△ !INTENTIONAL DEVIATION	Architectural choice(DIP pattern)

5.183.2 Rule 1 : Simplify Control Flow

Rule: Restrict all code to very simple control flow constructs – no `goto`, `setjmp`/`longjmp`, or recursion (direct or indirect).

Rationale: Creates predictable program structure for human understanding and static analysis. Recursion causes unbounded stack usage – dangerous in embedded systems.

STAR Compliance: ✓ COMPLIANT

```
    /* COMPLIANT - No goto, no recursion */
    esp_err_t
    star_motor_init(star_motor_handle_t *handle,
                    const star_motor_config_t *config) {
  if (handle == NULL || config == NULL) {
    return ESP_ERR_INVALID_ARG;
 }

  /* All control flow uses if/while/for */
  esp_err_t ret = configure_timer(handle, config);
  if (ret != ESP_OK) {
    return ret;
 }

  return configure_gpio(handle, config);
}
```

5.183.3 Rule 2 : Fixed Loop Upper - Bounds

Rule : All loops must have a fixed upper bound that static analysis tools can trivially prove.

Rationale : Prevents runaway code, guarantees termination, enables real - time deadline verification.

STAR Compliance: ✓ COMPLIANT

```
/* COMPLIANT - Fixed iteration count */
#define MAX_RETRIES (3)

    for (uint8_t i = 0; i < MAX_RETRIES; i++) {
  if (sensor_read() == ESP_OK) {
    break;
 }
  vTaskDelay(pdMS_TO_TICKS(100));
}

/* COMPLIANT - Array with known size */
#define BQ7850_MAX_CELLS (16)

for (uint8_t cell = 0; cell < BQ7850_MAX_CELLS; cell++) {
```

```
  read_cell_voltage(cell, &voltages[cell]);
}
```

5.183.4 Rule 3 : No Dynamic Memory After Initialization

Rule : Prohibit `malloc` /`free` after initialization phase.

Rationale : Dynamic allocation causes unpredictable performance, memory leaks, fragmentation, and corruption.

STAR Compliance: ✓ **COMPLIANT**

Best Practice : Use pre - allocated static pools instead of runtime memory requests.

```
/* COMPLIANT - Static allocation with fixed limits */
#define MAX_ITEMS (8)
#define MAX_DESC_LEN (64)

    typedef struct {
  char items[MAX_ITEMS][MAX_DESC_LEN]; /* Static 2D array */
  uint8_t item_count;
} static_pool_t;

/* Usage */
if (pool->item_count >= MAX_ITEMS) {
  return ESP_ERR_NO_MEM;
}
strncpy(pool->items[pool->item_count], desc, MAX_DESC_LEN - 1);
pool->items[pool->item_count][MAX_DESC_LEN - 1] = '\0';
pool->item_count++;
```

5.183.5 Rule 4 : Keep Functions Short

Rule : Functions should not exceed ~60 lines(one printed page, one statement per line) .

Rationale : Represents single verifiable logical units; improves comprehension, review, and testability.

STAR Compliance: ✓ **MOSTLY COMPLIANT**

```
    /* COMPLIANT - Single responsibility, short function */
    esp_err_t
    rx_motor_get_current_ma(rx_motor_handle_t *handle,
                            float *current_ma) {
  if (handle == NULL || current_ma == NULL || !handle->initialized) {
    return ESP_ERR_INVALID_ARG;
 }

  int raw_value = 0;
  esp_err_t ret = read_adc_channel(handle, &raw_value);
  if (ret != ESP_OK) {
    return ret;
 }

  *current_ma = (float)raw_value * handle->ki_propi / 1000.0F;
  return ESP_OK;
}
```

5.183.6 Rule 5 : Use Assertions Generously

Rule : Minimum two assertions per function; assertions must be side - effect - free.

Rationale : Acts as executable documentation verifying pre - conditions, post - conditions, and invariants.

STAR Compliance: ✓ **COMPLIANT**

Note: STAR uses explicit parameter validation instead of assertions for production code (assertions disabled in release builds).

```
/* COMPLIANT - Explicit validation (better than assert for production) */
esp_err_t star_pid_compute(star_pid_handle_t* handle, float setpoint,
                           float measurement, float dt, float* output)
{
  /* Pre-condition checks (effectively runtime assertions) */
  if (handle == NULL) {
    return ESP_ERR_INVALID_ARG;
 }
  if (!handle->initialized) {
    return ESP_ERR_INVALID_STATE;
 }
  if (output == NULL) {
    return ESP_ERR_INVALID_ARG;
 }
  if (dt <= 0.0F || dt > 1.0F) { /* Sanity check time delta */
    return ESP_ERR_INVALID_ARG;
 }

  /* Function logic */
  float error = setpoint - measurement;
  handle->integral += error * dt;

  /* Post-condition: integral within bounds */
  if (handle->integral > handle->integral_max) {
    handle->integral = handle->integral_max;
 } else if (handle->integral < handle->integral_min) {
    handle->integral = handle->integral_min;
 }

  *output = handle->kp * error + handle->ki * handle->integral;
  return ESP_OK;
}
```

5.183.7 Rule 6 : Declare Data at Smallest Scope

Rule : Minimize variable scope to reduce accidental corruption risks.

Rationale : Limits chances of accidental reference or corruption from other code.

STAR Compliance: ✓ **COMPLIANT**

```
    /* COMPLIANT - Variables declared at minimal scope */
    void
    process_sensors(void) {
  /* Loop variable declared in for statement */
  for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
```

```
    /* Temporary variable for single sensor */
    float temp_c = 0.0F;
    if (read_sensor(i, &temp_c) == ESP_OK) {
      /* Process immediately, temp_c out of scope after block */
      update_telemetry(i, temp_c);
    }
  }
}
```

5.183.8 Rule 7 : Check All Return Values and Parameters

Rule : Validate all function returns and input parameters; treat warnings as errors.

Rationale : Forces explicit failure - condition handling rather than ignoring error codes.

STAR Compliance: ✓ **COMPLIANT**

```
    /* COMPLIANT - All returns checked, all parameters validated */
    esp_err_t
    star_bus_manager_add_bus(star_bus_manager_t *manager,
                             star_bus_config_t *config) {
  /* Parameter validation */
  if (manager == NULL || config == NULL) {
    return ESP_ERR_INVALID_ARG;
  }

  /* Check return value and propagate errors */
  esp_err_t ret = star_bus_config_init(config);
  if (ret != ESP_OK) {
    ESP_LOGE(TAG, "Bus initialization failed: %s", esp_err_to_name(ret));
    return ret;
  }

  /* Mutex operation checked */
  if (xSemaphoreTake(manager->mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
    ESP_LOGE(TAG, "Failed to acquire mutex");
    return ESP_ERR_TIMEOUT;
  }

  /* Critical section */
  add_to_list(manager, config);
  xSemaphoreGive(manager->mutex);

  return ESP_OK;
}
```

5.183.9 Rule 8 : Limit Preprocessor Use

Rule : Restrict to header inclusion and simple macros; forbid token pasting, recursive macros, and incomplete syntactic units.

Rationale : Preprocessor complexity obscures operations, fooling developers and static analysis tools.

STAR Compliance: ✓ **COMPLIANT**

```c
/* COMPLIANT - Simple constant macros */
#define MAX_RETRY_COUNT (3)
#define MOTOR_PWM_FREQ_HZ (20000)
#define TIMEOUT_MS (1000)

    /* COMPLIANT - Simple inline functions (better than complex macros) */
    static inline float
    voltage_mv_to_v(uint16_t mv) {
  return (float)mv / 1000.0F;
}

/* AVOID - Complex token-pasting macros */
// #define CREATE_HANDLER(name) handler_##name##_init()  /* Too complex */
```

5.183.10 Rule 9 : Restrict Pointer Use

Rule : Maximum one dereferencing level(`*ptr` OK, `**ptr` forbidden); no function pointers.

Rationale : Multiple indirection and function pointers make static data - flow analysis impossible.

STAR Compliance: △ **!INTENTIONAL DEVIATION**

Intentional Architectural Deviation: STAR uses function pointers for **Dependency Inversion Principle(DIP)** – an industry-standard pattern for testability and modularity. This is a conscious trade-off for better architecture.

DIP Pattern(Intentional Function Pointer Use) :

```c
    /* INTENTIONAL DEVIATION - DIP interface with function pointers */
    typedef struct star_error_interface {
  esp_err_t (*record_error)(void *ctx, esp_err_t error, const char
     *message,
                            const char *file, int line, const char *func);
  bool (*can_retry)(void *ctx);
  esp_err_t (*reset_state)(void *ctx);
  void *ctx; /* Implementation context */
} star_error_interface_t;

/* Why we keep this:
 * 1. Enables mock implementations for unit testing
 * 2. Allows swapping error handlers without recompilation
 * 3. Follows SOLID principles (Dependency Inversion)
 * 4. Industry-standard pattern (used in Linux kernel, RTOS, etc.)
 */
```

5.183.11 Rule 10 : Compile with Maximum Warnings

Rule : Enable all compiler warnings at highest pedantry; achieve zero - warning builds with static analyzers .

Rationale : Modern compilers catch bugs before runtime.Zero warnings, no excuses.

STAR Compliance: ✓ **COMPLIANT**

```c
#platformio.ini - Compiler flags
    build_flags = -Wall #Enable all warnings - Wextra #Extra warnings -
```

```
3                   Werror #Treat warnings as errors - Wno - unused -
4                   parameter #Allow unused params in interface
                        implementations -
5                   Wno - missing - field - initializers
```

CI / CD Enforcement : Build fails on any compiler warning.

5.183.12 Summary: STAR Compliance Matrix

Table 24: NASA Power of 10 Compliance Status

Rule	Description	STAR Status
1	No `goto` / recursion	✓ COMPLIANT
2	Fixed loop bounds	✓ COMPLIANT
3	No dynamic memory	✓ COMPLIANT
4	Short functions(<60 lines)	✓ COMPLIANT
5	Use assertions	✓ COMPLIANT
6	Minimal variable scope	✓ COMPLIANT
7	Check all returns	✓ COMPLIANT
8	Limit preprocessor	✓ COMPLIANT
9	Restrict pointers	△ !INTENTIONAL(DIP)
10	Maximum warnings	✓ COMPLIANT

5.183.13 Defensive Coding Measures

Beyond the Power of 10 rules, STAR implements additional defensive coding measures for electrical noise resilience in the RX72N motor controller firmware.

Independent Watchdog Timer(IWDT) :

- 120 kHz dedicated oscillator (independent of main clock)
- 1 second timeout, fed every 4ms from 250Hz control loop
- Detects infinite loops, deadlocks, and system hangs
- Logs reset cause on startup for diagnostics

Register Guard(ESD / EMI Protection) :

- Captures “golden” register values after initialization
- Periodically refreshes critical registers (PORT PDR, MSTPCR)
- Recovers from transient bit-flips caused by electrical noise
- Tracks correction count for field diagnostics

IRQ Digital Filtering:

- Hardware noise rejection on interrupt pins
- Configurable filter clock divisor(PCLK / 1 to PCLK / 64)

- 3 - sample debounce prevents spurious interrupts

```
    /* Defensive coding in motor control loop */
    while (true) {
  /* ... control logic ... */

  rx_iwdt_feed(); /* Feed watchdog every 4ms */

  if (++guard_counter >= 250) { /* Every 1 second */
    rx_register_guard_refresh();
    guard_counter = 0;
 }
}
```

5.183.14 References

- Holzmann, Gerard J. *"The Power of 10: Rules for Developing Safety-Critical Code."* IEEE Computer, 39(6), pp. 95-97, June 2006.
- JPL Institutional Coding Standard for the C Programming Language (JPL DOCID D-60411)
- MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems

== == == == == == == == == == == == = =

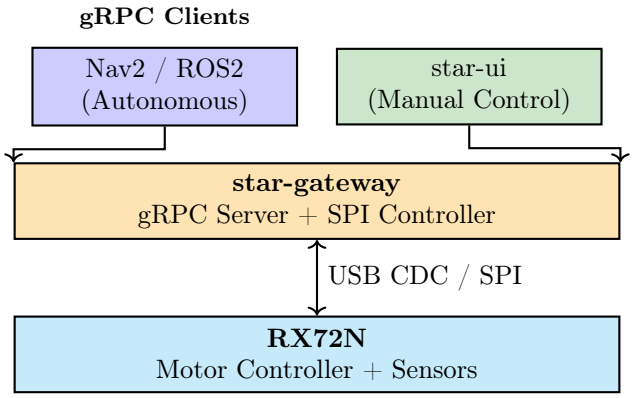
6 Gateway Architecture

This section documents the `star-gateway` Go service that bridges the Raspberry Pi 5 control stack with the RX72N motor controller.

6.1 Overview

The gateway service runs on the Raspberry Pi 5 and serves as the communication bridge between:

- **High-level side:** gRPC clients (Nav2/ROS2 for autonomous navigation, UI for manual control)
- **Low-level side:** RX72N motor controller via USB CDC (primary) or SPI (backup)



6.2 Protocol Stack Implementation

The gateway implements all layers of the communication protocol(see Section **??**) .

Table 25: Gateway Protocol Stack Implementation

Layer	Name	Go Package	Status
5	Application	`internal/service/`	Implemented
4	Serialization	`star-proto/gen/go/`	Implemented
3	HARQ + FEC	`internal/link/, internal/harq/, internal/fec/`	Implemented
2	Framing	`internal/frame/`	Implemented
1	Transport	`internal/transport/`	Implemented

6.3 Directory Structure

```
star - gateway / +--cmd / | +--star - gateway / | |
   +--main.go #Entry point | +--virtual_rx72n / |
   +--main.go #RX72N simulator + --internal / |
   +--transport / #Layer 1 : Physical transport | |
   +--spi.go #SPI transport(periph.io) | | +--cdc.go #USB CDC transport | |
   +--socket.go #Socket transport(simulation) |
   +--frame / #Layer 2 : Frame protocol | | +--frame.go #Constants and types |
   | +--encoder.go #Frame encoder | | +--decoder.go #Stream decoder |
   +--link / #Layer 3 : Reliability(HARQ) | | +--spi.go #SPILink(HARQ) |
   +--harq / #HARQ interface and types | | +--harq.go | +--fec / #FEC codec | |
```

```
+--convolutional.go #Rate 1 / 2,
K = 7 encoder | | +--viterbi.go #Viterbi decoder | |
    +--combiner.go #Chase Combiner | +--manager / #Transport management | |
    +--manager.go #TransportManager + SessionState |
    +--service / #Layer 5 : gRPC services | | +--motor_control.go | |
    +--telemetry.go | | +--battery.go | | +--configuration.go | |
    +--gateway.go #UI bridge service | +--app / #Application wiring | |
    +--gateway.go | +--server / #gRPC server |
    +--server.go + --go.mod + --CLAUDE.md
```

6.4 Frame Package

The `internal / frame` package defines the wire protocol constants and types.

6.4.1 Frame Constants

Table 26: Frame Protocol Constants

Constant	Value	Description
`SyncWord`	`0x55AA`	Frame sync marker(alternating bits : `[0x55, 0xAA]`)
`SyncSize`	2 bytes	Sync word size
`HeaderSize`	6 bytes	SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1)
`CRCSize`	4 bytes	CRC - 32 checksum
`MaxPayloadSize`	1024 bytes	Maximum protobuf payload
`MaxFrameSize`	1036 bytes	Total frame size
`MinFrameSize`	12 bytes	Empty payload frame

6.4.2 Frame Types

- `FrameTypeCommand` –Command from controller to peripheral
- `FrameTypeResponse` –Response from peripheral to controller
- `FrameTypeAck` –Acknowledgment of successful receipt
- `FrameTypeNack` –Negative acknowledgment(CRC error, etc.)

6.5 Transport Package

The `internal/transport` package provides physical transport implementations for communicating with the RX72N.

6.5.1 Available Transports

Table 27: Transport Implementations

Transport	Device	Speed	Usage
SPI	`/ dev / spidev0 .0`	10 MHz	Backup(priority 5)
USB CDC	`/ dev / ttyACM0`	12 Mbps(Full - Speed)	Primary(priority 10)
Socket	Unix socket	N / A	Simulation mode

6.5.2 SPI Configuration

Table 28: SPI Configuration Constants

Parameter	Value	Description
Device	`/dev/spidev0.0`	SPI device path
Speed	10 MHz	SPI clock frequency
Mode	0	CPOL = 0, CPHA = 0
Bits per word	8	Word size
Timeout	100 ms	Default operation timeout

6.5.3 Transport Interface

```
type Transport interface {
 Send(data[] byte)(int, error) Receive(maxLen int)([] byte, error)
     Transfer(txData[] byte)([] byte, error) Close() error
}
```

6.6 Link Layer (HARQ)

The `internal/link` package implements the reliability layer with two link types optimized for their respective transports.

6.6.1 SPILink

`SPILink` provides full Hybrid ARQ(HARQ) with forward error correction for the SPI transport, where the physical channel is more error-prone.

- **HARQ with ACK / NACK** –Retransmission on negative acknowledgment or timeout
- **FEC encoding / decoding** –Convolutional code(K = 7, Rate 1 / 2) with Viterbi decoding
- **Chase Combining** – Soft-combining of retransmitted frames for improved error recovery

6.6.2 CDCLink

`CDCLink` provides a lightweight link layer for USB CDC, since USB handles reliability at the hardware level.

- **No HARQ** –USB protocol provides its own error correction and retransmission
- **Sequence tracking** – Maintains sequence numbers for ordering

6.6.3 Shared Session State

Both link types use the shared `SessionState` from `internal/manager/` to maintain sequence number continuity across transport failovers.

6.6.4 HARQ Parameters

Table 29: HARQ Protocol Parameters

Parameter	Value	Description
Max Retries	3	Maximum transmission attempts
ACK Timeout	10 ms	ACK wait timeout
Sequence Max	0xFFFF	16 - bit wraparound
FEC Codec	Convolutional K = 7, Rate 1 / 2	Forward error correction

6.7 Transport Manager

The `internal / manager` package provides priority - based transport selection and automatic failover.

6.7.1 Transport Priorities

Table 30: Transport Manager Priority Configuration

Transport	Priority	Role
USB CDC	10	Primary transport
SPI	5	Backup transport

6.7.2 Failover Behavior

- **Shared SessionState** –Sequence continuity maintained across transports
- **Automatic failover** –Health monitoring detects transport failures
- **Primary timeout** –200 ms implicit timeout before failover
- **Secondary probe** –1 s PING probe to detect recovery of higher - priority transport

6.8 gRPC Services

The gateway exposes five gRPC services (see Section **??** for detailed specifications):

Table 31: Gateway gRPC Services

Service	Description
MotorControlService	Motor velocity commands, emergency stop, and control loop configuration
TelemetryService	Real-time sensor data streaming (encoders, IMU, battery)
BatteryManagementService	Battery monitoring, state of charge, and safety limits
ConfigurationService	Runtime parameter updates and system configuration
FirmwareUpdateService	Over-the-air firmware update for RX72N controller

6.9 Build and Test

```
#Build the gateway binary
    cd star -
    gateway go build./ cmd / star -
    gateway

#Run all tests
            go test./
        ...

#Run with verbose output
        go test -
    v./
        ...

#Format code
        go fmt./
        ...

#Static analysis
        go vet./
        ...
```

6.10 Dependencies

The gateway depends on :

- `github.com / Locked - Inc / star - proto / gen / go` –Generated protobuf and gRPC code
- `google.golang.org / grpc` –gRPC framework
- `google.golang.org / protobuf` – Protocol Buffers runtime

The `go.work` file at the repository root enables workspace mode for seamless cross-module development:

```
go 1.21

use (
    ./star-proto/gen/go
    ./star-proto/tests/go
    ./star-gateway
)
```

For the complete end - to - end communication path from UI to motor controller, see Section **??**.

== == == == == == == == == == == == = =

7 ROS2 Integration and Sensor Fusion Architecture

7.1 Overview

The STAR platform utilizes ROS2 Jazzy Jalisco as the primary high-level orchestration layer. The integration strategy focuses on providing a modular, reliable, and deterministic environment for SLAM, navigation, and mission control. This section details the architecture for Visual-LiDAR sensor fusion, coordinate frame definitions, and custom node interfaces.

7.2 Visual-LiDAR Sensor Fusion with RTAB-Map

The platform implements a heterogeneous sensor fusion strategy combining geometric data from LiDAR and appearance-based data from an RGB-D camera using **RTAB-Map** (Real-Time Appearance-Based Mapping).

7.2.1 Fusion Architecture

The fusion process is divided into two main layers:

1. **Local State Estimation(EKF) :** The `robot_localization` package runs an Extended Kalman Filter(EKF) fusing :
   - Wheel Odometry(from RX72N via `star_spi_bridge`)
   - IMU Linear Acceleration and Angular Velocity(from OAK - D or dedicated IMU)
2. **Global SLAM and Loop Closure(RTAB - Map) :** The `rtabmap_ros` node consumes :
   - Laser Scan data from RPLiDAR C1.
   - RGB - D images and PointClouds from the OAK - D camera.
   - Filtered Odometry from the EKF.

7.2.2 Mapping Strategy

RTAB-Map performs loop closure detection using a bag-of-words approach. When a loop is detected, it optimizes the pose graph to minimize drift. The resulting map is published as a 2D occupancy grid for Nav2 and a 3D octomap for obstacle avoidance.

7.3 Coordinate Frames (REP-105)

The STAR platform adheres to the **REP-105** standard for coordinate frames. The transform tree (`tf2`) is structured as follows:

- `map`**:** The global coordinate frame. Fixed to the world.
- `odom`**:** The local coordinate frame. Drift-prone, but continuous. The `map` → `odom` transform is published by RTAB-Map (Loop Closure).
- `base_link`**:** The center of the robot's drive axis. The `odom` → `base_link` transform is published by the EKF.
- `laser_frame`**:** The reference frame for the RPLiDAR C1.
- `camera_link`**:** The reference frame for the OAK-D Depth Camera.

7.4 Custom ROS2 Nodes and Interfaces

Three primary custom nodes facilitate the interaction between the STAR hardware and the ROS2 ecosystem.

7.4.1 star_spi_bridge

Bridges the low - level SPI protocol(RX72N) to ROS2 topics.

- **Subscribed Topics:** / cmd_vel(geometry_msgs / Twist)
- **Published Topics:** / odom / unfiltered(nav_msgs / Odometry), / joint_states(sensor_msgs / JointState)
- **Parameters:** spi_device, spi_speed_hz, gear_ratio

7.4.2 star_gateway_bridge

Provides a bridge between the gRPC / WebSocket gateway and the ROS2 computational graph.

- **Subscribed Topics:** / robot_status, / battery_state
- **Published Topics:** /teleop/cmd_vel (overrides for manual control)
- **Services:** / set_pid_gains, / trigger_calibration

7.4.3 star_safety_monitor

Ensures platform integrity and manages emergency states.

- **Logic :** Monitors battery voltage, motor current, and heartbeat from the RPi5 and RX72N.
- **Actions :** Publishes / emergency_stop or resets the motor controller if a timeout occurs.

7.5 Multi-Machine Communication

To ensure isolation and prevent interference in multi-robot environments, the ROS_DOMAIN_ID environment variable must be set uniquely for each STAR unit (e.g., export ROS_DOMAIN_ID=42).

7.6 Hardware Persistence (udev rules)

Consistent device naming is critical for reliable startup. The following udev rules are required in /etc/udev/rules.d/99-star.rules:

```
#RPLiDAR C1
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", SYMLINK+="ttyLidar"
#RX72N Motor Controller(via USB - Serial if applicable, or SPI identification)
SUBSYSTEM=="spidev", KERNEL=="spidev0.0", GROUP=="spi", MODE=="0660"
```

Note : The spi group must exist on the system before applying the udev rules .The STAR Docker container automatically creates this group via groupadd - f spi during build.On bare - metal installations, create the group manually :

```
sudo groupadd spi sudo usermod - aG spi $USER
```

8 ROS2 C++ Style Guide

8.1 Overview

The STAR project uses different coding conventions for ROS2 C++ code compared to RX72N firmware. This section documents the ROS2 C++ style guide, which supplements the C firmware style guide (Section **??**).

8.1.1 When to Use This Guide

- **ROS2 packages** (`star - ros2 / src /*/`): Use ROS2 C++ style
- **RX72N firmware** (`star-rx72n-firmware/`): Use C firmware style
- **Gateway** (`star-gateway/`): Follow Go conventions

8.1.2 Key Differences

Feature	C Firmware (RX72N)	ROS2 C++
Classes	N/A	`CamelCase`
Methods	`snake_case`	`snake_case` (same)
Variables	`snake_case`	`snake_case` (same)
Member vars	`s_` prefix	trailing `_`
Headers	`.h`	`.hpp`
Line limit	100 characters	120 characters
Namespaces	Not used	`star::package_name::`
Error handling	Return codes	Exceptions
Logging	`uart_puts()`	`RCLCPP_INFO/WARN/ERROR`
Include guards	`STAR_RX72N_FILE_H`	`PACKAGE_FILE_HPP_`
Doxygen	`/**` and `/**<`	`/**` and `/**<` (same)

Table 32: C vs C++ Style Comparison

8.2 Naming Conventions

8.2.1 Classes and Types

Use `CamelCase` for classes following ROS2 conventions:

```
// CamelCase for classes (ROS2 convention)
class StarGatewayBridgeNode : public rclcpp::Node {
// ...
};

// CamelCase for structs used as types
struct TelemetryData {
double battery_voltage_;
int32_t current_ma_;
};

// Type aliases use CamelCase
```

```
using TelemetryPtr = std::shared_ptr<TelemetryData>;
```

Listing 1: Class naming examples

8.2.2 Methods and Functions

Use `snake_case` for method names (same as C firmware):

```
// snake_case for methods (same as C firmware)
void publish_telemetry(const TelemetryData & data);
bool is_connected() const;
void on_battery_state_received(const
   sensor_msgs::msg::BatteryState::SharedPtr
msg);

// Use verb-based names that clarify actions
void check_for_errors();      // Good: Clear intent
void error_check();           // Bad: Noun-first is confusing
```

Listing 2: Method naming examples

8.2.3 Variables

Use `under_scored` for variables with trailing underscores for member variables:

```
// under_scored for variables
int loop_counter = 0;
std::string node_name = "gateway_bridge";

// Member variables with trailing underscore
class MyNode : public rclcpp::Node {
private:
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
std::shared_ptr<grpc::Channel> grpc_channel_;
bool is_connected_;
};

// Constants: ALL_CAPITALS
const int MAX_RETRIES = 3;
const double DEFAULT_TIMEOUT_S = 5.0;
```

Listing 3: Variable naming examples

8.2.4 Namespaces

Package-based namespaces use `under_scored` naming:

```
namespace star { namespace spi_bridge {

class SpiDriverNode : public rclcpp::Node {
// ...
};

}  // namespace spi_bridge
```

```
} // namespace star
```

Listing 4: Namespace examples

8.3 File Naming and Organization

8.3.1 Header Files

C++ headers use the `.hpp` extension (not `.h`):

- `star_gateway_bridge_node.hpp`
- `message_converter.hpp`
- `grpc_client.hpp`

Include guards follow the pattern: `PACKAGE_FILE_NAME_HPP_`

```
#ifndef STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_
#define STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_

// ... code ...

#endif  // STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_
```

Listing 5: Include guard example

8.3.2 Source Files

Source files use the `.cpp` extension with `under_scored` naming:

- `star_gateway_bridge_node.cpp`
- `message_converter.cpp`
- `grpc_client.cpp`

8.4 Header Organization

Headers must be organized in a standard order (enforced by `ament_cpplint`, ROS2's official include-order checker):

1. License and copyright
2. Include guard
3. ROS2 core includes (`<rclcpp/...>`)
4. ROS2 message includes (`<geometry_msgs/...>`, etc.)
5. Project includes (`"star/..."`)
6. System C++ includes (`<memory>`, `<string>`, etc.)

7. System C includes (`<cstdint>`, etc.)
8. Namespace declaration
9. Class/function declarations

See CLAUDE.md for complete example.

8.5 Code Formatting

8.5.1 Line Length

- **ROS2 C++**: 100 characters (enforced by `ament_uncrustify`, ROS2's official formatter)
- **C firmware**: 100 characters (`star-rx72n-firmware/.clang-format`)

Note: The ROS2 packages do NOT carry a `star-ros2/.clang-format` file. ROS2 uses `ament_uncrustify` (uncrustify with a bundled `ament_code_style.cfg`) and `ament_cpplint`, NOT clang-format. Running clang-format on `star-ros2/` sources will introduce drift that `ament_uncrustify -check` rejects in CI. To format ROS2 code locally, source ROS2 and run `./scripts/ros2/format-ros2.sh`.

8.5.2 Indentation

All code uses 2-space indentation (same as C firmware):

```
class MyNode : public rclcpp::Node { public: MyNode() :
Node("my_node") { timer_ = this->create_wall_timer(
std::chrono::milliseconds(100),
std::bind(&MyNode::timerCallback, this));
}

private:
void timerCallback() {
// ...
}

rclcpp::TimerBase::SharedPtr timer_;
};
```

Listing 6: Indentation example

8.5.3 Braces

- **Functions**: Braces on new line (same as C firmware)
- **Control statements**: Cuddled braces (`if (x) {`)
- **Namespaces**: No indentation inside namespace

```
// Functions: Braces on new line
void myFunction()
{
// ...
```

```
}

// Control statements: Cuddled braces
if (condition) {
// ...
} else {
// ...
}

// Namespaces: No indentation inside
namespace star {
namespace spi_bridge {

// Content at zero indent
class MyClass {
};

}  // namespace spi_bridge
}  // namespace star
```

Listing 7: Brace style examples

8.6 ROS2-Specific Patterns

8.6.1 Node Inheritance

Inherit from `rclcpp::Node` for basic nodes or `rclcpp_lifecycle::LifecycleNode` for safety-critical nodes:

```
// Basic node
class StarGatewayBridgeNode : public rclcpp::Node {
public:
StarGatewayBridgeNode();

private:
void telemetryCallback();

rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
rclcpp::TimerBase::SharedPtr telemetry_timer_;
};

// Safety-critical lifecycle node
class StarSpiDriverNode : public rclcpp_lifecycle::LifecycleNode {
public:
using CallbackReturn =
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;

StarSpiDriverNode();

// Lifecycle transitions
CallbackReturn on_configure(const rclcpp_lifecycle::State &) override;
CallbackReturn on_activate(const rclcpp_lifecycle::State &) override;
CallbackReturn on_deactivate(const rclcpp_lifecycle::State &) override;
CallbackReturn on_cleanup(const rclcpp_lifecycle::State &) override;
```

```
};
```

Listing 8: Node inheritance

8.6.2 Publishers and Subscribers

```
class MyNode : public rclcpp::Node { public: MyNode() :
Node("my_node") {
// Publisher: use SharedPtr
telemetry_pub_ = this->create_publisher<std_msgs::msg::String>(
"/telemetry",
10  // QoS depth
);

// Subscriber: use std::bind
cmd_sub_ = this->create_subscription<geometry_msgs::msg::Twist>(
"/cmd_vel",
10,
std::bind(&MyNode::cmdVelCallback, this, std::placeholders::_1)
);
}

private:
void cmdVelCallback(const geometry_msgs::msg::Twist::SharedPtr msg) {
// Handle message
}

rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_sub_;
};
```

Listing 9: Publishers and subscribers

8.6.3 Timers

Use `create_wall_timer` for periodic operations:

```
timer_ = this->create_wall_timer(
std::chrono::milliseconds(100),  // 10 Hz
std::bind(&MyNode::timerCallback, this)
);
```

Listing 10: Timer usage

8.7 Error Handling

ROS2 uses exceptions (different from C firmware return codes):

```
// Throw exceptions for errors
void publishTelemetry()
{
if (!grpc_channel_) {
throw std::runtime_error("gRPC channel not initialized");
```

```
}
// ...
}

// Catch exceptions in callbacks (avoid crashing node)
void timerCallback()
{
try {
publishTelemetry();
} catch (const std::exception & e) {
RCLCPP_ERROR(this->get_logger(), "Failed to publish: %s", e.what());
}
}
```

Listing 11: Exception handling

8.8 Logging

Use `RCLCPP_*` macros for logging, not `printf` or `cout`:

```
// ROS2 logging macros (preferred)
RCLCPP_INFO(this->get_logger(), "Node started");
RCLCPP_WARN(this->get_logger(), "Connection lost, retrying...");
RCLCPP_ERROR(this->get_logger(), "Failed to initialize: %s",
    error_msg.c_str());
RCLCPP_DEBUG(this->get_logger(), "Processing message %d", count);

// Throttled logging (max once per 5 seconds)
RCLCPP_WARN_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
"Command stale (%ldms > %dms)", cmd_age_ms, timeout_ms_);
```

Listing 12: ROS2 logging

Important: Never use `printf()` or `std::cout` in ROS2 nodes.

8.9 Documentation

All public APIs must have Doxygen comments using `/**` and `/**<` (same as C firmware):

```
/**
* @brief Brief description of class
*
* Detailed description can span multiple lines. Explain purpose,
* usage, and any important details.
*/
              class StarGatewayBridgeNode : public rclcpp::Node {
  public:
    /**
     * @brief Constructor for gateway bridge node
     *
     * @param options Node options for configuration
     */
    explicit StarGatewayBridgeNode(
        const rclcpp::NodeOptions &options = rclcpp::NodeOptions());

```

```
  private:
    /**
     * @brief Callback for telemetry publishing timer
     *
     * Forwards latest telemetry to Gateway via gRPC. Called at 10 Hz.
     */
    void telemetry_timer_callback();

    rclcpp::TimerBase::SharedPtr telemetry_timer_; /**< Timer for
        telemetry */
};
```

Listing 13: Doxygen documentation

8.10 Constants

Prefer enums and `const`(same philosophy as C firmware) :

```
        // Enum for related constants
        enum class MotorState {
          kIdle = 0,
          kRunning = 1,
          kError = 2
       };

// const for single values
const int DEFAULT_QOS_DEPTH = 10;
const double MAX_LINEAR_VELOCITY_MPS = 1.0;

// static const for class-specific constants
class MyNode : public rclcpp::Node {
private:
  static constexpr int kMaxRetries = 3;
  static constexpr double kTimeoutS = 5.0;
};
```

Listing 14: Constants

8.11 Formatting Enforcement

All ROS2 C++ code uses `clang - format` with configuration in `star - ros2 /.clang - format` :

- 2-space indentation (same as C firmware)
- 120-character line limit (ROS2 standard)
- Cuddled braces for control statements
- Function braces on new line
- No indentation inside namespaces
- ROS2-specific include order (core, messages, system, project)

8.11.1 Manual Formatting

Format code before committing :

```
cd star -
     ros2 find src - name '*.cpp' - o - name '*.hpp' |
     xargs clang - format - i
```

8.11.2 CI Enforcement

The `.github / workflows / ros2.yml` workflow runs `clang - format` checks on all PRs :

```
find src -
     name '*.cpp' - o - name '*.hpp' |
     xargs clang - format-- dry -
         run-- Werror
```

If formatting check fails, the build will fail with instructions to fix.

8.12 References

- **ROS2 C++ Style Guide** : https://docs.ros.org/en/rolling/The-ROS2-Project/Contributing/Code-Style-Language-Versions.html
- **ROS C++ Best Practices** : https://wiki.ros.org/CppStyleGuide
- **Google C++ Style Guide** : https://google.github.io/styleguide/cppguide.html
- **STAR C Firmware Style** : CLAUDE.md Section "Code Style"
- **STAR Project Documentation** : `docs / star_documentation.pdf`

8.13 Integration with Existing Tools

The ROS2 C++ style guide integrates seamlessly with existing STAR project tools :

Tool	Purpose	Configuration
`clang - format`	Code formatting	`star - ros2 /.clang - format`
`ament_cppcheck`	Static analysis	ROS2 workflow
`ament_cpplint`	Style checking	ROS2 workflow
`colcon build`	Build system	CMakeLists.txt
`colcon test`	Unit testing	gtest integration
GitHub Actions	CI / CD enforcement	`.github / workflows / ros2.yml`

Table 33: ROS2 Development Tools

8.14 Summary

The STAR project maintains separate but complementary style guides for C firmware and C++ ROS2 code:

- **C firmware** follows embedded best practices with strict NASA Power of 10 compliance

- **ROS2 C++** follows ROS community conventions while maintaining STAR project philosophy
- Both styles share common principles : enums over macros, error checking, documentation
- Automated tooling enforces consistency across all code

For complete examples and additional details, refer to CLAUDE.md.

9 Communication Stack

9.1 Overview

The STAR platform exposes three independent communication paths between the Raspberry Pi 5 and the RX72N motor controller. All three carry the same frame-level protocol described in §1. The host side picks whichever it needs; the firmware multiplexes them via `rx_comm_manager`.

Path	Host consumer	Link
USB CDC (USB0)	`star-gateway`, log viewers	USBb → `ttyACM{1,2,3}` (PROD) or `{4,5,6}` (TOM)
UART-over-USB	`star-gateway` (Go, gRPC / WS)	SCI9 → CY7C65213 → `ttyUSB0`
SPI	`star_spi_bridge` (ROS2)	RSPI2 → `spidev0.0`

USB CDC is the primary transport; it bench-verified working on both PROD (crystal clock) and TOM (HOCO-PLL clock) on 2026-04-24. SCI9 over the CY7C65213 bridge chip remains as a fallback / bring-up path; it is independent of the RX72N's USB peripheral state, which makes it useful during clock-tree debug. The SPI path is dedicated to the ROS2 motor-control bridge.

9.2 Why three paths

- **USB CDC (3-port)** is plug-and-play, supports the composite-device pattern (PROTO / DECODED / LOG ports), and gives the gateway a single USB cable for both command/telemetry and runtime logs. It is the path the gateway prefers when present.
- **UART-over-USB** via the CY7C65213 bridge is independent of the RX72N USB controller, so it stays alive across USB peripheral resets. Useful for bring-up, single-port host tools, and CI rigs without a working USB CDC stack.
- **SPI** delivers sub-millisecond, jitter-bounded round-trips for closed-loop 100 Hz velocity control on four wheels. It also permits hardware FEC and Chase Combining HARQ under the PWM-noisy motor environment.

9.3 Layers

Layer	Responsibility	Implementation
5 Application	Command handlers, telemetry aggregation	Gateway services, ROS2 nodes
4 Serialization	Protocol Buffers	generated Go / C++ / nanopb
3 Link (SPI)	FEC + Chase Combining, ARQ retries	`rx_spi_link`, gateway FEC
2 Framing	SYNC + SEQ + LEN + TYPE + FLAGS + CRC-32	`rx_frame`, `internal/frame`
1 Transport	USB CDC, UART @ 921 600, or SPI @ 10 MHz	`rx_usb`, `rx_uart_comm`, `rx_spi_comm`

The USB CDC and UART paths skip Layer-3 because the host-side USB stack (or the CY7C65213 bridge plus USB CDC driver) already provides lossless, in-order byte delivery; the framing-layer CRC alone is sufficient. The SPI path uses the full link layer because SPI itself is ECC-free and sensitive to motor-driver EMI.

9.4 Log multiplexing

The USB CDC composite device dedicates port 3 to firmware runtime logs, so log lines never share a byte stream with framed protocol data on USB. On the SCI9 UART path – which is single-port – log frames are inlined into the same byte stream as protobuf frames and distinguished by the TYPE byte:

- TYPE $\in \{0x10, 0x11, 0xFF, \ldots\}$: application or control frames, payload is protobuf-encoded or empty.
- TYPE $= 0x20$ (`LOG_MESSAGE`): payload is ASCII log text emitted by the RX72N's `rx_log_*` macros. The gateway extracts the payload and writes it to its own log stream with a `[RX72N]` prefix.

SPI does NOT carry log frames. The firmware sets `channel = k_comm_channel_uart` on every log emission in `comm_task.c`, and the ROS2 SPI driver's `is_valid_frame_type()` rejects `0x20` on the SPI wire (see `star_spi_bridge/src/spi_driver.cpp`). Use UART or the USB CDC log port for runtime log capture; SPI is reserved for protobuf command/response and HARQ control frames only.

There is no risk of the sync word `0x55AA` appearing inside log text and fooling the decoder because log bytes never leave the framing layer as raw bytes – every log chunk is a complete, CRC-checked frame.

10 USB Modes and Bench Behavior

10.1 Transport Selection at the Gateway

The Go gateway picks a transport via the `TransportMode` enum defined in `star-gateway/internal/manager/types` Four modes are valid:

Mode	Wire-format invariant	When to use
`auto`	Same framed protocol, any transport	Default; prefers USB CDC, falls back to SPI
`force-usb`	Full HARQ + reset handshake on USB	Strict-host environments where SPI is intentionally off
`simple-usb`	Framed protocol, no Layer-3 HARQ	Pi 5 xHCI hosts where USB CDC is already lossless
`force-spi`	Full HARQ over SPI; USB hot-plug off	Closed-loop motor-control rigs that need bounded jitter

10.1.1 Simple USB mode

What it is: The gateway opens the USB CDC port (typically `/dev/ttyACM{1,2,3}` on PROD or `/dev/ttyACM{4,5,6}` on TOM) and treats the byte stream as already-reliable. It performs:

- CRC-32 frame validation as defense in depth.
- No reset handshake at the link layer.

- No strict sequence-validation (gaps are tolerated rather than triggering NACK + retransmit storms).
- Relaxed health monitoring (lower retry budgets, longer dead timers).

Why it is safe: USB 2.0 Full-Speed already delivers a CRC-16 on every packet and re-transmits transparently in the host controller. Layering HARQ on top is wasted work and introduces latency that is nontrivial under typical CDC ACM scheduling (every-frame ACK + retry budget). On a Pi 5 xHCI host, simple-usb has been bench-verified to enumerate cleanly and survive arbitrary disconnect/reconnect cycles.

When to avoid: Strict-host environments (some Windows HCDs, some macOS releases, embedded controller hubs) may be picky enough about USB framing or clock accuracy that the loss-tolerant assumption fails. In that case prefer `force-usb` (or fall back to SPI).

10.1.2 Complex (full) USB mode – `force-usb`

This is the original transport design: a full link-layer state machine with reset handshake (RST/RST-ACK), strict 16-bit sequence checking, NACK-driven retransmits with Chase-Combining HARQ, and deadline monitors. It is the canonical mode on hardware that also has the SPI link active and where deterministic recovery is more important than raw throughput.

10.1.3 Forcing SPI – `force-spi`

Used by the ROS2 motor-control bridge (`star_spi_bridge`). USB hot-plug is disabled so that wandering USB cables can never knock the closed-loop control off the deterministic SPI path. The full HARQ stack runs on this transport because SPI itself has no error correction and PWM/motor noise is real.

10.2 USB enumeration topology

The RX72N exposes a single composite USB device that the host sees as three CDC-ACM serial endpoints in a single `idVendor / idProduct` tuple. Bench-verified mappings (commit `341bd09ce` on `feat/usb0`, 2026-04-24):

- **PROD board** (E2L `5AS071312B`, 24 MHz crystal): `/dev/ttyACM1` = PROTO, `/dev/ttyACM2` = DECODED, `/dev/ttyACM3` = LOG.
- **TOM board** (E2L `5AS071321B`, HOCO PLL): `/dev/ttyACM4` = PROTO, `/dev/ttyACM5` = DECODED, `/dev/ttyACM6` = LOG.
- Both enumerate as `idVendor=045b idProduct=0235`, and both heartbeat on all three ports under `p0/p1/p2`.

10.3 Re-enumeration: when ttyACM* changes

The OS assigns `/dev/ttyACM*` indices in order of appearance. Anything that drops the device and re-enumerates it can shift the mapping. Common causes on the bench:

- Reflash via E2 Lite – the firmware re-init cycles USB power and the USBb peripheral, so the host sees a disconnect followed by a fresh enumerate. The new `ACM` numbers usually come back in the same order, but if another USB-CDC device was plugged in between cycles, indices can advance.

- Power-cycling the board (USB unplug or barrel-jack toggle).
- Hot-plugging another CDC ACM device (an Arduino, an ESP32, another RX72N) while the gateway is running.

To make the assignment stable, install `scripts/71-star-symlinks.rules` via `scripts/setup-pi5-host.sh`. That udev rule creates persistent symlinks (e.g. `/dev/star-rx72n-proto`, `/dev/star-rx72n-decoded`, `/dev/star-rx72n-log`) keyed on the device's serial / interface index, so the gateway can open those symlinks regardless of what `/dev/ttyACM*` number the kernel happens to pick on a given boot.

10.4 Bench JTAG interaction (E2 Lite vs CY7C65213)

On the STAR bench, the Renesas E2 Lite debugger and the Cypress CY7C65213 USB-UART bridge are mutually exclusive on the host:

- While the E2 Lite is attached and holding the JTAG/FINE interface, the CY7C65213 does *not* enumerate as `/dev/cu.usbmodem*` or `/dev/cu.usbserial-*`. This is bench-harness wiring (shared USB power gating), not a fault.
- After flashing, physically disconnect the E2 Lite to expose the bridge to the host before running picocom or starting the gateway against the SCI9 fallback path.
- The USB0 path (`/dev/ttyACM*`) is not affected by E2 Lite presence – it runs on the RX72N USBb peripheral on the dedicated USB-C connector and is independent of SCI9.

10.5 Clock-source notes (TOM vs PROD)

The TOM board has no external main crystal and runs USB0 from HOCO → PLL (PLLSRCSEL=1, multiplier 12, `k_pll_multiplier_12_hoco = 0x1710`). The Renesas HUM explicitly recommends an 8–24 MHz crystal as the USB clock source (HUM page 345 PLLCR Note 1, section 9.1, Table 63.18). HOCO is ±2.44 % accurate at $T_a \geq -20°C$; USB 2.0 Full-Speed is specified at ±0.25 %, so HOCO is roughly 10x off-spec on paper.

In practice, the Pi 5's xHCI controller accepts the HOCO-driven clock without complaint, and TOM enumerates the 3-port CDC composite cleanly. Treat the HUM warnings as a portability risk (some macOS revisions or Windows HCDs may reject the clock source) rather than a blocking defect for ship-to-Pi deployments. For field deployments on unknown hosts, prefer the 24 MHz crystal-clocked PROD configuration.

10.6 References

- `star-gateway/internal/manager/types.go` – transport mode definitions.
- `star-rx72n-firmware/libs/rx_usb/` – USBb peripheral driver.
- `scripts/71-star-symlinks.rules` – udev rules for stable `/dev/star-rx72n-*` symlinks.
- `scripts/setup-pi5-host.sh` – one-time Pi 5 host setup that installs the rules above plus the USB autosuspend overrides.

11 Repository Layout: A Newcomer's Map

This section is a guided tour of the STAR repository. It is intended to be the first thing a new developer reads, before they need to find any specific file.

11.1 Project shape, in one paragraph

STAR is a four-language distributed system: a real-time motor-control firmware (C, Renesas RX72N + ThreadX), a Go gateway service (running on a Raspberry Pi 5), a ROS2 Jazzy autonomy workspace (C++, also on the Pi 5), and a TypeScript operator UI. They communicate via a shared Protocol Buffers schema (`star-proto/`) over USB CDC, SCI9 UART, or SPI. A separate Python ROS2 package (`star-compliance/`) consumes sensor data to generate ADA compliance audit reports. The repository also contains the KiCad PCB sources, the bench / Pi-deployment infrastructure scripts, the capstone deliverable PDFs, and the engineering documentation (including this document).

11.2 Top-level layout

Path	Role	What it contains
`star-rx72n-firmware/`	Embedded firmware (C, RX72N, ThreadX)	Production firmware tree: `src/`, `libs/` (26 component libraries), `tests/` (Unity host tests), bench-test sandboxes, build scripts. Largest single subproject in the repo.
`star-gateway/`	Pi 5-side Go service	gRPC + WebSocket bridge between the UI/ROS2 and the RX72N. Implements the wire protocol's framing, HARQ, FEC, and transport-mode selection (`auto` / `force-usb` / `simple-usb` / `force-spi`).
`star-ros2/`	ROS2 Jazzy workspace (C++)	Autonomy + safety stack on the Pi 5. Packages: `star_bringup` (launch orchestration), `star_supervisor` (e-stop/autonomy gating), `star_safety_monitor`, `star_gateway_bridge`, `star_spi_bridge`, `star_simple_bridge`, `star_simulation`, `star_perception`.
`star-proto/`	Wire protocol contract	`.proto` schemas in `proto/star/v1/`, generated code under `gen/` (per language), nanopb config in `nanopb/`. The single source of truth for every message that crosses a process boundary.
`star-ui/`	Operator UI (TypeScript / React + Vite)	Browser-based cockpit. Talks to the gateway over WebSocket. Includes telemetry panels, joystick / gamepad input, PID tuning UI, transport diagnostics.
`star-compliance/`	ADA compliance engine (Python ROS2 pkg)	Detectors (door, ramp, jamb, obstacle), engines (plane segmentation, floor frame), nodes (compliance_monitor, ramp_slope, etc.), report PDF generator. Runs on the Pi 5 alongside ROS2.
`matlab/`	Motor system identification	First / second-order motor models, PID design scripts, discretization, robustness sweeps. Output is the gain set the firmware compiles in.
`schematic/`	KiCad PCB sources	STAR_MCU schematic + layout, gerbers, symbol / footprint libraries, TOM breakout. The hardware ground truth.
`final_docs/`	Capstone deliverables	SDD (Software Description Document), SSUM (System and Software User Manual), Test Report, BOM, schematic exports, mechanical designs, presentation deck and demo handout, all as compiled PDFs plus their LaTeX / pptx sources.
`docs/`	Engineering documentation	This file plus every other engineering reference: `ARCHITECTURE.md`, `PI_DEPLOYMENT.md`, ADRs, bench wiring, the compiled `star_documentation.pdf`, and the section sources you are reading right now.
`infra/`	Pi 5 system files	Caddy reverse-proxy config, systemd

11.3 Top-level files

File	Purpose
`README.md`	60-second project overview. Points to this document for depth.
`CLAUDE.md`	Project-wide coding policy: terminology, no-backward-compatibility rule, ASCII-only mandate, NASA Power of 10, Doxygen requirements, embedded-bring-up discipline rules. Loaded automatically by Claude Code.
`LICENSE`	MIT.
`Makefile`	Top-level dev-loop targets: `build-rx72n`, `test-rx72n`, `coverage-rx72n`, `proto-gen`, `doxygen-html`, `ci-rx72n`, dev-container wrappers, plus the bare-metal `blinky` flow and the `motorN-forward,reverse` flash targets.
`Dockerfile`	Devcontainer base image (Debian + GNURX + ROS2 + Go + Node + buf + nanopb). Pulled by `.devcontainer/devcontainer.json`.
`start.sh` / `stop.sh`	One-command robot launch and shutdown on the Pi 5.
`dev.sh`	Wraps `start.sh` with `STAR_SIMULATION_MODE=true -rviz` for desk-side dev with the virtual RX72N.
`bootstrap_pi.sh`	First-time Pi 5 setup. Installs ROS2, Go, buf, golangci-lint, the compliance engine.
`build-ros2.sh`	Regenerates protobuf + colcon-builds the ROS2 workspace.
`test-ros2.sh`	Runs `colcon test` for the workspace.
`run-docker-build.sh`	Host-side wrapper that builds the devcontainer image and runs `build-ros2.sh` inside it.
`go.work`	Multi-module Go workspace: `star-gateway`, `star-proto/gen/go`, `star-proto/tests/go`.
`buf.gen.yaml`	Lives at `star-proto/buf.gen.yaml`. The repo-root copy was deleted; use the canonical one.
`STAR.code-workspace`	VS Code multi-root workspace (firmware + ROS2 + proto + gateway + UI + compliance).
`qodana.yaml` / `.coderabbit.yaml` / `.golangci.yml` / `.pre-commit-config.yaml` / `.clang-format` / `.dockerignore` / `.gitattributes` / `.gitignore` / `.gitmodules`	Linter and tool configuration. Most are unsurprising. Two non-obvious bits: `.gitattributes` declares the LFS-tracked installers and `.f3z` CAD files, and `.gitmodules` declares two submodules (nanopb and the Slamtec sllidar SDK).

A small number of large vendor-blob installers also live at repo root: `gcc-14.2.0.202511-GNURX-ELF.run` (215 MB, GNURX cross compiler) and `RFP_CLI_Linux_V32200_x64.tgz` (53 MB, Renesas Flash Programmer CLI). Both are tracked via Git LFS and are required by `Dockerfile` (which fetches them from the GitHub raw URL) and by the `firmware-toolchain-image` CI workflow.

11.4 Inside `star-rx72n-firmware/`

Path	What's there
`src/`	Production firmware source. `src/main.c` owns startup; `src/tasks/` holds ThreadX tasks (motor_control, comm, telemetry, imu, temp_sensor, obstacle_detect, led_status, watchdog_monitor, serial_bringup, usb); `src/shared/` holds shared_data; `src/boot/` the SMC-generated boot vectors and BSP.
`libs/`	Twenty-six in-house C libraries (see §11.4.1) plus the vendored Azure ThreadX RTOS in `libs/threadx/`. Every `rx_*` lib has `inc/` (public header), `src/` (impl), and is built as part of the .elf.
`tests/`	Host-compiled Unity test suite (60 binaries, 100 % line/function/branch coverage gate on the `libs/` subset). Built with the system Clang; uses the host CMakeLists in `tests/CMakeLists.txt`.
`cmake/`	Cross toolchain file (`toolchain-rx72n.cmake`) – pulled in by the firmware`CMakeLists.txt` for the GNURX build.
`scripts/`	Firmware-local scripts: linker-section dumpers, Doxygen helpers, the AD2 capture harness for `gpio_test/`.
`docs/`, `Doxyfile.main`, `Doxyfile.pdf.base`, `latex-extras/`	Doxygen sources for the firmware-only HTML and PDF reference. Runs from the top-level Makefile target `doxygen-html` / `doxygen-pdfs`.
`blinky/`, `blinky_rtos/`, `encoder_test/`, `encoder_sim/`, `gpio_test/`, `imu_test/`, `motor_spin_test/`, `motors_rtos/`, `pwm_test/`, `pwm_test_fit/`, `pwm_test_hal/`, `uart_test/`	Bench-test sandboxes. Each is a self-contained bare-metal or RTOS firmware that drives one peripheral end-to-end. Useful both for bring-up and as known-good reference init code per CLAUDE.md "trust known-good bench tests over fresh code."
`STAR Debug.launch` / `STAR HardwareDebug.launch` / `star-rx72n-firmware.elf.launch` / `STAR.scfg` / `BOARD_CONFIG.md`	e2 studio Smart Configurator project + JTAG / FINE / E2 Lite debug-launch shims. Used by Renesas e2 studio if anyone needs to step through the firmware.

11.4.1 The 26 firmware libraries

Every library follows the same layout: `libs/rx_<name>/inc/` public header, `libs/rx_<name>/src/` implementation. Mock versions live under `tests/mocks/`. Categorized:

Group	Libraries
Core utilities	`rx_core` (logging, error codes, check macros), `rx_crc` (CRC-32 with hw / sw backends), `rx_dmaca` (DMAC channel allocator).
HAL	`rx_hal` (PORT / MPC / SCI / RIIC / RSPI / GPTW / MTU / TPU / CMT / TMR / IWDT / POEG / CAC / DOC / ECCRAM / LPC / MPC / DTC accessors and inline init code).
Bus abstractions	`rx_bus` (bus-manager interface), `rx_i2c_comm`, `rx_spi_comm`, `rx_uart_comm`, `rx_comm_manager` (multi-transport multiplexer).
Wire protocol	`rx_frame` (binary framer), `rx_frame_ascii` (fallback ASCII framer for debug), `rx_session` (sequence numbers + reset handshake), `rx_spi_link` (HARQ link layer over SPI), `rx_fec` (rate-1/2 K=7 convolutional FEC with Viterbi), `rx_harq` (Chase-Combining HARQ state machine), `rx_nanopb` (the vendored nanopb runtime + generated message code).
USB CDC	`rx_usb` (USB0 USBb peripheral driver: enumeration, endpoints, 3-port CDC composite).
Motor / sensors	`rx_motor` (DRV8263 + GPTW PWM + current sense), `rx_drv8263` (chip-level driver), `rx_encoder` (MTU and TPU phase-counting backends), `rx_pid` (pure PID algorithm), `rx_obstacle_detect`.
External devices	`rx_bmp280` (I2C baro), `rx_bno055` (I2C IMU), `rx_ds18b20` (1-wire temp), `rx_hcsr04` (sonar trig/echo via ICU).
RTOS	`libs/threadx/` – vendored Azure ThreadX. Treat as read-only; updates come from upstream.

11.4.2 Where firmware tasks live

Task	Role
`motor_control_task.c`	100 Hz closed-loop PID on all four wheels. Reads encoder velocities, runs PID, writes duty cycles to DRV8263.
`comm_task.c`	RX path: parse incoming frames, dispatch to handlers (set_velocity, ping, reset, etc.). Active over USB CDC + UART + SPI via comm-mgr.
`telemetry_task.c`	TX path: aggregate motor / IMU / baro / temp / sonar data, serialize, send.
`imu_task.c`	Polls the BNO055 over RIIC0; publishes to shared_data.
`temp_sensor_task.c`	DS18B20 1-wire temperature poll.
`obstacle_detect_task.c`	HC-SR04 trigger / echo state machine; publishes per-sonar range_mm and obstacle bool.
`led_status_task.c`	Heartbeat LED + comm-pulse LED on `rx_log` traffic.
`watchdog_monitor_task.c`	Kicks the IWDT, monitors stack high-watermarks, asserts on starvation.
`serial_bringup_task.c`	ASCII bring-up CLI on SCI9 (only enabled in bring-up builds).
`usb_task.c`	USB CDC enumeration + 3-port composite-device service loop.

11.5 Inside `star-gateway/`

Standard Go layout: `cmd/` for binaries, `internal/` for non-exported packages, `test/` for cross-package tests.

- `cmd/star-gateway/` – the main daemon. Other `cmd/` entries (`cmd_debug`, `ping_test`, `vel_test`, `ascii_test`, `cmd_listen`, `sensor_dump`, `virtual_rx72n`) are debug / bring-up tools.
- `internal/transport/` – byte-stream layer (USB CDC + SPI).
- `internal/frame/` – framing layer (SYNC + CRC + decoder with resync).
- `internal/fec/` – rate-1/2 K=7 convolutional encoder, Viterbi decoder, soft-bit Chase combiner.
- `internal/harq/` – HARQ state machine (Chase Combining Type I).
- `internal/dispatcher/` – routes typed frames to service handlers.
- `internal/manager/` – transport-mode selection, hot-plug, simple-USB vs full HARQ.
- `internal/server/` – gRPC + WebSocket + HTTP servers.
- `internal/controller/` – joystick / Twist conversion.
- `internal/app/` – top-level wiring of all of the above.
- `test/e2e/` – end-to-end HIL tests.

11.6 Inside star-ros2/

Standard ROS2 / colcon workspace.

Package	Role
star_bringup	Launch orchestration: slam.launch.py / slam_mvp.launch.py, static_transforms.launch.py, explore.launch.py (frontier exploration), stereo_camera.launch.py.
star_supervisor	E-stop / autonomy gating between /cmd_vel (manual), /nav2/cmd_vel (autonomous), and /cmd_vel_out (hardware bridge). Dual-enforced with the firmware-side e-stop.
star_safety_monitor	Lifecycle node consuming sonar topics; publishes /star/estop on threshold breach.
star_gateway_bridge	ROS2 -> Go-gateway shim. Wraps the gateway's gRPC API as ROS2 publishers/subscribers/services.
star_spi_bridge	Direct SPI-to-RX72N bridge (skips the gateway). Used when ROS2 needs the deterministic SPI link.
star_simple_bridge	Lightweight HTTP/static map server (/map.{png,pgm,yaml}, POST /map/reset?confirm=yes, custom metrics).
star_compliance_msgs	Message + service definitions used by star-compliance/.
star_perception	Hailo-accelerated perception (door-state classifier, object detection). Models live under src/star_perception/models/.
star_simulation	Gazebo simulation launch + world files, used by dev.sh.
sllidar_ros2/	Vendored Slamtec RPLiDAR SDK / driver (third party, do not modify).

11.7 Inside star-proto/

- proto/star/v1/*.proto – canonical schemas. Field numbers, enum zero values, units (_mps, _rad, etc.) are governed by the Boston-Dynamics-style guide in docs/sections/04_style_guide.tex.
- nanopb/star/v1/*.options – nanopb size-budget config for firmware codegen.
- gen/ – generated code per language. *Not* committed by default (per star-proto/gen/.gitignore); regenerated by make proto-gen or buf generate.
- tests/go/, tests/typescript/, tests/nanopb/ – per-language round-trip tests.
- scripts/ – proto-gen drivers.
- buf.gen.yaml / buf.gen.web.yaml – buf codegen configuration. The repo-root buf.gen.yaml that used to also exist was redundant and has been removed.

11.8 Inside `scripts/`

Path	What's there
`git/`	Tracked Git hooks. Opt in with `git config core.hooksPath scripts/git`.
`ros2/`	ROS2 build / format / review scripts: `format-ros2.sh` (uncrustify), `format-ros2-docker.sh`, `review-ros2.sh`, `fix-header-guards.sh`, `cpplint-on-save.sh`.
`proto/`	Proto-gen drivers and the `verify-nanopb.sh` sanity check.
`utils/`	Encoding fixer (`fix-encoding.py` – the ASCII-only enforcer), copyright check, doxygen helpers.
`devtools/`	Misc dev-loop helpers (e.g. `run-docker-build.sh` wrapper helpers).
`clion/`	CLion / IDE integration helpers (compile-commands rewriter).
`door_state_training/`	YOLOv8 training pipeline for the door-state classifier consumed by `star-compliance`.
`pi5_gpio_test/`	Pi 5-side GPIO sweep used during Pi-side bring-up.
`flash-rx72n.sh`	Firmware flash via E2 Lite (`rfp-cli`).
`slam-mvp.sh`	SLAM minimum-viable-product launcher (`start` / `start-nav` / `stop`).
`setup-pi5-host.sh`	One-time Pi 5 host setup: udev rules, CPU governor, log rotation, optional boot service.
`setup-lichtblick-web.sh`	Cockpit web bundle setup.
`star-ap.sh` / `star-eth.sh`	Wi-Fi AP and Ethernet helpers for field deployment.
`star-help.sh`	Operator help screen (the "what can this robot do" cheat sheet).
`give-head.sh` / `take-head.sh`	Branch-state transfer helpers.
`merge-verification.sh`	Pre-merge gate that runs the full firmware + gateway + ROS2 test matrix.
`verify-parallel.sh`	Multi-suite parallel test runner.
`71-star-symlinks.rules`	udev rules for stable `/dev/star-rx72n-*` symlinks (installed by `setup-pi5-host.sh`).

11.9 Inside `final_docs/`

The capstone-deliverable tree, organized by submission category:

- `software_description_document/` – SDD (LaTeX + compiled PDF).
- `system_software_user_manual/` – SSUM.
- `test_report/` – the IEEE-29119-3 test report.
- `bill_of_materials/` – BOM with cost totals.
- `electrical_schematics_pcb/` – PDF exports of the schematic, electrical design, and the PCB stack-up.
- `mechanical_designs/` – chassis CAD (`.f3z` via LFS), Gazebo meshes (.stl), exported PDFs.

- `final_demo/` – handout PDF + demo script + 5-minute demo timing + contingency plan.
- `final_presentation/` – slide deck (.pptx + PDF) and the chart generators in `charts/` that feed into it.

11.10 Where do I look if I want to...?

Goal	Start here
Add a new ROS2 topic the gateway publishes	`star-gateway/internal/server/` + `star-ros2/src/star_gateway_bridge/`
Add a new wire-protocol message	`star-proto/proto/star/v1/` (.proto) + add nanopb config in `star-proto/nanopb/star/v1/` + run `make proto-gen`
Tune motor PIDs	`matlab/cascaded_pid_design.m`, then update `star-rx72n-firmware/libs/rx_motor/inc/rx_motor.h` config defaults
Add a new GPIO peripheral	`star-rx72n-firmware/src/inc/hardware_config.h` (pin map) + new lib under `star-rx72n-firmware/libs/rx_<peripheral>/`
Trace a USB CDC bug	`star-rx72n-firmware/libs/rx_usb/` + §1 + §11
Find the active firmware build	`star-rx72n-firmware/build/` (cross) or `star-rx72n-firmware/build-tom/` (TOM variant). Both are git-ignored.
Bench-test a new peripheral in isolation	Copy a sibling `star-rx72n-firmware/<peripheral>_test/` dir, edit `main.c`
Run host-side unit tests	`cd star-rx72n-firmware/tests && cmake -B build -S . && cmake -build build && ctest -test-dir build`
Run ROS2 tests	`./test-ros2.sh`
Run Go gateway tests	`cd star-gateway && go test -race ./...`
Re-flash firmware on the bench	`scripts/flash-rx72n.sh` (E2 Lite + rfp-cli)
Start the robot in simulation	`./dev.sh`
Start the robot on real hardware	`./start.sh`
Bring up a new Pi 5	`./bootstrap_pi.sh`, then `sudo bash scripts/setup-pi5-host.sh`
Find the canonical pinout	§?? (this document)
Find the wire protocol	§1
Find the e-stop / autonomy semantics	`docs/ARCHITECTURE.md` "supervisor safety model"
Find the bench-rig wiring	`docs/bench/ad2_wiring.md`
Read the operator runbook	`docs/PI_DEPLOYMENT.md`
Read the official capstone deliverables	`final_docs/` (each subdir has a `README.md` pointing at the primary PDF)

11.11 Branch + commit conventions

The repository uses linear-history Git on `main`, with no backward-compatibility shims (see `CLAUDE.md` – "ZERO backward compatibility requirements"). Breaking changes are encouraged when they im-

prove quality. CI gates each PR with firmware unit tests, ROS2 build/test, gateway test, proto regen verification, copyright header check, ASCII-only check, and a clang-format / clang-tidy pass.

11.12 Things this repository does not contain

To make the boundary clear:

- No vendor toolchains compiled in source – they are pulled by `Dockerfile` from LFS-tracked installers (GNURX + rfp-cli).
- No private secrets, credentials, or per-host configuration – secrets live in environment variables on the Pi 5; see `docs/PI_DEPLOYMENT.md`.
- No pre-built binaries committed to source control – `star-gateway/cmd/virtual_rx72n` is built locally via `go build`; the firmware `.elf` / `.mot` outputs land in `star-rx72n-firmware/build*/` which are git-ignored.
- No abandoned alternative implementations – the previous STM32 peripheral target and BeagleBone Blue secondary-compute target were removed in commit `b34f27c98`. Bring-up sandboxes that survived in `star-rx72n-firmware/` are still referenced from documentation or the Makefile; the rest were deleted in commit `e51306747`.

12 Boot Sequence and Clock Tree

This section walks through the RX72N's power-on sequence and the clock tree it ends up in. Anyone touching peripheral init, reset handling, or low-power transitions should read this first.

12.1 Power-on reset path

1. **Reset vector and OFS1.** On power-on the RX72N reads the OFS1 byte from option-setting memory (placed in the `.ofs1` section by `src/boot/smc/vecttbl.c`). OFS1 controls HOCO enable, IWDT autostart, and startup mode. STAR's OFS1 is configured so HOCO comes up automatically and IWDT is software-started (*not* autostart) so the firmware can program its timeout before the watchdog can fire.
2. **Custom boot.** `src/boot/` (alongside the Renesas-vendored SMC boot tree under `src/boot/smc/`) is responsible for setting up MSTPCR / register protection and getting to `main()` cleanly.
3. **Clock initialization.** `src/rx_clock_power_init.c` runs early in `main()`. It selects the clock source (crystal vs HOCO), programs the PLL, sets the sub-clock dividers, and unlocks `PRCR`-protected registers via the `rx_register_protection` helpers.
4. **Pin multiplexer release.** Once clocks are stable, `src/hardware_init.c` unlocks PFS / PWPR (the Renesas pin-protection register), routes every peripheral pin through the MPC PSEL table, and locks PFS again.
5. **Peripheral init.** Each library's `*_init()` runs in dependency order: HAL accessors first, then sensors, bus managers, actuators, and the wire-protocol stack.
6. **ThreadX scheduler.** `tx_application_define` creates each task's stack + thread, then `tx_kernel_enter` hands control to the scheduler. Every task has its own priority and stack size; nothing dynamic.

7. **IWDT armed.** The watchdog-monitor task arms the IWDT only after every other task has crossed its first deadline-monitored checkpoint, so a slow startup never trips the watchdog.

12.2 Clock tree

The RX72N has multiple parallel clock domains. Constants live in `libs/rx_hal/inc/rx72n_clock.h` as enums (`k_iclk_hz`, `k_pclka_hz`, `k_pclkb_hz`, `k_pclkd_hz`).

Clock	Rate	Source on PROD board	Source on TOM board
ICLK	240 MHz	24 MHz xtal → PLL	HOCO 16 MHz → PLL
PCLKA	120 MHz	ICLK / 2	ICLK / 2
PCLKB	60 MHz	ICLK / 4	ICLK / 4
PCLKD	60 MHz	ICLK / 4	ICLK / 4
FCLK	60 MHz	ICLK / 4	ICLK / 4
USB clk	48 MHz	main PLL via PACKCR.UPLLSEL=0	main PLL via PACKCR.UPLLSEL=0
IWDTCLK	15 kHz	dedicated low-speed oscillator	dedicated low-speed oscillator

12.2.1 Which peripheral runs on which clock

Domain	Peripherals	Notes
ICLK 240 MHz	CPU, Cache, ROM/RAM accessors	Driving the 100 Hz motor-
PCLKA 120 MHz	SCI7-SCI11 (extended SCIi), GPTW, CMT0/1, MTU3, TPU	Encoder phase counters, m
PCLKB 60 MHz	SCI0-SCI6 + SCI12, RIIC, RSPI, IWDT (rate-divider source)	SCI9 UART bring-up, hos
PCLKD 60 MHz	ADC12 (S12AD)	Motor current sense (IPR
USB 48 MHz	USB0 USBb peripheral	Sourced from main PLL v
IWDTCLK 15 kHz	IWDT	Independent of any other

Always trust the header. `rx72n_clock.h` is the single source of truth for these rates. Every BRR / divider / period math in the firmware derives from `k_pclkX_hz`. CLAUDE.md elevates this to a P0 rule: trust headers over comments. PCLKB is 60 MHz, not 120 MHz.

12.3 TOM vs PROD clock-source notes

PROD has a 24 MHz crystal as the main clock; TOM has no crystal and runs everything from HOCO → PLL (PLLSRCSEL=1, multiplier 12). HOCO accuracy is ±2.44 % at $T_a \geq -20°C$; USB 2.0 Full-Speed is specified at ±0.25 %. The Pi 5 xHCI host accepts the HOCO-driven clock and TOM enumerates the 3-port CDC composite cleanly. See §10 for the portability discussion.

12.4 Reset sources

The Reset Status Register 2 (`RSTSR2`) preserves the cause of the most recent reset across boot. STAR firmware reads it once during `main()` startup and logs the cause:

Bit	Source	Meaning
PORF	Power-on	Cold start
LVD0RF	LVD0	Voltage detection level 0 fired
LVD1RF	LVD1	Voltage detection level 1 fired
LVD2RF	LVD2	Voltage detection level 2 fired
SWRF	Software	Firmware called `SYSTEM.SWRR.WORD = 0xA501`
WDTRF	WDT	Watchdog reset
IWDTRF	IWDT	Independent watchdog reset
DPSRSTF	Deep Sleep	Wake from Deep Sleep mode
CWSF	Cold/Warm flag	Distinguishes cold vs warm boot

When `IWDTRF=1`, the firmware halts via `internal_rx_fatal_error` so an in-circuit debugger can inspect the stack from before the watchdog fired.

12.5 Where to look in code

Topic	Source
Clock-rate constants (the truth)	`star-rx72n-firmware/libs/rx_hal/inc/rx72n_clock.h`
Clock + power init (after reset)	`star-rx72n-firmware/src/rx_clock_power_init.c`
Pin-mux / PFS / PWPR unlock sequence	`star-rx72n-firmware/src/hardware_init.c`
OFS1 / boot vectors	`star-rx72n-firmware/src/boot/smc/vecttbl.c` (placed in `.ofs1` per linker script)
Reset-cause inspection	`star-rx72n-firmware/src/main.c` (RSTSR2 read + logging)
USB clock routing	`star-rx72n-firmware/libs/rx_usb/src/rx_usb_hw.c` (PACKCR.UPLLSEL, SCKCR2.UCK)

13 Build System and CI

This section explains how every artifact in the repository is produced, what gates the CI pipeline enforces, and how to reproduce each step locally.

13.1 Toolchain inventory

Tool	Version	Used to build
GNURX (rx-elf-gcc)	14.2.0.202511	Renesas RX72N firmware (`star-rx72n-firmware/`); LFS-tracked installer at `gcc-14.2.0.202511-GNURX-ELF.run`
Clang / system C compiler	18+ (host)	Host-side firmware unit tests
clang-tidy	16+	Optional firmware lint pass (`cmake -DENABLE_CLANG_TIDY=ON`)
clang-format	18	C / Go formatter for non-ROS2 code
ament_uncrustify	jazzy	ROS2 C++ formatter (see `docs/ROS2_FORMATTING.md`)
Go	1.26.2	`star-gateway/` + `virtual_rx72n` + proto Go output
golangci-lint	latest pinned in CI	Go static analysis
Buf	1.50+	Protobuf codegen (`star-proto/buf.gen.yaml`)
nanopb generator	0.4.x	Firmware protobuf codegen
ROS2 / colcon	Jazzy	Builds `star-ros2/`
Cyclone DDS	jazzy	DDS RMW middleware (`config/cyclonedds.xml`)
Doxygen	1.9+	Firmware reference HTML / PDF
KiCad CLI	10.0.1+	Schematic / PCB / BOM generation
Renesas Flash Programmer CLI	3.22	Firmware flash via E2 Lite (`RFP_CLI_Linux_V32200_x64.tgz`)

13.2 Devcontainer (the canonical build environment)

The `Dockerfile` at the repo root produces a Debian image with every cross-language toolchain pre-installed (GNURX, ROS2 Jazzy, Go, Node, buf, nanopb, Cyclone DDS). VS Code's "Reopen in Container" opens it; `run-docker-build.sh` wraps it for headless host use. The `firmware-toolchain-image.yml` CI workflow keeps a baked image warm at `ghcr.io/locked-inc/star/gnurx`.

GNURX is fetched at image-build time from the LFS-tracked installer via the `Dockerfile` `GNURX_INSTALLER_URL` ARG; rfp-cli the same way.

13.3 Top-level Makefile entry points

The repo-root `Makefile` is the documented dev loop. A selection:

Target	What it does
`make help`	Print every target with a one-line description.
`make build-image`	Build the devcontainer image.
`make build-rx72n`	Cross-compile the firmware in the devcontainer.
`make build-rx72n-release`	Same with `-DCMAKE_BUILD_TYPE=Release` – the canonical artifact for flashing.
`make test-rx72n`	Host-compile + run the Unity unit-test suite (60 binaries).
`make coverage-rx72n`	Same plus `-coverage`; runs gcovr against `libs/`.
`make format-rx72n`	Run clang-format against firmware sources.
`make check-rx72n`	Pre-merge firmware gate: build, test, coverage, format, ASCII, copyright.
`make ci-rx72n`	Same with the strictness CI uses.
`make proto-gen`	`buf generate` on `star-proto/proto/`.
`make proto-gen-firmware`	Subset: nanopb only.
`make proto-check-nanopb-sync`	Verify committed nanopb output matches `buf generate`.
`make doxygen-html`	Firmware HTML reference into `star-rx72n-firmware/docs/doxygen/html/`.
`make doxygen-pdfs`	Firmware PDF reference (one PDF per major library).
`make blinky`	Cross-compile + flash the bare-metal LED test via E2 Lite.
`make motorN-forward / -reverse`	Drive each individual motor open-loop via flashed bench firmware.
`make motor-stop`	Flash a do-nothing firmware to release all motors after a bench session.

13.4 Build-time feature flags

A small set of CMake options gate non-default behaviour. Pass them with `-D<NAME>=ON` on the configure line.

Flag	Default	Effect
`STAR_BENCH_PROBES`	OFF	Compile in the bench-only MPU-6050 WHO_AM_I probe and the data-flash erase/program self-test in `src/main.c`. Default OFF so production boots do not write data-flash on every reset.
`ENABLE_CLANG_TIDY`	OFF	Run `clang-tidy` as part of the host test build (matches the CI `clang-tidy (SEI CERT C)` workflow).

13.5 Per-subproject build

Subproject	How it builds
`star-rx72n-firmware/`	CMake + GNURX inside the devcontainer (`cmake -toolchain cmake/toolchain-rx72n.cmake`). Host tests via system Clang from `tests/CMakeLists.txt`.
`star-gateway/`	`go build ./...`; `go.work` resolves the proto-gen module.
`star-ros2/`	`./build-ros2.sh` – wraps `buf generate` + `colcon build -symlink-install`.
`star-proto/`	`buf generate`; output in `star-proto/gen/`.
`star-ui/`	Vite (`npm install && npm run build`).
`star-compliance/`	Setuptools + colcon when used as a ROS2 package.
`schematic/`	KiCad CLI for BOM / PDF / Gerber regeneration.
`docs/`	`cd docs && make` drives `pdflatex star_documentation.tex` (twice for TOC).

13.6 CI workflows

The active workflows live under `.github/workflows/`.

Workflow	What it gates
`firmware-toolchain-image.yml`	Rebuilds the GNURX devcontainer image (LFS pull, Docker build, push to GHCR).
`firmware-build-verify.yml`	Cross-compiles the Release build inside the toolchain image. Hard fail on `-Werror`.
`firmware-unit-tests.yml`	Host Unity tests + gcovr 100 % line/function/branch gate on the included `libs/`.
`firmware-hil-nightly.yml`	Hardware-in-the-loop nightly run on the bench (E2 Lite + AD2 + production board).
`firmware.yml`	Top-level firmware orchestration that pulls the above into one PR-status check.
`gateway.yml`	Go gateway: build, race tests, golangci-lint, govulncheck, cross-compile linux/amd64 + linux/arm64.
`ros2.yml`	ROS2 workspace: proto-gen, colcon build, colcon test on x86 plus a Pi 5 self-hosted ARM64 runner.
`proto.yml`	buf format / lint / build / breaking, regen Go/TypeScript/nanopb, round-trip serialization tests.
`proto-gen.yml`	Lockstep check: regenerate proto outputs and fail if the working tree changes.
`compliance-engine-ci.yml`	Python `star-compliance/`: lint, unit, integration, coverage XML.
`ascii-check.yml`	7-bit-ASCII enforcement (CLAUDE.md mandate).
`copyright-check.yml`	Header presence and consistency.
`since-version-check.yml`	`@since` tags on public APIs.

13.7 What "green" means at PR time

A PR is mergeable to `main` when:

- Every workflow above is green.
- Firmware host coverage at 100 % line / function / branch.
- Proto regen lockstep workflow has no diff.
- No non-ASCII characters in any source file.
- No `master/slave` or `MOSI/MISO` terminology.
- Doxygen builds without warnings (when `WARN_AS_ERROR` is enabled).

The repo deliberately has no backward-compatibility shims, so a breaking change to a proto / API / library is fine *provided* the same PR updates every consumer in lockstep.

13.8 Reproducing CI locally

What CI runs	Local equivalent
firmware-build-verify.yml	`make ci-rx72n`
firmware-unit-tests.yml	`make test-rx72n` then `make coverage-rx72n`
gateway.yml	`cd star-gateway && go test -race -cover ./... && golangci-lint run`
ros2.yml	`./build-ros2.sh && ./test-ros2.sh`
proto.yml + proto-gen.yml	`make proto-gen && git diff -exit-code`
ascii-check.yml	`python3 scripts/utils/fix-encoding.py -check .`
copyright-check.yml	`python3 scripts/utils/check-copyright.py`

14 Motor Control Loop

This section walks the motor control loop from the wheel rim to the gateway and back. It is the project's largest and most timing-sensitive subsystem, and the only place where every layer of the stack has to behave under hard real-time constraints.

14.1 Loop topology in one picture

```
Wheel (front-left, front-right, back-right, back-left -- 4x)
  |
  | 6 V brushed DC gearmotor + 341 PPR Hall encoder
  v
DRV8263H H-bridge (per motor) ------ IPROPI sense (per motor)
  ^                                          |
  | PWM (GPTW0..3, 20 kHz)                   | analog
  | DIR / SLEEP / DRVOFF GPIO                v
  |                                       S12AD (12-bit ADC)
  |                                          |
  |  Encoder phase A/B  -> MTU1, MTU2, TPU1, TPU2 (phase counting)
  |                                          |
  +----- motor_control_task @ 100 Hz <-------+
            |                |  reads encoders, runs PID,
            |                |  writes PWM, samples current
```

```
    |                |
    v                v
shared_data (atomic, lockless)
    |
    | command in / telemetry out
    v
comm_task / telemetry_task
    |
    | nanopb framed protocol over USB CDC / UART / SPI
    v
star-gateway (Go on Pi5)
    |
    v
ROS2 / UI consumer
```

14.2 The loop body, step by step

The motor-control task runs a single 10 ms tick (100 Hz). Each tick:

1. **Read encoder counts.** `rx_encoder_get_count(channel, &count)` reads each of the four hardware phase counters (MTU1, MTU2, TPU1, TPU2). Hall encoders are 341 PPR; with 4x quadrature decode that's 1364 counts/rev (about 13.64 counts per degree of wheel rotation).

2. **Compute velocity.** `rx_encoder_compute_velocity_mps` converts `delta_counts / dt_sec` to meters per second using the wheel circumference defined in `rx_motor_config_t`.

3. **Update setpoint.** The current command is read from `shared_data` (set by `comm_task` on receipt of a `SetVelocityRequest`).

4. **Run PID.** `rx_pid_compute(handle, setpoint, measured, dt_sec, &output)` runs the discrete PID with backward-Euler integration, anti-windup clamping, and derivative low-pass filtering. Gains come from `rx_pid_set_gains`.

5. **Apply duty.** The PID output is in $[-1.0, +1.0]$ (signed normalized PWM duty). Sign becomes the direction signal (DIR + SLEEP), magnitude becomes the PWM duty fed to the GPTW channel.

6. **Sample current (optional).** If the IPROPI ADC channel for this motor is enabled, `rx_drv8263_adc_to_amps` converts the latest sample to amps and stores it in `shared_data` for telemetry to read.

7. **Write back to shared_data.** Per-wheel commanded / measured velocity, duty, current, and motor-state are written to `shared_data`'s motor block via the lockless accessor pattern.

8. **Sleep until the next tick.** `tx_thread_sleep` waits for the next 10 ms tick. The scheduler ensures other tasks get to run during the wait window.

14.3 Where commands come from

`comm_task` owns the receive path. Each tick (10 ms or event-driven on USB CDC RX-ready) it pulls bytes from the active transport, runs them through `rx_frame_decode_with_resync`, validates the

sequence number against the per-channel session, decodes the protobuf payload via nanopb, and dispatches by `frame_type_t` to a handler. The handler updates the relevant fields in `shared_data`; the next motor-control tick will see the new values.

14.4 Where telemetry goes

`telemetry_task` runs at 50 Hz. It reads each motor's state from `shared_data`, builds a `star_v1_TelemetryReport` protobuf message, and pushes it via `rx_comm_manager_send`. The gateway parses and republishes as gRPC or ROS2 topic.

14.5 Concurrency model

ThreadX priorities (lower number = higher priority):

- `motor_control_task` – highest (100 Hz, hard real-time)
- `watchdog_monitor_task` – high (200 Hz IWDT kick)
- `comm_task` – medium (event-driven)
- `imu_task`, `telemetry_task`, `temp_sensor_task`, `obstacle_detect_task` – normal
- `led_status_task`, `serial_bringup_task` – low

Inter-task data flow goes exclusively through `src/shared/shared_data.c`. Each producer-consumer pair has a dedicated block; reads and writes use lockless atomic sequencing (write a "begin" generation counter, write data, write an "end" generation counter; readers retry if begin and end disagree). This avoids ThreadX mutex calls in the hot path.

14.6 Safety interactions

The motor-control task does NOT have an autonomous estop check; it trusts `shared_data.estop_active`. The flag is set by:

- The supervisor / safety_monitor publishing `/star/estop=true`, which the gateway forwards through the comm path.
- A watchdog event (the watchdog task asserts `estop_active` before reset).
- A motor fault (DRV8263 `nFAULT` pin trips, the `rx_motor` layer raises a per-motor fault).

When `estop_active` is true, the motor-control task forces zero duty on every channel and writes `k_motor_state_estop` into the shared block. The gateway side dual-enforces by also publishing zero Twist on `/cmd_vel_out`, so a bug in either layer alone cannot move the robot.

14.7 Tuning workflow

Motor tuning is offline: fit a model in MATLAB and compile the gains in.

1. Run a step-response capture with the bench firmware (`star-rx72n-firmware/motor_spin_test/`) and dump encoder velocity + commanded duty to UART or USB CDC.

2. Import the trace in `matlab/firmware_match_sim.m` / `matlab/motor_model_1st_order.m`.
3. Run `matlab/cascaded_pid_design.m` to design Kp / Ki / Kd against the identified plant.
4. Run `matlab/pid_discretize.m` to convert continuous gains to discrete (forward-Euler) form at 100 Hz.
5. Update PID config defaults in `star-rx72n-firmware/libs/rx_motor/inc/rx_motor.h`.
6. Rebuild firmware, reflash, re-capture step response, sanity-check rise time / overshoot.

The motor model in use: $G(s) = \frac{3.665}{0.075s+1}$ – a first-order DC motor with gain 3.665 rad/s/V and time constant 75 ms.

14.8 Where to look in code

Topic	Source
Loop body	`star-rx72n-firmware/src/tasks/motor_control_task.c`
PID algorithm	`star-rx72n-firmware/libs/rx_pid/src/rx_pid.c`
PWM / duty / direction primitives	`star-rx72n-firmware/libs/rx_motor/src/rx_motor.c`
DRV8263 chip driver	`star-rx72n-firmware/libs/rx_drv8263/src/rx_drv8263.c`
Encoder backends (MTU + TPU phase count)	`star-rx72n-firmware/libs/rx_encoder/`
Shared data block	`star-rx72n-firmware/src/shared/shared_data.c`
Command receive path	`star-rx72n-firmware/src/tasks/comm_task.c`
Telemetry transmit path	`star-rx72n-firmware/src/tasks/telemetry_task.c`
Motor model + PID design	`matlab/cascaded_pid_design.m`, `motor_model_1st_order.m`
Bench validation firmware	`star-rx72n-firmware/motor_spin_test/`, `pwm_test_hal/`

15 Safety Architecture

The STAR robot has actuators that can move heavy chassis at meters per second; "fail safe" is not optional. This section describes the overlapping safety mechanisms so a future engineer can identify, at review time, which layer actually owns a given safety guarantee.

15.1 Defense in depth, in three layers

1. **ROS2 supervisor (Pi 5).** `star_supervisor` gates command flow between manual / autonomous sources and the hardware bridge. It is the only thing that publishes on `/cmd_vel_out`.
2. **Gateway and firmware comm layer (Pi 5 + RX72N).** The gateway forwards `/star/estop` into the protobuf `EmergencyStopRequest` message; the firmware `comm_task` immediately writes `shared_data.estop_active = true`.
3. **Firmware motor / watchdog layer (RX72N).** `motor_control_task` forces zero duty when `estop_active` is set; the IWDT independently resets the chip if the watchdog task ever stops kicking; DRV8263 `nFAULT` / OLP detection triggers a per-motor disable.

A single bug in any one layer cannot, by itself, cause the robot to move when the operator has called e-stop.

15.2 The supervisor model

The robot hardware bridge reads `/cmd_vel_out`, never `/cmd_vel` directly. `star_supervisor` arbitrates:

Source topic	Forwarded to /cmd_vel_out when
`/cmd_vel` (manual)	`autonomy_enable=false` AND `estop=false`
`/nav2/cmd_vel`	`autonomy_enable=true` AND `estop=false`
`/goal_pose` -> `/nav2/goal_pose`	`autonomy_enable=true` AND `estop=false`

`/star/autonomy_enable` and `/star/estop` are both latched booleans (`TransientLocal` QoS) so a node that joins late still sees the current state.

When `estop=true` the supervisor publishes a zero `Twist` on `/cmd_vel_out` continuously (not just once), so the hardware bridge cannot drift back to a stale non-zero command if a publisher misbehaves.

15.3 Estop signal flow

```
operator clicks "STOP" in UI / safety_monitor trips a sonar threshold
        |
        v
ROS2 publishes /star/estop = true (TransientLocal latched)
        |
        +---> star_supervisor: zeros /cmd_vel_out (Layer 1)
        |
        +---> star_gateway_bridge -> Go gateway:
        |       sends EmergencyStopRequest over USB CDC / SPI
        |
        v
RX72N comm_task receives, calls internal_handle_estop,
        sets shared_data.estop_active = true (Layer 2)
        |
        v
RX72N motor_control_task next tick:
        if (shared_data.estop_active) duty = 0; state = ESTOP (Layer 3)
        rx_motor_emergency_stop() drives DRVOFF high on every
        DRV8263 channel as a hardware-level disable
```

Recovery requires explicit operator action: clear the latched `/star/estop` (UI button, or `ros2 topic pub`). The firmware does not auto-clear.

15.4 Watchdog architecture

The RX72N has two watchdog peripherals:

Feature	IWDT (Independent)	WDT
Clock source	15 kHz dedicated low-speed oscillator	PCLKB (60 MHz)
Stoppable in software	No, always on	Yes
Use case in STAR	Hard hang detection	Reserved for soft-debug timeouts
Reset flag	RSTSR2.IWDTRF	RSTSR2.WDTRF
On-reset behavior	`internal_rx_fatal_error` halts	Logs warning, returns boot status code
Period	1 s with current divider	–

`watchdog_monitor_task` runs at 200 Hz and:

1. Reads each task's last-tick timestamp from `shared_data.task_heartbeats`.
2. If any task is more than 3 ticks late, logs an error and asserts `shared_data.estop_active`.
3. Calls `rx_iwdt_feed()` only after every check has passed.
4. Reads stack high-watermarks and asserts on starvation (less than 256 B remaining on any task).

Because the IWDT can never be stopped, a permanently-stuck `watchdog_monitor_task` can only fail to kick, after which the IWDT will reset the chip. The firmware on next boot reads RSTSR2.IWDTRF, logs the failure, and halts so a debugger can read the post-mortem state.

15.5 Hardware-level safety: DRV8263 + DRVOFF

Each DRV8263H motor driver has three control pins:

- `nSLEEP`: low forces the chip into sleep, all outputs Hi-Z. Pulled high during normal operation.
- `DRVOFF`: high forces all outputs Hi-Z without sleeping the chip. Used as a fast disable on fault.
- `nFAULT`: open-drain output asserted by the chip on overcurrent / thermal / OLP / Vm undervoltage. RX72N reads it as a GPIO input.

`rx_motor_emergency_stop` drives DRVOFF high before attempting any PWM cleanup, so even if the GPTW PWM keeps running, the motor is electrically isolated within microseconds.

The Open Load Protection (OLP) diagnostic in `rx_drv8263` runs at startup and on operator request; it cycles a 3-pattern test across each motor with DRVOFF high to detect open / shorted loads before they become runtime faults.

15.6 ThreadX FPU and stack safety

ThreadX RXv3 port supports per-thread FPU context, but only for threads that opt in. Tasks that touch `float` / `double` must declare `TX_THREAD_EXTENSION_2` and be created with `TX_ENABLE_FPU_SUPPORT`. If a task uses floats without the extension, you get silent corruption when another FPU-using task preempts it.

Every motor / PID / IMU / telemetry task in STAR enables FPU support; the LED status / serial bringup tasks do not. The `watchdog_monitor_task` verifies stack high-watermarks on every tick to catch overflow before it corrupts another task's stack.

15.7 Lifecycle nodes (ROS2 side)

`star_safety_monitor` is a `LifecycleNode`; it transitions:

```
UNCONFIGURED  --on_configure-->  INACTIVE
INACTIVE      --on_activate-->   ACTIVE
ACTIVE        --on_deactivate--> INACTIVE
INACTIVE      --on_cleanup-->    UNCONFIGURED
```

While `INACTIVE` the node listens for diagnostics but does not publish `/star/estop`. While `ACTIVE` it monitors sonar topics, IMU jolts, and the supervisor's bond, and trips the estop when any threshold breaches. The `on_shutdown` callback gracefully publishes `/star/estop=true` as a final "safe stop" before exit.

Reconfigurability: `on_cleanup()` is required by REP-2003 to leave the node in a state where a subsequent `on_configure()` succeeds. The implementation `undeclare_parameter()`s all 12 declared parameters in addition to releasing publishers, subscribers and the bond – otherwise the second `declare_parameter()` call would throw `AlreadyDeclaredParameterException`.

Safe-rate guard: `on_activate()` clamps `publish_rate` to a ≥ 0.1 Hz floor (10 s monitoring period) and rejects `NaN`/$\pm\infty$. A user override of 0.0 would otherwise produce a 0 ms timer (busy-spin) via the 1000.0/publish_rate division – the safety monitor must never be the cause of a CPU peg. The default rate is 10 Hz.

15.8 Where to look in code

Topic	Source
ROS2 supervisor / arbitration	`star-ros2/src/star_supervisor/`
ROS2 lifecycle safety monitor	`star-ros2/src/star_safety_monitor/`
Estop forwarding (gateway)	`star-gateway/internal/service/`
Firmware estop hot path	`star-rx72n-firmware/src/tasks/motor_control_task.c`
Firmware estop receive path	`star-rx72n-firmware/src/tasks/comm_task.c`
Watchdog monitor	`star-rx72n-firmware/src/tasks/watchdog_monitor_task.c`
IWDT driver	`star-rx72n-firmware/libs/rx_core/src/rx_iwdt.c`
DRV8263 hardware-level disable	`star-rx72n-firmware/libs/rx_drv8263/` + DRVOFF GPIO in hardware_config.h
RSTSR2 reset-cause inspection	`star-rx72n-firmware/src/main.c`
ARQ -> HARQ migration	`docs/decisions/ADR-001-arq-to-harq.md`

16 Test Strategy

This section enumerates every layer of automated testing in the project, what it verifies, where it runs, and how to invoke it locally.

16.1 Layer 1: Firmware host unit tests (Unity)

Where: `star-rx72n-firmware/tests/`. **Framework:** Unity (single-source vendored). **Compiler:** system Clang (*not* GNURX). **Run:** `cmake -B build -S star-rx72n-firmware/tests && cmake -build build -j 8 && ctest -test-dir build -j 8`, or the shortcut `make test-rx72n`.

Sixty test executables, each linking the production source for the library under test against mock substitutes. Mocks live under `tests/mocks/`.

Coverage gate: `firmware-unit-tests.yml` enforces 100 % line / function / branch coverage on the `libs/`-filtered subset. Hardware-only sources that cannot run on the host (`rx_usb/*`, `rx_crc_hw.c`, `rx_hcsr04` hw/isr/icu, several `rx_hal/src` files that drive registers directly) are explicitly excluded so the gate honestly reflects what is host-testable.

The `test_coverage_compile_all` target is a no-op test that links every host-compilable production file – its job is to ensure `lcov -initial` sees every translation unit so the denominator of coverage is correct.

16.2 Layer 2: Cross-build verification

Where: `star-rx72n-firmware/CMakeLists.txt`. **Compiler:** GNURX 14.2.0 (cross). **Run:** `make build-rx72n` or `make ci-rx72n`.

This proves the firmware tree compiles for the actual target with `-Wall -Wextra -Werror` and links cleanly. It does not run the firmware – there is no on-target test harness in this layer.

Optional clang-tidy: `cmake -DENABLE_CLANG_TIDY=ON` runs the project's curated clang-tidy ruleset. NOLINT suppressions for `readability-*` checks are forbidden; failing checks must be refactored.

16.3 Layer 3: Gateway Go tests (race + coverage)

Where: `star-gateway/internal/`. **Framework:** Go's built-in `testing` + testify. **Run:** `cd star-gateway && go test -race -cover ./...`.

Coverage requirement: $\geq 75\,\%$. Most packages well above; some are at 100 % (`controller`, `fec`, `frame`).

Fuzz tests: `frame/fuzz_test.go`, `dispatcher/fuzz_test.go` run under `go test -fuzz=Fuzz` on demand and are part of the periodic HIL run.

16.4 Layer 4: Hardware-in-the-Loop (HIL)

Where: `star-gateway/test/e2e/hil_test.go`. **Setup:** Pi 5 + RX72N over USB CDC + AD2 capture rig. **Run:** `go test -tags=hardware ./test/e2e/...`. **CI:** `firmware-hil-nightly.yml` on the self-hosted Pi 5 runner.

Tests command / response round-trip latency, transport failover, USB hot-plug, the simple-USB vs full-HARQ mode selection, and end-to-end estop propagation.

16.5 Layer 5: ROS2 colcon tests

Where: `star-ros2/src/<package>/test/`. **Framework:** gtest for C++, pytest for Python. **Run:** `./test-ros2.sh`.

Per-package focus:

- `star_safety_monitor`: lifecycle transitions, sonar threshold breach, bond failure handling.
- `star_supervisor`: estop / autonomy gating truth-table coverage.
- `star_spi_bridge`: frame round-trip with mock spidev0.0.
- `star_gateway_bridge`: gRPC stub server integration tests.
- `star_compliance_msgs`: serialization round-trip.

16.6 Layer 6: Compliance engine tests

Where: `star-compliance/tests/`. **Framework:** pytest. **Run:** `cd star-compliance && PYTHONPATH=. pytest tests/` or via `compliance-engine-ci.yml`.

Detector unit tests (cane_zone_filter, corridor_medial_axis, doorway_lidar_detector, jamb_plane_fitter, imu_jolt_detector, obstacle_clusterer), engine tests (door_offset_calibration, plane_segmentation, floor_frame), and node smoke / behavior tests.

16.7 Layer 7: Proto round-trip tests

Where: `star-proto/tests/{go,typescript,nanopb}/`. **Run:** per-language test commands; CI in `proto.yml`.

Verifies that a message encoded by one language decodes byte- identically in every other. `proto-gen.yml` additionally fails if regenerating produces a non-empty diff (lockstep gate).

16.8 Layer 8: UI tests

Where: `star-ui/src/test/`. **Framework:** Vitest + React Testing Library. **Run:** `cd star-ui && npm test`.

16.9 Layer 9: Static analysis

Tool	Scope
clang-format	Repo-wide on `.c/.h/.cpp/.hpp` except ROS2 packages.
ament_uncrustify	ROS2 packages only (`docs/ROS2_FORMATTING.md`).
clang-tidy	Optional pass on firmware host build. NOLINT for `readability-*` is forbidden.
golangci-lint	Run from `gateway.yml`; full suite per `.golangci.yml`.
govulncheck	Go-side vulnerability scanner.
ament_cppcheck, ament_cpplint	ROS2 default analyzers.
buf lint	Proto schemas; enforced by `proto.yml`.
buf breaking	Detects non-additive proto changes.
ASCII purity check	`scripts/utils/fix-encoding.py -check .`; CI `ascii-check.yml`.
Copyright header check	`scripts/utils/check-copyright.py`; CI `copyright-check.yml`.
`@since` version check	CI `since-version-check.yml`.

16.10 What you should run before every commit

1. Whatever subset of the above touches the code you changed.
2. For C firmware edits: `make test-rx72n` at minimum.
3. For proto edits: `make proto-gen` and verify the diff includes the regenerated outputs.
4. For ROS2 edits: `./build-ros2.sh && ./test-ros2.sh`.
5. For Go edits: `cd star-gateway && go test -race ./... && golangci-lint run`.

6. Always: `python3 scripts/utils/fix-encoding.py -check .`

The repo includes a tracked pre-commit hook at `scripts/git/pre-commit` that enforces ASCII purity. Install it with `git config core.hooksPath scripts/git` (opt-in; `.git/hooks/` is not auto-populated).

16.11 Coverage gaps to be honest about

- There is no firmware on-target test harness. Cross-build is verified, but the firmware itself runs only on hardware and the only on-target tests are the bench-test sandboxes (`star-rx72n-firmware/<periphe`
and the HIL e2e suite.
- There is no soak-test infrastructure that runs for hours. The HIL run is a one-shot pass.
- There is no automated PCB / electrical test.

17 Troubleshooting and Known Gotchas

This section is the lessons-learned dump. Every entry below cost real bench-debug time; if you hit a similar symptom, read the corresponding entry first before reaching for an oscilloscope.

17.1 USB / CDC enumeration

17.1.1 ttyACM* indices change after re-flash

Symptom: after `rfp-cli` reflash the gateway fails to open `/dev/ttyACM1` (or whichever index it had before).

Why: the firmware re-init cycles the USBb peripheral, the host sees a disconnect + re-enumerate, and Linux assigns the next free index. Hot-plugging another CDC device between the cycles shifts the numbers further.

Fix: install the udev symlinks: `sudo bash scripts/setup-pi5-host.sh` (which deploys `scripts/71-star-sym`
The gateway can then open `/dev/star-rx72n-proto / -decoded / -log` and not care what `ttyACM*` number it ends up as.

17.1.2 No CY7C65213 UART bridge while E2 Lite is plugged in

Symptom: `ls /dev/cu.usbmodem* /dev/cu.usbserial-*` shows no Cypress-bridge serial device while the E2 Lite is connected.

Why: bench harness shares USB power gating between the E2 Lite and the Cypress bridge. This is hardware wiring, not a fault.

Fix: disconnect the E2 Lite physically after flashing before running anything that opens the SCI9 fallback path. The USB0 path on `/dev/ttyACM*` is independent and does *not* suffer from this constraint.

17.1.3 TOM enumerates fine on Pi 5 despite HUM warnings

Symptom: The Renesas HW manual says PLLSRCSEL must be 0 when using USB; TOM violates this (PLLSRCSEL=1, HOCO source) and HOCO accuracy is ±2.44 % vs USB FS spec ±0.25 %.

Reality: The Pi 5 xHCI controller accepts the HOCO-driven clock without issue. Bench-verified 2026-04-24 on commit `341bd09ce`: TOM enumerates the 3-port CDC composite cleanly, idVendor=045b idProduct=0235, no EPIPE / EPROTO.

Caveat: stricter xHCI variants (some macOS revisions, some Windows HCDs, embedded controller hubs) MAY reject this clock. For ship-to-Pi deployments, TOM USB is fine. For field deployments on unknown hosts, the 24 MHz crystal-clocked PROD configuration is the safer choice.

17.1.4 USB ISR not firing despite ICU/IER configured

Symptom: `g_usb_isr_entry_count == 0` even though INTSTS0 / BRDY / BEMP latch correctly inside the USB peripheral. Vector 144 (USBI0) configured cleanly; nothing fires.

Why: two HUM-mandated steps were missing.

1. **SLIPRCR.WPRC was never written.** HUM 15.7.7 step (5) makes `SLIPRCR.WPRC = 1` mandatory after assigning a software-configurable interrupt. Without it the ICU silently drops USBI0.
2. **SYSCFG.USBE was set BEFORE SYSCFG.SCKE.** HUM 40.3.1.1 requires SCKE first; reverse order silently no-ops half the SYSCFG follow-on writes because the USB module clock is gated until SCKE=1 latches.

Fix: both fixes shipped on `feat/usb0` commit `f23b15eb5`; they are now in main. If you see this symptom again, check that those two specific writes are present and in the right order in your bring-up code.

17.2 Bench-rig pitfalls

17.2.1 AD2 Single-mode buffer is 4096 samples, period

Symptom: an AD2 capture longer than 4 ms appears to show the MUT going dead. You set `FDwfDigitalInBufferSizeSet(8192)` and run a 20-second sweep; only the first 4 ms are useful.

Why: `FDwfDigitalInBufferSizeSet(n)` silently clamps to 4096 samples no matter what you ask for. With `SampleFormat=8` (one byte per word), 4096 samples at 1 MHz = 4 ms (sometimes 8 ms with byte-format packing). The AD2 isn't recording past that point, and your firmware looks mute.

Fix: pick the sample rate to match the desired window: `sample_hz = buffer_size / desired_seconds`. For a 20 s debug-trace, that's 200 Hz. For high-frequency signals, switch to Record mode (`FDwfDigitalInAcquisitio mode=2`) which streams continuously to the host.

17.2.2 sci9_debug_puts hangs from a ThreadX task

Symptom: debug logging via `sci9_debug_puts` freezes the calling task; everything else continues but that task is wedged.

Why: the helper busy-polls TDRE before writing. SCI9 runs on PCLKA (120 MHz) but its BRR was being computed against PCLKB (60 MHz) – so the configured baud was halved and TDRE never came back true at the expected rate within the busy-poll budget.

Fix until BRR/PCLK is corrected: skip `sci9` debug from any ThreadX task; use the rx_log path or USB CDC log port 3 instead.

17.3 Build / toolchain

17.3.1 Devcontainer rebuild loop

Symptom: VS Code rebuilds the devcontainer image every session; takes minutes.

Why: the GHCR cache is missing or stale for the user.

Fix: pull the cached image manually: `docker pull ghcr.io/locked-inc/star/gnurx:latest`. The `firmware-toolchain-image.yml` workflow keeps it warm.

17.3.2 Local rx-elf-gcc fails; should I use the devcontainer?

Memory rule: use the devcontainer only if the local `rx-elf-gcc` actually fails. If it works, build directly – it's faster. There is no `devcontainer-exec.sh` script and you should not create one.

17.4 Coding-policy reminders

17.4.1 No NOLINT for readability-* clang-tidy checks

Rule: `readability-function-size`, `readability-function-cognitive-complexity`, `readability-magic-numb` etc. must be REFACTORED, never suppressed with `NOLINT`.

Why: the user's standing rule. Suppressions accumulate silently and the codebase loses the structural feedback the lint gave for free.

Exception: path-sensitive checks like `clang-analyzer-optin.core.EnumCastOutOfRange` or `clang-analyzer-security.insecureAPI.DeprecatedOrUnsafeBufferHandling` where there is genuinely no source-level workaround – a NOLINT is acceptable there. `readability-*` is NOT in that exception class.

Common refactor patterns (see commit `9b5a0d53e`):

- *magic-numbers in static_asserts*: replace `static_assert(k_x == 0xLITERAL, ...)` with `static_assert(k_x != k_y && (k_x & k_y) == 0, ...)` so the relationship between named constants is asserted instead of re-stating the literal.
- *non-const-parameter on stub callbacks*: have the stub actually use the parameter (e.g. `if (buf_size > 0U) out_buf[0] = '0';`) so clang-tidy sees a real non-const use, preserving the function-pointer signature contract.
- *math-missing-parentheses*: wrap each operand cluster in its own pair of parens so the precedence is obvious to the reader and to clang-tidy (`((a * b) + c) / d` instead of `a * b + c / d`).

17.4.2 Trust headers over comments

Rule: when a stripped-down test file's comment and the `libs/rx_hal/inc/*.h` header disagree, the header wins. The clock tree comment in `encoder_test/` (or anywhere) claiming PCLKB = 120 MHz is wrong; `rx72n_clock.h` says 60 MHz and that is the truth.

17.4.3 Trust known-good bench tests over fresh code

Rule: if `motor_spin_test/`, `encoder_test/`, or `imu_test/` already initialize a peripheral successfully on the bench, diff your new code against theirs. Do not freelance an init sequence when a proven one exists. CLAUDE.md elevates this to a P0 rule because guessing-and-flashing has cost the project real days.

17.5 Common red herrings

17.5.1 "It looks like the MCU halts 8 ms after rfp-cli disconnect"

It does not. The AD2 buffer simply ran out (see 4096-sample issue above). Re-sample at `buffer_size / desired_seconds` and the trace will run for the full window.

17.5.2 "USB enumeration is broken because BRR is wrong"

Not necessarily. Before blaming firmware, verify the host side:

```
stty -F /dev/ttyACM1 -a
```

Old stty state from a previous session can sit on a tty and look exactly like a baud-rate mismatch. CLAUDE.md's "Embedded Gotchas" list calls this out for the SCI9 path; the same lesson applies to USB CDC re-enumerations.

17.5.3 "The peripheral isn't responding to register writes"

Most likely it is in module-stop. The MSTPCRA / MSTPCRB / MSTPCRC / MSTPCRD bit for that peripheral has to be cleared (`0`) before any of its registers are addressable. Silent failure mode.

Other common silent failures:

- **PFS locked**: writes to `P{n}PFS` require `PWPR.B0WI=0` then `PWPR.PFSWE=1`. Missing unlock = pin-mux never changes = no signal at the pad.
- **MPC PSEL wrong**: PSEL value differs per pin. Check the MPC chapter's pin-function table, not a sibling pin.
- **Wrong PCLK**: SCI uses PCLKB; GPTW uses PCLKA; ADC12 uses PCLKD. Mixing silently miscomputes the divider.
- **FPU context save not enabled**: a task using floats without `TX_THREAD_EXTENSION_2` silently corrupts another task's FPU state on preemption.
- **Encoder TMDR=0 instead of phase-counting mode**: gives a free-running timer, not an encoder.

CLAUDE.md's "Embedded Gotchas to Pattern-Match Instantly" list is the expanded version; this section is the curated subset that has actually bitten this project.

17.6 Where to look in code

Topic	Source
udev symlink rules	`scripts/71-star-symlinks.rules`
USB autosuspend disable	`scripts/setup-pi5-host.sh` (installs `70-star-usb-no-autosuspend.rules`)
USB ISR routing fix	`star-rx72n-firmware/libs/rx_usb/src/rx_usb_hw.c`
SCI9 debug helper	`star-rx72n-firmware/encoder_test/sci9.c` (and similar in sibling bench tests)
AD2 capture harness	`star-rx72n-firmware/gpio_test/host/`
Embedded gotchas (canonical)	`CLAUDE.md` "Engineering Discipline" section

18 Glossary

Acronyms and project-specific terms, alphabetical.

ADA Americans with Disabilities Act. The federal regulation the compliance engine in `star-compliance/` audits environments against.

AD2 Digilent Analog Discovery 2. The 16-channel logic analyzer used on the bench (7 of them = 112 channels). See `docs/bench/ad2_wiring.md`.

Ament The build tool ROS2 uses (`colcon` + `ament_cmake` + `ament_python`). `ament_uncrustify` is the formatter ROS2 packages use; `ament_cppcheck`, `ament_cpplint` are the static analyzers.

ARQ Automatic Repeat Request. Original error-handling scheme for the wire protocol. Replaced by HARQ; see ADR-001.

BNO055 Bosch 9-axis IMU on the I2C bus.

BMP280 Bosch barometric pressure sensor.

BSP Board Support Package. The Renesas-vendored boot / startup code under `src/boot/smc/`.

Buf The protobuf tooling we use; replaces `protoc`. `buf generate`, `buf lint`, `buf breaking`.

Caddy Reverse proxy on the Pi 5; serves the cockpit web bundle and the SLAM map endpoint over HTTPS with an internal CA. See `infra/pi5/etc/caddy/Caddyfile`.

CDC Communications Device Class – the USB class our 3-port composite device exposes. Each port becomes a `/dev/ttyACM*` on Linux.

CIPO Controller In, Peripheral Out (legacy: MISO). Project terminology mandate.

CMT Compare-Match Timer. Used for ThreadX system tick on CMT0.

colcon ROS2 multi-package build tool. We invoke it from `build-ros2.sh`.

Composite USB device A single USB device exposing multiple function classes (here: three CDC-ACM serial ports under one idVendor / idProduct). See §10.

COPI Controller Out, Peripheral In (legacy: MOSI). Project terminology mandate.

Cyclone DDS The DDS RMW middleware ROS2 Jazzy uses. Configured by `config/cyclonedds.xml`.

DRV8263H Texas Instruments H-bridge motor driver. One per wheel (4 total). See `star-rx72n-firmware/libs/`

DRVOFF Active-high pin on the DRV8263 that puts the bridge into Hi-Z. The hardware-level e-stop lever.

DS18B20 Dallas/Maxim 1-wire temperature sensor.

E2 Lite The Renesas E2 Lite debugger / flasher. Plugs into the JTAG header. See bench docs.

FEC Forward Error Correction. Convolutional rate-1/2 K=7; used by the SPI link layer.

FINE Renesas single-wire debug protocol; alternative to full JTAG. The E2 Lite supports both.

Foxglove Web cockpit framework; embedded in the Lichtblick bundle the Pi serves over Caddy.

FPU Floating-Point Unit. RX72N has a single-precision FPU. ThreadX needs `TX_THREAD_EXTENSION_2` to context- switch FPU registers per task.

Gateway The Go service in `star-gateway/` that bridges the UI / ROS2 and the RX72N.

Generation counter Lockless synchronization pattern in `shared_data` for atomic-ish data exchange between producer and consumer tasks.

gRPC Google RPC framework, used by the gateway for ROS2 / UI service interfaces.

GNURX Renesas's GCC port for the RX architecture. We use 14.2.0.202511.

GPTW General PWM Timer (whose initials happen to be GPT). We use four channels for 20 kHz motor PWM.

HARQ Hybrid Automatic Repeat Request. Combines ARQ retries with FEC and Chase Combining. See ADR-001.

HC-SR04 Ultrasonic range sensor. Trigger / echo timing via the RX72N ICU peripheral.

HOCO High-speed On-Chip Oscillator. RX72N's 16 MHz internal clock; TOM uses HOCO → PLL because it has no external crystal.

ICU Interrupt Controller Unit. Sometimes also "Input Capture Unit" – context dependent.

IMU Inertial Measurement Unit. The BNO055 gives 9-axis + fused orientation.

IPROPI DRV8263 current-mirror output (1 A motor = 202 μA IPROPI). Sensed by the RX72N S12AD ADC.

IWDT Independent Watchdog Timer. Cannot be stopped in software; always-on safety net. See §15.

KiCad Open-source EDA suite; the schematic and PCB are KiCad sources under `schematic/`.

Lichtblick Web cockpit (Foxglove fork) served from the Pi.

LFS Git Large File Storage. Used for the GNURX / rfp-cli installers and the Fusion 360 `.f3z` CAD.

Linker script `star-rx72n-firmware/src/linker_script.ld`. Defines memory regions, OFS1-OFS8 sections, the boot vectors.

MTU3 Multi-Function Timer Pulse Unit 3. Used for two of the four encoder phase-counting channels.

Nav2 ROS2 Navigation 2 stack. Consumes the SLAM map and publishes `/nav2/cmd_vel`.

nanopb Embedded-C Protocol Buffers runtime; static-allocation only. `star-rx72n-firmware/libs/rx_nanopb/` hosts it.

OFS1 Option-Setting Function 1 byte. Read by the RX72N at reset; controls HOCO enable, IWDT autostart, etc.

OLP Open Load Protection. Diagnostic on the DRV8263 that detects open-circuit / short-to-VM / short-to-GND conditions per motor channel.

PCLKA / PCLKB / PCLKD Peripheral clock domains. PCLKA at 120 MHz, PCLKB at 60 MHz, PCLKD at 60 MHz. See §12.

PFS / PWPR Pin-Function-Select Register / Pin-Write-Protection Register. Required unlock dance before changing pin-mux.

PID Proportional-Integral-Derivative controller. The motor loop's heart. `libs/rx_pid/`.

PLL Phase-Locked Loop. Multiplies the main clock (crystal on PROD, HOCO on TOM) up to 240 MHz ICLK.

POEG Port Output Enable for GPTW. Hardware safety to disable PWM outputs on fault.

PROD The production RX72N PCB variant. Has a 24 MHz crystal. Compile flag `STAR_BOARD_PROD`.

Protobuf Protocol Buffers, the wire-format / RPC schema language. Schemas live in `star-proto/proto/star/v1/`

RFP-CLI Renesas Flash Programmer CLI. Used to flash the firmware via the E2 Lite. `scripts/flash-rx72n.sh`.

RIIC Renesas I-square-C peripheral. Two channels: RIIC0 (host I2C), RIIC1 (sensors).

RMW ROS Middleware. The DDS abstraction; we use `rmw_cyclonedds_cpp`.

ROS2 Robot Operating System 2; we run Jazzy.

RSPI Renesas SPI peripheral. RSPI2 talks to the Pi 5 `/dev/spidev0.0`.

RTOS Real-Time Operating System. We use Azure ThreadX.

SCI Serial Communications Interface. UART hardware on the RX72N. We use SCI9 with the Cypress CY7C65213 USB-UART bridge.

SLIPRCR Software-configurable Interrupt PR-write-protection Control Register. Has to be unlocked + WPRC=1 latched after SLIBR writes; missing this latch was the USB-ISR-not-firing root cause.

Star As a prefix: every top-level subproject begins with `star-` (e.g. `star-rx72n-firmware/`, `star-gateway/`). Vendored third-party packages keep their original names (`sllidar_ros2`, `threadx`).

STAR Spatial Topography Accessibility Robot. The product.

Supervisor The ROS2 node that gates command flow through `/cmd_vel_out`. See §15.

ThreadX Azure RTOS / Microsoft's real-time OS for embedded targets. Vendored under `libs/threadx/`.

TOM An RX72N PCB variant used for breakout-style bench tests. Has no external crystal (HOCO-driven). Compile flag `STAR_BOARD_TOM`.

TPU Timer Pulse Unit. Used for two encoder channels (the other two are MTU3).

Twist ROS2 `geometry_msgs/Twist` message; linear + angular velocity command.

USBb The full-speed USB peripheral on the RX72N (USB0 module).

USB CDC See "CDC" above.

Viterbi Soft-decision decoder for the K=7 convolutional code. `star-gateway/internal/fec/viterbi.go` and `star-rx72n-firmware/libs/rx_fec/`.

xHCI eXtensible Host Controller Interface. The Pi 5's USB 3 controller. The "permissive enough to accept TOM's HOCO clock" host – see §10.

19 Compliance Engine

19.1 Executive summary

The STAR compliance engine is a ROS2 Jazzy Python package (`star_compliance`) that turns the Pi 5's fused sensor stream into ADA-2010 accessibility audit reports. It runs entirely on the Raspberry Pi 5, consuming the live SLAM map, the RPLiDAR C1 `/scan`, the BNO055 IMU `/imu/data` (forwarded from the RX72N via `star_spi_bridge`), the IMX219 stereo cloud `/stereo/points2`, the rectified left image, the EKF-fused `/odom`, and the optional Hailo-8L YOLO 3D detection feed `/perception/detections_3d`. It produces per-check ROS2 messages on the `/compliance/*` topic tree, appends rows to `extras/validation_log.csv`, and – on demand – builds a PDF audit report of every flagged violation and every passing measurement. The engine is the only piece of STAR whose output is human-facing: it is what a Certified Access Specialist (CASp) actually reads after a scan. See the package map in §11 for where it lives in the repo.

19.2 Pipeline in one picture

```
Sensors (Pi 5 inputs)
  |
  |  /scan          (RPLiDAR C1, 10 Hz)
  |  /imu/data      (BNO055, 200 Hz, forwarded over SPI)
  |  /odom          (robot_localization EKF, 100 Hz)
  |  /map           (slam_toolbox async, 0.5 Hz)
  |  /stereo/points2 + /cam0/.../image_rect_color (IMX219 stereo)
  |  /perception/detections_3d (Hailo YOLO 3D, optional)
  v
Detectors (pure-numpy / sklearn / Open3D, no rclpy)
  |
  |  cane_zone_filter, corridor_medial_axis, doorway_lidar_detector,
  |  imu_jolt_detector, jamb_plane_fitter, obstacle_clusterer,
  |  wall_plane_fitter, door_state_classifier (ONNX)
  v
Engines (cross-cutting math)
  |
  |  floor_frame, plane_segmentation, imu_cross_validate,
  |  door_offset_calibration
  v
Nodes (rclpy glue, one per ADA check)
  |
```

```
  |  ramp_slope_node, door_clear_width_node, door_threshold_node,
  |  path_blockage_node, protruding_objects_node,
  |  dynamic_obstacle_node, ...
  v
compliance_monitor_node (CPU safety net, throttles ADA 307)
  |
  v
/compliance/* topics + extras/validation_log.csv
  |
  v
report/pdf_generator.py
  |
  v
star_audit_report_sample.pdf  (CASp-readable PDF)
```

The pipeline is strictly one-way: detectors never call into nodes, nodes never call detectors back. Detectors and engines are pure Python (no `rclpy` import) so they unit-test against synthetic inputs. ROS2 wiring is confined to the `nodes/` layer.

19.3 Detectors

Detectors live under `star-compliance/star_compliance/detectors/`. Each is a pure-Python module (numpy, sklearn, optionally Open3D or OpenCV-DNN) that takes raw sensor arrays and returns geometric features. None of them import `rclpy`.

Module	Sensor inputs	What it detects
`cane_zone_filter`	3D point cloud + floor plane	Points whose height above the floor falls in the ADA 307 "cane-detectable zone" (27–80 in / 0.686–2.032 m). Filters fixed protrusions from transient occupants (person, chair, backpack).
`corridor_medial_axis`	`nav_msgs/OccupancyGrid` (slam_toolbox)	Medial-axis skeleton of free space plus per-pixel local clearance via 2x distance transform. Returns the minimum corridor width along each accessible segment.
`door_state_classifier`	Rectified left RGB image	Door state (open / closed / ajar / none) via a YOLOv8n ONNX model trained on DoorDet. Falls back to a geometric heuristic on the disparity cloud when no ONNX weights are present.
`doorway_lidar_detector`	Rolling `sensor_msgs/LaserScan` window	Two parallel-wall RANSAC lines on either side of the path; flags local minima of corridor width below 1.1 m as doorway candidates for the stereo jamb fitter.
`imu_jolt_detector`	`/imu/data` (200 Hz) + `/odom`	Sustained 25 ms bursts of gravity-compensated z-axis acceleration above a 2.0 m/s^2 rolling-mean threshold. Used as ADA 404.2.5 door-threshold presence cue.
`jamb_plane_fitter`	3D point cloud + floor plane	Two parallel vertical planes (door jambs) inside the handle-height band (85–100 cm), reporting their perpendicular distance. RANSAC + verticality gate.
`obstacle_clusterer`	`/scan` + `/map` + `/odom`	DBSCAN clusters of LaserScan returns after slam_toolbox background subtraction. Output is the dynamic-obstacle set (centroid, radius, density).
`wall_plane_fitter`	3D point cloud (room scan)	Iterative RANSAC of vertical wall planes; returns up to 8 walls with normal, offset, and inlier count. Used as the reference surface for ADA 307 protrusion measurement.

19.4 Engines

Engines live under `star-compliance/star_compliance/engines/` and provide cross-cutting math reused by multiple detectors and nodes. They are pure-Python and unit-tested independently.

Module	Purpose
`door_offset_calibration`	Calibrated door-thickness + stop-depth + hinge-offset constants (default 2.5 in / 0.0635 m total) applied to the frame-opening width to produce the ADA 404.2.3 "between the face of the door and the stop" clear-width estimate. JSON-loadable per deployment; the offset is echoed in the audit PDF.
`floor_frame`	Rolling floor-plane estimate in the map frame, sourced from BNO055 quaternion (robot pitch/roll) and optional forward-floor LiDAR returns. Exposes "height above the floor" for any 3D point. Fails loud if the floor normal deviates by more than 18 deg from vertical.
`imu_cross_validate`	0.5 s rolling window of IMU pitch samples plus an agreement gate: a LiDAR-plane slope is only published when its agreement with the IMU window is within 0.5 deg. Used by the ramp-slope node as a sanity check.
`plane_segmentation`	Open3D RANSAC plane fitter shared by every ramp-related ADA check. Returns plane normal, offset `d`, inlier count, and the slope (angle between normal and gravity) in degrees.

19.5 Nodes (one ROS2 node per ADA check)

Nodes live under `star-compliance/star_compliance/nodes/` and form the only `rclpy`-aware layer. Each implements a single ADA section's measurement workflow, subscribes to the sensors listed in its module docstring, runs the relevant detectors and engines, and publishes a typed `star_compliance_msgs/*` message plus a CSV row to `extras/validation_log.csv`.

Node	ADA section	Audited check
`compliance_monitor_node`	–	Orchestrator and CPU safety net. Reads the Pi 5 1-minute load average; when normalized load > 80% it disables `star_protruding_objects_node` (the ADA 307 RANSAC pipeline is the heaviest consumer); re-enables below 50%. Publishes `/compliance/monitor/status`.
`door_clear_width_node`	404.2.3	Door clear width must be $\geq$ 32 in (0.8128 m), measured between the face of the door and the stop with the door open 90 deg. Pipeline: LiDAR doorway candidate $\rightarrow$ time-synced stereo cloud + image $\rightarrow$ jamb-plane RANSAC at handle height $\rightarrow$ YOLO door-state $\rightarrow$ calibrated door+stop+hinge offset.
`door_threshold_node`	404.2.5	Door threshold height; STAR reports presence only (no precise height). Watches a 2 s window after each `/compliance/door_clear_width` event and fires `detected=True` if `ImuJoltDetector` confirms a $> 2.0\,\mathrm{m/s^2}$ z-axis burst. The PDF report carries the mandatory CASp-follow-up disclosure.
`dynamic_obstacle_node`	–	DBSCAN clusterer fed by `/scan` + `/map` + `/odom`, optionally fused with Hailo YOLO 3D detections (`yolo+lidar`, `lidar`, `yolo`-only sources). Not ADA-specific; feeds path-blockage and Nav2 behavior tree pedestrian-pause actions.
`path_blockage_node`	403.5	Live corridor clear width below the 36 in ADA threshold. Compares slam_toolbox baseline width (medial-axis transform on `/map`) to the instantaneous `/scan`-derived width along the robot's heading; flags when live $<$ 36 in AND map-minus-live $>$ 0.15 m for 3 consecutive scans.
`path_width_node`	403.5 (stretch)	Standalone medial-axis path-width auditor. Stub in the capstone build; production algorithm uses `skimage.morphology.medial_axis` + `distance_transform_edt` on the OccupancyGrid.
`protruding_objects_node`	307	Fixed objects protruding $>$ 4 in (0.1016 m) into circulation paths within the 27–80 in cane-detectable zone. Consumes `/cloud_map` (RTAB-Map consolidated, 1 Hz) plus optional Hailo YOLO 3D class labels; transient classes (person, chair, backpack) are emitted with `flagged_violation=false`. Heaviest CPU consumer; gated by `compliance_monitor_node`.
`ramp_landing_node`	405.7	Architected only. Designed algorithm: 60 in $\times$ 60 in inscribed-rectangle test on flat regions adjacent to ramp endpoints. Placeholder so the package launches with the full topology in place.
`ramp_slope_node`	405.2	Ramp running slope must not exceed 1:12 (4.76 deg). Forward LiDAR fan ($\pm$15 deg, $\leq$ 2 m)

19.6 Report generation

`star_compliance/report/pdf_generator.py` is a single-shot PDF builder. It loads `extras/validation_log.csv` (the rolling append-only log every node writes to), partitions rows by the `flagged` column into violations and passing measurements, and emits a one-or-two-page PDF via `reportlab`. The default output path is `star-compliance/star_audit_report_sample.pdf`; an operator can re-run `python3 -m star_compliance.report.pdf_generator` after any scan to refresh it.

The PDF contains:

- Title block with generation timestamp.
- One-paragraph summary (total measurements, violations, passing).
- Violations table (run id, time, location, measured slope, ground-truth slope, IMU/LiDAR agreement, notes).
- Passing-measurements table with the same columns.
- Footer disclosure that "measurements are observational; final legal attestation of ADA compliance remains with the Certified Access Specialist".

The CSV schema is shared across nodes: `run_id`, `date`, `time`, `location`, `ada_section`, `measurement` (per-node columns such as `ramp_slope_lidar_deg`, `ada_clear_width_m`), `flagged` (yes/no), and `notes`. Comment rows beginning with `#` in the `run_id` field are skipped by the loader.

19.7 Bring-up: `compliance.launch.py`

`star_compliance/bringup/compliance.launch.py` brings up the full compliance stack with `ros2 launch star_compliance compliance.launch.py`. It declares the following launch arguments (each gates one or more nodes via `IfCondition`):

Argument	Default	Effect
`use_stereo`	`true`	Enable stereo-dependent nodes (`door_clear_width_node`). Disable when running headless without IMX219.
`use_ada_307`	`true`	Enable `protruding_objects_node`. Disable on CPU-constrained scans.
`use_path_blockage`	`true`	Enable `path_blockage_node` (requires `/map`).
`use_dynamic_obstacles`	`true`	Enable the DBSCAN `dynamic_obstacle_node`.
`use_compliance_monitor`	`true`	Enable the `compliance_monitor_node` CPU safety net.
`ada_307_input_topic`	`/cloud_map`	Switch the ADA 307 cloud source to `/stereo/points2` for higher-rate / higher-CPU operation.

The `ramp_slope_node` and `door_threshold_node` are unconditional; they have no stereo or map dependency. The architected stubs (`ramp_width_node`, `ramp_landing_node`, `path_width_node`, `trip_hazard_node`) are not launched by the capstone bring-up but ship in the package so the topology is in place for the deployment phase.

Per-node parameters worth noting:

- `compliance_monitor_node`: `cpu_count` (defaults to `os.cpu_count()`) tunes the load-normalization denominator.
- `protruding_objects_node`: `enabled` (bool) is set live by the monitor; `input_cloud_topic` comes from the launch argument.
- `door_threshold_node`: `STAR_THRESHOLD_CSV` environment variable redirects the per-node CSV log.
- `door_clear_width_node`: `STAR_VALIDATION_CSV` env redirects the shared validation log; `DEFAULT_SENSOR_HE` (0.25) is the LiDAR mounting height for floor-anchor cross-checks.
- `ramp_slope_node`: `IMU_AGREEMENT_GATE_DEG` (0.5 deg), `SCAN_FORWARD_MAX_M` (2.0 m), `SCAN_FORWARD_HALF_A` (15 deg), and `MIN_POINTS_FOR_FIT` (50). Output PNGs land under `/tmp/star_compliance/`.

For sensor-side context (camera intrinsics, stereo baseline, LiDAR mounting height), see `docs/imx219_stereo_c` and the hardware-pinout section in §11.

19.8 Where to look in code

Topic	Source
Package manifest	`star-compliance/package.xml`, `setup.py`, `setup.cfg`
Detectors (pure-Python)	`star-compliance/star_compliance/detectors/`
Engines (cross-cutting math)	`star-compliance/star_compliance/engines/`
Nodes (rclpy glue)	`star-compliance/star_compliance/nodes/`
Bring-up launch file	`star-compliance/star_compliance/bringup/compliance.launch.py`
PDF audit report builder	`star-compliance/star_compliance/report/pdf_generator.py`
Sample audit PDF	`star-compliance/star_audit_report_sample.pdf`
ONNX door-state model location	`star-compliance/star_compliance/models/`
Validation log (append-only)	`extras/validation_log.csv`
Per-node tests	`star-compliance/tests/`
Architecture / deployment / tuning	`star-compliance/docs/ARCHITECTURE.md`, `DEPLOYMENT.md`, `TUNING.md`
Stereo camera bench notes	`docs/imx219_stereo_camera.md`
Repo-wide layout	§11

20 Operator UI

20.1 Executive summary

The STAR operator UI is a browser-based cockpit written in TypeScript on top of React 19 and bundled with Vite. The runtime is a single-page application: the operator opens the Pi 5 in a browser, the page fetches its bundle from the static-asset server, and the client opens a single WebSocket to the Go gateway (§11). All telemetry and command traffic flows through that one WebSocket as binary-encoded `STAREnvelope` messages – the same Protocol Buffer wire contract as the firmware (§1) but multiplexed at the UI level into a single envelope per direction. A Zustand store fans incoming envelopes out to per-panel selectors so a new lidar scan does not re-render the battery

panel. Layout is a tiling window manager built on `react-grid-layout` with persisted per-view overrides. Three panels (`Nav2GoalPanel`, `PidTuningPanel`, `TransportDiagPanel`) are explicitly placeholder UIs: they render a yellow "Not implemented yet" banner and disable their action buttons, because the gateway does not yet expose the corresponding RPC over the UI WebSocket. They exist so the layout slots are reserved; they do not fake state.

20.2 Layout overview

The UI is a flat tree under `star-ui/src/`:

Path	Role
`main.tsx`	React entry point. Mounts `App` inside a custom `ErrorBoundary` that prints uncaught exceptions to a monospace fallback page.
`App.tsx`	Top-level component. Resolves the active view, builds the panel registry, runs the responsive grid, owns the maximize-overlay UX.
`theme.ts`	A single `COLORS` record of semantic accent / status colors plus theme-aware CSS variable references.
`components/`	17 panels + the status bar + the joystick `ControllerView`. Each panel is self-contained and reads from one or two store selectors.
`hooks/`	`useGamepad` (Gamepad API polling) and `useSTARConnection` (WebSocket lifecycle + reconnect + REST fallback for E-Stop).
`store/`	Two Zustand stores: `dashboardStore` (telemetry, alerts, e-stop) and `useWindowStore` (active view, per-view layout overrides).
`services/GatewayService.ts`	Packet ring buffer + envelope-preview formatter for the packet analyzer.
`proto/star/v1/`	Generated TypeScript bindings (`protobuf-ts`) for every `star.v1` message used on the wire.
`test/setup.ts`, `hooks/useGamepad.test.ts`	Vitest setup + the only unit test currently shipped (gamepad polling, deadzone, axes mapping).

The data flow inside the browser:

```
Gateway (Go on Pi5)
  |
  |  binary STAREnvelope (Protocol Buffers)
  v
WebSocket (ws://<pi-host>:8080/ws)
  |
  v
useSTARConnection                    navigator.getGamepads()
  |  STAREnvelope.fromBinary             |
  |                                      v
  |                                  useGamepad
  |                                      |  { linearVel, angularVel }
  v                                      v
dashboardStore                       TeleopPanel @ 50 Hz
  | telemetry / motors /                  |  ControllerState envelope
  | battery / odometry /                  v
  | lidar / system /                  useSTARConnection.sendControllerState
```

```
  | alerts / eStop                        |  STAREnvelope.toBinary
  v                                       v
useShallow / .subscribe()              WebSocket (TX path)
  |
  v
Panels (MotorPanel, BatteryPanel, OdometryPanel, ...)
  |
  |  StatusBar -> sendEStop / sendEStopRelease
  |   (REST /api/estop fallback if WS is down)
  |
useWindowStore (zustand + persist via localStorage)
  |
  v
ResponsiveGridLayout (react-grid-layout)
```

20.3 Panels

Every entry under `star-ui/src/components/` is one row below. The *Status* column distinguishes three categories. *Live* means the panel renders real gateway data and has no stub banner. *Read-only* means the panel only displays data and never writes back. *Placeholder (banner)* means the panel renders a yellow "Not implemented yet" banner and disables its action buttons because the corresponding gateway RPC is not yet wired to the UI WebSocket.

Panel	Purpose	Status	Data source
`TeleopPanel`	Joystick / gamepad teleop. 50 Hz `ControllerState` TX.	Live	`useGamepad` + `sendControllerState`
`ControllerView`	Visual joystick + connection pill rendered inside `TeleopPanel`.	Live (read-only)	Props from `TeleopPanel`
`MotorPanel`	Per-wheel velocity bars (FL / FR / BL / BR).	Live (read-only)	`dashboardStore.motors`
`BatteryPanel`	Pack voltage, SOC, temperature, segmented battery icon.	Live (read-only)	`dashboardStore.battery`
`OdometryPanel`	2D odometry trail on a canvas plus x / y / heading readouts.	Live (read-only)	`dashboardStore.odometry`
`LidarPanel`	Polar LiDAR scatter on a canvas; `store.subscribe` (no re-render).	Live (read-only)	`dashboardStore.lidarScan`
`ImuPanel`	Roll / pitch / yaw attitude indicator + accel + gyro readouts.	Live (read-only)	`dashboardStore.telemetry.imu`
`GpsPanel`	Lat / lon / alt / accuracy + fix-quality badge.	Live (read-only)	`dashboardStore.telemetry.gps`
`SystemHealthPanel`	Uptime, free heap, CPU, temperature, firmware version, WiFi RSSI.	Live (read-only)	`dashboardStore.systemStatus` + `telemetry`
`AlertsPanel`	Severity-colored alert cards with relative age.	Live (read-only)	`dashboardStore.alerts`
`DiagLogPanel`	Monospace scrolling log of every alert (newest first).	Live (read-only)	`dashboardStore.alerts`
`TimeSeriesPanel`	1 Hz sparklines: FL / FR motor velocity, CPU %, temperature.	Live (read-only)	`dashboardStore` (polled at 1 Hz via getState)
`PacketAnalyzer`	Last 30 envelopes (TX + RX) with seq / type / size / decoded preview.	Live (read-only)	`services/GatewayService.getRecentPac` ring buffer (200 ms poll)
`StatusBar`	Connection pill + STALE / robot mode / E-Stop / RESUME / Layouts / theme.	Live	`dashboardStore` + `sendEStop` + `sendEStopRelease`
`CameraFeed`	Camera placeholder (icon + reticle); no WebRTC / MJPEG client yet.	Placeholder (no banner)	None – static SVG only
`FirmwarePanel`	OTA UI shell with simulated progress bar (`simulateUpload`).	Placeholder (no banner)	Local state only – no gateway RPC wired
`Nav2GoalPanel`	Inputs for X / Y / theta navigation goals; Send button disabled.	Placeholder (banner)	None – gateway has no Nav2 goal RPC
`PidTuningPanel`	Kp / Ki / Kd inputs per motor (FL / FR / BL / BR); Apply button disabled.	Placeholder (banner)	None – no UI-side channel to `SetMotorPidConfig`
`TransportDiagPanel`	Static SPI / USB CDC	Placeholder	None – gateway

Note on placeholders: `CameraFeed` and `FirmwarePanel` are placeholder UIs but do not render the "banner" pattern – the camera shows a "No Camera Stream" empty state, and the firmware panel exposes interactive buttons that drive a local simulated progress timer. The three banner-bearing panels (`Nav2GoalPanel`, `PidTuningPanel`, `TransportDiagPanel`) are the ones honest about being stubs.

20.4 State management

Two Zustand stores own all UI state. Both are kept deliberately small and per-slice; panels select only the field they need via `useShallow` or a stable getter so a new envelope re-renders only the relevant subtree.

20.4.1 `store/dashboardStore.ts`

The telemetry feed. It owns connection state, the last seq number, the "is the data stale" flag, and one slice per major payload kind. The critical method is `updateFromEnvelope(env)`: each `case` in the oneof switch performs exactly one `set()` call so React re-renders only the slice that changed. A sequence-gap detector raises a `SEQ_GAP WARN` alert when `seq > lastSeq + 1`.

Field	Type	Source
`connectionState`	`ConnectionState` (4-state)	`useSTARConnection` on open / close.
`lastSeq`	`number`	`env.seq` for gateway-originated envelopes.
`seqGapDetected`	`boolean`	Set when a gap is observed (for the Status bar).
`dataIsStale`	`boolean`	Set on `ws.onclose`; cleared on first `telemetry` envelope.
`telemetry`	`TelemetryData|null`	oneof case `telemetry`.
`motors`	`MotorStatus[]|null` (length 4)	oneof case `motors`.
`battery`	`BatteryState|null`	oneof case `battery`.
`odometry`	`OdometryData|null`	oneof case `odometry`.
`lidarScan`	`LidarScan|null`	oneof case `lidar`.
`systemStatus`	`SystemStatus|null`	oneof case `system`.
`alerts`	`Alert[]` (capped at 200)	oneof case `alert` + locally-injected `SEQ_GAP` entries.
`eStopActive`	`boolean`	`triggerEStop` / `releaseEStop` from the StatusBar.

20.4.2 `store/useWindowStore.ts`

The layout / view manager. Six named views (`OVERVIEW`, `TELEOP`, `NAVIGATION`, `DEBUG`, `CONFIG`, `FULL`); each declares its panel set and a default `react-grid-layout` for the `lg` breakpoint. The `FULL` view is the only scrollable one. The store is wrapped in `persist(...)` so per-view drag / resize edits survive a reload under the localStorage key `robot-dashboard-v6`. Reset clears the overrides for the active view, not all views.

Field	Type	Purpose
activeView	ViewName	One of OVERVIEW / TELEOP / NAVIGATION / DEBUG / CONFIG / FULL.
viewLayouts	Partial<Record<ViewName, Layouts>>	Per-view user overrides; absent keys fall back to the view's default.
maximizedPanel	string|null	The key of the currently maximized panel (Esc closes).
setView	action	Switch view; clears maximizedPanel.
updateLayouts	action	Persist a drag / resize for the current view.
toggleMaximize	action	Open / close the maximize overlay.
resetLayout	action	Drop overrides for the active view only.

20.5 Custom hooks

Hook	Behaviour
useGamepad	Polls navigator.getGamepads() on every animation frame. Picks the first non-null gamepad, applies a 0.1 deadzone to both axes, returns { connected, linearVel, angularVel } where linearVel = -axes[1] (forward stick = +1) and angularVel = axes[0]. Disconnect events fall back to the polling tick rather than firing immediately.
useSTARConnection	Owns the lifetime of a single WebSocket. Sets binaryType = 'arraybuffer' *before* any messages arrive (the browser default blob would crash Uint8Array construction). On every message it calls STAREnvelope.fromBinary, then updateFromEnvelope on the store, then recordPacket into the analyzer ring. On close it marks data stale and reconnects with exponential backoff (500 ms base, 30 s cap, ±30 % jitter). Returns three TX functions: sendControllerState(state), sendEStop(reason), sendEStopRelease(). The two e-stop senders fall back to a REST POST /api/estop?action=...&reason=... if the WebSocket is not open – this is the safety-critical guarantee that operator e-stop never silently fails.

The repository ships exactly these two custom hooks (plus the useGamepad.test.ts Vitest suite). All other state lives in useDashboardStore or useWindowStore.

20.6 Service layer

services/GatewayService.ts is intentionally narrow. It does *not* own the WebSocket, the encoder, or the decoder. It owns one data structure: a RING_SIZE = 100 entry ring buffer of PacketRecords, written by recordPacket(env, dir, sizeBytes, errorType?) and read by getRecentPackets(). The ring is module-scoped (not in Zustand) so that high-rate envelope arrivals never trigger a store re-render – the PacketAnalyzer polls the buffer at 200 ms intervals instead. Each record is { seq, type, direction, sizeBytes, tsMs, preview }, and formatPreview pretty-prints the oneof variant for the DECODED column (e.g. for a lidar payload it renders "<n> pts"; for controller

it renders `"lin=... ang=... [TX]"`). The file also exports `MV_PER_V` as the canonical `milliVolt -> Volt` divisor used by `BatteryPanel` so battery math is consistent across the UI.

The actual gRPC-over-WebSocket plumbing – `STAREnvelope.toBinary` on TX and `STAREnvelope.fromBinary` on RX – lives in `useSTARConnection`, not here. The `*.client.ts` files under `src/proto/star/v1/` (`configuration.client.ts`, `firmware_update.client.ts`, `motor_control.client.ts`, `telemetry.client.ts`) are protobuf-ts gRPC clients that the UI *generates* but does not yet instantiate. The intent is that configuration / OTA / PID-set RPCs will be served via gRPC-Web through a separate transport once those panels are de-stubbed; today the only live transport is the multiplexed `STAREnvelope` on `/ws`.

20.7 WebSocket protocol

The UI side speaks one wire format: `star.v1.STAREnvelope`, encoded as Protocol Buffers binary (no length prefix – WebSocket already frames). The envelope carries metadata (`seq`, `ts_ms`) plus a `oneof payload`.

Direction	Sender	Payload variants used
RX (gateway -> UI)	Gateway	`telemetry`, `motors`, `battery`, `odometry`, `lidar`, `system`, `alert`.
TX (UI -> gateway)	UI	`controller` (50 Hz from `TeleopPanel`), `estop` (operator E-Stop / Resume from `StatusBar`).

Latency expectations: the TX cadence is 50 Hz (20 ms period, `sendIntervalMs = 1000 / sendRateHz` in `TeleopPanel.tsx`). The RX cadence is whatever the gateway chooses to push; per-payload rates are governed gateway-side. The UI makes no rate assumption on RX – the `seq` field plus the `seqGapDetected` flag are the only freshness signals it exposes. On `ws.onclose` the store is marked stale immediately; panels render their last-known value but the `StatusBar` shows the `STALE` pill so the operator does not trust it.

Out-of-band fallback: the only TX path with a non-WebSocket fallback is operator E-Stop, which posts to `/api/estop` via `fetch()` when the WebSocket is closed. This is the dual-channel safety-critical path required by §15.

20.8 Build and run

Command	What it does
`npm install`	Install dependencies (React 19, Zustand 5, react-grid-layout, protobuf-ts, uPlot, Vite 7, Vitest 4).
`npm run dev`	Vite dev server on `0.0.0.0:5173` (`strictPort: true`). HMR enabled.
`npm run build`	`tsc -b && vite build`. Strict TypeScript build first, then Vite production bundle into `dist/`.
`npm run preview`	Serve the built `dist/` bundle (production smoke test).
`npm run lint`	ESLint over the whole project (`eslint .`).
`npm run test`	`vitest run` – single-shot Vitest run with the `jsdom` environment.

Environment variables: the only env var the runtime reads is `VITE_WS_PORT` (parsed in `App.tsx` line 31). If absent, the UI defaults to port 8080. Protocol (`ws://` vs `wss://`) is auto-selected from `window.location.protocol`. Hostname always tracks `window.location.hostname`, so the same bundle works whether the operator opens the Pi directly (`http://star.local`) or through Caddy

(`https://star.local`). There is no `VITE_STAR_WS_URL` override – the URL is constructed from the three components above.

20.9 Theming and layout

20.9.1 Theme

`src/theme.ts` exports a `COLORS` record. Static accent colors (`accent`, `warning`, `success`, `danger`, `estopActive`) are literal hex values and stay the same in both themes – they are SF / iOS system colors. Theme-aware colors (`textPrimary`, `panelBg`, `border`, etc.) are not literals at all, they are the strings `var(-color-text)`, `var(-panel-bg)`, etc., so they resolve at runtime through CSS custom properties defined in `index.css`. The `StatusBar` writes `document.documentElement.dataset.theme = 'dark'|'light'` and persists the choice to `localStorage['star-theme']`.

20.9.2 Layout engine

`App.tsx` renders one `<ResponsiveGridLayout>` from `react-grid-layout`. Five breakpoints (`lg`/`md`/`sm`/`xs`/`xxs` with column counts `24`/`18`/`12`/`8`/`6`) let panels reflow on tablets and phones. Row height for non-scrollable views is recomputed by a `ResizeObserver` so 18 rows always fill the viewport minus the 80 px top bar and the 12 px gaps. Drag is restricted to `.panel-header`; resize handles are exposed on all eight edges. A maximize button on each panel opens a fullscreen overlay (Esc closes). All panel chrome (the glass background, the hover, the stagger-fade-in animation) lives in `App.css` / `index.css`, not in TypeScript.

20.10 Testing

The Vitest setup is intentionally light. `src/test/setup.ts` imports `@testing-library/jest-dom` for matchers; the environment is `jsdom` (declared in `vitest.config.ts`). The repository ships exactly one test suite, `src/hooks/useGamepad.test.ts`, covering the three gamepad behaviours (initial disconnected state, axes-to-velocity mapping, 0.1 deadzone). `npm run test` runs the suite once and exits. There are no Playwright / Cypress browser tests today; UI-side end-to-end coverage is delegated to the gateway-level e2e harness in `star-gateway/test/e2e/`.

20.11 Where to look in code

If you want to...	Open
Add a new panel	`star-ui/src/components/<Name>Panel.tsx`, then register the key in `App.tsx usePanelRegistry` and add a layout entry to one or more views in `store/useWindowStore.ts`.
Add a new RX payload kind	Extend the `oneof` in `star-proto/proto/star/v1/ui.proto`, regen, then add a `case` branch in `store/dashboardStore.ts updateFromEnvelope`.
Change WebSocket URL / port	`App.tsx` (the `WS_PORT` / `WS_PROTOCOL` / `WS_URL` block) and / or set `VITE_WS_PORT`.
Tune the gamepad deadzone or rate	`src/hooks/useGamepad.ts` (deadzone constant) and `src/components/TeleopPanel.tsx` (`sendRateHz`).
De-stub a placeholder panel	`Nav2GoalPanel.tsx` / `PidTuningPanel.tsx` / `TransportDiagPanel.tsx` – remove the banner and wire the disabled button to the gateway RPC.
Add a new view (panel preset)	`store/useWindowStore.ts` – add a `ViewName` entry to `views` with its panel list and `lg` layout. The `StatusBar` dropdown picks them up automatically.
Reset persisted layouts	In the browser devtools, clear `localStorage['robot-dashboard-v6']`; or click "Reset to Default" in the Layouts dropdown for the active view only.
Inspect TX / RX traffic	Open the `DEBUG` view – the `PacketAnalyzer` panel shows the last 30 envelopes with type, size, age and a decoded preview.
Add a unit test	`src/hooks/<thing>.test.ts`; `npm run test`. The `useGamepad` suite is the template.

21 Operations Runbook

21.1 Scope

This section is the in-the-field guide. It assumes a Pi 5 has already been provisioned per `docs/PI_DEPLOYMENT.md` (one-time hardware-and-OS setup) and that the topology is the one drawn in `docs/ARCHITECTURE.md` (cockpit, Caddy, supervisor mux, k3s). **This file answers only one question**: what does the operator do during a session, when something is on fire, or when the routine "power-on, drive, power-off" sequence drifts. Setup, build, and topology are explicitly out of scope.

Throughout the runbook, `${STAR_ROOT}` is whatever absolute path the repo lives at on the Pi 5 (`/workspaces/STAR` on the development Pi; the value is interchangeable). All paths in this section use that placeholder.

21.2 Daily start sequence

Power up. Plug the chassis battery, give it 10 seconds for the Pi 5 to come out of POST and finish boot. You should see the Pi 5's PWR LED solid red and the activity LED blinking; the RX72N

PCB's 3.3 V LED solid green; the RPLiDAR LED solid green (the rotor will spin up only when SLAM activates it).

Wait for Pi services. The Pi runs three units that come up on boot, before any operator action:

- `star-cpu-governor.service` (one-shot, pins governor to `schedutil`).
- `caddy.service` (TLS termination on `https://star.local`, binds to the tailscale interface `100.64.0.6`).
- `star-cockpit-api.service` (HTTP API at `127.0.0.1:9102` that the Grafana cockpit toggles autonomy / e-stop / speed against).

These are *infrastructure* – they always run. They do not launch ROS2. To confirm the Pi is ready before you launch the stack:

```
systemctl is-active caddy star-cockpit-api star-cpu-governor
```

All three should print `active`.

Run `./start.sh`. From `${STAR_ROOT}` on the Pi (over SSH or local console):

```
./start.sh
```

`start.sh` is the canonical robot launcher. It:

1. Calls `require_host_setup` – warns (but does not block) if the Pi 5 host setup from `scripts/setup-pi5-host.` is incomplete.
2. Brings up the WiFi AP `STAR-Robot` on `wlan0` unless `-no-ap` is passed. The Pi becomes `192.168.50.1`.
3. Stops anything already running (`./stop.sh`), flushes the Cyclone DDS daemon's stale participant cache.
4. Probes for an RX72N over SPI for 4 seconds. Falls back to `virtual_rx72n` (DEV mode) if the probe fails.
5. Detects RPLiDAR on `/dev/rplidar` or `/dev/ttyUSB0`; stops `ModemManager` so it does not grab the port.
6. Launches in order: `virtual_rx72n` (DEV only), `star-gateway` (gRPC :50051, WS :8080), `star_spi_bridge`, fake odom TF (DEV only), SLAM stack (`slam.launch.py`), compliance engine, `star_gateway_bridge_main`, UI (Vite, :5173), RViz (optional), Lichtblick web (:8081).

The operator-facing flags are summarized below.

Flag	Meaning
(no flags)	Auto-detect everything; full stack. Default field choice.
`-no-lidar`	Skip LiDAR / SLAM / EKF even when `/dev/rplidar` is present. Use for indoor bench tests where the rotor is in the way.
`-no-ui`	Skip the Vite dev server. Use when no operator laptop is connected; saves 150 MB RAM.
`-no-compliance`	Skip the seven compliance nodes (ramp slope, doorway, protrusions, etc.). Use during pure-mobility testing.
`-no-ap`	Do not broadcast `STAR-Robot` AP. Use when the Pi is connected via Ethernet and you want to keep `wlan0` on a station network.
`-rviz`	Launch `rviz2` with `slam_lidar.rviz` after startup.
`-help`	Print usage.

Environment overrides: `STAR_SIMULATION_MODE=true|false` forces DEV / HW mode and skips the SPI probe; `STAR_WIFI_SSID`, `STAR_WIFI_PASS`, and `STAR_WIFI_IFACE` override the AP defaults.

Verify health. After `start.sh` prints its summary banner, sanity-check the stack:

Check	Expected
`nc -z localhost 50051`	gateway gRPC reachable.
`nc -z localhost 8080`	gateway WS reachable.
`nc -z localhost 8765`	`foxglove_bridge` listening.
`nc -z localhost 5173`	UI dev server reachable (HW WiFi mode prints `http://192.168.50.1:5173`).
`ros2 topic hz /scan`	10 Hz when LiDAR is up.
`ros2 topic hz /odom/unfiltered`	50 Hz from `simple_bridge` or `star_gateway_bridge`.
`ros2 topic echo /map -n 1 -field info`	a non-empty origin / resolution after the robot has moved.
`curl -s https://star.local/api/state`	`{"autonomy":..., "estop":..., "speed":...}` (cockpit API live).

21.3 Daily stop sequence

From `${STAR_ROOT}`:

```
./stop.sh
./stop.sh -stop-ap    (also tear down the WiFi AP)
```

`stop.sh` unwinds in the reverse of startup, with process-group SIGTERM first, a 3–5 second grace, then SIGKILL on survivors:

1. ROS2 client first: `star_gateway_bridge_main`.
2. ROS2 launch groups (`slam.launch.py`, `compliance.launch.py`, `star_spi_bridge.launch.py`, `stereo_camera.launch.py`).
3. Individual ROS2 nodes that may have outlived their launch parent (`sllidar`, `slam_toolbox`, `ekf_node`, `robot_state_publisher`, `static_transform_publisher`, `gscam`, `foxglove_bridge`).
4. Go binaries: `star-gateway`, `virtual_rx72n` (a 2-second sleep here lets the serial port release before the next `start.sh`).

5. Visualization: Vite UI, RViz, Lichtblick.
6. A final pgrep sweep for orphans.
7. By default, leaves the WiFi AP up so SSH from the operator laptop survives. `-stop-ap` tears it down.

When to use Ctrl-C vs `stop.sh` vs power-cycle. Ctrl-C in `start.sh`'s shell triggers the script's INT trap, which calls `./stop.sh` – equivalent. Use Ctrl-C if you ran `start.sh` in the foreground and want a clean unwind. Use `./stop.sh` from any other shell. Use a power-cycle only when `stop.sh` fails to release `/dev/ttyACM*` or `/dev/ttyUSB0` (rare; usually means a node is stuck in an uninterruptible kernel I/O wait, which only a hardware reset clears).

21.4 Dev mode

`./dev.sh` is a one-liner wrapper:

```
exec env STAR_SIMULATION_MODE=true ${STAR_ROOT}/start.sh -rviz "$@"
```

It forces the SPI probe to skip, brings up `virtual_rx72n` (the Go simulator that emits synthesized telemetry), launches RViz, and otherwise runs the same launch order as `start.sh`. Use it when:

- The robot hardware is not connected (desk-side dev).
- You want to test a new SLAM tuning on a recorded bag without the real motors.
- You want to debug the gateway or UI without an RX72N responding.

DEV mode publishes a static `odom -> base_link` TF (`robot stationary at origin`) so the SLAM TF chain stays intact. Never `./dev.sh` on a robot whose wheels are on the ground.

21.5 Field deployment checklist

Phase	Item	Pass criterion
Pre-flight	Power cable	Battery shows >11.5 V at the chassis distribution block.
Pre-flight	USB CDC enumerated	`ls /dev/star-rx72n-proto /dev/ttyACM*` resolves; if missing, see the E2 Lite recovery path inside `scripts/slam-mvp.sh`'s `ensure_mcu()`.
Pre-flight	RPLiDAR powered	`ls /dev/rplidar /dev/ttyUSB0`; LED on the lidar should be solid green.
Pre-flight	IMU calibration	BNO055 reaches sys / gyro / accel calib level 3 – check via the cockpit IMU panel or `ros2 topic echo /star/imu/calib_status -n 1`.
Pre-flight	Host setup	`./start.sh` prints no `require_host_setup` warnings. If it does, run `sudo bash ${STAR_ROOT}/scripts/setup-pi5-host.sh`.
Operational	E-stop test	`ros2 topic pub -once /star/estop std_msgs/Bool '{data: true}'` – wheels must brake within ≤50 ms. Confirm with cockpit / Grafana.
Operational	Autonomy toggle	`POST https://star.local/api/autonomy/toggle` – the cockpit or `ros2 topic echo /star/state/autonomy_enable` should flip.
Operational	Supervisor mux	With autonomy off, `/cmd_vel` drives the robot; with autonomy on, only `/nav2/cmd_vel` drives. Use `ros2 topic echo /cmd_vel_out`.
Post-flight	Log archive	`tar czf run-$(date +%Y%m%d-%H%M).tgz /tmp/star-logs/` – copy off the Pi before shutdown.
Post-flight	Mission report	Compliance engine writes a PDF under `${STAR_ROOT}/star-compliance/reports/` – archive alongside the logs.

21.6 Cockpit URLs and ports

These are the ports operators interact with. All bind to `0.0.0.0` on the Pi 5 unless noted otherwise (so they are reachable from a laptop joined to `STAR-Robot` or over the tailnet).

Port	Service	What it serves; credential strategy
443	Caddy on Pi 5	TLS terminator. Bound to `100.64.0.6` (tailscale interface) only. Routes `/api/*` to the cockpit API at `100.64.0.6:9102`; everything else proxies to `star_simple_bridge` on `localhost:9101`. Internal CA – one-time install on operator laptop.
50051	gateway gRPC	Used by the ROS2 `star_gateway_bridge` only. Localhost-only by convention; no auth.
8080	gateway WS	UI -> gateway WebSocket. `WS_STRICT_ORIGIN=false` during dev. No auth (lab-only).
8765	`foxglove_bridge`	ROS topic visualizer used by both the bundled Lichtblick at `:8081` and any external Foxglove Studio install (`ws://<pi5>:8765`). Plaintext; relies on tailscale or the AP for confidentiality.
5173	UI Vite dev server	The TypeScript operator UI. No auth, plaintext. AP-network only by default.
8081	Lichtblick web (`npx serve`)	Self-hosted Foxglove Studio fork. Requires one-time install via `scripts/setup-lichtblick-web.sh`.
9100	node_exporter	Prometheus host metrics scraped by Alloy on the Pi and pushed to k3s. No auth.
9101	`star_simple_bridge`	Serves `/map.{png,pgm,yaml}`, accepts `POST /map/reset?confirm=yes`, exposes its own custom metrics. Fronted by Caddy on `star.local`.
9102	`star_cockpit_api`	The Python HTTP backend for the cockpit toggles. Binds to `100.64.0.6:9102`; reachable only via Caddy at `https://star.local/api/*`. Endpoints: `GET /api/state`, `POST /api/autonomy/toggle`, `POST /api/estop/toggle`, `POST /api/slam/{start,stop}`, `POST /api/cmd_vel`, `POST /api/speed`.

The Grafana cockpit dashboard (panels: SLAM map, Motors / FL.. BR velocity / duty / faults, IMU roll / pitch / heading / gyro / accel, Odometry linear / angular / pose, LiDAR closest obstacle / bearing / range stats, Bridge health line-rate / parse errors / serial reopens) runs in the homelab k3s cluster, not on the Pi. It scrapes via Alloy and embeds `https://star.local/map.png` directly from the operator's browser.

21.7 Systemd unit reference

Unit	What it runs	Status / log access
`caddy.service`	`caddy run -config /etc/caddy/Caddyfile` as user `caddy`. Provides TLS on `star.local`, binds to `100.64.0.6`. `Restart=always`.	`systemctl status caddy`; `journalctl -u caddy -f`.
`star-cockpit-api.service`	Sources ROS2, exports `rmw_cyclonedds_cpp` + `CYCLONEDDS_URI`, runs `python3 /opt/star_cockpit_api/server.py`. Subscribes to latched supervisor state topics, publishes to `/star/autonomy_enable`, `/star/estop`, `/star/speed_scale`. `Restart=always`, `RestartSec=3`.	`systemctl status star-cockpit-api`; `journalctl -u star-cockpit-api -f`.
`star-slam-mvp.service`	*Optional, opt-in.* Runs `scripts/slam-mvp.sh start` as user `star` after `network-online.target` and a 8-second sleep so USB devices have enumerated. `Type=forking`, `RemainAfterExit=yes`, `Restart=no`. **Not enabled by default.** Enable with `sudo systemctl enable -now star-slam-mvp.service`.	`systemctl status star-slam-mvp`; log files live under `/tmp/slam-mvp-logs/<node>.log`.
`star-cpu-governor.service`	One-shot at very early boot (`DefaultDependencies=no`, `Before=base.target`); writes `schedutil` into every CPU's `scaling_governor`. `RemainAfterExit=yes` so `systemctl status` reflects "applied".	`systemctl status star-cpu-governor`; not in journal beyond the initial run.
`star-robot.service`	Optional auto-boot wrapper around `./start.sh -no-ui` / `./stop.sh`. Installed by `setup-pi5-host.sh`	Enabled only for field deployments; `journalctl -u star-robot` captures `/tmp/star-logs/systemd.log`.

The full safety model behind autonomy / e-stop arbitration is in §15; the supervisor topic mux table in `docs/ARCHITECTURE.md` is the authoritative version.

21.8 Wi-Fi and Ethernet helpers

Two helper scripts live on the Pi 5 `PATH` (installed by `setup-pi5-host.sh`, sourced from `${STAR_ROOT}/scripts/`):

Helper	Behaviour
`star-ap up`	Brings up the `STAR-Hotspot` NetworkManager profile on `wlan0`. Pi becomes `192.168.50.1` broadcasting SSID `STAR-Robot`. **Drops any existing wlan0 SSH session.**
`star-ap down`	Tears down the AP and lets NetworkManager rejoin the previous station network.
`star-ap status`	Reports current Wi-Fi mode (AP / station / down).
`star-eth up`	Force-activates the `netplan-eth0` shared profile. Pi becomes `10.42.0.1`, runs dnsmasq for the laptop on the other end. The autoconnect default is on, so this is rarely needed – plug in the cable and `star-eth status` should already report `UP`.
`star-eth down`	Drops the eth0 shared profile (cable can stay plugged in).
`star-eth auto on|off`	Toggles `connection.autoconnect` on the profile. `auto off` means the cable does nothing until you call `star-eth up`.
`star-eth status`	Shows carrier, IPv4, peer DHCP lease, and the autoconnect setting.

The two are mutually exclusive in practice – `star-ap up` owns `wlan0`; `star-eth` owns `eth0`. Running both is fine; the Pi just routes per-interface. SSH options are summarized in the cheat sheet at `scripts/star-help.sh` (run `star-help` on the Pi for a colorized rendering).

21.9 Common runtime issues

For the deep gotcha catalog – USB enumeration, AD2 buffer caps, FPU context save, MSTPCR module-stop – see §17. The few entries below are operations-specific and not duplicated there.

Robot moves but no telemetry on the cockpit / UI. `/cmd_vel_out` is being consumed (motors spin) but the UI shows stale topics. Almost always the WebSocket origin or URL: the gateway runs with `WS_STRICT_ORIGIN=false` during dev; verify with `ss -tlnp | grep 8080` that the gateway is on `:8080`, then check the UI is pointing at `ws://192.168.50.1:8080/ws` (AP) or `ws://10.42.0.1:8080/ws` (Eth). If the cockpit (Grafana panels) has no telemetry but Lichtblick over `ws://<pi>:8765` is fine, the bridge to k3s Prometheus is the issue, not the robot.

Robot freezes mid-motion. The RX72N IWDT will reset the chip if `watchdog_task` ever stops kicking, which forces motor outputs to zero on power-up. See §15 for the three layers (supervisor / comm / firmware-watchdog) and which one likely caught the fault. Recovery: power-cycle the chassis. The Pi 5 stays up; only the RX72N reboots.

Compass / yaw is bad after start. BNO055 needs the calibration loop to converge. Wave the chassis in a figure-eight for 15–30 seconds; sys / gyro / accel calib levels should each climb to 3. Until they do, the EKF heading drifts and SLAM matches poorly. The cockpit IMU panel exposes the calib state directly; `ros2 topic echo /star/imu/calib_status -n 1` is the CLI equivalent.

LiDAR reports zero ranges across the whole scan. Power, not data. Either:

- The RPLiDAR's 5 V rail dropped (battery low; chassis distribution loose) – LED on the lidar will be off or amber.
- `ModemManager` grabbed `/dev/ttyUSB0` during boot before `start.sh` could stop it. Run `sudo systemctl stop ModemManager` and `./start.sh` again.
- Permissions: confirm `ls -l /dev/ttyUSB0` shows group-writable; the udev rule from `setup-pi5-host.sh` should have set this.

21.10 Log locations

Source	Path	Access pattern
`start.sh` children	`/tmp/star-logs/<node>.log`	`tail -f /tmp/star-logs/gateway.log`, etc. Rotated daily by `/etc/logrotate.d/star-robot`, 3 generations, 20 MB cap.
`slam-mvp.sh` children	`/tmp/slam-mvp-logs/<node>.log`	`scripts/slam-mvp.sh logs <node>` or `tail -f /tmp/slam-mvp-logs/<node>.log`.
`star-cockpit-api.service`	journald only	`journalctl -u star-cockpit-api -f`; backs to `/var/log/journal/`.
`caddy.service`	journald only	`journalctl -u caddy -f`.
`star-slam-mvp.service`	journald + per-node files	`journalctl -u star-slam-mvp` for the systemd wrapper; per-node detail under `/tmp/slam-mvp-logs/`.
`star-robot.service` (if enabled)	`/tmp/star-logs/systemd.log` + `journalctl -u star-robot`	As specified in `StandardOutput=append:` of the unit.
`rosout` log files	`~/.ros/log/<run-id>/`	ROS2 daemon writes one directory per run; useful for crash post-mortems.
`nginx-style HTTP access` (Caddy)	journald only	`journalctl -u caddy | grep "GET /api"` for cockpit-API requests.

To grep across all robot logs from one shell:

```
grep -RIn ERROR /tmp/star-logs/ /tmp/slam-mvp-logs/
```

21.11 Ops references

The placeholder `${STAR_ROOT}` is whatever absolute path the repository lives at on the host – `/workspaces/STAR` on the development Pi 5, `~/STAR` on a fresh checkout, etc. Substitute as appropriate; the systemd units in `infra/pi5/etc/systemd/system/` hardcode `/workspaces/STAR` and must be re-templated if the repo lives elsewhere.

For a deeper map of every directory referenced in this section (`infra/pi5/`, `scripts/`, `star-ros2/src/<pkg>/la` the systemd units, `monitoring/`), see §11. For setup steps that run once per Pi (deploy keys, apt packages, host-level udev / governor), see `docs/PI_DEPLOYMENT.md`. For the supervisor mux topology and TLS / hostname strategy, see `docs/ARCHITECTURE.md`.

22 Logging Architecture

22.1 Scope

This section consolidates how a single log byte travels from a call site such as `rx_log_info("MOTOR", "...")` in the firmware to a line on an operator's screen. The same story is touched in §1 (`LOG_MESSAGE` frame definition), §9 (log multiplexing on shared transports) and §17 (operator-side log paths); this section is the central reference and the others cross-link back here.

22.2 Layers

The producer side runs in many ThreadX tasks; the single drainer runs in `comm_task` at 100 Hz; the gateway is the final consumer. The path is the same regardless of which transport is active.

```
rx_log_info("MOTOR", "...")            (call site, any task)
        |
        v
compile-time level filter              (LOG_LEVEL macro)
        |
        v
rx_log_uart ring buffer (4096 B)       (libs/rx_core/src/rx_log_uart.c)
        |
        v
comm_task drain @ 100 Hz               (internal_ship_log_frames)
        |
        v
LOG_MESSAGE frame, TYPE = 0x20         (libs/rx_frame/, gateway frame.go)
        |
        v
UART (SCI9 -> CY7C65213) | USB CDC port 3 | SPI MISO
        |
        v
Gateway TransportManager.handle()      (FrameTypeLogMessage case)
        |
        v
log.Printf("[RX72N] %s", payload)      (stderr / log file)
```

The ISR producer is a side branch into the same drain path – see §22.7 below.

22.3 Log levels

The level enum lives in `libs/rx_core/inc/rx_log.h`:

Level	Value	Compiled in when LOG_LEVEL >=
k_log_none	0	Nothing emitted; everything compiled out
k_log_error	1	Operation-preventing failures only
k_log_warn	2	Recoverable but worth noting
k_log_info	3	Default at -DCMAKE_BUILD_TYPE=Release
k_log_debug	4	Default at -DCMAKE_BUILD_TYPE=Debug
k_log_verbose	5	Maximum detail; trace-level

Filtering is compile-time, not runtime. Each macro expands inside an `#if (LOG_LEVEL >= k_log_*)` guard; if the level is below threshold the macro becomes `((void)0)` and the compiler removes the call site entirely. With `-DLOG_LEVEL=k_log_info` the binary contains zero bytes of debug-level log code – not zero overhead at runtime, but zero text section as well. This matters because log strings live in flash and add up quickly across the firmware.

Override at configure time:

```
cmake -DLOG_LEVEL=k_log_warn ...    # field-test build
cmake -DLOG_LEVEL=k_log_debug ...   # bench / development build
cmake -DLOG_LEVEL=k_log_none ...    # silent production build
```

22.4 Tagging convention

Every translation unit that logs declares a file-scope tag:

```
static const char* const s_tag = "MOTOR";
```

The double-`const` (pointer is const, the bytes pointed to are const) is mandatory: a round-1 source audit found tags declared as plain `static const char* s_tag` which placed the pointer itself in RAM as a writable variable, even though the string literal was in flash. That allowed a runaway pointer in another module to swap the tag at runtime. The hardened form parks both pointer and bytes in `.rodata`.

Tag strings are short (3-8 characters), upper-case, and module-scoped: `"MOTOR"`, `"COMM"`, `"USB_CDC"`, `"USB_ISR"`, `"ISR_LOG"`, etc. The gateway prefixes forwarded log lines with `[RX72N]`; the firmware tag is the *second* field, which lets log-grep workflows like `grep '\[RX72N\].*MOTOR'` pull per-module subsets.

22.5 Three logging API tiers

The macros in `rx_log.h` fan out into three categories with distinct safety and performance properties.

Tier	Caller context	What it does
rx_log_info(tag, msg)	Task only	Plain string log, [INFO] [TAG] msg\n
rx_log_info_str(tag, msg, str)	Task only	Logs msg followed by a bounded string (256 char cap)
rx_log_info_val(tag, msg, v)	Task only	C23 _Generic dispatch; uint8 / 16 / 32 decimal
rx_log_info_hex(tag, msg, v)	Task only	8-digit hex; common for rx_err_t or register dumps
rx_isr_log_push(event_id, v)	ISR or task	Lock-free SPSC enqueue; deferred emit at next drain

The same `_str`, `_val`, `_hex` suffixes exist for every level (`rx_log_error_*`, `rx_log_warn_*`, etc.). The `_val` family uses C23 `_Generic` to dispatch on the argument's type – `uint8_t` prints as decimal, `uint16_t` as decimal, `int32_t` as 8-digit hex (because `rx_err_t` is `int32_t` and is the dominant signed caller). To force decimal on a signed value, cast to `uint32_t` explicitly.

String logs (`_str`) are bounded by `k_log_str_max_len` (256) per NASA Power-of-10 Rule 2; longer inputs are silently truncated rather than blocked from logging.

22.6 Banner: ISR vs task context

Never call `rx_log_*` from anything reachable from an ISR. The `rx_log_uart` backend acquires a ThreadX mutex with `TX_WAIT_FOREVER`; that is legal from a task and a deadlock (plus undefined behavior under ThreadX) from interrupt context. Every USB and SCI ISR runs at priority 5 and would block the entire peripheral while waiting on a mutex held by a lower-priority task.

This rule is enforced by audit and by a banner comment at the top of every ISR-reachable source file. The two canonical examples are:

- `libs/rx_usb/src/rx_usb_isr.c` – USB0 vector 144, every function in the file is reachable from `usb0_usbi_isr()`. Banner enforces zero `rx_log_*` call sites.
- `libs/rx_usb/src/rx_usb_cdc.c` – CDC class handlers invoked from the same ISR. Same banner, same rule.

The CI / source-audit step grep-checks these files for any `rx_log_` substring; finding one is a hard failure.

The escape hatch is `rx_isr_log_push`, described next.

22.7 The deferred-ISR-log machinery

The deferred path lives in `libs/rx_core/inc/rx_isr_log.h` and `src/rx_isr_log.c`. It is a lock-free single-producer / single-consumer ring buffer sized at `k_rx_isr_log_capacity = 32` records of 8 bytes each (256 bytes total).

Field	Purpose
`event_id` (uint8)	Index into a static `s_isr_log_event_table[]` of {tag, message, level, has_value} tuples
`value` (uint32)	One auxiliary integer (port id, error code, register snapshot); event-specific

The producer side (ISR) does not allocate, format, or block. It reads `tail`, computes `(head + 1) & 31`, writes the record, and advances `head`. If the ring is full it bumps `s_total_dropped` and returns `false` – preferable to corrupting the ring or stalling the ISR.

The consumer side is `rx_isr_log_drain()`, called every 10 ms by `comm_task` (the same tick that ships log frames). It walks `tail` forward to the snapshot of `head` at entry, looks up each record's metadata in the static table, and emits via the regular `rx_log_*` macros. This means an ISR-pushed event ends up in exactly the same `rx_log_uart` ring as task-context logs, framed identically, with the right level prefix. After draining, if drops occurred since the previous tick the drain emits a single summary line `"ISR log overflow: dropped N records"` so observers know events were lost.

Adding a new ISR log event:

1. Append the new `k_isr_log_event_*` value to the enum in `rx_isr_log.h`.
2. Add the matching row to `s_isr_log_event_table[]` in `rx_isr_log.c` with the desired tag, message, level, and `has_value`.
3. In ISR code, call `rx_isr_log_push(k_isr_log_event_xxx, value)`.

The 32-record ring at the 100 Hz drain rate absorbs 3200 events/s before any drops, well above the steady-state USB ISR rate of 10-20 events/s during enumeration and 0/s after. The drop counter and high-water mark are exposed via `rx_isr_log_get_stats()`.

The deferred queue is what allows the `nFAULT`/`DRVOFF` / e-stop instrumentation in §15 to log from interrupt context without violating the no-mutex-from-ISR rule.

22.8 LOG_MESSAGE wire format

The byte-level format – SYNC (0x55AA) + header + TYPE = 0x20 + payload + CRC-32 – is documented in §1, and the `rx_frame_type_t` enum lives in `libs/rx_frame/inc/rx_frame.h`. The payload of a `LOG_MESSAGE` frame is 1..1024 bytes of raw ASCII log text, *not* null-terminated and may span multiple lines. The framing layer protects every chunk with the same CRC-32 used by command and response frames, so log bytes cannot fool the decoder by accidentally reproducing the sync word inside their payload.

22.9 Multi-transport log multiplexing

The same producer ring (`rx_log_uart`) feeds three different transports depending on which one the gateway has opened. The wire behavior is summarized below; see §9 for the framing rationale.

Transport	Log channel	Multiplex strategy
USB CDC composite (USB0)	Dedicated CDC port 3	Dedicated; logs never share a byte stream with proto frames
SCI9 UART (CY7C65213)	Same byte stream, framed	TYPE = 0x20 distinguishes log frames from proto frames
SPI (CIPO from RX72N)	Same byte stream, framed	TYPE = 0x20; gateway's frame decoder filters on type

The dedicated USB CDC log port is host-visible at `/dev/ttyACM3` on PROD boards or `/dev/ttyACM6` on TOM boards (the third interface of the 3-port composite – see §10 for the topology and re-enumeration story). Because the LOG port is its own USB interface, the operator can attach a plain serial terminal (`minicom -D /dev/ttyACM3`) and read firmware logs without going through the gateway at all – useful when the gateway itself is the suspect.

On UART and SPI the gateway does the demultiplexing: every received frame whose TYPE byte equals 0x20 is routed through the log path instead of the protobuf application path.

22.10 Gateway-side log forwarding

The gateway's `TransportManager.handle()` method (in `star-gateway/internal/manager/manager.go`) recognizes frame type `frame.FrameTypeLogMessage` (0x20). The handler:

1. Updates the implicit heartbeat (the firmware is alive enough to be emitting logs).

2. Calls `log.Printf("[RX72N] %s", string(result.Payload))`, writing the payload verbatim to the gateway's `stderr` (or whatever log sink the `log` package was redirected to under systemd).

3. Returns `nil` so the receive loop does not attempt to unmarshal the bytes as protobuf.

Notably, `LOG_MESSAGE` is intentionally absent from the gateway's HARQ allow-list – log frames are best-effort, not acknowledged, and not retransmitted.

22.11 Operator-visible log paths

Path	What you see there
`/tmp/star-logs/star-gateway.log`	Gateway log file when launched by the launch scripts (Pi 5)
`journalctl -u star-cockpit-api`	Same log stream when running as a systemd unit
Foxglove cockpit "Logs" panel	Live tail of gateway stderr forwarded over the cockpit websocket
`minicom -D /dev/ttyACM3` (PROD)	Raw firmware log stream straight off the dedicated USB CDC log port
`minicom -D /dev/ttyACM6` (TOM)	Same, on TOM-board hardware

The first three paths get the same bytes (the gateway is the single forwarder). The last two skip the gateway entirely and read directly from the firmware over USB; useful when the gateway is hung or has crashed and there is still a need to read RX72N logs.

22.12 Performance constraints

The producer side is cheap: `rx_log_info(...)` formats inline, acquires the ring mutex, memcpys the bytes, and returns. There is no I/O on the call path. That said, two rules from prior audits apply.

No `rx_log_debug` on the 100 Hz hot path. Round-3 review found that issuing a debug log from `motor_control_task`'s control loop pushes 80 bytes per tick – 8 kB/s – into the ring. Combined with `comm_task` also generating logs, this saturates the 4 KiB ring within 0.5 s on any moderately verbose build; bytes start dropping silently. Rule: the motor control loop logs only on transitions and faults, never per-tick.

No string logs on the hot path. `rx_log_*_str` iterates up to 256 characters per call; even though it is cycle-bounded it is the slowest log API. Use `_val` or `_hex` when the operand is an integer (almost always).

Drops are silent at the producer. The `rx_log_uart` ring counts dropped bytes in `rx_log_uart_stats_t.dro`
Inspect it via `rx_log_uart_get_stats()` during bring-up to confirm the build is not over-logging. The ISR-side ring exposes the same counter through `rx_isr_log_get_stats()`.

22.13 Where to look in code

Topic	Source
Log macros, level enum, `_Generic` dispatch	`star-rx72n-firmware/libs/rx_core/inc/rx_log.h`
UART backend ring buffer + drain API	`star-rx72n-firmware/libs/rx_core/src/rx_log_uart.c`
ISR-safe deferred queue (header)	`star-rx72n-firmware/libs/rx_core/inc/rx_isr_log.h`
ISR-safe deferred queue (impl + table)	`star-rx72n-firmware/libs/rx_core/src/rx_isr_log.c`
Drain + ship as LOG_MESSAGE frames	`star-rx72n-firmware/src/tasks/comm_task.c` (`internal_ship_log_frames`, `rx_isr_log_drain` call site)
Frame type 0x20 definition	`star-rx72n-firmware/libs/rx_frame/inc/rx_frame.h` (`k_frame_type_log_message`)
ISR logging discipline banner	`star-rx72n-firmware/libs/rx_usb/src/rx_usb_isr.c`, `rx_usb_cdc.c`
Gateway-side LOG_MESSAGE handler	`star-gateway/internal/manager/manager.go` (`FrameTypeLogMessage` case)
Frame type Go enum	`star-gateway/internal/frame/frame.go` (`FrameTypeLogMessage = 0x20`)